TODO PROJESİ · STAJ ÇALIŞMA NOTU

İş Akışı ve Satır Satır Kod Rehberi

Dört ana iş akışı (açılış, giriş/oturum, görev oluşturma, hatırlatma maili) ve bu akışlarda kullanılan 117 dosyanın her fonksiyonu, her satırı kısa notlarla.

.NET 8 Clean Architecture API · Angular 21 · SQL Server · Hangfire
Kod, projenin 8 Ekim 2026 tarihli halinden birebir alınmıştır.

Bu rehber nasıl okunur?

Her bölüm bir iş akışını anlatır. Bölümün başında, isteğin hangi dosyadan hangi dosyaya geçtiğini gösteren numaralı bir akış vardır. Ardından o akışta kullanılan dosyalar, kullanılma sırasıyla gelir. Bir dosya birden fazla akışta geçiyorsa yalnızca ilk geçtiği yerde açıklanır.

Sol sütun: kod
Dosyadan birebir alınmıştır, satır numaraları gerçek dosyadakiyle aynıdır.

Sağ sütun: açıklama
O satırın (ya da aynı işi yapan satır grubunun) ne yaptığı, gerekiyorsa neden öyle yazıldığı.

ƒ Mavi bant
Bir fonksiyonun, constructor'ın ya da tipin başladığı yer. Bantta tüm fonksiyonun tek cümlelik özeti vardır.

Soluk satırlar
Sadece `{` `}` gibi parantezlerden oluşan satırlar. Anlamları yoktur, açıklanmamıştır.

Katman renkleri

Web (Angular) tarayıcıda çalışan arayüz **API** HTTP kapısı (controller, middleware) **Application** use case'ler, kurallar, arayüzler **Domain** iş nesneleri ve kuralları **Infrastructure** veritabanı, mail, JWT gibi dış dünya

Web
Sayfalar, servisler, interceptor, guard. API'ye HTTP ile konuşur.

API
İsteği karşılar, request → command çevirir, hataları ProblemDetails yapar.

Application
Use case'ler iş akışını yönetir. Dış dünyayı sadece arayüzlerle (port) bilir.

Domain
TodoList, TodoItem, User... Hiçbir şeye bağımlı değildir.

Infrastructure
Application'daki arayüzleri uygular: EF Core, MailKit, JWT, Google.

Bağımlılık yönü: Web → API → Application → Domain. Infrastructure, Application'daki arayüzleri uygular (dependency inversion). API, her şeyi Program.cs'te birbirine bağlar (composition root).

İçindekiler

Bölüm 1 — Uygulamanın açılışı

1. Program.cs
2. DependencyInjection.cs
3. DependencyInjection.cs
4. TodoDbContextConfiguration.cs
5. AuthenticationSetup.cs
6. JwtSettings.cs
7. AuthorizationPolicies.cs
8. RateLimitingPolicies.cs
9. SecurityHeadersMiddleware.cs
10. BackgroundJobsSetup.cs
11. main.ts
12. app.config.ts
13. app.ts
14. app.routes.ts
15. auth.guards.ts
16. api-url.ts

Bölüm 2 — Giriş ve oturum

1. login-page.ts
2. auth.models.ts
3. auth.service.ts
4. auth.interceptor.ts
5. google-identity.service.ts
6. AuthController.cs
7. RequireClientHeaderAttribute.cs
8. LoginRequest.cs
9. RegisterRequest.cs
10. GoogleLoginRequest.cs
11. AuthConfigResponse.cs
12. RefreshTokenCookie.cs
13. LoginWithPasswordCommand.cs
14. LoginWithPasswordCommandValidator.cs
15. LoginWithPasswordUseCase.cs
16. EmailNormalizer.cs
17. IUserRepository.cs
18. UserRepository.cs
19. User.cs
20. UserRole.cs
21. IPasswordHasher.cs
22. AspNetCorePasswordHasher.cs
23. SessionIssuer.cs
24. AuthSession.cs
25. RefreshTokenSettings.cs
26. IAccessTokenGenerator.cs
27. AccessToken.cs
28. JwtAccessTokenGenerator.cs
29. ISecureTokenGenerator.cs
30. SecureTokenGenerator.cs
31. IRefreshTokenRepository.cs
32. RefreshTokenRepository.cs
33. RefreshToken.cs
34. AuthResultDto.cs
35. UserDto.cs
36. AuthenticationFailedException.cs
37. RefreshSessionUseCase.cs
38. LogoutUseCase.cs
39. GetCurrentUserUseCase.cs
40. ICurrentUser.cs
41. HttpContextCurrentUser.cs
42. RegisterWithPasswordCommand.cs
43. RegisterWithPasswordCommandValidator.cs
44. RegisterWithPasswordUseCase.cs
45. EmailAlreadyRegisteredException.cs
46. LoginWithGoogleCommand.cs
47. LoginWithGoogleCommandValidator.cs
48. LoginWithGoogleUseCase.cs
49. IGoogleTokenValidator.cs
50. GoogleUserInfo.cs
51. GoogleIdTokenValidator.cs
52. GoogleAuthenticationSettings.cs
53. IRoleAssignmentPolicy.cs
54. ConfiguredRoleAssignmentPolicy.cs

Bölüm 3 — Görev oluşturma (ve hata yolları)

1. todo.models.ts
2. todo-page.ts

3. todo-api.service.ts
4. TodosController.cs
5. CreateTodoRequest.cs
6. UpdateTodoRequest.cs
7. SetTodoCompletionStatusRequest.cs
8. CreateTodoCommand.cs
9. CreateTodoCommandValidator.cs
10. SharedValidationRules.cs
11. ValidatorExtensions.cs
12. RequestValidationException.cs
13. CreateTodoUseCase.cs
14. ITodoListRepository.cs
15. TodoListRepository.cs
16. TodoListAccessPolicy.cs
17. TodoList.cs
18. TodoItem.cs
19. TodoPriority.cs
20. NameComparison.cs
21. DomainRuleViolationException.cs
22. DuplicateNameException.cs
23. TodoDbContext.cs
24. DatabaseErrorTranslator.cs
25. ConflictException.cs
26. DataAccessException.cs
27. ConcurrencyConflictException.cs
28. TodoItemDto.cs
29. ValidationExceptionHandler.cs
30. DomainExceptionHandler.cs
31. ApplicationExceptionHandler.cs
32. DatabaseExceptionHandler.cs
33. UnhandledExceptionHandler.cs
34. ProblemDetailsWriter.cs

Bölüm 4 — Hatırlatma maili (arka plan işi)

1. TodoReminderJob.cs
2. SendDueTodoRemindersUseCase.cs
3. ReminderSettings.cs
4. ReminderRunResult.cs
5. ITodoReminderRepository.cs
6. TodoReminderRepository.cs
7. TodoReminderEmailComposer.cs
8. IEmailSender.cs
9. EmailMessage.cs
10. EmailRejectedException.cs
11. EmailServiceUnavailableException.cs
12. MailKitEmailSender.cs
13. SmtpSettings.cs

Ek — Aynı kalıpla çalışan diğer dosyalar

Bölüm 1 — Uygulamanın açılışı

HerSeyiBaslat.bat önce veritabanını günceller (dotnet ef database update), sonra API'yi ve Angular sitesini başlatır. Bu bölüm, ilk istek gelmeden önce iki tarafta neyin hazırlandığını anlatır: hangi servisler DI'a kaydedilir, hangi middleware'ler hangi sırayla dizilir, Angular hangi ayarlarla ayağa kalkar.

Akış adım adım

1. Serilog kurulur, `WebApplication` builder oluşturulur. Bütün kayıtlar buradan tetiklenir.
   `todo.API/Program.cs`
2. `AddApplication()`: use case'ler ve FluentValidation validator'ları DI'a kaydedilir, doğrulama mesajları Türkçe yapılır.
   `todo.Application/DependencyInjection.cs`
3. `AddInfrastructure()`: DbContext, repository'ler, JWT, şifre, Google, mail servisleri ve ayar nesneleri kaydedilir.
   `todo.Infrastructure/DependencyInjection.cs`
4. SQL Server bağlantısı ve geçici hatalarda otomatik yeniden deneme ayarlanır.
   `todo.Infrastructure/Persistence/TodoDbContextConfiguration.cs`
5. JWT doğrulama kuralları, varsayılan 'giriş zorunlu' politikası, AdminOnly politikası ve hız limiti tanımlanır.
   `todo.API/Authentication/AuthenticationSetup.cs`
6. Hangfire kurulur; SMTP ayarlıysa hatırlatma işi 5 dakikada bir çalışacak şekilde kaydedilir.
   `todo.API/BackgroundJobs/BackgroundJobsSetup.cs`
7. Her cevaba güvenlik header'ları ekleyen middleware. Program.cs middleware sırasının en başına koyar.
   `todo.API/Security/SecurityHeadersMiddleware.cs`
8. Tarayıcı tarafı: Angular uygulaması başlatılır.
   `todo.Web/src/main.ts`
9. Zoneless çalışma, router, HttpClient + interceptor ve açılışta oturumu geri yükleme ayarlanır.
   `todo.Web/src/app/app.config.ts`
10. Hangi adreste hangi sayfanın açılacağı ve hangi guard'ın kapıda duracağı belirlenir.
    `todo.Web/src/app/app.routes.ts`

Bu bölümde açıklanan dosyalar (kullanım sırasıyla): Program.cs, DependencyInjection.cs, DependencyInjection.cs, TodoDbContextConfiguration.cs, AuthenticationSetup.cs, JwtSettings.cs, AuthorizationPolicies.cs, RateLimitingPolicies.cs, SecurityHeadersMiddleware.cs, BackgroundJobsSetup.cs, main.ts, app.config.ts, app.ts, app.routes.ts, auth.guards.ts, api-url.ts

API `todo.API/Program.cs`

> Uygulamanın giriş noktasıdır: tüm servisleri DI kabına kaydeder, HTTP istek hattını (middleware sırasını) kurar ve sunucuyu başlatır. API katmanı diğer katmanları burada birbirine bağlar.

`using System.Text.Json.Serialization;`
`using Serilog;`
`using Serilog.Events;`
	JSON enum dönüştürücüsü ve Serilog loglama kütüphanesi içeri alınır.

`using TodoApi.API.Authentication;`
`using TodoApi.API.BackgroundJobs;`
`using TodoApi.API.ExceptionHandling;`
`using TodoApi.API.Security;`
`using TodoApi.Application;`
`using TodoApi.Infrastructure;`
	Projenin kendi katmanları içeri alınır: API'nin kimlik doğrulama, arka plan işleri, hata yönetimi, güvenlik klasörleri ile Application ve Infrastructure katmanları.

`Log.Logger = new LoggerConfiguration()`
`    .WriteTo.Console()`
`    .CreateLogger();`
	Uygulama daha ayağa kalkmadan konsola yazan geçici bir logger kurulur; böylece başlangıçta çıkan hatalar da loglanabilir.

`try` `{`	Tüm başlatma kodu `try` bloğuna alınır; açılışta bir hata olursa aşağıdaki `catch` onu yakalayıp loglayacak.
`    var builder = WebApplication.CreateBuilder(args);`	`WebApplication.CreateBuilder` appsettings, user secrets, ortam değişkenleri gibi ayarları okuyan ve servis kaydı yapılan builder nesnesini oluşturur.

`    builder.Services.AddSerilog((services, logger) => logger`
`        .ReadFrom.Configuration(builder.Configuration)`
`        .ReadFrom.Services(services)`
`        .Enrich.FromLogContext()`
`        .MinimumLevel.Override("Microsoft.AspNetCore.Diagnostics.ExceptionHandlerMiddleware", LogEventLevel.Fatal));`
	Serilog asıl logger olarak kaydedilir: ayarları config'den okur, DI servislerini kullanabilir, LogContext bilgisini ekler. ExceptionHandlerMiddleware logu susturulur çünkü hatalar kendi handler'larımızda zaten loglanıyor.

`builder.WebHost.ConfigureKestrel(options => { options.AddServerHeader = false; });`	Kestrel web sunucusu yanıtlara `Server` başlığı eklemez; saldırgana sunucu teknolojisi hakkında bilgi verilmemiş olur.
`builder.Services .AddControllers() .AddJsonOptions(options => { options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()); });`	Controller'lar DI'a eklenir. `JsonStringEnumConverter` enum değerlerini JSON'da sayı yerine metin (ör. "High") olarak yazar ve okur.
`builder.Services.AddProblemDetails(options => { options.CustomizeProblemDetails = context => { context.ProblemDetails.Extensions["traceId"] = ProblemDetailsWriter.GetTraceId(context.HttpContext); }; });`	Standart hata formatı ProblemDetails (RFC 7807) açılır; her hata yanıtına `traceId` eklenir ki hata loglarda kolayca bulunabilsin. `=>` lambda yani kısa fonksiyon tanımıdır.
`builder.Services.AddExceptionHandler<ValidationExceptionHandler>(); builder.Services.AddExceptionHandler<DomainExceptionHandler>(); builder.Services.AddExceptionHandler<ApplicationExceptionHandler>(); builder.Services.AddExceptionHandler<DatabaseExceptionHandler>(); builder.Services.AddExceptionHandler<UnhandledExceptionHandler>();`	Hata yakalayıcılar sırayla kaydedilir: önce doğrulama, sonra domain, uygulama, veritabanı, en son beklenmeyen hatalar. Sıra önemlidir; ilk uygun handler hatayı üstlenir. `<T>` generic tip parametresidir.
`string[] allowedOrigins = builder.Configuration .GetSection("Cors:AllowedOrigins") .GetChildren() .Select(child => child.Value) .Where(origin => string.IsNullOrWhiteSpace(origin) == false) .Select(origin => origin!) .ToArray();`	appsettings'teki `Cors:AllowedOrigins` listesi okunur, boş değerler atılır ve dizi yapılır. `origin!` derleyiciye 'bu null değil' der.
`if (allowedOrigins.Length == 0) { throw new InvalidOperationException("At least one Cors:AllowedOrigins entry is required."); }`	Hiç izinli adres yoksa uygulama bilerek açılmaz; yanlış yapılandırma sessizce geçmek yerine hemen fark edilir (fail-fast).
`builder.Services.AddCors(options => { options.AddPolicy("TodoWeb", policy => { policy .WithOrigins(allowedOrigins) .WithHeaders("Content-Type", "Authorization", RequireClientHeaderAttribute.HeaderName) .WithMethods("GET", "POST", "PUT", "PATCH", "DELETE") .AllowCredentials(); }); });`	"TodoWeb" adlı CORS politikası: sadece izinli Angular adresleri, belirli başlıklar (`X-Todo-Client` dahil) ve HTTP metodları kabul edilir. `AllowCredentials` refresh token çerezinin gönderilmesine izin verir.
`builder.Services.AddApplication(); builder.Services.AddInfrastructure(builder.Configuration); builder.Services.AddTodoAuthentication(builder.Configuration);`	Katmanların kendi kayıt metodları çağrılır: use case'ler (Application), DbContext/repository/servisler (Infrastructure) ve JWT, yetki, hız sınırı (API).
`builder.Services.AddTodoBackgroundJobs(builder.Configuration);`	Hangfire arka plan işleri (hatırlatma e-postaları) servis olarak eklenir.
`var app = builder.Build();`	`Build()` ile servis kaydı biter ve çalıştırılabilir uygulama nesnesi oluşur; bundan sonra middleware sırası tanımlanır.
`app.UseSecurityHeaders();`	Güvenlik başlıklarını ekleyen middleware en başa konur ki hata yanıtları dahil her yanıtta bulunsun.
`app.UseSerilogRequestLogging(options => { options.MessageTemplate = "HTTP {RequestMethod} {RequestPath} responded {StatusCode} in {Elapsed:0} ms (user {UserId})"; options.EnrichDiagnosticContext = (diagnosticContext, httpContext) => { diagnosticContext.Set("UserId", httpContext.User.FindFirst("sub")?.Value ?? "anonymous"); }; });`	Her HTTP isteği için tek satırlık özet log yazılır (metod, yol, durum kodu, süre). `?.` null ise durur, `??` null ise "anonymous" yazar.
`if (app.Environment.IsProduction()) { app.UseHsts(); app.UseHttpsRedirection(); }`	Sadece canlı ortamda HSTS (tarayıcıya hep HTTPS kullan der) ve HTTP'den HTTPS'e yönlendirme açılır; geliştirmede gereksiz zorluk çıkarmasın diye.
`app.UseExceptionHandler(); app.UseCors("TodoWeb");`	Yukarıda kaydedilen exception handler'lar devreye alınır, ardından "TodoWeb" CORS politikası uygulanır.
`app.UseRateLimiter(); app.UseAuthentication(); app.UseAuthorization();`	Hız sınırlayıcı, kimlik doğrulama (JWT okunur) ve yetkilendirme (izin kontrolü) çalışır. Authentication her zaman Authorization'dan önce gelmelidir.
`app.UseTodoBackgroundJobs(); app.MapControllers();`	Hatırlatma işi zamanlanır ve Hangfire paneli eklenir; ardından controller'lardaki route'lar uç nokta olarak bağlanır.
`app.Run();`	`app.Run()` sunucuyu başlatır ve uygulama kapanana kadar istekleri dinler.

```
}
catch (Exception exception) when (exception is not HostAbortedException)
{
    Log.Fatal(exception, "The application failed to start.");
    throw;
}
finally
{
    Log.CloseAndFlush();
}
```

Açılışta oluşan hata Fatal seviyede loglanıp tekrar fırlatılır. `when (... is not HostAbortedException)` EF araçlarının normal durdurmasını hata saymamak içindir.

`finally` her durumda çalışır; bellekte bekleyen loglar dosyaya/konsola yazılıp logger kapatılır, böylece son loglar kaybolmaz.

ƒ class Program — Top-level kodun gizli Program sınıfını public yapar.

```
public partial class Program
{
}
```

`partial` ile derleyicinin ürettiği Program sınıfı public açılır; entegrasyon testleri `WebApplicationFactory<Program>` ile uygulamayı başlatabilsin diye.

Application `todo.Application/DependencyInjection.cs`

> Application katmanının tüm servislerini (FluentValidation doğrulayıcıları ve use case'ler) DI kabına kaydeden tek giriş noktasıdır. Program.cs bunu `AddApplication()` ile çağırır.

```
using System;
using System.Globalization;
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
```

Temel .NET tipleri, kültür (dil) ayarları, FluentValidation ve DI arayüzleri içeri alınır.

```
using TodoApi.Application.Features.Authentication.Common;
using TodoApi.Application.Features.Authentication.GetCurrentUser;
using TodoApi.Application.Features.Authentication.LoginWithGoogle;
using TodoApi.Application.Features.Authentication.LoginWithPassword;
using TodoApi.Application.Features.Authentication.Logout;
using TodoApi.Application.Features.Authentication.RefreshSession;
using TodoApi.Application.Features.Authentication.RegisterWithPassword;
using TodoApi.Application.Features.Reminders.SendDueTodoReminders;
using TodoApi.Application.Features.TodoLists.CreateTodoList;
using TodoApi.Application.Features.TodoLists.DeleteTodoList;
using TodoApi.Application.Features.TodoLists.GetTodoListById;
using TodoApi.Application.Features.TodoLists.GetTodoLists;
using TodoApi.Application.Features.TodoLists.RenameTodoList;
using TodoApi.Application.Features.Todos.CreateTodo;
using TodoApi.Application.Features.Todos.DeleteTodo;
using TodoApi.Application.Features.Todos.GetTodoById;
using TodoApi.Application.Features.Todos.GetTodosInList;
using TodoApi.Application.Features.Todos.SetTodoCompletionStatus;
using TodoApi.Application.Features.Todos.UpdateTodo;
using TodoApi.Application.Features.Users.GetUsers;
```

Kaydedilecek use case sınıflarının bulunduğu özellik (feature) klasörleri içeri alınır; her klasör tek bir iş akışını temsil eder (vertical slice).

```
namespace TodoApi.Application;
```

Dosya kapsamlı namespace: bu dosyadaki her şey `TodoApi.Application` ad alanına aittir.

ƒ class DependencyInjection — Application katmanının servis kayıtlarını toplayan statik yardımcı sınıf.

```
public static class DependencyInjection
{
```

`static class` örneği oluşturulamaz, sadece statik metod içerir; extension metod yazmak için gereklidir.

ƒ AddApplication — Doğrulayıcıları ve tüm use case'leri DI'a ekleyip aynı koleksiyonu geri döndürür.

```
    public static IServiceCollection AddApplication(this IServiceCollection services)
    {
        if (services == null)
        {
            throw new ArgumentNullException(nameof(services));
        }

        ValidatorOptions.Global.LanguageManager.Culture = CultureInfo.GetCultureInfo("tr-TR");

        services.AddValidatorsFromAssemblyContaining<CreateTodoListCommandValidator>();

        services.AddScoped<CreateTodoListUseCase>();
        services.AddScoped<GetTodoListByIdUseCase>();
        services.AddScoped<GetTodoListsUseCase>();
        services.AddScoped<RenameTodoListUseCase>();
        services.AddScoped<DeleteTodoListUseCase>();
        services.AddScoped<CreateTodoUseCase>();
        services.AddScoped<GetTodosInListUseCase>();
        services.AddScoped<GetTodoByIdUseCase>();
        services.AddScoped<UpdateTodoUseCase>();
        services.AddScoped<SetTodoCompletionStatusUseCase>();
        services.AddScoped<DeleteTodoUseCase>();
```

`this IServiceCollection` bunu extension metod yapar; böylece `builder.Services.AddApplication()` şeklinde çağrılabilir.

Koleksiyon null gelirse anlaşılır bir hata fırlatılır; `nameof` parametre adını metne çevirir.

FluentValidation'ın hazır hata mesajları Türkçe üretilsin diye kültür tr-TR yapılır; bu ayar tüm uygulama için geçerlidir.

`CreateTodoListCommandValidator`'ın bulunduğu derlemedeki tüm doğrulayıcılar otomatik bulunup kaydedilir; tek tek yazmaya gerek kalmaz.

Liste ve todo use case'leri `AddScoped` ile kaydedilir: her HTTP isteği için bir örnek oluşur ve aynı istekteki DbContext ile ömrü uyumlu olur.

```
        services.AddScoped<SessionIssuer>();
        services.AddScoped<RegisterWithPasswordUseCase>();
        services.AddScoped<LoginWithPasswordUseCase>();
        services.AddScoped<LoginWithGoogleUseCase>();
        services.AddScoped<RefreshSessionUseCase>();
        services.AddScoped<LogoutUseCase>();
        services.AddScoped<GetCurrentUserUseCase>();
```

Kimlik doğrulama use case'leri kaydedilir; `SessionIssuer` giriş akışlarının ortak kullandığı, access ve refresh token üreten yardımcıdır.

```
        services.AddScoped<GetUsersUseCase>();
        services.AddScoped<SendDueTodoRemindersUseCase>();
```

Admin için kullanıcı listeleme ve Hangfire işinin çağırdığı hatırlatma e-postası gönderme use case'leri kaydedilir.

```
        return services;
```

Aynı koleksiyon geri döndürülür ki çağıran taraf başka `Add...` metodlarını zincirleyebilsin (fluent kullanım).

```
    }
}
```

Infrastructure `todo.Infrastructure/DependencyInjection.cs`

> Infrastructure katmanının servislerini (DbContext, repository'ler, JWT, Google, şifre, e-posta servisleri ve ayar nesneleri) DI kabına kaydeder. Application'daki arayüzler burada somut sınıflara bağlanır.

```
using System;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
```

Temel tipler, EF Core, yapılandırma okuma ve DI kütüphaneleri içeri alınır; `Extensions` paketi `TryAddSingleton` için gereklidir.

```
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Email;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Application.Features.Authentication.Common;
using TodoApi.Application.Features.Reminders.SendDueTodoReminders;
```

Application katmanındaki arayüzler ve ayar tipleri içeri alınır; Infrastructure onları uygular (bağımlılık içe doğru akar).

```
using TodoApi.Infrastructure.Persistence;
using TodoApi.Infrastructure.Services.Authentication;
using TodoApi.Infrastructure.Services.Email;
```

Infrastructure'ın kendi somut sınıfları: veritabanı, kimlik doğrulama ve e-posta servisleri.

```
namespace TodoApi.Infrastructure;
```

Dosya kapsamlı namespace tanımı.

ƒ class DependencyInjection — Infrastructure servislerini ve ayarlarını DI'a kaydeden statik sınıf.

```
public static class DependencyInjection
```

Extension metod barındırabilmesi için `static` tanımlanmış sınıf.

```
{
```

ƒ AddInfrastructure — Veritabanı, repository, ayar ve dış servis kayıtlarını yapar; eksik ayarda uygulamayı durdurur.

```
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
```

`this IServiceCollection` ile extension metod olur; ayrıca ayarları okumak için `IConfiguration` alır.

```
    {
        if (services == null)
        {
            throw new ArgumentNullException(nameof(services));
        }

        if (configuration == null)
        {
            throw new ArgumentNullException(nameof(configuration));
        }
```

İki parametre için null kontrolü yapılır; hatalı çağrı anlaşılır bir mesajla hemen fark edilir.

```
        string? connectionString = configuration.GetConnectionString("TodoDatabase");

        if (string.IsNullOrWhiteSpace(connectionString))
        {
            throw new InvalidOperationException(
                "The TodoDatabase connection string is missing.");
        }
```

"TodoDatabase" bağlantı metni okunur; yoksa uygulama açılmaz. `string?` değişkenin null olabileceğini belirtir.

```
        services.AddDbContext<TodoDbContext>(options =>
        {
            options.UseTodoSqlServer(connectionString);
        });
```

`TodoDbContext` scoped olarak kaydedilir ve SQL Server'a, tekrar deneme özelliğiyle bağlanacak şekilde ayarlanır (`UseTodoSqlServer`).

```
        services.AddScoped<ITodoListRepository, TodoListRepository>();
        services.AddScoped<IUserRepository, UserRepository>();
        services.AddScoped<IRefreshTokenRepository, RefreshTokenRepository>();
        services.AddScoped<ITodoReminderRepository, TodoReminderRepository>();
```

Repository arayüzleri somut sınıflara bağlanır. Use case'ler sadece arayüzü bilir; EF Core detayı Infrastructure'da kalır (Clean Architecture).

```
        JwtSettings jwtSettings = JwtSettings.FromConfiguration(configuration);
        GoogleAuthenticationSettings googleSettings =
            GoogleAuthenticationSettings.FromConfiguration(configuration);
        ConfiguredRoleAssignmentPolicy roleAssignmentPolicy =
            ConfiguredRoleAssignmentPolicy.FromConfiguration(configuration);
        RefreshTokenSettings refreshTokenSettings = ReadRefreshTokenSettings(configuration);
        SmtpSettings smtpSettings = SmtpSettings.FromConfiguration(configuration);
        ReminderSettings reminderSettings = ReadReminderSettings(configuration);
```

JWT, Google, rol atama, refresh token, SMTP ve hatırlatma ayarları açılışta okunup doğrulanır; hatalı ayar varsa uygulama hemen hata verir.

```
        services.AddSingleton(jwtSettings);
        services.AddSingleton(googleSettings);
        services.AddSingleton<IRoleAssignmentPolicy>(roleAssignmentPolicy);
        services.AddSingleton(refreshTokenSettings);
        services.AddSingleton(smtpSettings);
        services.AddSingleton(reminderSettings);
```

Ayar nesneleri singleton kaydedilir: uygulama boyunca tek örnek yeterlidir çünkü değişmezler. Rol politikası arayüz tipiyle kaydedilir.

```
66          services.TryAddSingleton(TimeProvider.System);
```

Sistem saati `TimeProvider` olarak eklenir; `TryAdd` testler sahte saat kaydettiyse onu ezmez. Saati soyutlamak test yazmayı kolaylaştırır.

```
68          services.AddSingleton<IPasswordHasher, AspNetCorePasswordHasher>();
69          services.AddSingleton<IAccessTokenGenerator, JwtAccessTokenGenerator>();
70          services.AddSingleton<IGoogleTokenValidator, GoogleIdTokenValidator>();
71          services.AddSingleton<ISecureTokenGenerator, SecureTokenGenerator>();
72          services.AddSingleton<IEmailSender, MailKitEmailSender>();
```

Şifre hashleme, JWT üretme, Google token doğrulama, güvenli rastgele token ve MailKit e-posta servisleri durumsuz oldukları için singleton kaydedilir.

```
74          return services;
```

Zincirleme çağrı için aynı servis koleksiyonu döndürülür.

```
75      }
```

ƒ ReadRefreshTokenSettings — Refresh token ömrünü (gün) config'den okuyup doğrular ve ayar nesnesi oluşturur.

```
77      private static RefreshTokenSettings ReadRefreshTokenSettings(IConfiguration configuration)
```

`private static` sadece bu sınıf içinden çağrılabilen, örneğe ihtiyaç duymayan yardımcı metoddur.

```
78      {
```

```
79          string lifetimeText = configuration["Jwt:RefreshTokenLifetimeDays"] ?? string.Empty;
80          bool lifetimeIsNumber = int.TryParse(lifetimeText, out int lifetimeDays);
```

`Jwt:RefreshTokenLifetimeDays` metni okunur ve sayıya çevrilmeye çalışılır; `TryParse` hata fırlatmaz, başarıyı `bool` döndürür, değeri `out` ile verir.

```
82          if (lifetimeIsNumber == false || lifetimeDays <= 0)
83          {
84              throw new InvalidOperationException(
85                  "The Jwt:RefreshTokenLifetimeDays setting must be a positive whole number.");
86          }
```

Değer sayı değilse veya sıfır/negatifse açıklayıcı bir hata ile uygulama durdurulur.

```
88          return new RefreshTokenSettings(TimeSpan.FromDays(lifetimeDays));
```

Gün sayısı `TimeSpan`'e çevrilerek `RefreshTokenSettings` oluşturulur.

```
89      }
```

ƒ ReadReminderSettings — "Reminders" bölümündeki hatırlatma ayarlarını varsayılanlarla okuyup `ReminderSettings` üretir.

```
91      private static ReminderSettings ReadReminderSettings(IConfiguration configuration)
```

Hatırlatma e-postası ayarlarını okuyan yardımcı metodun imzası.

```
92      {
```

```
93          IConfigurationSection section = configuration.GetSection("Reminders");
```

appsettings'teki "Reminders" bölümü alınır.

```
95          bool isEnabled = ReadBoolean(section, "Enabled", defaultValue: true);
96          int leadTimeHours = ReadPositiveInteger(section, "LeadTimeHours", defaultValue: 24);
97          string schedule = section["Schedule"] ?? "*/5 * * * *";
98          int batchSize = ReadPositiveInteger(section, "BatchSize", defaultValue: 50);
99          string timeZoneId = section["DisplayTimeZone"] ?? "Europe/Istanbul";
100         int maximumAttempts = ReadPositiveInteger(section, "MaximumAttempts", defaultValue: 3);
101         int retryDelayMinutes = ReadPositiveInteger(section, "RetryDelayMinutes", defaultValue: 15);
```

Her ayar okunur, yoksa varsayılan kullanılır: açık, 24 saat önce, 5 dakikada bir (cron), 50'lik grup, İstanbul saati, 3 deneme, 15 dk bekleme.

```
103         TimeZoneInfo displayTimeZone;
```

Saat dilimi değişkeni try bloğunda atanacağı için önceden tanımlanır.

```
105         try
106         {
107             displayTimeZone = TimeZoneInfo.FindSystemTimeZoneById(timeZoneId);
108         }
```

Saat dilimi kimliği işletim sisteminde aranır; e-postadaki tarihler kullanıcının yerel saatiyle gösterilsin diye.

```
109         catch (TimeZoneNotFoundException exception)
110         {
111             throw new InvalidOperationException(
112                 $"The Reminders:DisplayTimeZone setting '{timeZoneId}' is not a known time zone.",
113                 exception);
114         }
```

Bilinmeyen saat dilimi daha açıklayıcı bir hataya sarılır; orijinal hata `exception` olarak içeride saklanır. `$"..."` metin içine değişken koyar.

```
116         return new ReminderSettings(
117             isEnabled,
118             TimeSpan.FromHours(leadTimeHours),
119             schedule,
120             batchSize,
121             displayTimeZone,
122             maximumAttempts,
123             TimeSpan.FromMinutes(retryDelayMinutes));
```

Okunan değerler uygun tiplere (saat, dakika `TimeSpan`) çevrilerek değişmez `ReminderSettings` nesnesi oluşturulur.

```
124     }
```

ƒ ReadBoolean — Bölümden true/false ayarı okur; boşsa varsayılanı döner, geçersizse hata fırlatır.

```
126     private static bool ReadBoolean(IConfigurationSection section, string key, bool defaultValue)
```

Genel amaçlı bool ayar okuyucunun imzası.

```
127     {
```

```
128         string? value = section[key];
129
130         if (string.IsNullOrWhiteSpace(value))
131         {
132             return defaultValue;
133         }
```

Değer okunur; boşsa varsayılan değer döndürülür.

```
135         if (bool.TryParse(value, out bool result) == false)
136         {
137             throw new InvalidOperationException($"The {section.Path}:{key} setting must be true or false.");
138         }
```

Değer `true` / `false` değilse hangi ayarın bozuk olduğunu tam yoluyla (`section.Path`) belirten hata fırlatılır.

```
140         return result;
```

Geçerli bool değer döndürülür.

```
    }
```

ƒ ReadPositiveInteger — Bölümden pozitif tam sayı ayarı okur; boşsa varsayılanı döner, geçersizse hata fırlatır.

```
    private static int ReadPositiveInteger(IConfigurationSection section, string key, int defaultValue)
```
Genel amaçlı pozitif sayı okuyucunun imzası.

```
    {
        string? value = section[key];

        if (string.IsNullOrWhiteSpace(value))
        {
            return defaultValue;
        }
```
Değer okunur; boşsa varsayılan değer döndürülür.

```
        if (int.TryParse(value, out int result) == false || result <= 0)
        {
            throw new InvalidOperationException($"The {section.Path}:{key} setting must be a positive whole number.");
        }
```
Sayıya çevrilemezse veya sıfır/negatifse ayarın yolunu içeren açıklayıcı hata fırlatılır.

```
        return result;
```
Geçerli sayı döndürülür.

```
    }
}
```

Infrastructure `todo.Infrastructure/Persistence/TodoDbContextConfiguration.cs`

> DbContext'in SQL Server'a nasıl bağlanacağını tek yerde tanımlar. Hem uygulama (DI) hem de EF tasarım araçları aynı ayarı kullansın diye ayrı tutulmuştur.

```
using System;
using Microsoft.EntityFrameworkCore;
```
`TimeSpan` için System ve EF Core içeri alınır.

```
namespace TodoApi.Infrastructure.Persistence;
```
Dosya kapsamlı namespace: Persistence (veri saklama) klasörü.

ƒ class TodoDbContextConfiguration — SQL Server bağlantı ve tekrar deneme ayarlarını içeren statik yardımcı sınıf.

```
public static class TodoDbContextConfiguration
```
Extension metod içerdiği için `static class` olarak tanımlanır.

```
{
    public const int MaximumRetryCount = 3;
    public static readonly TimeSpan MaximumRetryDelay = TimeSpan.FromSeconds(5);
```
En fazla 3 deneme ve denemeler arası en fazla 5 saniye. `const` derleme anında sabit; `static readonly` ise çalışma anında bir kez atanır (TimeSpan const olamaz).

ƒ UseTodoSqlServer — DbContext seçeneklerine SQL Server bağlantısını ve geçici hata tekrar denemesini ekler.

```
    public static void UseTodoSqlServer(this DbContextOptionsBuilder options, string connectionString)
```
`this DbContextOptionsBuilder` ile `options.UseTodoSqlServer(...)` şeklinde çağrılabilen extension metoddur.

```
    {
        options.UseSqlServer(connectionString, sqlServer =>
```
EF Core'a SQL Server sağlayıcısı ve bağlantı metni verilir; lambda ile SQL Server'a özel ayarlar yapılır.

```
        {
            sqlServer.EnableRetryOnFailure(
                maxRetryCount: MaximumRetryCount,
                maxRetryDelay: MaximumRetryDelay,
                errorNumbersToAdd: null);
```
Ağ kopması gibi geçici hatalarda sorgu otomatik tekrar denenir. `errorNumbersToAdd: null` EF'in bildiği hata kodları dışında ek kod eklenmediğini belirtir.

```
        });
    }
}
```

API `todo.API/Authentication/AuthenticationSetup.cs`

> API'nin güvenlik altyapısını kurar: JWT ile kimlik doğrulama, varsayılan olarak giriş zorunluluğu, AdminOnly politikası, giriş uçları için hız sınırı ve `ICurrentUser` servisi. Program.cs `AddTodoAuthentication` ile çağırır.

```
using System.Text;
using System.Threading.RateLimiting;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Authorization;
using Microsoft.IdentityModel.Tokens;
```
Metin kodlama, hız sınırlama, JWT Bearer, yetkilendirme ve token doğrulama kütüphaneleri içeri alınır.

```
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Domain.Enums;
using TodoApi.Infrastructure.Services.Authentication;
```
Projenin `ICurrentUser` arayüzü, `UserRole` enum'u ve Infrastructure'daki JWT ayarları/üreticisi içeri alınır.

```
namespace TodoApi.API.Authentication;
```
Dosya kapsamlı namespace tanımı.

ƒ class AuthenticationSetup — Kimlik doğrulama, yetkilendirme ve hız sınırı kayıtlarını toplayan statik sınıf.

```
public static class AuthenticationSetup
```
Extension metodlar için `static class`.

```
{
    private const int DefaultAuthenticationRequestsPerMinute = 10;
```
Ayar verilmezse giriş uçlarına dakikada en fazla 10 istek izni; `private const` sınıf içi sabit değerdir.

ƒ AddTodoAuthentication — JWT doğrulamasını, yetki politikalarını, hız sınırlayıcıyı ve mevcut kullanıcı servisini DI'a kaydeder.

```
    public static IServiceCollection AddTodoAuthentication(
        this IServiceCollection services,
        IConfiguration configuration)
```
`IServiceCollection` üzerinde çağrılan extension metod; ayarları okumak için `IConfiguration` alır.

```
    {
```

`        JwtSettings jwtSettings = JwtSettings.FromConfiguration(configuration);`
`        int authenticationRequestsPerMinute = ReadAuthenticationRequestsPerMinute(configuration);`
`        byte[] signingKeyBytes = Encoding.UTF8.GetBytes(jwtSettings.SigningKey);`
	JWT ayarları okunup doğrulanır, istek limiti okunur ve imza anahtarı byte dizisine çevrilir (şifreleme algoritmaları byte ile çalışır).

`        services`
`            .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)`
`            .AddJwtBearer(options =>`
	Varsayılan kimlik doğrulama şeması JWT Bearer yapılır: istekteki `Authorization: Bearer <token>` başlığı okunacak.

`            {`	
`                options.MapInboundClaims = false;`	`MapInboundClaims = false` claim adlarının uzun Microsoft URI'lerine çevrilmesini engeller; `sub`, `role` gibi kısa adlar aynen kalır.
`                options.TokenValidationParameters = new TokenValidationParameters` `                {`	Gelen token'ın hangi kurallarla doğrulanacağı nesne başlatıcısı (`new X { ... }`) ile tanımlanır.

`                    ValidateIssuer = true,`
`                    ValidIssuer = jwtSettings.Issuer,`
`                    ValidateAudience = true,`
`                    ValidAudience = jwtSettings.Audience,`
	Token'ı bizim sunucumuzun ürettiği (Issuer) ve bizim API için üretildiği (Audience) kontrol edilir.

`                    ValidateIssuerSigningKey = true,`
`                    IssuerSigningKey = new SymmetricSecurityKey(signingKeyBytes),`
`                    ValidAlgorithms = new[] { SecurityAlgorithms.HmacSha256 },`
	İmza gizli anahtarla doğrulanır ve sadece HMAC-SHA256 kabul edilir; böylece 'none' gibi zayıf algoritma saldırıları engellenir.

`                    ValidateLifetime = true,` `                    ClockSkew = TimeSpan.FromSeconds(30),`	Süresi dolmuş token reddedilir; sunucu saat farkları için sadece 30 saniye tolerans tanınır (varsayılan 5 dakikadır).
`                    NameClaimType = "name",` `                    RoleClaimType = JwtAccessTokenGenerator.RoleClaimType`	Kullanıcı adının "name", rolün "role" claim'inden okunacağı söylenir; `[Authorize(Roles=...)]` bu sayede çalışır.
`                };`	
`            });`	
`        services.AddAuthorization(options =>` `        {`	

`            options.FallbackPolicy = new AuthorizationPolicyBuilder()`
`                .RequireAuthenticatedUser()`
`                .Build();`
	`FallbackPolicy`: üzerinde özel kural olmayan her uç nokta giriş ister. Herkese açık uçlar `[AllowAnonymous]` ile açıkça işaretlenmelidir (güvenli varsayılan).

`            options.AddPolicy(AuthorizationPolicies.AdminOnly, policy =>`
`            {`
`                policy.RequireAuthenticatedUser();`
`                policy.RequireRole(UserRole.Admin.ToString());`
`            });`
	"AdminOnly" politikası: giriş yapmış ve Admin rolüne sahip kullanıcı ister; `[Authorize(Policy = ...)]` ile kullanılır.

`        });`	

`        services.AddRateLimiter(options =>`
`        {`
`            options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;`
	Hız sınırlayıcı eklenir; limit aşılınca 429 Too Many Requests döner.

`            options.AddPolicy(RateLimitingPolicies.Authentication, httpContext =>`
`            {`
`                string clientAddress = httpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown";`
	"Authentication" politikası her istemci IP'si için ayrı sayaç tutar; IP yoksa "unknown" kullanılır. Kaba kuvvet şifre denemelerine karşı koruma.

`                return RateLimitPartition.GetFixedWindowLimiter(`
`                    clientAddress,`
`                    _ => new FixedWindowRateLimiterOptions`
`                    {`
`                        PermitLimit = authenticationRequestsPerMinute,`
`                        Window = TimeSpan.FromMinutes(1),`
`                        QueueLimit = 0`
`                    });`
	Sabit pencere: her IP dakikada belirlenen sayıda istek yapabilir; `QueueLimit = 0` fazlası beklemeye alınmaz, direkt reddedilir. `_` kullanılmayan parametredir.

`            });` `        });`	
`        services.AddHttpContextAccessor();` `        services.AddScoped<ICurrentUser, HttpContextCurrentUser>();`	`HttpContext`'e erişim açılır ve `ICurrentUser` istek başına kaydedilir; Application katmanı HTTP bilmeden giriş yapan kullanıcıyı öğrenir.
`        return services;`	Zincirleme kullanım için koleksiyon geri döndürülür.
`    }`	

ƒ ReadAuthenticationRequestsPerMinute — Giriş uçları için dakikalık istek limitini config'den okur ve doğrular.

`    private static int ReadAuthenticationRequestsPerMinute(IConfiguration configuration)`	Sadece bu sınıfta kullanılan yardımcı metodun imzası.
`    {`	
`        string? configuredValue = configuration["Security:AuthenticationRequestsPerMinute"];` ``	`Security:AuthenticationRequestsPerMinute` okunur; boşsa varsayılan 10 döndürülür.

`        if (string.IsNullOrWhiteSpace(configuredValue))`
`        {`
`            return DefaultAuthenticationRequestsPerMinute;`
`        }`	

`        bool isNumber = int.TryParse(configuredValue, out int requestsPerMinute);` ``	Değer sayıya çevrilir; sayı değilse veya pozitif değilse uygulama açıklayıcı hata ile durdurulur.

`        if (isNumber == false || requestsPerMinute <= 0)`
`        {`
`            throw new InvalidOperationException(`
`                "The Security:AuthenticationRequestsPerMinute setting must be a positive whole number.");`
`        }`	

`        return requestsPerMinute;`	Geçerli limit döndürülür.

```
    }
}
```

Infrastructure `todo.Infrastructure/Services/Authentication/JwtSettings.cs`

> JWT ayarlarını (issuer, audience, imza anahtarı, token ömrü) config'den okuyup doğrulayan değişmez ayar nesnesidir. Hem token üretimi hem de API'deki token doğrulaması bunu kullanır.

```
using System;
using System.Text;
using Microsoft.Extensions.Configuration;
```

Temel tipler, byte sayımı için metin kodlama ve yapılandırma okuma kütüphanesi içeri alınır.

```
namespace TodoApi.Infrastructure.Services.Authentication;
```

Dosya kapsamlı namespace tanımı.

ƒ class JwtSettings — Doğrulanmış, değiştirilemez JWT yapılandırma değerlerini taşır.

```
public sealed class JwtSettings
```

`sealed` ile miras alınamaz; ayar nesnesinin davranışı değiştirilemez.

```
{
    public const string SectionName = "Jwt";
```

appsettings'teki bölüm adı "Jwt"; `public const` dışarıdan da kullanılabilen sabittir.

```
    private const int MinimumSigningKeyByteCount = 32;
```

İmza anahtarı en az 32 byte (256 bit) olmalı; HMAC-SHA256 için kısa anahtar kolay kırılır.

```
    private readonly string _issuer;
    private readonly string _audience;
    private readonly string _signingKey;
    private readonly int _accessTokenLifetimeMinutes;
```

Değerler `private readonly` alanlarda tutulur; constructor'dan sonra değiştirilemez (immutable).

ƒ constructor — Dört JWT ayarını alır, her birini doğrular ve alanlara atar.

```
    public JwtSettings(
        string issuer,
        string audience,
        string signingKey,
        int accessTokenLifetimeMinutes)
```

Constructor dört değeri parametre olarak alır; geçersiz bir `JwtSettings` nesnesi hiç oluşamaz.

```
    {
        if (string.IsNullOrWhiteSpace(issuer))
        {
            throw new InvalidOperationException("The Jwt:Issuer setting is missing.");
        }

        if (string.IsNullOrWhiteSpace(audience))
        {
            throw new InvalidOperationException("The Jwt:Audience setting is missing.");
        }
```

Issuer ve Audience boşsa hangi ayarın eksik olduğunu söyleyen hata fırlatılır.

```
        if (string.IsNullOrWhiteSpace(signingKey))
        {
            throw new InvalidOperationException(
                "The Jwt:SigningKey setting is missing. In development, store it with " +
                "'dotnet user-secrets set Jwt:SigningKey <value> --project todo.API'. " +
                "On a server, provide it through the Jwt__SigningKey environment variable or a secret store.");
        }
```

İmza anahtarı yoksa hata, geliştirmede user-secrets, sunucuda ortam değişkeni ile nasıl verileceğini anlatır; gizli anahtar appsettings'e yazılmaz.

```
        if (Encoding.UTF8.GetByteCount(signingKey) < MinimumSigningKeyByteCount)
        {
            throw new InvalidOperationException(
                $"The Jwt:SigningKey setting must be at least {MinimumSigningKeyByteCount} bytes long.");
        }
```

Anahtar uzunluğu karakter değil byte olarak ölçülür (Türkçe karakterler 2 byte olabilir); 32 byte'tan kısaysa reddedilir.

```
        if (accessTokenLifetimeMinutes <= 0)
        {
            throw new InvalidOperationException(
                "The Jwt:AccessTokenLifetimeMinutes setting must be a positive number.");
        }
```

Access token ömrü sıfır veya negatif olamaz.

```
        _issuer = issuer;
        _audience = audience;
        _signingKey = signingKey;
        _accessTokenLifetimeMinutes = accessTokenLifetimeMinutes;
```

Tüm kontroller geçince değerler alanlara atanır.

```
    }
```

ƒ Issuer — Token'ı üreten tarafın adını döndüren salt okunur özellik.

```
    public string Issuer
    {
        get
        {
            return _issuer;
        }
    }
```

Sadece `get` içeren özellik: dışarıdan okunur, değiştirilemez.

ƒ Audience — Token'ın hedef kitlesini döndüren salt okunur özellik.

```
    public string Audience
    {
        get
        {
            return _audience;
        }
    }
```

Token'ın hangi uygulama için üretildiğini döndürür.

ƒ SigningKey — İmza için gizli anahtarı döndüren salt okunur özellik.

```
    public string SigningKey
    {
        get
        {
            return _signingKey;
        }
    }
```

Token imzalama ve doğrulamada kullanılan gizli anahtarı döndürür.

ƒ AccessTokenLifetimeMinutes — Access token geçerlilik süresini dakika olarak döndürür.

```
    public int AccessTokenLifetimeMinutes
    {
        get
        {
            return _accessTokenLifetimeMinutes;
        }
    }
```

Access token'ın kaç dakika geçerli olacağını döndürür; kısa ömür çalınan token'ın zararını sınırlar.

ƒ FromConfiguration — "Jwt" bölümünü okuyup doğrulanmış `JwtSettings` nesnesi oluşturan fabrika metodu.

```
    public static JwtSettings FromConfiguration(IConfiguration configuration)
```

`static` fabrika metodu: nesne yerine sınıf üzerinden çağrılır, `JwtSettings.FromConfiguration(config)`.

```
    {
        if (configuration == null)
        {
            throw new ArgumentNullException(nameof(configuration));
        }
```

Yapılandırma null ise `ArgumentNullException` fırlatılır.

```
        IConfigurationSection section = configuration.GetSection(SectionName);
```

"Jwt" bölümü alınır.

```
        string lifetimeText = section["AccessTokenLifetimeMinutes"] ?? string.Empty;
        bool lifetimeIsNumber = int.TryParse(lifetimeText, out int accessTokenLifetimeMinutes);
```

Token ömrü metin olarak okunur (yoksa boş metin, `??` sayesinde) ve tam sayıya çevrilmeye çalışılır.

```
        if (lifetimeIsNumber == false)
        {
            throw new InvalidOperationException(
                "The Jwt:AccessTokenLifetimeMinutes setting must be a whole number.");
        }
```

Sayıya çevrilemezse hata fırlatılır; pozitiflik kontrolünü constructor yapar.

```
        JwtSettings settings = new JwtSettings(
            section["Issuer"] ?? string.Empty,
            section["Audience"] ?? string.Empty,
            section["SigningKey"] ?? string.Empty,
            accessTokenLifetimeMinutes);
```

Okunan değerlerle nesne oluşturulur; eksik değerler boş metin olarak gider ve constructor'daki doğrulamaya takılır.

```
        return settings;
```

Doğrulanmış ayar nesnesi döndürülür.

```
    }
}
```

API todo.API/Authentication/AuthorizationPolicies.cs

> Yetki politikası adlarını sabit olarak tutar. Böylece politika hem tanımlanırken hem controller'da kullanılırken aynı isim yazılır, yazım hatası riski kalmaz.

```
namespace TodoApi.API.Authentication;
```

Dosya kapsamlı namespace: API katmanının Authentication klasörü.

ƒ class AuthorizationPolicies — Yetki politikası adlarını içeren sabitler sınıfı.

```
public static class AuthorizationPolicies
```

`static class` olduğu için örneği oluşturulmaz; sadece sabit değer barındırır.

```
{
    public const string AdminOnly = "AdminOnly";
```

"AdminOnly" politika adı; `AuthenticationSetup`'ta tanımlanır, admin uçlarında `[Authorize(Policy = AuthorizationPolicies.AdminOnly)]` ile kullanılır.

```
}
```

API todo.API/Authentication/RateLimitingPolicies.cs

> Hız sınırı (rate limiting) politika adlarını sabit olarak tutar; tanım ve kullanım yerinde aynı ismin kullanılmasını garanti eder.

```
namespace TodoApi.API.Authentication;
```

Dosya kapsamlı namespace: API katmanının Authentication klasörü.

ƒ class RateLimitingPolicies — Hız sınırı politika adlarını içeren sabitler sınıfı.

```
public static class RateLimitingPolicies
```

Sadece sabit barındıran `static class`.

```
{
```

```
    public const string Authentication = "Authentication";
```
"Authentication" politika adı; giriş/kayıt uçlarında `[EnableRateLimiting(...)]` ile kullanılarak IP başına istek sayısını sınırlar.

```
}
```

API `todo.API/Security/SecurityHeadersMiddleware.cs`

> Her HTTP yanıtına tarayıcı güvenlik başlıklarını ekleyen özel middleware'dir. Program.cs'te hattın en başına `UseSecurityHeaders()` ile eklenir.

```
using TodoApi.API.BackgroundJobs;
```
Hangfire panel yolunu (`DashboardPath`) bilmek için BackgroundJobs klasörü içeri alınır.

```
namespace TodoApi.API.Security;
```
Dosya kapsamlı namespace: API katmanının Security klasörü.

ƒ class SecurityHeadersMiddleware — Yanıtlara nosniff, frame engeli, CSP, referrer ve cache başlıklarını ekleyen middleware.

```
public sealed class SecurityHeadersMiddleware
```
`sealed` bu sınıftan miras alınamayacağını belirtir; tasarımı kapalı tutar.

```
{
```

```
    private const string DashboardContentSecurityPolicy =
        "default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; " +
        "img-src 'self' data:; connect-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'";
```
Hangfire paneli için daha esnek CSP: paneli çalıştıran kendi script ve stillerine izin verir, dış kaynakları ve iframe içine gömülmeyi yasaklar.

```
    private readonly RequestDelegate _next;
```
Hattaki bir sonraki middleware'i tutan alan. `private readonly` sadece bu sınıf görür ve constructor dışında değiştirilemez.

ƒ constructor — Hattaki sonraki adımı (`RequestDelegate`) alıp saklar.

```
    public SecurityHeadersMiddleware(RequestDelegate next)
    {
        _next = next;
    }
```
ASP.NET Core middleware oluştururken sonraki adımı otomatik verir; burada alana kaydedilir.

ƒ InvokeAsync — Yanıt başlamadan güvenlik başlıklarını ekleyecek kodu kaydeder ve isteği sonraki adıma iletir.

```
    public async Task InvokeAsync(HttpContext httpContext)
```
Her istekte çağrılan metod. `async Task` asenkron çalışır; `await` ile beklerken iş parçacığı serbest kalır.

```
    {
```

```
        bool isDashboardRequest = IsDashboardRequest(httpContext);
```
İsteğin Hangfire paneline mi gittiği önceden hesaplanır; hangi CSP'nin kullanılacağını belirler.

```
        httpContext.Response.OnStarting(() =>
        {
            IHeaderDictionary headers = httpContext.Response.Headers;
```
`OnStarting` başlıkları yanıt gönderilmeye başlamadan hemen önce ekler; böylece hata yanıtları dahil hiçbir yanıt bunları atlamaz.

```
            headers["X-Content-Type-Options"] = "nosniff";
```
`nosniff` : tarayıcı içerik tipini tahmin etmeye çalışmaz; metnin script gibi çalıştırılmasını önler.

```
            headers["X-Frame-Options"] = "DENY";
```
`DENY` : sayfa başka sitelerin iframe'i içinde gösterilemez; clickjacking saldırısını engeller.

```
            headers["Content-Security-Policy"] = isDashboardRequest
                ? DashboardContentSecurityPolicy
                : "default-src 'none'; frame-ancestors 'none'";
```
Panel için esnek CSP, API JSON yanıtları için her şeyi yasaklayan sıkı CSP seçilir. `koşul ? a : b` kısa if-else'dir.

```
            headers["Referrer-Policy"] = "no-referrer";
```
`no-referrer` : başka siteye geçerken adres bilgisi (Referer) gönderilmez; URL'deki bilgiler sızmaz.

```
            headers["Cache-Control"] = "no-store";
```
`no-store` : yanıtlar tarayıcı veya proxy önbelleğinde saklanmaz; kişisel todo verileri diskte kalmaz.

```
            return Task.CompletedTask;
```
Geri çağırma senkron iş yaptığı için tamamlanmış bir `Task` döndürür.

```
        });
```

```
        await _next(httpContext);
```
İstek hattaki sonraki middleware'e iletilir ve bitmesi beklenir.

```
    }
```

ƒ IsDashboardRequest — İstek yolunun Hangfire paneliyle (/hangfire) başlayıp başlamadığını döner.

```
    private static bool IsDashboardRequest(HttpContext httpContext)
```
Yalnızca bu sınıf içinde kullanılan statik yardımcı metod.

```
    {
```

```
        return httpContext.Request.Path.StartsWithSegments(BackgroundJobsSetup.DashboardPath);
```
`StartsWithSegments` yolu parça parça karşılaştırır; "/hangfire/jobs" eşleşir ama "/hangfirex" eşleşmez.

```
    }
}
```

ƒ class SecurityHeadersMiddlewareExtensions — Middleware'i okunaklı `UseSecurityHeaders()` adıyla eklemeyi sağlayan extension sınıfı.

```
public static class SecurityHeadersMiddlewareExtensions
```
Extension metod barındıran statik sınıf.

```
{
```

ƒ UseSecurityHeaders — `SecurityHeadersMiddleware` 'i istek hattına ekler.

```
    public static IApplicationBuilder UseSecurityHeaders(this IApplicationBuilder app)
```
`this IApplicationBuilder` sayesinde Program.cs'te `app.UseSecurityHeaders()` yazılabilir.

```
    {
```

```
        return app.UseMiddleware<SecurityHeadersMiddleware>();
```
`UseMiddleware<T>` middleware sınıfını hatta ekler ve zincir için builder'ı döndürür.

```
    }
}
```

API `todo.API/BackgroundJobs/BackgroundJobsSetup.cs`

> Hangfire arka plan iş sistemini kurar: servisleri kaydeder, hatırlatma e-postası işini zamanlar ve geliştirmede izleme panelini açar. Program.cs iki aşamada çağırır: `AddTodoBackgroundJobs` ve `UseTodoBackgroundJobs`.

```
using Hangfire;
using Hangfire.Dashboard;
using Hangfire.SqlServer;
```

Hangfire çekirdeği, panel (dashboard) ve SQL Server depolama paketleri içeri alınır.

```
using TodoApi.Application.Abstractions.Email;
using TodoApi.Application.Features.Reminders.SendDueTodoReminders;
```

Application katmanındaki `IEmailSender` arayüzü ve `ReminderSettings` ayar tipi içeri alınır.

```
namespace TodoApi.API.BackgroundJobs;
```

Dosya kapsamlı namespace tanımı.

ƒ class BackgroundJobsSetup — Hangfire kurulumunu ve hatırlatma işinin zamanlanmasını yapan statik sınıf.

```
public static class BackgroundJobsSetup
```

Extension metodlar için `static class`.

```
{
```

```
    public const string DashboardPath = "/hangfire";
```

Panelin adresi "/hangfire"; güvenlik middleware'i de bu sabiti kullanır.

ƒ AddTodoBackgroundJobs — Hatırlatmalar açıksa Hangfire'ı, iş sunucusunu ve `TodoReminderJob`'u DI'a kaydeder.

```
    public static IServiceCollection AddTodoBackgroundJobs(
        this IServiceCollection services,
        IConfiguration configuration)
```

Servis kayıt aşamasında çağrılan extension metod.

```
    {
```

```
        if (AreRemindersEnabled(configuration) == false)
        {
            return services;
        }
```

Hatırlatmalar ayardan kapatılmışsa Hangfire hiç kurulmaz; gereksiz kaynak harcanmaz.

```
        string connectionString = configuration.GetConnectionString("TodoDatabase")
            ?? throw new InvalidOperationException("The TodoDatabase connection string is missing.");
```

Hangfire işleri aynı veritabanında saklayacağı için bağlantı metni alınır; yoksa `?? throw` ile anında hata fırlatılır.

```
        services.AddHangfire(hangfire =>
        {
            hangfire.SetDataCompatibilityLevel(CompatibilityLevel.Version_180);
            hangfire.UseSimpleAssemblyNameTypeSerializer();
            hangfire.UseRecommendedSerializerSettings();
```

Hangfire yapılandırılır: güncel veri formatı (1.8), kısa tip adlarıyla serileştirme ve önerilen JSON ayarları.

```
            hangfire.UseSqlServerStorage(connectionString, new SqlServerStorageOptions
            {
                SqlClientFactory = Microsoft.Data.SqlClient.SqlClientFactory.Instance,
                PrepareSchemaIfNecessary = true
            });
        });
```

İşler SQL Server'da saklanır; yeni `Microsoft.Data.SqlClient` kullanılır ve Hangfire tabloları yoksa otomatik oluşturulur.

```
        services.AddHangfireServer(options =>
        {
            options.ServerName = "todo-api";
            options.WorkerCount = 2;
        });
```

İşleri çalıştıran Hangfire sunucusu uygulama içinde başlatılır; "todo-api" adıyla ve aynı anda en fazla 2 iş çalıştırır.

```
        services.AddScoped<TodoReminderJob>();
```

Hangfire'ın çalıştıracağı `TodoReminderJob` sınıfı scoped kaydedilir; her çalıştırmada yeni DbContext ile çalışır.

```
        return services;
```

Zincirleme kullanım için koleksiyon döndürülür.

```
    }
```

ƒ UseTodoBackgroundJobs — Uygulama kurulduktan sonra hatırlatma işini zamanlar veya kaldırır, geliştirmede paneli açar.

```
    public static WebApplication UseTodoBackgroundJobs(this WebApplication app)
```

`WebApplication` üzerinde çağrılan extension metod; `Build()` sonrası çalışır.

```
    {
```

```
        if (AreRemindersEnabled(app.Configuration) == false)
        {
            return app;
        }
```

Hatırlatmalar kapalıysa hiçbir şey yapılmaz; zaten Hangfire servisleri de kaydedilmemiştir.

```
        IRecurringJobManager recurringJobs = app.Services.GetRequiredService<IRecurringJobManager>();
        IEmailSender emailSender = app.Services.GetRequiredService<IEmailSender>();
        ReminderSettings reminderSettings = app.Services.GetRequiredService<ReminderSettings>();
        ILogger logger = app.Services.GetRequiredService<ILoggerFactory>().CreateLogger(nameof(BackgroundJobsSetup));
```

Gerekli servisler DI kabından alınır: zamanlayıcı, e-posta gönderici, ayarlar ve logger. `GetRequiredService` bulamazsa hata fırlatır.

```
        if (emailSender.IsConfigured)
        {
            recurringJobs.AddOrUpdate<TodoReminderJob>(
                TodoReminderJob.RecurringJobId,
                job => job.RunAsync(CancellationToken.None),
                reminderSettings.Schedule);
```

SMTP ayarlıysa iş cron zamanlamasıyla (varsayılan 5 dakikada bir) kaydedilir; `AddOrUpdate` aynı kimlikle tekrar çağrılınca kopya oluşturmaz, günceller.

```
            logger.LogInformation(
                "Todo reminders scheduled with '{Schedule}' for todos due within {LeadTime}.",
                reminderSettings.Schedule,
                reminderSettings.LeadTime);
```

Zamanlamanın yapıldığı yapılandırılmış logla bildirilir; `{Schedule}` gibi yer tutucular Serilog'da ayrı alan olarak saklanır.

```
        }
```

```
        else
        {
            recurringJobs.RemoveIfExists(TodoReminderJob.RecurringJobId);
```
SMTP ayarlı değilse önceden kalmış zamanlanmış iş varsa silinir; e-posta gönderemeyen iş boşuna çalışmaz.

```
            logger.LogWarning(
                "Todo reminders are not scheduled because SMTP is not configured. " +
                "Set Email:Smtp:Username and Email:Smtp:Password with 'dotnet user-secrets'.");
        }
```
Geliştiriciye SMTP kullanıcı adı ve şifresini user-secrets ile girmesi gerektiğini söyleyen uyarı loglanır.

```
        if (app.Environment.IsDevelopment())
        {
```
İzleme paneli sadece geliştirme ortamında açılır; canlıda iş verileri dışarı açılmaz.

```
            app.MapHangfireDashboard(DashboardPath, new DashboardOptions
            {
                DashboardTitle = "Todo Arka Plan İşleri",
                Authorization = new IDashboardAuthorizationFilter[] { new LocalRequestsOnlyAuthorizationFilter() }
            })
```
Panel "/hangfire" adresine Türkçe başlıkla eklenir; `LocalRequestsOnlyAuthorizationFilter` sadece bu bilgisayardan gelen isteklere izin verir.

```
            .AllowAnonymous();
```
`AllowAnonymous` JWT fallback politikasını panel için kapatır; tarayıcı panele token gönderemez, koruma yerel istek filtresiyle sağlanır.

```
        }
        return app;
    }
```
Zincirleme kullanım için uygulama nesnesi döndürülür.

ƒ AreRemindersEnabled — `Reminders:Enabled` ayarını okur; yoksa true kabul eder.

```
    private static bool AreRemindersEnabled(IConfiguration configuration)
    {
        return configuration.GetValue("Reminders:Enabled", defaultValue: true);
    }
}
```
Her iki aşamada aynı kontrolü yapan yardımcı metod.

`GetValue` ayarı bool'a çevirir; ayar yoksa varsayılan `true` döner.

Web (Angular) `todo.Web/src/main.ts`

> Angular uygulamasının giriş noktası. Tarayıcı ilk bu dosyayı çalıştırır; kök bileşen `App` ve `appConfig` ayarlarıyla uygulamayı ayağa kaldırır.

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';
```
Uygulamayı başlatan `bootstrapApplication` fonksiyonunu, tüm global ayarları tutan `appConfig` nesnesini ve kök bileşen `App`'i içeri alır.

```
bootstrapApplication(App, appConfig)
```
Standalone kök bileşen `App`'i `appConfig` içindeki provider'larla (HTTP, router, oturum geri yükleme vb.) başlatır. NgModule gerekmez.

```
  .catch((err) => console.error(err));
```
Başlatma bir Promise döndürür; açılışta bir hata olursa (ör. başlatıcı patlarsa) hatayı tarayıcı konsoluna yazar.

Web (Angular) `todo.Web/src/app/app.config.ts`

> Uygulama genelindeki servisleri (provider) tek yerde tanımlar: HTTP istemcisi ve interceptor, router, açılışta oturum geri yükleme ve Türkçe dil ayarları.

```
import { registerLocaleData } from '@angular/common';
import tr from '@angular/common/locales/tr';
```
Tarih/sayı biçimlendirmede Türkçe kuralları kaydetmek için `registerLocaleData` fonksiyonunu ve Türkçe yerel veri paketini içeri alır.

```
import { provideHttpClient, withInterceptors } from '@angular/common/http';
```
`HttpClient`'ı uygulamaya ekleyen `provideHttpClient` ve fonksiyon tabanlı interceptor bağlayan `withInterceptors` içeri alınır.

```
import {
  ApplicationConfig,
  LOCALE_ID,
  inject,
  provideAppInitializer,
  provideBrowserGlobalErrorListeners,
} from '@angular/core';
```
Angular çekirdeğinden yapılandırma tipi, dil token'ı `LOCALE_ID`, bağımlılık almak için `inject`, açılış başlatıcısı ve global hata dinleyicisi alınır.

```
import { provideRouter } from '@angular/router';
import { provideNzI18n, tr_TR } from 'ng-zorro-antd/i18n';
```
Router'ı kuran `provideRouter` ile ng-zorro bileşen kütüphanesinin Türkçe metinlerini sağlayan `provideNzI18n` ve `tr_TR` alınır.

```
import { routes } from './app.routes';
import { authInterceptor } from './interceptors/auth.interceptor';
import { AuthService } from './services/auth.service';
```
Projenin kendi rota listesi, her HTTP isteğine araya giren `authInterceptor` ve oturumu yöneten `AuthService` içeri alınır.

```
registerLocaleData(tr);
```
Türkçe yerel veriyi Angular'a kaydeder; böylece `date` gibi pipe'lar tarihleri Türkçe ay/gün adlarıyla gösterebilir.

```
export const appConfig: ApplicationConfig = {
  providers: [
```
`appConfig` dışa aktarılır; `main.ts` bunu kullanır. `providers` dizisi tüm uygulamaya açık servisleri listeler.

```
    provideBrowserGlobalErrorListeners(),
```
Yakalanmamış tarayıcı hatalarını ve reddedilen Promise'leri Angular'ın hata işleyicisine yönlendirir; hatalar sessizce kaybolmaz.

```
    provideHttpClient(withInterceptors([authInterceptor])),
```
`HttpClient`'ı kurar ve tüm istekleri `authInterceptor`'dan geçirir; token ekleme ve 401'de yenileme tek merkezde yapılır.

```
    provideRouter(routes),
```
`app.routes.ts` içindeki rotalarla router'ı etkinleştirir; URL'ye göre hangi sayfanın açılacağı buradan belirlenir.

```
    provideAppInitializer(() => inject(AuthService).restoreSession()),
    provideNzI18n(tr_TR),
    { provide: LOCALE_ID, useValue: 'tr-TR' },
  ],
};
```

Uygulama ekrana gelmeden önce `restoreSession` çalışır; HttpOnly çerezdeki refresh token ile oturum geri yüklenir, böylece guard'lar doğru karar verir. `inject()` burada servisi DI'dan alır.

ng-zorro bileşenlerinin (tarih seçici, butonlar vb.) iç metinlerini Türkçe yapar.

Angular'ın varsayılan dilini `tr-TR` yapar; pipe'lar biçimlendirmede bu dili kullanır.

Web (Angular) **todo.Web/src/app/app.ts**

> Kök bileşen. Kendi içeriği yoktur; sadece router'ın o anki sayfayı yerleştireceği `<router-outlet />` alanını sağlar.

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  template: '<router-outlet />',
})
```

Bileşen tanımlamak için `Component` dekoratörünü ve sayfaların yerleşeceği `RouterOutlet` yönergesini içeri alır.

`@Component` dekoratörü altındaki sınıfı bir Angular bileşeni olarak işaretler ve ayarlarını verir.

`index.html` içindeki `<app-root>` etiketi bu bileşenle değiştirilir.

Standalone bileşen olduğu için kullandığı `RouterOutlet`'i doğrudan `imports` içinde belirtir; NgModule yoktur.

Şablon yalnızca `<router-outlet />`; URL'ye göre giriş, todo veya admin sayfası buraya çizilir.

ƒ **App** — Uygulamanın kök bileşeni; tüm sayfaları router aracılığıyla barındıran boş kabuk.

```
export class App {}
```

Sınıf boştur çünkü tüm mantık sayfa bileşenlerinde ve servislerdedir; kök bileşen sadece çerçeve görevi görür.

Web (Angular) **todo.Web/src/app/app.routes.ts**

> URL ile sayfa bileşenleri arasındaki eşleşmeyi ve her sayfanın hangi guard ile korunduğunu tanımlar.

```
import { Routes } from '@angular/router';

import { adminGuard, authGuard, guestGuard } from './guards/auth.guards';

import { AdminUsersPage } from './pages/admin/admin-users-page';
import { LoginPage } from './pages/login/login-page';
import { TodoPage } from './pages/todos/todo-page';

export const routes: Routes = [
  {
    path: 'login',
    component: LoginPage,
    canActivate: [guestGuard],
    title: 'Giriş | Todo',
  },
  {
    path: 'admin/users',
    component: AdminUsersPage,
    canActivate: [adminGuard],
    title: 'Kullanıcılar | Todo',
  },
  {
    path: '',
    component: TodoPage,
    canActivate: [authGuard],
    pathMatch: 'full',
    title: 'Todo | Gününü düzenle',
  },
  { path: '**', redirectTo: '' },
];
```

Rota listesinin tipi `Routes` içeri alınır; yanlış alan yazılırsa TypeScript derlemede uyarır.

Sayfalara kimin girebileceğine karar veren üç guard alınır: giriş yapmış, misafir ve admin kontrolü.

Rotaların göstereceği üç sayfa bileşeni alınır: admin kullanıcı listesi, giriş sayfası ve todo sayfası.

`routes` dizisi dışa aktarılır; `app.config.ts` içinde `provideRouter(routes)` ile kullanılır.

`/login` adresi giriş sayfasını açar. `guestGuard` zaten giriş yapmış kullanıcıyı ana sayfaya yollar; `title` tarayıcı sekme başlığıdır.

`/admin/users` adresi kullanıcı yönetim sayfasıdır; `adminGuard` yalnızca Admin rolündekilerin girmesine izin verir.

Boş yol (ana sayfa) todo listesini açar; `authGuard` giriş gerektirir. `pathMatch: 'full'` sadece tam boş URL'de eşleşmesini sağlar.

`**` joker rotası tanımsız her adresi ana sayfaya yönlendirir; listenin sonunda olmalıdır çünkü rotalar sırayla denenir.

Web (Angular) **todo.Web/src/app/guards/auth.guards.ts**

> Rotalara girişi kontrol eden üç fonksiyonel guard içerir. Oturum bilgisini `AuthService`'ten okuyup izin verir veya uygun sayfaya yönlendirir.

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';
```

Bağımlılık almak için `inject`, guard tipi `CanActivateFn`, yönlendirme için `Router` ve oturum durumunu bilen `AuthService` alınır.

ƒ **authGuard** — Sadece giriş yapmış kullanıcıyı sayfaya alır; değilse giriş sayfasına yönlendirir.

```
export const authGuard: CanActivateFn = () => {
```

`CanActivateFn` tipinde bir ok fonksiyonu; sınıf yazmadan guard tanımlamanın modern Angular yolu.

```
  const auth = inject(AuthService);
  const router = inject(Router);
```

`inject()` ile servisler alınır. Guard fonksiyonları Angular'ın DI bağlamında çalıştığı için `inject` burada kullanılabilir.

```
  if (auth.isAuthenticated()) {
    return true;
  }
```

Bellekte geçerli bir oturum varsa `true` döner ve sayfa açılır.

```
  return router.createUrlTree(['/login']);
```

Oturum yoksa `/login` için bir `UrlTree` döner; router bunu görünce gezinmeyi iptal edip giriş sayfasına gider.

```
};
```

ƒ guestGuard — Giriş sayfasını sadece oturumu olmayanlara açar; giriş yapmışları ana sayfaya gönderir.

```
export const guestGuard: CanActivateFn = () => {
```

Misafir guard'ı; giriş yapmış birinin tekrar login ekranını görmesini engeller.

```
  const auth = inject(AuthService);
  const router = inject(Router);
```

Oturum servisi ve router `inject()` ile alınır.

```
  if (auth.isAuthenticated()) {
    return router.createUrlTree(['/']);
  }
```

Kullanıcı zaten giriş yapmışsa ana sayfaya (`/`) yönlendirilir.

```
  return true;
```

Oturum yoksa giriş sayfası açılabilir.

```
};
```

ƒ adminGuard — Sadece Admin rolündeki kullanıcıları admin sayfasına alır; diğerlerini login veya ana sayfaya yollar.

```
export const adminGuard: CanActivateFn = () => {
```

Admin sayfasını koruyan guard. Bu sadece arayüz kolaylığıdır; asıl yetki kontrolü API tarafında yapılır.

```
  const auth = inject(AuthService);
  const router = inject(Router);
```

Oturum servisi ve router `inject()` ile alınır.

```
  if (!auth.isAuthenticated()) {
    return router.createUrlTree(['/login']);
  }
```

Önce giriş yapılmış mı bakılır; yapılmamışsa giriş sayfasına yönlendirilir.

```
  if (auth.isAdmin()) {
    return true;
  }
```

Giriş yapmış kullanıcının rolü Admin ise sayfa açılır. `isAdmin` bir `computed` signal olduğu için `()` ile okunur.

```
  return router.createUrlTree(['/']);
```

Giriş yapmış ama admin olmayan kullanıcı ana sayfaya gönderilir.

```
};
```

Web (Angular) `todo.Web/src/app/core/api-url.ts`

> API'nin temel adresini tek bir yerden belirler. Servisler ve interceptor adresi buradan alır, böylece adres değişince tek dosya güncellenir.

```
declare global {
```

`declare global` ile tarayıcının global tiplerine ekleme yapılır; yeni kod üretmez, sadece TypeScript'e bilgi verir.

```
  interface Window {
    __TODO_API_URL__?: string;
  }
```

`Window` arayüzüne isteğe bağlı (`?`) `__TODO_API_URL__` alanı eklenir; böylece `window.__TODO_API_URL__` yazınca tip hatası çıkmaz.

```
}
```

ƒ resolveApiUrl — API temel adresini döndürür: window üzerinde tanımlıysa onu, yoksa yerel geliştirme adresini.

```
export function resolveApiUrl(): string {
```

Dışa aktarılan fonksiyon; dönüş tipi `string` olarak açıkça belirtilmiştir.

```
  if (typeof window !== 'undefined' && window.__TODO_API_URL__) {
    return window.__TODO_API_URL__;
  }
```

Global bir API adresi tanımlıysa onu kullanır; e2e testleri (`e2e/fixtures.ts`) bunu ayarlayıp test API'sine yönlendirir. `typeof window` kontrolü tarayıcı dışı ortamda hatayı önler.

```
  return 'http://localhost:7070/api';
```

Aksi halde yerel geliştirme API adresi `http://localhost:7070/api` kullanılır.

```
}
```

Bölüm 2 — Giriş ve oturum

Kullanıcı e-posta/şifre ya da Google ile giriş yapar. Sunucu iki şey verir: 15 dakikalık access token (JWT, sadece bellekte tutulur) ve 7 günlük refresh token (HttpOnly cookie). Token süresi dolunca Angular sessizce yeniler, sayfa yenilenince oturumu geri yükler.

Akış adım adım

1. Form doğrulanır, `auth.login()` çağrılır.
   `todo.Web/src/app/pages/login/login-page.ts`
2. `POST /api/auth/login` isteği hazırlanır.
   `todo.Web/src/app/services/auth.service.ts`
3. İsteğe `X-Todo-Client` header'ı ve cookie izni eklenir.
   `todo.Web/src/app/interceptors/auth.interceptor.ts`
4. `Login` : header kontrolü, hız limiti; `LoginRequest` → `LoginWithPasswordCommand` .
   `todo.API/Controllers/AuthController.cs`
5. Doğrulama, kullanıcıyı bulma, şifre hash kontrolü (gerekirse yeniden hash).
   `todo.Application/Features/Authentication/LoginWithPassword/LoginWithPasswordUseCase.cs`
6. PBKDF2 ile şifre doğrulaması.
   `todo.Infrastructure/Services/Authentication/AspNetCorePasswordHasher.cs`
7. Access token (JWT) + refresh token üretilir, refresh token'ın sadece hash'i kaydedilir.
   `todo.Application/Features/Authentication/Common/SessionIssuer.cs`
8. Refresh token HttpOnly + Secure + SameSite=Strict cookie olarak yazılır; gövdede access token ve kullanıcı bilgisi döner.
   `todo.API/Authentication/RefreshTokenCookie.cs`
9. Oturum signal'e yazılır, ana sayfaya gidilir. Sonraki isteklere interceptor `Bearer` token ekler.
   `todo.Web/src/app/services/auth.service.ts`
10. Token bitince: cookie ile yenileme, token rotasyonu, çalınma (reuse) tespiti.
    `todo.Application/Features/Authentication/RefreshSession/RefreshSessionUseCase.cs`
11. Google ile girişte: Google token'ı doğrulanır, hesap bulunur/bağlanır/oluşturulur, admin rolü belirlenir.
    `todo.Application/Features/Authentication/LoginWithGoogle/LoginWithGoogleUseCase.cs`

Bu bölümde açıklanan dosyalar (kullanım sırasıyla): login-page.ts, auth.models.ts, auth.service.ts, auth.interceptor.ts, google-identity.service.ts, AuthController.cs, RequireClientHeaderAttribute.cs, LoginRequest.cs, RegisterRequest.cs, GoogleLoginRequest.cs, AuthConfigResponse.cs, RefreshTokenCookie.cs, LoginWithPasswordCommand.cs, LoginWithPasswordCommandValidator.cs, LoginWithPasswordUseCase.cs, EmailNormalizer.cs, IUserRepository.cs, UserRepository.cs, User.cs, UserRole.cs, IPasswordHasher.cs, AspNetCorePasswordHasher.cs, SessionIssuer.cs, AuthSession.cs, RefreshTokenSettings.cs, IAccessTokenGenerator.cs, AccessToken.cs, JwtAccessTokenGenerator.cs, ISecureTokenGenerator.cs, SecureTokenGenerator.cs, IRefreshTokenRepository.cs, RefreshTokenRepository.cs, RefreshToken.cs, AuthResultDto.cs, UserDto.cs, AuthenticationFailedException.cs, RefreshSessionUseCase.cs, LogoutUseCase.cs, GetCurrentUserUseCase.cs, ICurrentUser.cs, HttpContextCurrentUser.cs, RegisterWithPasswordCommand.cs, RegisterWithPasswordCommandValidator.cs, RegisterWithPasswordUseCase.cs, EmailAlreadyRegisteredException.cs, LoginWithGoogleCommand.cs, LoginWithGoogleCommandValidator.cs, LoginWithGoogleUseCase.cs, IGoogleTokenValidator.cs, GoogleUserInfo.cs, GoogleIdTokenValidator.cs, GoogleAuthenticationSettings.cs, IRoleAssignmentPolicy.cs, ConfiguredRoleAssignmentPolicy.cs

Web (Angular) `todo.Web/src/app/pages/login/login-page.ts`

Giriş ve kayıt ekranının mantığı. Formu doğrular, `AuthService` ile giriş/kayıt yapar, Google butonunu hazırlar ve hataları Türkçe mesajlara çevirir.

```ts
import { HttpErrorResponse } from '@angular/common/http';
```

HTTP hata yanıtlarını tanımak için `HttpErrorResponse` alınır; durum koduna göre mesaj seçilecek.

```ts
import {
  Component,
  ElementRef,
  OnInit,
  effect,
  inject,
  signal,
  untracked,
  viewChild,
} from '@angular/core';
```

Bileşen araçları alınır: `signal` reaktif durum, `effect` değişime tepki, `untracked` takip dışı çalıştırma, `viewChild` şablondaki elemana erişim, `ElementRef` DOM elemanı sarmalayıcısı.

```
12 import { FormsModule } from '@angular/forms';
13 import { Router } from '@angular/router';
14 import { Observable, firstValueFrom } from 'rxjs';
```

`[(ngModel)]` form bağlama için `FormsModule`, yönlendirme için `Router` ve akışı Promise'e çeviren `firstValueFrom` alınır.

```
15 import { NzButtonModule } from 'ng-zorro-antd/button';
16 import { NzInputModule } from 'ng-zorro-antd/input';
```

ng-zorro buton ve input bileşen modülleri; şablonda hazır stilli bileşenler kullanmak için.

```
17 import { AuthResult } from '../../models/auth.models';
18 import { AuthService } from '../../services/auth.service';
19 import { GoogleIdentityService } from '../../services/google-identity.service';
```

Giriş sonucu tipi, oturum servisi ve Google buton servisi alınır.

ƒ LoginMode — Ekranın modu (giriş/kayıt) ve kullanılan giriş yöntemi (şifre/Google) için birleşim tipleri.

```
21 type LoginMode = 'login' | 'register';
22 type SignInMethod = 'password' | 'google';
```

Sadece belirtilen metinlere izin veren tipler; yazım hatası derleme anında yakalanır.

```
24 const MINIMUM_PASSWORD_LENGTH = 8;
```

En az şifre uzunluğu; API'deki kuralla aynı tutulur ki kullanıcı hatayı sunucuya gitmeden görsün.

```
26 @Component({
27   selector: 'app-login-page',
28   imports: [FormsModule, NzButtonModule, NzInputModule],
29   templateUrl: './login-page.html',
30   styleUrl: './login-page.scss',
31 })
```

Bileşen ayarları: etiket adı, standalone `imports` listesi, ayrı HTML şablon ve SCSS stil dosyası.

ƒ LoginPage — Giriş/kayıt formu ile Google girişini yöneten sayfa bileşeni.

```
32 export class LoginPage implements OnInit {
```

`implements OnInit` sınıfın `ngOnInit` yaşam döngüsü metodunu yazacağını TypeScript'e söyler.

```
33   private readonly auth = inject(AuthService);
34   private readonly googleIdentity = inject(GoogleIdentityService);
35   private readonly router = inject(Router);
```

Oturum servisi, Google servisi ve router `inject()` ile alınır.

```
36   private readonly googleButtonContainer = viewChild<ElementRef<HTMLElement>>('googleButton');
```

`viewChild` şablondaki `#googleButton` elemanını signal olarak verir; eleman sadece Google hazırken çizildiği için başta `undefined` olabilir.

```
38   readonly mode = signal<LoginMode>('login');
39   readonly isSubmitting = signal(false);
40   readonly errorMessage = signal<string | null>(null);
41   readonly googleStatus = signal<'loading' | 'ready' | 'disabled' | 'unavailable'>('loading');
```

Ekran durumu signal'leri: mod, gönderim sürüyor mu, hata mesajı ve Google butonunun durumu. Değişince şablon kendiliğinden güncellenir (zoneless).

```
42   private readonly googleClientId = signal<string | null>(null);
```

API'den gelen Google Client ID; `private` çünkü şablon değil sadece bileşen mantığı kullanır.

```
44   email = '';
45   displayName = '';
46   password = '';
```

Form alanları; şablonda `ngModel` ile iki yönlü bağlanır, kullanıcı yazdıkça güncellenir.

ƒ constructor — Google butonu elemanı ve Client ID hazır olunca butonu çizen effect'i kurar.

```
48   constructor() {
```

`effect` DI bağlamı gerektirdiği için constructor içinde oluşturulur.

```
49     effect(() => {
50       const container = this.googleButtonContainer()?.nativeElement;
51       const clientId = this.googleClientId();
```

`effect` içinde okunan signal'ler takip edilir; buton elemanı veya Client ID değişince fonksiyon yeniden çalışır. `?.` eleman yoksa `undefined` döner.

```
53       if (container === undefined || clientId === null) {
54         return;
55       }
```

İkisinden biri henüz hazır değilse hiçbir şey yapmadan çıkılır; ikisi de hazır olunca effect tekrar tetiklenir.

```
57       untracked(() => void this.renderGoogleButton(container, clientId));
```

`untracked` içindeki signal okumaları takip edilmez; böylece buton çizimi effect'i gereksiz yere yeniden tetiklemez. `void` Promise'in bilerek beklenmediğini gösterir.

```
58     });
59   }
```

ƒ ngOnInit — Sayfa açılınca Google giriş ayarlarını API'den yüklemeyi başlatır.

```
61   ngOnInit(): void {
```

Angular bileşen hazırlandığında bu metodu bir kez çağırır.

```
62     void this.setUpGoogleSignIn();
```

Google ayar yüklemesi arka planda başlatılır; sayfa beklemeden çizilir.

```
63   }
```

ƒ switchMode — Giriş ve kayıt modları arasında geçiş yapar, eski hata mesajını temizler.

```
65   switchMode(mode: LoginMode): void {
```

Şablondaki giriş/kayıt sekmeleri bu metodu çağırır.

```
66     this.mode.set(mode);
67     this.errorMessage.set(null);
```

Signal'e `set` ile yeni mod yazılır ve önceki hata mesajı silinir.

```
68   }
```

ƒ submit — Formu doğrular, moda göre giriş veya kayıt isteği gönderir.

```
70   async submit(): Promise<void> {
```

Form gönderilince çağrılan `async` metot.

```
71     const validationError = this.validateForm();
72
73     if (validationError !== null) {
74       this.errorMessage.set(validationError);
75       return;
76     }
```

Önce ön yüz doğrulaması yapılır; hata varsa mesaj gösterilir ve sunucuya istek atılmaz.

```
78     const email = this.email.trim();
```

E-postanın başındaki/sonundaki boşluklar `trim` ile temizlenir.

```
80     if (this.mode() === 'login') {
81       await this.signIn(this.auth.login({ email, password: this.password }), 'password');
```

Giriş modundaysa e-posta ve şifreyle `login` isteği hazırlanır ve ortak `signIn` ile gönderilir.

```
    } else {
      await this.signIn(
        this.auth.register({ email, displayName: this.displayName.trim(), password: this.password }),
        'password',
      );
    }
  }
```

Kayıt modundaysa e-posta, temizlenmiş ad ve şifreyle `register` isteği gönderilir.

ƒ setUpGoogleSignIn — API'den Google Client ID'yi alır ve Google butonunun durumunu belirler.

```
  private async setUpGoogleSignIn(): Promise<void> {
    try {
      const config = await firstValueFrom(this.auth.getAuthConfig());
      if (config.googleClientId === null) {
        this.googleStatus.set('disabled');
        return;
      }
      this.googleClientId.set(config.googleClientId);
      this.googleStatus.set('ready');
    } catch {
      this.googleStatus.set('unavailable');
    }
  }
```

Sonuç `googleStatus` ve `googleClientId` signal'lerine yazılır.

`/auth/config` isteği yapılır ve sonucu `await` ile beklenir.

Sunucuda Google yapılandırılmamışsa buton devre dışı gösterilir.

Client ID saklanır ve durum `ready` olur; bu, şablonda buton alanını açar ve constructor'daki effect butonu çizer.

API'ye ulaşılamazsa Google girişi kullanılamaz olarak işaretlenir; şifreyle giriş yine çalışır.

ƒ renderGoogleButton — Google butonunu çizer; kullanıcı hesap seçince Google ile girişi başlatır.

```
  private async renderGoogleButton(container: HTMLElement, clientId: string): Promise<void> {
    try {
      await this.googleIdentity.renderButton(container, clientId, (idToken) => {
        void this.signIn(this.auth.loginWithGoogle(idToken), 'google');
      });
    } catch {
      this.googleStatus.set('unavailable');
    }
  }
```

Effect tarafından eleman ve Client ID hazır olduğunda çağrılır.

Servis butonu çizer; Google ID token verince `loginWithGoogle` isteği ortak `signIn` ile gönderilir.

Google betiği yüklenemezse (ör. ağ engeli) durum `unavailable` yapılır.

ƒ signIn — Giriş/kayıt isteğini çalıştırır, başarıda ana sayfaya gider, hatada mesaj gösterir.

```
  private async signIn(request: Observable<AuthResult>, method: SignInMethod): Promise<void> {
    this.isSubmitting.set(true);
    this.errorMessage.set(null);
    try {
      await firstValueFrom(request);
      await this.router.navigate(['/']);
    } catch (error: unknown) {
      this.errorMessage.set(this.describeError(error, method));
    } finally {
      this.isSubmitting.set(false);
    }
  }
```

Şifre ve Google girişi aynı akışı kullanır; tekrarlanan kod tek yerde toplanır.

Gönderim başlar: buton kilitlenir ve önceki hata silinir.

İstek beklenir; servis oturumu belleğe yazınca ana sayfaya yönlendirilir.

Hata olursa durum koduna göre anlaşılır bir Türkçe mesaj gösterilir.

`finally` başarıda da hatada da çalışır; gönderim bitti işaretlenir ve buton tekrar açılır.

ƒ validateForm — Form alanlarını kontrol eder; ilk bulunan hata mesajını ya da null döndürür.

```
  private validateForm(): string | null {
    const email = this.email.trim();

    if (email.length === 0 || !email.includes('@')) {
      return 'Geçerli bir e-posta adresi gir.';
    }

    if (this.mode() === 'register' && this.displayName.trim().length === 0) {
      return 'Adını boş bırakma.';
    }

    if (this.password.length === 0) {
      return 'Şifreni gir.';
    }

    if (this.mode() === 'register') {
      const hasLetter = /\p{L}/u.test(this.password);
      const hasDigit = /\d/.test(this.password);
      if (this.password.length < MINIMUM_PASSWORD_LENGTH || !hasLetter || !hasDigit) {
        return `Şifre en az ${MINIMUM_PASSWORD_LENGTH} karakter olmalı, harf ve rakam içermeli.`;
      }
    }

    return null;
  }
```

Hızlı geri bildirim içindir; asıl doğrulama yine API'de yapılır.

E-posta boşsa veya `@` içermiyorsa hata döner.

Kayıt modunda görünen ad boş bırakılamaz.

Şifre boş olamaz.

Kayıtta şifre gücü kontrol edilir: `\p{L}` her dilden harfi (Türkçe dahil), `\d` rakamı yakalar; `u` bayrağı Unicode desteği açar.

Şifre 8 karakterden kısaysa ya da harf veya rakam içermiyorsa açıklayıcı hata döner.

Hata yoksa `null` döner; form gönderilebilir.

ƒ describeError — HTTP hatasını kullanıcıya gösterilecek Türkçe mesaja çevirir.

```
  private describeError(error: unknown, method: SignInMethod): string {
```

`unknown` tipi hatanın ne olduğunu bilmediğimizi söyler; kullanmadan önce kontrol etmek zorunludur.

```
    if (!(error instanceof HttpErrorResponse)) {
      return 'Giriş yapılamadı. Lütfen tekrar dene.';
    }
```
Hata HTTP hatası değilse (ör. kod hatası) genel bir mesaj döner.

```
    if (error.status === 0) {
      return 'API sunucusuna bağlanılamadı. HerSeyiBaslat.bat ile uygulamayı aç.';
    }
```
Durum 0, sunucuya hiç ulaşılamadığı anlamına gelir; kullanıcıya API'yi başlatması söylenir.

```
    if (error.status === 401) {
      return method === 'google'
        ? 'Google ile giriş doğrulanamadı.'
        : 'E-posta adresi veya şifre hatalı.';
    }
```
401: kimlik doğrulanamadı. Yönteme göre Google veya e-posta/şifre mesajı seçilir; hangi alanın yanlış olduğu bilerek söylenmez.

```
    if (error.status === 409) {
      return 'Bu e-posta adresiyle zaten bir hesap var. Giriş yapmayı dene; hesabın Google ile açtıysan "Google ile devam et" butonunu kullan.';
    }
```
409: bu e-postayla hesap zaten var; kullanıcı girişe veya Google butonuna yönlendirilir.

```
    if (error.status === 429) {
      return 'Çok fazla deneme yapıldı. Bir dakika bekleyip tekrar dene.';
    }
```
429: API'nin hız sınırı (rate limit) aşıldı; kaba kuvvet denemelerine karşı koruma.

```
    const problem = error.error as { detail?: string; title?: string } | null;
```
Diğer hatalarda gövde, API'nin döndürdüğü ProblemDetails biçiminde varsayılır; `as` tip dönüşümüdür.

```
    return problem?.detail ?? problem?.title ?? `Sunucu yanıtı: ${error.status}`;
```
Sırayla `detail`, `title` ya da durum kodu gösterilir. `?.` null güvenli erişim, `??` ilk dolu değeri seçer.

```
  }
}
```

Web (Angular) `todo.Web/src/app/models/auth.models.ts`

> Giriş/oturum ile ilgili API sözleşmesinin TypeScript tipleri. Backend'deki DTO'ların ön yüzdeki karşılığıdır; derleme anında tip güvenliği sağlar.

ƒ UserRole — Kullanıcı rolü: yalnızca 'User' veya 'Admin' olabilir.

```
export type UserRole = 'User' | 'Admin';
```
Birleşim (union) tipi; değişkene bu iki metin dışında değer atanırsa TypeScript hata verir.

ƒ CurrentUser — Giriş yapmış kullanıcının ön yüzde gösterilen bilgileri.

```
export interface CurrentUser {
```
`interface` sadece verinin şeklini tanımlar; çalışma zamanında kod üretmez.

```
  id: number;
  email: string;
  displayName: string;
  role: UserRole;
```
Kullanıcı kimliği, e-posta, görünen ad ve rol bilgisi.

```
  hasPassword: boolean;
  hasGoogleLogin: boolean;
```
Hesabın şifresi ve Google bağlantısı olup olmadığı; arayüz buna göre seçenek gösterebilir.

```
  createdAtUtc: string;
```
Hesabın oluşturulma zamanı; JSON'da tarih metin (ISO, UTC) olarak gelir.

```
}
```

ƒ AuthResult — Giriş, kayıt ve yenileme uçlarının döndürdüğü sonuç.

```
export interface AuthResult {
```
Refresh token bu gövdede yoktur; sunucu onu HttpOnly çerez olarak ayrıca gönderir.

```
  accessToken: string;
  expiresAtUtc: string;
  user: CurrentUser;
```
Kısa ömürlü access token, bitiş zamanı (UTC metin) ve kullanıcı bilgisi.

```
}
```

ƒ AuthConfig — API'nin giriş ayarları; Google Client ID içerir.

```
export interface AuthConfig {
```
`/auth/config` ucunun yanıt tipi.

```
  googleClientId: string | null;
```
Google girişi yapılandırılmamışsa `null` gelir; giriş sayfası o zaman Google butonunu kapatır.

```
}
```

ƒ LoginRequest — E-posta/şifre ile giriş isteğinin gövdesi.

```
export interface LoginRequest {
```
`/auth/login` ucuna gönderilen veri.

```
  email: string;
  password: string;
```
Kullanıcının girdiği e-posta ve şifre.

```
}
```

ƒ RegisterRequest — Yeni hesap oluşturma isteğinin gövdesi.

```
export interface RegisterRequest {
```
`/auth/register` ucuna gönderilen veri.

```
  email: string;
  displayName: string;
  password: string;
```
E-posta, görünen ad ve şifre; kurallar hem ön yüzde hem API'de doğrulanır.

```
}
```

Web (Angular) `todo.Web/src/app/services/auth.service.ts`

> Ön yüzdeki oturumun tek sahibi. Giriş, kayıt, Google girişi, oturum yenileme ve çıkış isteklerini yapar; access token'ı yalnızca bellekte (signal içinde) tutar.

```ts
import { HttpClient } from '@angular/common/http';
import { Injectable, computed, inject, signal } from '@angular/core';
import { Router } from '@angular/router';
```

API'ye istek atan `HttpClient`, Angular çekirdek araçları (`signal`, `computed`, `inject`, `Injectable`) ve yönlendirme için `Router` alınır.

```ts
import {
  Observable,
  catchError,
  finalize,
  firstValueFrom,
  map,
  of,
  shareReplay,
  tap,
} from 'rxjs';
```

RxJS araçları alınır: `map` dönüştürür, `tap` yan etki yapar, `of` sabit değer akışı üretir, `finalize` bitişte çalışır, `shareReplay` sonucu paylaşır, `firstValueFrom` akışı Promise'e çevirir.

```ts
import { resolveApiUrl } from '../core/api-url';
import {
  AuthConfig,
  AuthResult,
  CurrentUser,
  LoginRequest,
  RegisterRequest,
} from '../models/auth.models';
```

API adres yardımcısı ve sunucuyla konuşurken kullanılan veri tipleri (`AuthResult`, `CurrentUser`, istek gövdeleri) alınır.

ƒ Session — Bellekte tutulan oturum: access token, bitiş zamanı ve kullanıcı bilgisi.

```ts
interface Session {
  accessToken: string;
  expiresAtUtc: string;
  user: CurrentUser;
}
```

Dosyaya özel arayüz. Refresh token burada yok; o HttpOnly çerezde durur ve JavaScript ona hiç erişemez.

```ts
const EXPIRY_SAFETY_MARGIN_MS = 30_000;
```

Token bitişinden 30 saniye önce onu süresi dolmuş say; istek yoldayken token ölmesin diye güvenlik payı. `30_000` okunaklı yazılmış 30000'dir.

```ts
@Injectable({
  providedIn: 'root',
})
```

`@Injectable({providedIn: 'root'})` servisi uygulama genelinde tek örnek (singleton) yapar; ayrıca bir yerde kaydetmeye gerek kalmaz.

ƒ AuthService — Giriş, kayıt, Google girişi, oturum yenileme, çıkış ve bellekteki token yönetimini yapan servis.

```ts
export class AuthService {
```

Servis sınıfı dışa aktarılır; guard'lar, interceptor ve sayfalar bunu `inject` ile kullanır.

```ts
  private readonly http = inject(HttpClient);
  private readonly router = inject(Router);
  private readonly apiUrl = resolveApiUrl();
```

`HttpClient` ve `Router` `inject()` ile alınır, API adresi bir kez hesaplanır. `readonly` alanın sonradan yeniden atanamayacağını belirtir.

```ts
  private readonly session = signal<Session | null>(null);
```

Oturum bir `signal` içinde tutulur: değeri değişince ona bağlı ekranlar ve `computed` değerler otomatik güncellenir. Başta oturum yok (`null`).

```ts
  private refreshInFlight: Observable<Session | null> | null = null;
```

Devam eden yenileme isteğini saklar; aynı anda birden çok istek 401 alırsa hepsi aynı tek yenileme isteğini bekler.

```ts
  readonly currentUser = computed(() => this.session()?.user ?? null);
  readonly isAdmin = computed(() => this.currentUser()?.role === 'Admin');
```

`computed` başka signal'lerden türeyen salt okunur değerdir: o anki kullanıcı ve kullanıcının Admin olup olmadığı otomatik hesaplanır.

ƒ getAuthConfig — API'den giriş ayarlarını (Google Client ID) çeker.

```ts
  getAuthConfig(): Observable<AuthConfig> {
```

Giriş sayfası Google butonunu göstermek için önce bu ayarı ister.

```ts
    return this.http.get<AuthConfig>(`${this.apiUrl}/auth/config`);
```

`/auth/config` adresine GET isteği atar; `<AuthConfig>` generic'i yanıtın tipini belirtir.

```ts
  }
```

ƒ login — E-posta ve şifreyle giriş yapar, başarılıysa oturumu başlatır.

```ts
  login(request: LoginRequest): Observable<AuthResult> {
```

Dönen `Observable` çağıran abone olunca çalışır; HTTP istekleri tembeldir.

```ts
    return this.http
      .post<AuthResult>(`${this.apiUrl}/auth/login`, request)
      .pipe(tap((result) => this.startSession(result)));
```

`/auth/login`'e POST atar; `tap` başarılı yanıtta oturumu belleğe yazar. Refresh çerezini sunucu yanıtla birlikte ayarlar.

```ts
  }
```

ƒ register — Yeni hesap oluşturur ve kullanıcıyı hemen giriş yapmış hale getirir.

```ts
  register(request: RegisterRequest): Observable<AuthResult> {
```

Kayıt da girişle aynı sonucu (`AuthResult`) döner.

```ts
    return this.http
      .post<AuthResult>(`${this.apiUrl}/auth/register`, request)
      .pipe(tap((result) => this.startSession(result)));
```

`/auth/register`'a POST atar; başarılıysa oturum başlatılır.

```ts
  }
```

ƒ loginWithGoogle — Google'dan alınan ID token'ı API'ye gönderip giriş yapar.

```ts
  loginWithGoogle(idToken: string): Observable<AuthResult> {
```

Parametre Google'ın verdiği kimlik token'ıdır; doğrulamayı API sunucu tarafında yapar.

```ts
    return this.http
      .post<AuthResult>(`${this.apiUrl}/auth/google`, { idToken })
      .pipe(tap((result) => this.startSession(result)));
```

`/auth/google`'a `{ idToken }` gövdesiyle POST atar; başarılıysa oturum başlatılır.

```
64  }
```

ƒ restoreSession — Uygulama açılırken refresh çereziyle oturumu geri yüklemeye çalışır.

```
66  async restoreSession(): Promise<void> {
```

`app.config.ts` içindeki `provideAppInitializer` bunu çağırır; sayfa yenilense bile kullanıcı tekrar giriş yapmak zorunda kalmaz.

```
67    const restored = await firstValueFrom(this.refreshSession());
```

`firstValueFrom` Observable'ın ilk değerini Promise olarak verir; böylece `await` ile beklenebilir.

```
69    if (restored === null) {
70      await firstValueFrom(this.refreshSession());
71    }
```

İlk deneme başarısızsa bir kez daha denenir; geçici bir hata (ör. başka sekmenin aynı anda token döndürmesi) yüzünden kullanıcı çıkış yapmış görünmesin.

```
72  }
```

ƒ refreshSession — Refresh çereziyle yeni access token alır; aynı anda gelen çağrılar tek isteği paylaşır.

```
74  refreshSession(): Observable<Session | null> {
```

Başarılıysa yeni oturumu, başarısızsa `null` döndüren bir akış verir.

```
75    if (this.refreshInFlight === null) {
76      this.refreshInFlight = this.http.post<AuthResult>(`${this.apiUrl}/auth/refresh`, null).pipe(
```

Devam eden yenileme yoksa `/auth/refresh`'e POST başlatılır. Gövde boştur; kimlik HttpOnly çerezle kanıtlanır.

```
77        map((result) => this.startSession(result)),
```

Başarılı yanıt `startSession` ile belleğe yazılır ve oluşan oturum akışın değeri olur.

```
78        catchError(() => {
79          this.session.set(null);
80          return of(null);
81        }),
```

Hata olursa (çerez yok/süresi dolmuş) oturum temizlenir ve hata fırlatmak yerine `null` döndürülür.

```
82        finalize(() => {
83          this.refreshInFlight = null;
84        }),
```

İstek bitince (başarı ya da hata) paylaşılan istek sıfırlanır; sonraki yenileme yeni istek atar.

```
85        shareReplay(1),
```

`shareReplay(1)` tek HTTP isteğini tüm abonelerle paylaşır ve son sonucu saklar; aynı refresh token iki kez kullanılmaz.

```
86      );
87    }
89    return this.refreshInFlight;
```

Mevcut ya da yeni oluşturulan paylaşılan akış döndürülür.

```
90  }
```

ƒ isAuthenticated — Bellekte oturum olup olmadığını döndürür.

```
92  isAuthenticated(): boolean {
```

Guard'lar sayfaya izin verip vermeyeceğine bununla karar verir.

```
93    return this.session() !== null;
```

Signal `()` ile okunur; oturum `null` değilse kullanıcı giriş yapmıştır.

```
94  }
```

ƒ getAccessToken — Hâlâ geçerli olan access token'ı, yoksa null döndürür.

```
96  getAccessToken(): string | null {
```

Interceptor her API isteğinde token'ı buradan alır.

```
97    const session = this.session();
98
99    if (session === null) {
100     return null;
101   }
```

Oturum yoksa token da yoktur; `null` döner.

```
103   if (Date.parse(session.expiresAtUtc) - EXPIRY_SAFETY_MARGIN_MS <= Date.now()) {
104     return null;
105   }
```

Bitiş zamanına 30 saniyeden az kaldıysa token kullanılmaz; interceptor önce yenileme yapar.

```
107   return session.accessToken;
```

Token geçerliyse döndürülür.

```
108 }
```

ƒ logout — Sunucuda refresh token'ı iptal ettirir ve yerel oturumu kapatır.

```
110 async logout(): Promise<void> {
```

`async` fonksiyon Promise döndürür; içinde `await` kullanılabilir.

```
111   try {
112     await firstValueFrom(this.http.post<void>(`${this.apiUrl}/auth/logout`, null));
113   } catch {
114   }
```

`/auth/logout` çağrılır; sunucu refresh token'ı iptal edip çerezi siler. Hata olsa bile yerel çıkış yapılsın diye hata yutulur.

```
116   this.endSession();
```

Her durumda yerel oturum kapatılır ve giriş sayfasına gidilir.

```
117 }
```

ƒ endSession — Bellekteki oturumu siler ve kullanıcıyı giriş sayfasına yönlendirir.

```
119 endSession(): void {
```

Interceptor yenileme başarısız olunca da bunu çağırır.

```
120   this.session.set(null);
121   void this.router.navigate(['/login']);
```

Oturum `null` yapılır; token bellekten silinir. `void` yönlendirme Promise'inin bilerek beklenmediğini belirtir.

```
122 }
```

ƒ startSession — API'den gelen giriş sonucunu bellekteki oturuma çevirip saklar.

```
124 private startSession(result: AuthResult): Session {
```

`private` olduğu için sadece bu servis içinden çağrılır.

```
125   const session: Session = {
126     accessToken: result.accessToken,
127     expiresAtUtc: result.expiresAtUtc,
128     user: result.user,
129   };
```

Yanıttaki access token, bitiş zamanı ve kullanıcı bilgisiyle yeni oturum nesnesi oluşturulur.

```
131   this.session.set(session);
132
133   return session;
```

Oturum signal'e yazılır (bağlı ekranlar güncellenir) ve oluşturulan oturum döndürülür.

```
  }
}
```

Web (Angular) `todo.Web/src/app/interceptors/auth.interceptor.ts`

> API'ye giden her HTTP isteğinin arasına girer: auth isteklerine çerez ve anti-CSRF başlığı ekler, diğerlerine access token ekler, 401 gelirse oturumu yenileyip isteği bir kez tekrarlar.

```ts
import {
  HttpErrorResponse,
  HttpEvent,
  HttpHandlerFn,
  HttpInterceptorFn,
  HttpRequest,
} from '@angular/common/http';
```

Angular HTTP tipleri alınır: hata yanıtı, olay, sonraki işleyici, interceptor fonksiyon tipi ve istek nesnesi.

```ts
import { inject } from '@angular/core';
```

Servisleri DI'dan almak için `inject` fonksiyonu alınır.

```ts
import { Observable, catchError, switchMap, throwError } from 'rxjs';
```

RxJS araçları: `Observable` zamanla değer üreten akıştır; `catchError` hatayı yakalar, `switchMap` bir akışı başka akışa çevirir, `throwError` hata akışı üretir.

```ts
import { resolveApiUrl } from '../core/api-url';
import { AuthService } from '../services/auth.service';
```

API temel adresini veren yardımcı fonksiyon ve oturum/token bilgisini tutan `AuthService` alınır.

```ts
export const CLIENT_HEADER_NAME = 'X-Todo-Client';
```

Anti-CSRF başlık adı. API, `/api/auth` uçlarında bu başlığı zorunlu tutar; başka bir site tarayıcıya bu özel başlığı ekletemediği için sahte istekler engellenir.

```ts
const ENDPOINTS_WITHOUT_ACCESS_TOKEN = [
  '/auth/config',
  '/auth/login',
  '/auth/register',
  '/auth/google',
  '/auth/refresh',
  '/auth/logout',
];
```

Access token gerektirmeyen auth uçlarının listesi. Bunlar ya girişten önce çağrılır ya da kimliği HttpOnly refresh çereziyle kanıtlar.

ƒ authInterceptor — Her API isteğine uygun kimlik bilgilerini ekler ve 401 alınınca oturumu yenileyip isteği bir kez tekrar dener.

```ts
export const authInterceptor: HttpInterceptorFn = (request, next) => {
```

`HttpInterceptorFn` tipinde fonksiyon; `request` giden istek, `next` ise isteği zincirde bir sonraki adıma (sonunda sunucuya) iletir.

```ts
  const auth = inject(AuthService);
  const apiUrl = resolveApiUrl();
```

Oturum servisi `inject()` ile alınır ve API'nin temel adresi hesaplanır.

```ts
  if (!request.url.startsWith(apiUrl)) {
    return next(request);
  }
```

İstek bizim API'mize gitmiyorsa (ör. Google betiği) hiç dokunmadan iletilir; token başka sunuculara sızmaz.

```ts
  const isAuthRequest = request.url.startsWith(`${apiUrl}/auth/`);
```

İstek `/auth/` altındaki bir uca mı gidiyor diye bakılır. `${...}` template string içinde değişken yerleştirmedir.

```ts
  const needsAccessToken = !ENDPOINTS_WITHOUT_ACCESS_TOKEN.some((endpoint) =>
    request.url.endsWith(endpoint),
  );
```

URL listedeki uçlardan biriyle bitmiyorsa access token gerekir. `some` dizide en az bir eşleşme olup olmadığına bakar.

```ts
  const preparedRequest = isAuthRequest
    ? request.clone({ withCredentials: true, setHeaders: { [CLIENT_HEADER_NAME]: '1' } })
    : request;
```

Auth isteklerinde `withCredentials: true` ile refresh çerezi gönderilir ve anti-CSRF başlığı eklenir. İstek nesneleri değişmez olduğu için `clone` ile kopyalanır.

```ts
  if (!needsAccessToken) {
    return next(preparedRequest);
  }
```

Token gerektirmeyen istek hazırlanmış haliyle doğrudan gönderilir.

```ts
  const accessToken = auth.getAccessToken();
```

Bellekteki access token okunur; süresi dolmak üzereyse servis `null` döndürür.

```ts
  const firstAttempt =
    accessToken !== null
      ? sendWithToken(preparedRequest, next, accessToken)
      : auth
          .refreshSession()
          .pipe(switchMap((session) => sendWithToken(preparedRequest, next, session?.accessToken ?? null)));
```

Geçerli token varsa istek `Authorization: Bearer` başlığıyla gönderilir.

Token yoksa önce oturum yenilenir, sonra yeni token ile istek gönderilir. `?.` null ise durur, `??` soldaki null ise sağdakini kullanır.

```ts
  return firstAttempt.pipe(
    catchError((error: unknown) => {
      if (!(error instanceof HttpErrorResponse) || error.status !== 401) {
        return throwError(() => error);
      }
      return auth.refreshSession().pipe(
        switchMap((session) => {
          if (session === null) {
            auth.endSession();
            return throwError(() => error);
          }
          return sendWithToken(preparedRequest, next, session.accessToken);
```

İlk denemenin sonucuna `pipe` ile hata yakalayıcı eklenir; `pipe` RxJS operatörlerini sırayla bağlar.

Hata 401 (yetkisiz) değilse dokunulmaz, aynen yukarı fırlatılır; ekran kendi hata mesajını gösterir.

401 geldiyse token geçersiz olmuştur; refresh çereziyle yeni oturum istenir ve sonucuna göre devam edilir.

Yenileme başarısızsa (refresh token yok/geçersiz) oturum kapatılıp giriş sayfasına gidilir ve orijinal hata fırlatılır.

Yenileme başarılıysa istek yeni access token ile bir kez daha gönderilir; kullanıcı hiçbir şey fark etmez.

```
        }),
      );
    }),
  );
};
```

ƒ sendWithToken — Token varsa isteğe Bearer başlığı ekleyip gönderir, yoksa isteği olduğu gibi iletir.

```
function sendWithToken(
  request: HttpRequest<unknown>,
  next: HttpHandlerFn,
  accessToken: string | null,
): Observable<HttpEvent<unknown>> {
```

Yardımcı fonksiyon; dosya içinde kalır, dışa aktarılmaz. `<unknown>` generic parametresi gövde tipinin belirsiz olduğunu söyler.

```
  if (accessToken === null) {
    return next(request);
  }
```

Token yoksa istek başlıksız gönderilir; API 401 dönerse yukarıdaki yenileme mantığı devreye girer.

```
  return next(request.clone({ setHeaders: { Authorization: `Bearer ${accessToken}` } }));
```

İstek kopyalanıp `Authorization: Bearer <token>` başlığı eklenir ve zincire iletilir.

```
}
```

Web (Angular) todo.Web/src/app/services/google-identity.service.ts

> Google Identity Services betiğini gerektiğinde bir kez yükler ve "Google ile devam et" butonunu çizer. Kullanıcı seçim yapınca Google'ın verdiği ID token'ı çağırana iletir.

```
import { Injectable } from '@angular/core';
```

Servis tanımlamak için `Injectable` dekoratörü alınır.

ƒ GoogleCredentialResponse — Google'ın giriş sonrası verdiği yanıt; içinde ID token bulunur.

```
interface GoogleCredentialResponse {
  credential: string;
}
```

`credential` alanı Google'ın imzaladığı ID token'dır (JWT); API bunu doğrular.

ƒ GoogleIdentityApi — Google betiğinin `window.google` üzerinde sunduğu API'nin kullandığımız kısmının tipi.

```
interface GoogleIdentityApi {
  accounts: {
    id: {
```

Google betiği TypeScript tipi getirmediği için kullandığımız kısmı kendimiz tanımlarız; yanlış kullanımda derleme hatası alırız.

```
      initialize(options: {
        client_id: string;
        callback: (response: GoogleCredentialResponse) => void;
      }): void;
```

`initialize`: Client ID ve kullanıcı giriş yapınca çağrılacak `callback` fonksiyonuyla Google'ı hazırlar.

```
      renderButton(
        parent: HTMLElement,
        options: {
```

`renderButton`: Google butonunu verilen HTML elemanının içine çizer; ikinci parametre görünüm ayarlarıdır.

```
          type?: 'standard' | 'icon';
          theme?: 'outline' | 'filled_blue' | 'filled_black';
          size?: 'large' | 'medium' | 'small';
          text?: 'signin_with' | 'signup_with' | 'continue_with';
          shape?: 'rectangular' | 'pill';
```

Buton tipi, tema, boyut, yazı ve şekil seçenekleri; birleşim tipleri sadece geçerli değerlere izin verir. `?` alanın isteğe bağlı olduğunu belirtir.

```
          width?: number;
          locale?: string;
        },
      ): void;
```

Buton genişliği (piksel) ve dil ayarı; fonksiyon bir şey döndürmez (`void`).

```
    };
  };
}
```

```
declare global {
  interface Window {
    google?: GoogleIdentityApi;
  }
}
```

`declare global` ile `Window` tipine isteğe bağlı `google` alanı eklenir; betik yüklenince `window.google` dolu olur.

```
const GOOGLE_SCRIPT_URL = 'https://accounts.google.com/gsi/client';
```

Google'ın resmi giriş betiğinin adresi; sadece giriş sayfası açılınca yüklenir.

```
@Injectable({
  providedIn: 'root',
})
```

`providedIn: 'root'` ile servis tüm uygulamada tek örnek olur; betik yükleme durumu da paylaşılmış olur.

ƒ GoogleIdentityService — Google betiğini yükleyip giriş butonunu çizen ve ID token'ı ileten servis.

```
export class GoogleIdentityService {
```

Google'a özgü ayrıntılar bu serviste saklanır; giriş sayfası sadece basit bir metot çağırır.

```
  private scriptLoading: Promise<GoogleIdentityApi> | null = null;
```

Devam eden ya da biten betik yüklemesinin Promise'i; betiğin sayfaya iki kez eklenmesini önler.

ƒ renderButton — Google betiğini yükler, Client ID ile başlatır ve butonu verilen elemana çizer.

```
  async renderButton(
    container: HTMLElement,
    clientId: string,
    onCredential: (idToken: string) => void,
  ): Promise<void> {
```

Parametreler: butonun konacağı eleman, Google Client ID ve token gelince çağrılacak fonksiyon. `Promise<void>` değer döndürmeden biten async işlem demektir.

```
    const google = await this.loadScript();
```

Betik yüklenene kadar `await` ile beklenir ve Google API nesnesi alınır.

```
    google.accounts.id.initialize({
      client_id: clientId,
      callback: (response) => onCredential(response.credential),
    });
    google.accounts.id.renderButton(container, {
      type: 'standard',
      theme: 'outline',
      size: 'large',
      text: 'continue_with',
      shape: 'rectangular',
      width: Math.min(container.clientWidth || 320, 400),
      locale: 'tr',
    });
  }
```

Google başlatılır; kullanıcı hesap seçince `callback` çalışır ve yanıttaki ID token `onCredential`'a verilir.

Butonu standart, çerçeveli, büyük ve "Google ile devam et" yazılı olarak çizer.

Genişlik kapsayıcının genişliğidir (0 ise 320) ama en fazla 400; Google bu sınırın üstünü kabul etmez. `||` soldaki 0 ise sağı kullanır.

Buton metni Türkçe gösterilir.

ƒ loadScript — Google betiğini sayfaya bir kez ekler ve yüklenince `window.google` nesnesini döndürür.

```
  private loadScript(): Promise<GoogleIdentityApi> {
    if (this.scriptLoading !== null) {
      return this.scriptLoading;
    }
    this.scriptLoading = new Promise<GoogleIdentityApi>((resolve, reject) => {
      if (window.google !== undefined) {
        resolve(window.google);
        return;
      }
      const script = document.createElement('script');
      script.src = GOOGLE_SCRIPT_URL;
      script.async = true;
      script.defer = true;
      script.onload = () => {
        if (window.google !== undefined) {
          resolve(window.google);
        } else {
          reject(new Error('Google Identity Services yüklenemedi.'));
        }
      };
      script.onerror = () => {
        this.scriptLoading = null;
        reject(new Error('Google Identity Services yüklenemedi.'));
      };
      document.head.appendChild(script);
    });
    return this.scriptLoading;
  }
}
```

`private` yardımcı metot; sadece bu servis içinde kullanılır.

Yükleme zaten başlamışsa aynı Promise döndürülür; betik tekrar eklenmez.

Yeni bir Promise oluşturulur; `resolve` başarıyı, `reject` hatayı bildirir. Böylece callback tabanlı betik yükleme `await` ile kullanılabilir.

Betik daha önce başka yoldan yüklendiyse hemen mevcut nesneyle tamamlanır.

Yeni bir `<script>` etiketi oluşturulur; `async` / `defer` sayfanın çizimini bloklamadan arka planda yüklenmesini sağlar.

Betik yüklenince `window.google` varsa Promise başarıyla, yoksa hata ile tamamlanır.

Ağ hatasında saklanan Promise sıfırlanır ki sonraki denemede yeniden yüklenebilsin; ardından hata bildirilir.

Etiket sayfanın `<head>` bölümüne eklenir; tarayıcı bu anda indirmeye başlar.

Yükleme Promise'i döndürülür.

API `todo.API/Controllers/AuthController.cs`

> Giriş, kayıt, Google ile giriş, oturum yenileme, çıkış ve 'ben kimim' uç noktalarını sunan API controller'ı. İşi kendisi yapmaz; isteği use case'lere iletir ve refresh token çerezini yönetir.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;
using TodoApi.API.Authentication;
using TodoApi.API.Models.Requests;
using TodoApi.API.Models.Responses;
using TodoApi.Application.Contracts;
using TodoApi.Application.Exceptions;
using TodoApi.Application.Features.Authentication.Common;
using TodoApi.Application.Features.Authentication.GetCurrentUser;
using TodoApi.Application.Features.Authentication.LoginWithGoogle;
using TodoApi.Application.Features.Authentication.LoginWithPassword;
using TodoApi.Application.Features.Authentication.Logout;
using TodoApi.Application.Features.Authentication.RefreshSession;
using TodoApi.Application.Features.Authentication.RegisterWithPassword;
using TodoApi.Infrastructure.Services.Authentication;
namespace TodoApi.API.Controllers;
```

ASP.NET Core'un yetkilendirme, MVC (controller) ve hız sınırlama (rate limiting) özelliklerini içeri alır.

Projenin kendi katmanlarını içeri alır: API yardımcıları, istek/yanıt modelleri, Application use case'leri ve Google ayarları için Infrastructure.

Bu dosyadaki tiplerin ad alanı (namespace); klasör yapısıyla uyumlu tutulur.

ƒ AuthController — Kimlik doğrulama ile ilgili tüm HTTP uç noktalarını toplayan, işi use case'lere devreden controller.

```
[ApiController]
[Route("api/auth")]
```

`[ApiController]` bir attribute'tur (sınıfa eklenen etiket); otomatik model doğrulama ve 400 yanıtı gibi API davranışlarını açar.

`[Route("api/auth")]` bu controller'daki tüm uç noktaların adresinin `api/auth` ile başlayacağını belirtir.

```
[RequireClientHeader]
```

`[RequireClientHeader]` GET dışındaki isteklerde `X-Todo-Client` başlığını zorunlu kılar; çerezle çalışan uç noktaları CSRF saldırısına karşı korur.

```
public sealed class AuthController : ControllerBase
```

`sealed` sınıftan kalıtım alınmasını engeller. `ControllerBase` `Ok`, `Problem` gibi yanıt üreten yardımcı metotları sağlar.

```
{
    private readonly RegisterWithPasswordUseCase _registerWithPasswordUseCase;
    private readonly LoginWithPasswordUseCase _loginWithPasswordUseCase;
    private readonly LoginWithGoogleUseCase _loginWithGoogleUseCase;
    private readonly RefreshSessionUseCase _refreshSessionUseCase;
    private readonly LogoutUseCase _logoutUseCase;
    private readonly GetCurrentUserUseCase _getCurrentUserUseCase;
    private readonly GoogleAuthenticationSettings _googleSettings;
```

`private readonly` alanlar: sadece bu sınıf görür ve kurucuda bir kez atanır. Her uç noktanın çağıracağı use case burada tutulur.

ƒ AuthController (constructor) — Gerekli use case'leri ve Google ayarlarını Dependency Injection ile alıp alanlara kaydeder.

```
    public AuthController(
        RegisterWithPasswordUseCase registerWithPasswordUseCase,
        LoginWithPasswordUseCase loginWithPasswordUseCase,
        LoginWithGoogleUseCase loginWithGoogleUseCase,
        RefreshSessionUseCase refreshSessionUseCase,
        LogoutUseCase logoutUseCase,
        GetCurrentUserUseCase getCurrentUserUseCase,
        GoogleAuthenticationSettings googleSettings)
```

Kurucu metot; parametreleri ASP.NET Core'un Dependency Injection konteyneri otomatik olarak doldurur, controller nesneleri kendisi `new` ile oluşturmaz.

```
    {
        _registerWithPasswordUseCase = registerWithPasswordUseCase;
        _loginWithPasswordUseCase = loginWithPasswordUseCase;
        _loginWithGoogleUseCase = loginWithGoogleUseCase;
        _refreshSessionUseCase = refreshSessionUseCase;
        _logoutUseCase = logoutUseCase;
        _getCurrentUserUseCase = getCurrentUserUseCase;
        _googleSettings = googleSettings;
    }
```

Gelen bağımlılıkları ileride kullanmak üzere `readonly` alanlara atar.

ƒ GetConfig — Frontend'e Google Client Id gibi herkese açık giriş ayarlarını döner.

```
    [AllowAnonymous]
```

`[AllowAnonymous]` giriş yapmamış kullanıcının da bu uç noktayı çağırabilmesini sağlar; aksi halde FallbackPolicy her isteği kimlik doğrulamaya zorlar.

```
    [HttpGet("config")]
    public ActionResult<AuthConfigResponse> GetConfig()
```

`[HttpGet("config")]` GET /api/auth/config isteğini bu metoda bağlar. `ActionResult<T>` hem T verisi hem de hata yanıtı dönebilmeyi sağlar.

```
    {
        AuthConfigResponse response = new AuthConfigResponse
        {
            GoogleClientId = _googleSettings.ClientId
        };
```

Yanıt nesnesini object initializer ile oluşturur; Google ayarlı değilse `GoogleClientId` null olur ve frontend Google butonunu göstermez.

```
        return Ok(response);
```

`Ok(...)` nesneyi JSON'a çevirip 200 durum koduyla döner.

```
    }
```

ƒ Register — E-posta/şifre ile kayıt isteğini use case'e iletir, refresh token çerezini yazar, 201 döner.

```
    [AllowAnonymous]
    [EnableRateLimiting(RateLimitingPolicies.Authentication)]
```

Anonim erişime açık; `[EnableRateLimiting]` bu uç noktayı dakikada 10 istekle sınırlar, kaba kuvvet ve spam kayıtlara karşı koruma sağlar.

```
    [HttpPost("register")]
    public async Task<ActionResult<AuthResultDto>> Register(
```

`[HttpPost("register")]` POST /api/auth/register'a bağlar. `async Task<...>` metodun asenkron çalıştığını ve sonucu ileride döneceğini belirtir.

```
        [FromBody] RegisterRequest request,
        CancellationToken cancellationToken)
```

`[FromBody]` isteğin JSON gövdesini `RegisterRequest` nesnesine çevirir. `CancellationToken` istemci bağlantıyı keserse işi iptal etmeye yarar.

```
    {
        RegisterWithPasswordCommand command = new RegisterWithPasswordCommand
        {
            Email = request.Email,
            DisplayName = request.DisplayName,
            Password = request.Password
        };
```

API modelini Application katmanının komutuna çevirir; böylece Application katmanı HTTP modellerine bağımlı olmaz (Clean Architecture kuralı).

```
        AuthSession session = await _registerWithPasswordUseCase.ExecuteAsync(
            command,
            cancellationToken);
```

`await` use case bitene kadar bekler ama iş parçacığını bloklamaz. Use case doğrulama, kayıt ve oturum üretimini yapıp `AuthSession` döner.

```
        RefreshTokenCookie.Write(Response, session.RefreshToken, session.RefreshTokenExpiresAtUtc);
```

Refresh token'ı JavaScript'in okuyamadığı HttpOnly çereze yazar; access token ise yanıt gövdesinde gider.

```
        return CreatedAtAction(nameof(GetCurrentUser), null, session.Result);
```

`CreatedAtAction` 201 Created döner ve kullanıcı bilgisinin okunabileceği `me` uç noktasını gösterir. `nameof` metot adını derleme anında güvenle metne çevirir.

```
    }
```

ƒ Login — E-posta/şifre girişini use case'e iletir, refresh çerezini yazar, access token ve kullanıcıyı döner.

```
    [AllowAnonymous]
    [EnableRateLimiting(RateLimitingPolicies.Authentication)]
    [HttpPost("login")]
```

Anonim, dakikada 10 istekle sınırlı POST /api/auth/login uç noktası; şifre tahmin saldırılarını yavaşlatır.

89 `public async Task<ActionResult<AuthResultDto>> Login(` 90 `[FromBody] LoginRequest request,` 91 `CancellationToken cancellationToken)`	Gövdeden `LoginRequest` okur ve iptal belirteci alır; sonuç olarak `AuthResultDto` (access token + kullanıcı) döner.
92 `{`	
93 `LoginWithPasswordCommand command = new LoginWithPasswordCommand` 94 `{` 95 `Email = request.Email,` 96 `Password = request.Password` 97 `};`	HTTP isteğini Application katmanındaki `LoginWithPasswordCommand` komutuna dönüştürür.
99 `AuthSession session = await _loginWithPasswordUseCase.ExecuteAsync(` 100 `command,` 101 `cancellationToken);`	Giriş use case'ini çalıştırır; şifre yanlışsa use case istisna fırlatır ve global exception handler 401 döner.
103 `RefreshTokenCookie.Write(Response, session.RefreshToken, session.RefreshTokenExpiresAtUtc);` 104 105 `return Ok(session.Result);`	Yeni refresh token'ı çereze yazar ve access token ile kullanıcı bilgisini 200 OK olarak gövdede döner.
106 `}`	
ƒ LoginWithGoogle — Google'dan gelen ID token'ı use case'e iletir, oturum açar ve refresh çerezini yazar.	
108 `[AllowAnonymous]` 109 `[EnableRateLimiting(RateLimitingPolicies.Authentication)]` 110 `[HttpPost("google")]`	Anonim, hız sınırlı POST /api/auth/google uç noktası; Google ile giriş isteğini karşılar.
111 `public async Task<ActionResult<AuthResultDto>> LoginWithGoogle(` 112 `[FromBody] GoogleLoginRequest request,` 113 `CancellationToken cancellationToken)`	Gövdeden Google'ın verdiği ID token'ı içeren `GoogleLoginRequest` okunur.
114 `{`	
115 `LoginWithGoogleCommand command = new LoginWithGoogleCommand` 116 `{` 117 `IdToken = request.IdToken` 118 `};`	ID token'ı Application komutuna koyar; token'ın Google tarafından imzalandığını use case doğrular, controller değil.
120 `AuthSession session = await _loginWithGoogleUseCase.ExecuteAsync(` 121 `command,` 122 `cancellationToken);`	Google girişi use case'ini çalıştırır; kullanıcı yoksa oluşturulur, varsa hesabı bağlanır.
124 `RefreshTokenCookie.Write(Response, session.RefreshToken, session.RefreshTokenExpiresAtUtc);` 125 126 `return Ok(session.Result);`	Refresh token çerezini yazar ve sonucu 200 OK ile döner.
127 `}`	
ƒ Refresh — Çerezdeki refresh token ile yeni access token üretir, refresh token'ı döndürerek (rotation) yeniler.	
129 `[AllowAnonymous]` 130 `[HttpPost("refresh")]` 131 `public async Task<ActionResult<AuthResultDto>> Refresh(CancellationToken cancellationToken)`	POST /api/auth/refresh; access token süresi dolmuş olabileceği için anonim erişime açıktır, kimlik çerezle kanıtlanır.
132 `{`	
133 `string? refreshToken = RefreshTokenCookie.Read(Request);`	Çerezden refresh token'ı okur. `string?` değerin null olabileceğini belirtir; çerez yoksa null gelir.
135 `try`	`try` bloğu: içinde istisna olursa aşağıdaki `catch` bloğu devreye girer.
136 `{`	
137 `AuthSession session = await _refreshSessionUseCase.ExecuteAsync(` 138 `refreshToken,` 139 `cancellationToken);`	Refresh use case'i token'ın hash'ini veritabanında arar, geçerliyse eskisini iptal edip yenisini üretir.
141 `RefreshTokenCookie.Write(Response, session.RefreshToken, session.RefreshTokenExpiresAtUtc);` 142 143 `return Ok(session.Result);`	Yeni refresh token'ı çereze yazar (eskisi artık geçersiz) ve yeni access token'ı döner.
144 `}`	
145 `catch (AuthenticationFailedException exception)`	Token geçersiz, süresi dolmuş veya iptal edilmişse `AuthenticationFailedException` yakalanır.
146 `{`	
147 `RefreshTokenCookie.Delete(Response);`	Geçersiz çerezi tarayıcıdan siler; böylece istemci boşuna tekrar denemez.
149 `return Problem(` 150 `statusCode: StatusCodes.Status401Unauthorized,` 151 `title: "Authentication failed.",` 152 `detail: exception.Message);`	`Problem(...)` standart ProblemDetails formatında 401 Unauthorized yanıtı üretir; adlandırılmış parametrelerle durum kodu, başlık ve detay verilir.
153 `}`	
154 `}`	
ƒ Logout — Çerezdeki refresh token'ı veritabanında iptal eder ve çerezi silerek oturumu kapatır.	
156 `[AllowAnonymous]` 157 `[HttpPost("logout")]` 158 `public async Task<IActionResult> Logout(CancellationToken cancellationToken)`	POST /api/auth/logout; access token bitmiş olsa da çıkış yapılabilsin diye anonimdir. `IActionResult` gövdesiz yanıt dönebilir.
159 `{`	
160 `string? refreshToken = RefreshTokenCookie.Read(Request);`	Çerezdeki refresh token'ı okur; yoksa null olur.
162 `await _logoutUseCase.ExecuteAsync(refreshToken, cancellationToken);`	Logout use case'i token'ı veritabanında iptal eder; çerez çalınmış olsa bile artık kullanılamaz.

```
        RefreshTokenCookie.Delete(Response);

        return NoContent();
    }
```

Çerezi tarayıcıdan siler ve 204 No Content (başarılı, gövdesiz) döner.

ƒ GetCurrentUser — Geçerli access token sahibinin kullanıcı bilgilerini döner.

```
    [Authorize]
    [HttpGet("me")]
    public async Task<ActionResult<UserDto>> GetCurrentUser(CancellationToken cancellationToken)
    {
        UserDto? currentUser = await _getCurrentUserUseCase.ExecuteAsync(cancellationToken);

        if (currentUser == null)
        {
            return Unauthorized();
        }

        return Ok(currentUser);
    }
}
```

`[Authorize]` geçerli JWT access token ister. GET /api/auth/me, giriş yapan kullanıcının bilgisini verir.

Use case, token'daki kullanıcı kimliğiyle veritabanından kullanıcıyı getirir; `UserDto?` bulunamazsa null olabilir.

Token geçerli ama kullanıcı silinmişse 401 Unauthorized döner.

Kullanıcı bilgisini 200 OK ile döner.

API todo.API/Authentication/RequireClientHeaderAttribute.cs

> Durum değiştiren isteklerde `X-Todo-Client` başlığını zorunlu kılan bir action filter. Çerezle kimlik doğrulayan AuthController'ı CSRF saldırılarına karşı korur.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

namespace TodoApi.API.Authentication;

[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)]
```

MVC sonuç tiplerini ve action filter altyapısını içeri alır.

Dosyanın ad alanı; kimlik doğrulamayla ilgili API yardımcıları burada toplanır.

`[AttributeUsage]` bu attribute'un hem sınıflara hem metotlara eklenebileceğini belirtir. `|` iki seçeneği birleştirir.

ƒ RequireClientHeaderAttribute — GET dışındaki isteklerde özel istemci başlığı yoksa isteği 400 ile reddeden filtre.

```
public sealed class RequireClientHeaderAttribute : Attribute, IActionFilter
{
    public const string HeaderName = "X-Todo-Client";
```

Hem `Attribute` (etiket) hem `IActionFilter` (controller metodu çalışmadan önce/sonra kod çalıştıran filtre) olarak tanımlanır.

Başlık adı sabit olarak tutulur; CORS ayarı ve testler de aynı sabiti kullanır, yazım hatası riski kalmaz.

ƒ OnActionExecuting — Controller metodundan önce çalışır; başlık eksikse isteği 400 yanıtıyla keser.

```
    public void OnActionExecuting(ActionExecutingContext context)
    {
        string method = context.HttpContext.Request.Method;
        bool isReadOnlyRequest = HttpMethods.IsGet(method) || HttpMethods.IsHead(method) || HttpMethods.IsOptions(method);

        if (isReadOnlyRequest)
        {
            return;
        }

        bool hasClientHeader = context.HttpContext.Request.Headers.ContainsKey(HeaderName);

        if (hasClientHeader == false)
        {
            ProblemDetails problemDetails = new ProblemDetails
            {
                Status = StatusCodes.Status400BadRequest,
                Title = "The request is missing a required header.",
                Detail = $"The {HeaderName} header is required for this request."
            };

            context.Result = new BadRequestObjectResult(problemDetails);

        }
    }
```

Controller metodu çalışmadan hemen önce ASP.NET Core tarafından çağrılır.

İsteğin HTTP metodunu alır; GET, HEAD ve OPTIONS veri değiştirmediği için okuma isteği sayılır.

Okuma isteklerinde kontrol yapılmaz; `return` ile filtreden çıkılır ve istek normal devam eder.

İstek başlıklarında `X-Todo-Client` var mı bakar. Başka sitedeki bir form özel başlık ekleyemez; CORS izni gerekir, bu da CSRF'yi engeller.

Başlık yoksa isteği reddetme dalına girer.

Standart hata formatı olan ProblemDetails nesnesini 400 durumu, başlık ve açıklamayla doldurur. `$"..."` metin içine değişken yerleştirir.

`context.Result` atanınca controller metodu hiç çalışmaz; istemciye doğrudan 400 Bad Request döner.

ƒ OnActionExecuted — Controller metodundan sonra çalışır; burada yapılacak iş olmadığı için boştur.

```
    public void OnActionExecuted(ActionExecutedContext context)
    {
    }
}
```

Arayüz bu metodu zorunlu kıldığı için yazılmıştır; metottan sonra yapılacak bir işlem yoktur.

API todo.API/Models/Requests/LoginRequest.cs

> Giriş uç noktasına (POST /api/auth/login) gelen JSON gövdesinin API katmanındaki modeli. Controller bunu Application komutuna çevirir.

```
namespace TodoApi.API.Models.Requests;
```

API katmanındaki istek modellerinin ad alanı.

ƒ LoginRequest — Giriş isteğinin e-posta ve şifre alanlarını taşıyan basit veri sınıfı (DTO).

```
3  public sealed class LoginRequest
```

`sealed` sınıf; sadece veri taşır, iş kuralı içermez. Doğrulama Application katmanındaki validator'da yapılır.

```
4  {
5      public string Email { get; set; } = string.Empty;
```

JSON'daki `email` alanı buraya gelir. `{ get; set; }` otomatik özelliktir; `= string.Empty` alan gelmezse null yerine boş metin olmasını sağlar.

```
7      public string Password { get; set; } = string.Empty;
```

Kullanıcının girdiği düz şifre; sadece doğrulama için kullanılır, hiçbir yerde saklanmaz.

```
8  }
```

API todo.API/Models/Requests/RegisterRequest.cs

Kayıt uç noktasına (POST /api/auth/register) gelen JSON gövdesinin modeli. Controller bunu `RegisterWithPasswordCommand` 'a çevirir.

```
1  namespace TodoApi.API.Models.Requests;
```

API katmanındaki istek modellerinin ad alanı.

ƒ RegisterRequest — Kayıt için e-posta, görünen ad ve şifreyi taşıyan veri sınıfı.

```
3  public sealed class RegisterRequest
```

Sadece veri taşıyan `sealed` sınıf; rol alanı bilerek yoktur, istemci kendini admin yapamaz.

```
4  {
5      public string Email { get; set; } = string.Empty;
```

Kullanıcının e-posta adresi; boş gelirse `string.Empty` olur, kontrolü validator yapar.

```
7      public string DisplayName { get; set; } = string.Empty;
```

Uygulamada gösterilecek kullanıcı adı.

```
9      public string Password { get; set; } = string.Empty;
```

Düz şifre; use case içinde PBKDF2 ile hash'lenir, düz hali veritabanına yazılmaz.

```
10 }
```

API todo.API/Models/Requests/GoogleLoginRequest.cs

Google ile giriş uç noktasına (POST /api/auth/google) gelen gövdenin modeli. Frontend'in Google'dan aldığı ID token'ı taşır.

```
1  namespace TodoApi.API.Models.Requests;
```

API katmanındaki istek modellerinin ad alanı.

ƒ GoogleLoginRequest — Google ID token'ını sunucuya taşıyan veri sınıfı.

```
3  public sealed class GoogleLoginRequest
```

Tek alanlı basit veri sınıfı.

```
4  {
5      public string IdToken { get; set; } = string.Empty;
```

Google'ın imzaladığı ID token (JWT); sunucu bu imzayı ve hedef Client Id'yi doğrulamadan kullanıcıya güvenmez.

```
6  }
```

API todo.API/Models/Responses/AuthConfigResponse.cs

GET /api/auth/config yanıtının modeli. Frontend'e Google ile giriş için gereken herkese açık Client Id'yi bildirir.

```
1  namespace TodoApi.API.Models.Responses;
```

API katmanındaki yanıt modellerinin ad alanı.

ƒ AuthConfigResponse — Frontend'e giriş ayarlarını (Google Client Id) taşıyan yanıt sınıfı.

```
3  public sealed class AuthConfigResponse
```

Sadece veri taşıyan `sealed` yanıt sınıfı.

```
4  {
5      public string? GoogleClientId { get; set; }
```

`string?` null olabilir demektir; Google ayarlanmamışsa null döner ve frontend Google butonunu gizler. Client Id gizli değildir, secret ise asla gönderilmez.

```
6  }
```

API todo.API/Authentication/RefreshTokenCookie.cs

Refresh token çerezini (`todo_refresh`) yazma, okuma ve silme işlemlerini tek yerde toplayan yardımcı sınıf. Çerezin güvenlik ayarlarının her yerde aynı olmasını garanti eder.

```
1  namespace TodoApi.API.Authentication;
```

Dosyanın ad alanı; API katmanının kimlik doğrulama yardımcıları.

ƒ RefreshTokenCookie — Refresh token çerezini güvenli ayarlarla yazan, okuyan ve silen statik yardımcı.

```
3  public static class RefreshTokenCookie
```

`static class` nesnesi oluşturulamayan, sadece statik üyeler içeren sınıftır; durum tutmayan yardımcılar için uygundur.

```
4  {
5      public const string Name = "todo_refresh";
```

Çerezin adı: `todo_refresh` . `const` derleme anında sabitlenen, değişmeyen değerdir.

```
7      public const string Path = "/api/auth";
```

Çerez sadece `/api/auth` altındaki isteklere gönderilir; diğer API çağrılarında gereksiz yere taşınmaz.

ƒ Write — Refresh token'ı belirtilen bitiş zamanıyla güvenli çerez olarak yanıta ekler.

```
9      public static void Write(HttpResponse response, string refreshToken, DateTime
       expiresAtUtc)
10     {
```

Yanıta refresh token çerezini ekleyen metot; token değerini ve bitiş zamanını alır.

```
        response.Cookies.Append(Name, refreshToken, CreateOptions(expiresAtUtc));
    }
```

`Set-Cookie` başlığını güvenlik ayarlarıyla birlikte yanıta ekler.

ƒ Read — İstekteki refresh token çerezini okur; yoksa null döner.

```
    public static string? Read(HttpRequest request)
    {
        bool cookieExists = request.Cookies.TryGetValue(Name, out string? refreshToken);
        if (cookieExists == false)
        {
            return null;
        }
        return refreshToken;
    }
```

İstekten çerezi okur; `string?` dönüş tipi çerez yoksa null dönebileceğini belirtir.

`TryGetValue` çerez varsa true döner ve değeri `out` parametresiyle verir; yoksa istisna fırlatmaz.

Çerez yoksa null döner; çağıran taraf bunu geçersiz oturum olarak işler.

Çerez varsa içindeki token değerini döner.

ƒ Delete — Refresh token çerezini tarayıcıdan siler.

```
    public static void Delete(HttpResponse response)
    {
        response.Cookies.Delete(Name, CreateOptions(DateTime.UnixEpoch));
    }
```

Çıkışta veya geçersiz oturumda çerezi silen metot.

Silme için aynı Path ve ayarlar verilmelidir, yoksa tarayıcı çerezi silmez; geçmiş bir tarih (1970) ile süresi doldurulur.

ƒ CreateOptions — Çerezin HttpOnly, Secure, SameSite=Strict ve süre ayarlarını üretir.

```
    private static CookieOptions CreateOptions(DateTime expiresAtUtc)
    {
        return new CookieOptions
        {
            HttpOnly = true,
            Secure = true,
            SameSite = SameSiteMode.Strict,
            Path = Path,
            Expires = new DateTimeOffset(DateTime.SpecifyKind(expiresAtUtc, DateTimeKind.Utc)),
            IsEssential = true
        };
    }
}
```

Çerez ayarlarını tek yerden üreten özel (`private`) yardımcı metot.

Ayarları object initializer ile oluşturup döner.

`HttpOnly` JavaScript'in çerezi okumasını engeller; XSS saldırısında token çalınamaz.

`Secure` çerezin sadece HTTPS üzerinden gönderilmesini sağlar.

`SameSite=Strict` başka sitelerden gelen isteklerde çerezin gönderilmesini engeller; CSRF'ye karşı ek koruma.

Çerezin geçerli olduğu yol `/api/auth` olarak ayarlanır.

Bitiş zamanını UTC olarak işaretleyip `DateTimeOffset` 'e çevirir; saat dilimi kaymasıyla çerezin erken/geç bitmesi önlenir.

`IsEssential` çerezin çerez onayı (consent) politikasından etkilenmemesini sağlar; oturum için zorunludur.

Application `todo.Application/Features/Authentication/LoginWithPassword/LoginWithPasswordCommand.cs`

> Şifreyle giriş use case'inin girdi nesnesi (command). Application katmanına aittir; HTTP'den bağımsızdır.

```
namespace TodoApi.Application.Features.Authentication.LoginWithPassword;
```

Özellik klasörüne (feature folder) göre ad alanı; giriş ile ilgili tüm dosyalar aynı yerde durur.

ƒ LoginWithPasswordCommand — Giriş için gereken e-posta ve şifreyi taşıyan değişmez komut nesnesi.

```
public sealed class LoginWithPasswordCommand
{
    public string Email { get; init; } = string.Empty;
    public string Password { get; init; } = string.Empty;
}
```

Use case'e verilecek veriyi taşıyan `sealed` sınıf.

`init` özelliğe sadece nesne oluşturulurken değer atanabilir; sonra değiştirilemez, komut yolda bozulmaz.

Kullanıcının girdiği düz şifre; aynı şekilde `init` ile sadece oluşturulurken atanır.

Application `todo.Application/Features/Authentication/LoginWithPassword/LoginWithPasswordCommandValidator.cs`

> Giriş komutunun şekil kurallarını FluentValidation ile tanımlar. Use case bu kuralları ilk adımda çalıştırır; hata olursa 400 döner.

```
using FluentValidation;
```

FluentValidation kütüphanesini içeri alır; kuralları okunaklı zincir metotlarla yazmayı sağlar.

```
namespace TodoApi.Application.Features.Authentication.LoginWithPassword;
```

Giriş özelliğinin ad alanı.

ƒ LoginWithPasswordCommandValidator — Giriş komutundaki e-posta ve şifre için boşluk ve uzunluk kurallarını tanımlar.

```
public sealed class LoginWithPasswordCommandValidator : AbstractValidator<LoginWithPasswordCommand>
{
```

`AbstractValidator<T>` sınıfından türeyerek `LoginWithPasswordCommand` için doğrulayıcı olur. `<T>` generic tip parametresidir.

ƒ LoginWithPasswordCommandValidator (constructor) — Doğrulama kurallarını kurucu içinde kaydeder.

```
    public LoginWithPasswordCommandValidator()
    {
```

Kurallar kurucu metotta tanımlanır; nesne oluşunca hazır olur.

```
        RuleFor(command => command.Email)
            .NotEmpty()
            .MaximumLength(256)
            .WithName("E-posta");

        RuleFor(command => command.Password)
            .NotEmpty()
            .MaximumLength(128)
            .WithName("Şifre");
    }
}
```

E-posta boş olamaz ve en fazla 256 karakter olabilir. `command => command.Email` lambda ifadesidir. `WithName` hata mesajında Türkçe alan adı gösterir.

Şifre boş olamaz, en fazla 128 karakter; çok uzun şifreyle hash'i yavaşlatma saldırısı önlenir. Girişte biçim kuralı yok, sadece doğru/yanlış bakılır.

Application `todo.Application/Features/Authentication/LoginWithPassword/LoginWithPasswordUseCase.cs`

> E-posta ve şifreyle girişin iş akışını yönetir: doğrular, kullanıcıyı bulur, şifreyi kontrol eder ve yeni oturum (access + refresh token) üretir. Application katmanındadır; veritabanı ve hash detaylarını arayüzler üzerinden kullanır.

```
using System;
using System.Threading;
using System.Threading.Tasks;
```

Temel .NET tipleri, iptal belirteci (`CancellationToken`) ve asenkron `Task` tiplerini içeri alır.

```
using FluentValidation;
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Application.Common.Validation;
using TodoApi.Application.Exceptions;
using TodoApi.Application.Features.Authentication.Common;
using TodoApi.Domain.Entities;
```

FluentValidation, Application arayüzleri (repository, hasher), doğrulama yardımcısı, özel istisnalar, ortak oturum sınıfları ve Domain'deki `User` varlığı içeri alınır.

```
namespace TodoApi.Application.Features.Authentication.LoginWithPassword;
```

Giriş özelliğinin ad alanı.

ƒ LoginWithPasswordUseCase — Şifreyle giriş senaryosunu baştan sona yöneten use case sınıfı.

```
public class LoginWithPasswordUseCase
{
```

Use case sınıfı; tek bir iş senaryosunu (şifreyle giriş) temsil eder.

```
    private const string InvalidCredentialsMessage = "The email address or password is incorrect.";
```

Tek bir genel hata mesajı: e-posta mı şifre mi yanlış söylenmez, böylece saldırgan hangi e-postaların kayıtlı olduğunu öğrenemez.

```
    private readonly IUserRepository _userRepository;
    private readonly IPasswordHasher _passwordHasher;
    private readonly SessionIssuer _sessionIssuer;
    private readonly IValidator<LoginWithPasswordCommand> _validator;
```

Bağımlılıklar: kullanıcı deposu, şifre hash'leyici, oturum üretici ve doğrulayıcı. Arayüz kullanmak test etmeyi ve katman ayrımını kolaylaştırır.

ƒ LoginWithPasswordUseCase (constructor) — Bağımlılıkları DI ile alır, null kontrolü yapıp alanlara kaydeder.

```
    public LoginWithPasswordUseCase(
        IUserRepository userRepository,
        IPasswordHasher passwordHasher,
        SessionIssuer sessionIssuer,
        IValidator<LoginWithPasswordCommand> validator)
    {
        if (userRepository == null)
        {
            throw new ArgumentNullException(nameof(userRepository));
        }

        if (passwordHasher == null)
        {
            throw new ArgumentNullException(nameof(passwordHasher));
        }

        if (sessionIssuer == null)
        {
            throw new ArgumentNullException(nameof(sessionIssuer));
        }

        if (validator == null)
        {
            throw new ArgumentNullException(nameof(validator));
        }

        _userRepository = userRepository;
        _passwordHasher = passwordHasher;
        _sessionIssuer = sessionIssuer;
        _validator = validator;
    }
```

Kurucu metot; bağımlılıklar Dependency Injection konteynerinden gelir.

Her bağımlılık için null kontrolü yapılır; eksikse uygulama ilk anda `ArgumentNullException` ile anlaşılır şekilde patlar. `nameof` parametre adını verir.

Kontrol edilen bağımlılıklar `readonly` alanlara atanır.

ƒ ExecuteAsync — Komutu doğrular, kullanıcıyı ve şifreyi kontrol eder, başarılıysa yeni oturum döner.

```
    public async Task<AuthSession> ExecuteAsync(
        LoginWithPasswordCommand command,
        CancellationToken cancellationToken = default)
    {
        await _validator.ValidateOrThrowAsync(command, cancellationToken);

        string normalizedEmail = EmailNormalizer.Normalize(command.Email);
        string providedPassword = command.Password;

        User? user = await _userRepository.GetByEmailAsync(normalizedEmail, cancellationToken);
```

Asenkron ana metot; `AuthSession` döner. `= default` iptal belirtecinin verilmezse boş (iptal edilmez) olacağını belirtir.

Önce FluentValidation kuralları çalışır; geçersizse istisna fırlar ve global handler 400 döner. `await` işlemin bitmesini bloklamadan bekler.

E-posta küçük harfe çevrilip boşlukları kırpılır; veritabanındaki kayıtla aynı biçimde aranır. Şifre yerel değişkene alınır.

Kullanıcı e-postayla aranır; `User?` bulunamazsa null olabileceğini belirtir.

66	`        if (user == null || user.PasswordHash == null)`	Kullanıcı yoksa veya sadece Google ile kayıtlıysa (şifre hash'i yok) şifreyle giriş yapılamaz.
67	`        {`	
68	`            _passwordHasher.HashPassword(providedPassword);`	Sonucu kullanılmasa da şifre yine hash'lenir: cevap süresi var olan kullanıcıyla aynı olur, saldırgan süreden e-postanın kayıtlı olup olmadığını anlayamaz.
70	`            throw new AuthenticationFailedException(InvalidCredentialsMessage);`	Genel hata mesajıyla `AuthenticationFailedException` fırlatılır; API bunu 401 Unauthorized yanıtına çevirir.
71	`        }`	

73
74
75
	`        PasswordVerificationOutcome outcome = _passwordHasher.VerifyPassword(`
	`            user.PasswordHash,`
	`            providedPassword);`
		Girilen şifre, saklanan PBKDF2 hash'iyle karşılaştırılır; sonuç başarısız, başarılı veya başarılı ama yeniden hash gerekli olabilir.

77
78
79
80
	`        if (outcome == PasswordVerificationOutcome.Failed)`
	`        {`
	`            throw new AuthenticationFailedException(InvalidCredentialsMessage);`
	`        }`
		Şifre yanlışsa aynı genel mesajla 401 hatası verilir.

82
83
84
85
	`        if (outcome == PasswordVerificationOutcome.SucceededRehashNeeded)`
	`        {`
	`            user.ReplacePasswordHash(_passwordHasher.HashPassword(providedPassword));`
	`        }`
		Şifre doğru ama hash eski ayarlarla (ör. düşük iterasyon) üretilmişse, düz şifre elimizdeyken yeni ayarlarla tekrar hash'lenip güncellenir.

87	`        await _userRepository.SaveChangesAsync(cancellationToken);`	Değişiklikler (varsa yeni hash) veritabanına kaydedilir.

89
90
91
	`        AuthSession session = await _sessionIssuer.IssueAsync(user, cancellationToken);`
	
	`        return session;`
		Ortak `SessionIssuer` 15 dakikalık JWT access token ve hash'i saklanan yeni refresh token üretir; oturum controller'a döner.

92 93	`    }` `}`	

Application `todo.Application/Features/Authentication/Common/EmailNormalizer.cs`

> E-posta adreslerini tek bir standart biçime (kırpılmış, küçük harf) getiren yardımcı. Kayıt, giriş ve Google girişi aynı kuralı kullanır, böylece aynı adres iki hesap açamaz.

1	`namespace TodoApi.Application.Features.Authentication.Common;`	Kimlik doğrulama özelliklerinin ortak kullandığı sınıfların ad alanı.
	ƒ EmailNormalizer — E-posta adreslerini karşılaştırılabilir standart biçime çeviren statik yardımcı.	
3	`public static class EmailNormalizer`	`static class` nesnesi oluşturulmayan, durum tutmayan yardımcı sınıftır.
4	`{`	
	ƒ Normalize — E-postayı kırpar ve küçük harfe çevirir; null gelirse boş metin döner.	
5	`    public static string Normalize(string? email)`	`string?` parametresi null gelebileceğini belirtir; metot her durumda null olmayan metin döner.
6	`    {`	

7
8
9
10
	`        if (email == null)`
	`        {`
	`            return string.Empty;`
	`        }`
		E-posta null ise hata fırlatmak yerine boş metin döner; çağıranlar null kontrolüyle uğraşmaz.

12	`        return email.Trim().ToLowerInvariant();`	Baş/son boşlukları kırpar ve küçük harfe çevirir. `ToLowerInvariant` kültürden bağımsızdır; Türkçe "I" harfi sorununu önler.
13 14	`    }` `}`	

Application `todo.Application/Abstractions/Persistence/IUserRepository.cs`

> Kullanıcı verisine erişim portu (repository). Application SQL veya EF Core bilmez; gerçek sorgular Infrastructure'daki `UserRepository` içinde yazılır.

1
2
3
	`using System.Collections.Generic;`
	`using System.Threading;`
	`using System.Threading.Tasks;`
		`IReadOnlyList`, `CancellationToken` ve `Task` tipleri için isim alanları.

4	`using TodoApi.Domain.Entities;`	`User` Domain varlığı için isim alanı.
6	`namespace TodoApi.Application.Abstractions.Persistence;`	Veri erişim portlarının isim alanı.
	ƒ interface IUserRepository — Kullanıcıları okuma, ekleme ve kaydetme işlemlerinin sözleşmesi.	
8	`public interface IUserRepository`	`interface`: Repository deseni; iş mantığı veritabanı teknolojisinden ayrılır, testlerde sahte depo kullanılabilir.
9	`{`	
	ƒ GetByIdAsync — Kimliğe göre kullanıcıyı getirir, yoksa null.	
10	`    Task<User?> GetByIdAsync(int userId, CancellationToken cancellationToken);`	`Task<User?>`: asenkron çalışır ve kullanıcı bulunamazsa null döner.
	ƒ GetByEmailAsync — Normalize edilmiş e-postaya göre kullanıcıyı getirir.	
12	`    Task<User?> GetByEmailAsync(string email, CancellationToken cancellationToken);`	E-posta ile arama; kayıt ve şifreli giriş ile Google hesabı bağlamada kullanılır.

ƒ GetByGoogleSubjectIdAsync — Google subject id'ye göre bağlı kullanıcıyı getirir.

```
    Task<User?> GetByGoogleSubjectIdAsync(string googleSubjectId, CancellationToken cancellationToken);
```

Google girişinde önce bu değişmez Google kimliğiyle arama yapılır.

ƒ GetAllAsync — Tüm kullanıcıları salt okunur liste olarak döndürür.

```
    Task<IReadOnlyList<User>> GetAllAsync(CancellationToken cancellationToken);
```

`IReadOnlyList<User>`: çağıran listeyi değiştiremez. Admin'in kullanıcı listesi ekranında kullanılır.

ƒ AddAsync — Yeni kullanıcıyı kaydedilmek üzere ekler.

```
    Task AddAsync(User user, CancellationToken cancellationToken);
```

Yeni kullanıcıyı takibe alır; veritabanına yazma `SaveChangesAsync` çağrılınca olur.

ƒ SaveChangesAsync — Bekleyen tüm değişiklikleri veritabanına yazar.

```
    Task SaveChangesAsync(CancellationToken cancellationToken);
```

Bekleyen değişiklikleri tek seferde veritabanına yazar (Unit of Work fikri).

```
}
```

Infrastructure `todo.Infrastructure/Persistence/UserRepository.cs`

> Kullanıcıları veritabanından bulan ve kaydeden repository. Application katmanındaki `IUserRepository` arayüzünü EF Core ile uygular.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Domain.Entities;
```

Gerekli isim alanları: koleksiyonlar, LINQ, asenkron işlemler, EF Core'un asenkron sorgu metotları, Application arayüzü ve `User` entity'si.

```
namespace TodoApi.Infrastructure.Persistence;
```

Dosya kapsamlı namespace: Infrastructure katmanının veri erişim (Persistence) bölümü.

ƒ class UserRepository — Kullanıcı verisine EF Core üzerinden erişen repository sınıfı.

```
public sealed class UserRepository : IUserRepository
```

`IUserRepository` uygular; Application katmanı SQL veya EF Core detaylarını bilmeden kullanıcı arar ve kaydeder.

```
{
    private readonly TodoDbContext _todoDbContext;
```

Veritabanı bağlamı (`TodoDbContext`) alanı; tüm sorgular bu nesne üzerinden yapılır.

ƒ UserRepository — DbContext'i DI'dan alıp saklar.

```
    public UserRepository(TodoDbContext todoDbContext)
    {
        if (todoDbContext == null)
        {
            throw new ArgumentNullException(nameof(todoDbContext));
        }

        _todoDbContext = todoDbContext;
    }
```

Kurucu metot; `TodoDbContext` istek başına (scoped) DI tarafından verilir.

DbContext null ise açık bir istisna fırlatılır; değilse `readonly` alana atanır.

ƒ GetByIdAsync — Id'ye göre kullanıcıyı bulur; yoksa null döner.

```
    public async Task<User?> GetByIdAsync(int userId, CancellationToken cancellationToken)
    {
        User? user = await _todoDbContext.Users.FirstOrDefaultAsync(
            existingUser => existingUser.Id == userId,
            cancellationToken);

        return user;
    }
```

`Task<User?>`: asenkron çalışır ve kullanıcı bulunamazsa null dönebilir.

`FirstOrDefaultAsync` id eşleşen ilk kaydı getirir, yoksa null verir. Lambda (`existingUser => ...`) EF Core tarafından SQL WHERE koşuluna çevrilir.

ƒ GetByEmailAsync — E-posta adresine göre kullanıcıyı bulur; yoksa null döner.

```
    public async Task<User?> GetByEmailAsync(string email, CancellationToken cancellationToken)
    {
        User? user = await _todoDbContext.Users.FirstOrDefaultAsync(
            existingUser => existingUser.Email == email,
            cancellationToken);

        return user;
    }
```

Şifreyle girişte ve Google hesabı bağlamada kullanılır; e-posta küçük harfe normalize edilmiş olarak gelir.

E-postası eşleşen ilk kullanıcı aranır. Email sütununda benzersiz indeks olduğu için en fazla bir kayıt bulunur.

ƒ GetByGoogleSubjectIdAsync — Google hesap kimliğine (sub) göre kullanıcıyı bulur.

```
    public async Task<User?> GetByGoogleSubjectIdAsync(
        string googleSubjectId,
        CancellationToken cancellationToken)
    {
```

Google ile girişte önce bu metotla aranır; Google kimliği e-postadan daha güvenilir, çünkü değişmez.

```
        User? user = await _todoDbContext.Users.FirstOrDefaultAsync(
            existingUser => existingUser.GoogleSubjectId == googleSubjectId,
            cancellationToken);

        return user;
    }
```

`GoogleSubjectId` eşleşen ilk kullanıcı getirilir; yoksa null döner.

ƒ GetAllAsync — Tüm kullanıcıları id sırasına göre salt okunur liste olarak döner.

```
    public async Task<IReadOnlyList<User>> GetAllAsync(CancellationToken cancellationToken)
    {
        IQueryable<User> userQuery = _todoDbContext.Users;

        userQuery = userQuery.AsNoTracking();
        userQuery = userQuery.OrderBy(user => user.Id);

        List<User> users = await userQuery.ToListAsync(cancellationToken);

        IReadOnlyList<User> result = users;

        return result;
    }
```

Admin ekranında kullanıcı listesini göstermek için kullanılır. `IReadOnlyList` çağıranın listeyi değiştirememesini sağlar.

`IQueryable` henüz çalışmamış bir sorgudur; adım adım koşul eklenir, veritabanına ancak sonunda gidilir.

`AsNoTracking` EF Core'un değişiklik takibini kapatır, sadece okuma için daha hızlıdır. Ardından id'ye göre sıralanır.

`ToListAsync` sorguyu çalıştırıp sonuçları listeye alır; liste salt okunur arayüz tipinde döndürülür.

ƒ AddAsync — Yeni kullanıcıyı EF Core takibine ekler; kayıt SaveChanges ile yapılır.

```
    public async Task AddAsync(User user, CancellationToken cancellationToken)
    {
        if (user == null)
        {
            throw new ArgumentNullException(nameof(user));
        }

        await _todoDbContext.Users.AddAsync(user, cancellationToken);
    }
```

Kayıt veya Google ile ilk girişte yeni kullanıcı eklemek için çağrılır.

Kullanıcı null ise açık bir istisna fırlatılır.

Kullanıcı 'eklenecek' olarak işaretlenir; veritabanına henüz yazılmaz, böylece birden fazla değişiklik tek seferde kaydedilebilir.

ƒ SaveChangesAsync — Bekleyen tüm değişiklikleri veritabanına tek seferde yazar.

```
    public async Task SaveChangesAsync(CancellationToken cancellationToken)
    {
        await _todoDbContext.SaveChangesAsync(cancellationToken);
    }
}
```

Use case işini bitirince bu metodu çağırır; ekleme ve güncellemeler birlikte kaydedilir.

DbContext'teki tüm takip edilen değişiklikler (yeni, değişen kayıtlar) SQL komutlarına çevrilip tek işlemde çalıştırılır.

Domain `todo.Domain/Entities/User.cs`

> Sistemdeki kullanıcıyı temsil eden Domain entity'si. Kayıt, Google hesabı bağlama, şifre değiştirme ve admin yapma kuralları burada korunur; böylece hiçbir katman bu kuralları atlayamaz.

```
using TodoApi.Domain.Enums;
using TodoApi.Domain.Exceptions;

namespace TodoApi.Domain.Entities;
```

Domain içindeki `UserRole` enum'u ve `DomainRuleViolationException` için isim alanları. Domain katmanı dış kütüphanelere bağlı değildir.

Dosya kapsamlı namespace: Domain entity'leri.

ƒ class User — Kullanıcı kimliği, giriş yöntemleri ve rolünü kurallarıyla birlikte tutan entity.

```
public class User
{
    private const int MaximumEmailLength = 256;
    private const int MaximumDisplayNameLength = 100;
    private const int MaximumGoogleSubjectIdLength = 255;

    private int _id;
    private string _email = string.Empty;
    private string _displayName = string.Empty;
    private string? _passwordHash;
    private string? _googleSubjectId;
    private UserRole _role = UserRole.User;
    private DateTime _createdAtUtc;
```

Zengin domain modeli: property'ler `private set`, değişiklikler sadece anlamlı metotlarla (ör. `PromoteToAdmin`) yapılır.

`private const` uzunluk sınırları; veritabanı sütun uzunluklarıyla uyumludur. E-posta 256, görünen ad 100, Google kimliği 255 karakter.

Değerleri tutan `private` backing field'lar. `string?` null olabilir: şifresiz (sadece Google) veya Google'sız (sadece şifre) hesaplar olabilir. Rol varsayılanı User'dır.

ƒ User — Ortak alanları doğrulayıp kullanıcı oluşturan gizli kurucu.

```
    private User(string email, string displayName, UserRole role)
    {
        Email = ValidateAndNormalizeEmail(email);
        DisplayName = ValidateAndNormalizeDisplayName(displayName);
        Role = ValidateRole(role);
        CreatedAtUtc = DateTime.UtcNow;
    }
```

`private` kurucu: dışarıdan `new User(...)` yazılamaz, nesne sadece statik fabrika metotlarıyla oluşturulur. EF Core da veritabanından okurken bu kurucuyu kullanabilir.

E-posta, görünen ad ve rol doğrulanıp normalize edilerek atanır; oluşturulma zamanı UTC olarak kaydedilir.

ƒ **Id** — Veritabanının ürettiği kullanıcı kimliği.

```
28      public int Id
29      {
30          get
31          {
32              return _id;
33          }
34          private set
35          {
36              _id = value;
37          }
38      }
```

`private set`: herkes okuyabilir ama sadece sınıfın kendisi ve EF Core değer atayabilir. Id'yi veritabanı belirler.

ƒ **Email** — Küçük harfe çevrilmiş, benzersiz e-posta adresi.

```
40      public string Email
41      {
42          get
43          {
44              return _email;
45          }
46          private set
47          {
48              _email = value;
49          }
50      }
```

Sadece doğrulamadan geçmiş değer atanır; veritabanında benzersiz indeks vardır, aynı e-postayla iki hesap açılamaz.

ƒ **DisplayName** — Arayüzde gösterilen kullanıcı adı.

```
52      public string DisplayName
53      {
54          get
55          {
56              return _displayName;
57          }
58          private set
59          {
60              _displayName = value;
61          }
62      }
```

Boşlukları temizlenmiş, en fazla 100 karakterlik görünen ad.

ƒ **PasswordHash** — Şifrenin PBKDF2 hash'i; şifresiz hesapta null.

```
64      public string? PasswordHash
65      {
66          get
67          {
68              return _passwordHash;
69          }
70          private set
71          {
72              _passwordHash = value;
73          }
74      }
```

Düz şifre asla saklanmaz, sadece hash tutulur. Sadece Google ile giriş yapan hesaplarda null'dır.

ƒ **GoogleSubjectId** — Google hesabının değişmez kimliği (sub); bağlı değilse null.

```
76      public string? GoogleSubjectId
77      {
78          get
79          {
80              return _googleSubjectId;
81          }
82          private set
83          {
84              _googleSubjectId = value;
85          }
86      }
```

Google e-postayı değiştirse bile bu kimlik sabit kalır; kullanıcıyı Google ile eşleştirmenin güvenilir yoludur.

ƒ **Role** — Kullanıcının rolü: User veya Admin.

```
88      public UserRole Role
89      {
90          get
91          {
92              return _role;
93          }
94          private set
95          {
96              _role = value;
97          }
98      }
```

Dışarıdan doğrudan değiştirilemez; Admin yapmak için kurallı `PromoteToAdmin` metodu kullanılmak zorundadır.

ƒ CreatedAtUtc — Hesabın oluşturulma zamanı (UTC).

```
100     public DateTime CreatedAtUtc
101     {
102         get
103         {
104             return _createdAtUtc;
105         }
106         private set
107         {
108             _createdAtUtc = value;
109         }
110     }
```

Kurucuda bir kez atanır; saat dilimi karışıklığı olmasın diye UTC kullanılır.

ƒ HasPassword — Hesabın şifresi varsa true döner.

```
112     public bool HasPassword
113     {
114         get
115         {
116             return PasswordHash != null;
117         }
118     }
```

Hesaplanan property: veritabanında sütunu yoktur, `PasswordHash` null değilse true olur.

ƒ HasGoogleLogin — Hesap bir Google hesabına bağlıysa true döner.

```
120     public bool HasGoogleLogin
121     {
122         get
123         {
124             return GoogleSubjectId != null;
125         }
126     }
```

`GoogleSubjectId` doluysa true; admin olma kuralı bu değere dayanır.

ƒ IsAdmin — Kullanıcının rolü Admin ise true döner.

```
128     public bool IsAdmin
129     {
130         get
131         {
132             return Role == UserRole.Admin;
133         }
134     }
```

Rol karşılaştırmasını tek yerde toplar; kodun her yerinde `Role == UserRole.Admin` yazmaya gerek kalmaz.

ƒ RegisterWithPassword — E-posta ve şifre hash'iyle her zaman User rolünde yeni hesap oluşturur.

```
136     public static User RegisterWithPassword(
137         string email,
138         string displayName,
139         string passwordHash)
```

`static` fabrika metodu: nesne olmadan `User.RegisterWithPassword(...)` diye çağrılır. Rol parametresi yoktur, şifreyle kayıt olan kimse admin olamaz.

```
140     {
141         User user = new User(email, displayName, UserRole.User);
142         user.PasswordHash = ValidatePasswordHash(passwordHash);
143
144         return user;
145     }
```

Rol sabit olarak `UserRole.User` verilir; şifre hash'i doğrulanıp atanır ve yeni kullanıcı döndürülür.

ƒ RegisterWithGoogle — Google ile ilk girişte Google kimliği ve belirlenen rolle hesap oluşturur.

```
147     public static User RegisterWithGoogle(
148         string email,
149         string displayName,
150         string googleSubjectId,
151         UserRole role)
```

Rol, admin e-posta listesine göre Application katmanında belirlenip verilir; Google e-postayı doğruladığı için Admin olabilir.

```
152     {
153         User user = new User(email, displayName, role);
154         user.GoogleSubjectId = ValidateGoogleSubjectId(googleSubjectId);
155
156         return user;
157     }
```

Ortak alanlar kurucuda doğrulanır, Google kimliği ayrıca doğrulanıp atanır. Şifre yoktur, `PasswordHash` null kalır.

ƒ LinkGoogleAccount — Mevcut hesabı Google hesabına bağlar ve eski şifreyi siler.

```
159     public void LinkGoogleAccount(string googleSubjectId)
```

Aynı e-postayla önceden şifreyle açılmış hesaba ilk kez Google ile girildiğinde çağrılır.

```
160     {
161         string validatedGoogleSubjectId = ValidateGoogleSubjectId(googleSubjectI
    d);
```

Gelen Google kimliği önce doğrulanıp temizlenir.

```
163         if (GoogleSubjectId == validatedGoogleSubjectId)
164         {
165             return;
166         }
```

Hesap zaten bu Google kimliğine bağlıysa yapılacak bir şey yoktur; metot sessizce biter.

```
168         if (GoogleSubjectId != null)
169         {
170             throw new DomainRuleViolationException(
171                 "This user is already linked to a different Google account.");
172         }
```

Hesap başka bir Google hesabına bağlıysa üzerine yazılmaz; kural ihlali fırlatılır. Hesap ele geçirmeyi önler.

```
174         GoogleSubjectId = validatedGoogleSubjectId;
175         PasswordHash = null;
```

Google kimliği bağlanır ve şifre silinir. Neden: e-postayı başkası önceden şifreyle kaydetmiş olabilir; şifre silinince bu saldırgan giremez (pre-hijacking koruması).

```
176     }
```

ƒ ReplacePasswordHash — Şifresi olan hesabın hash'ini yenisiyle değiştirir.

```
178     public void ReplacePasswordHash(string passwordHash)
```

Girişte hash eski ayarlarla üretilmişse (rehash gerekli) yeni hash'i kaydetmek için kullanılır.

```
179     {
180         if (HasPassword == false)
181         {
182             throw new DomainRuleViolationException(
183                 "A password hash can only be replaced for an account that has a password.");
184         }
```

Şifresi olmayan hesaba bu yoldan şifre eklenemez; aksi halde Google hesabına gizlice şifre eklenebilirdi.

```
186         PasswordHash = ValidatePasswordHash(passwordHash);
```

Yeni hash doğrulanıp atanır.

```
187     }
```

ƒ PromoteToAdmin — Google ile doğrulanmış hesabı Admin rolüne yükseltir.

```
189     public void PromoteToAdmin()
```

Admin yetkisi vermenin tek yolu bu metottur; kural Domain içinde olduğu için hiçbir katman atlayamaz.

```
190     {
191         if (HasGoogleLogin == false)
192         {
193             throw new DomainRuleViolationException(
194                 "Only an account whose email address was verified by Google can become an admin.");
195         }
```

Hesap Google'a bağlı değilse istisna fırlatılır. Sadece e-postası Google tarafından doğrulanmış hesap admin olabilir; sahte e-postayla kayıt admin olamaz.

```
197         Role = UserRole.Admin;
```

Kural sağlandıysa rol Admin yapılır.

```
198     }
```

ƒ ValidateAndNormalizeEmail — E-postayı kontrol eder, küçük harfe çevirir; geçersizse istisna fırlatır.

```
200     private static string ValidateAndNormalizeEmail(string email)
```

`private static`: sadece bu sınıf içinde kullanılan, nesne verisine ihtiyaç duymayan yardımcı metot.

```
201     {
202         if (string.IsNullOrWhiteSpace(email))
203         {
204             throw new DomainRuleViolationException(
205                 "An email address cannot be empty.");
206         }
```

E-posta boş veya sadece boşluksa kural ihlali fırlatılır.

```
208         string normalizedEmail = email.Trim().ToLowerInvariant();
```

`Trim` boşlukları atar, `ToLowerInvariant` kültürden bağımsız küçük harfe çevirir; `A@x.com` ile `a@x.com` aynı hesap olur.

```
210         if (normalizedEmail.Length > MaximumEmailLength)
211         {
212             throw new DomainRuleViolationException(
213                 $"An email address cannot exceed {MaximumEmailLength} characters.");
214         }
```

Uzunluk sınırı aşılırsa kural ihlali fırlatılır. `$"...{...}"` metin içine değer yerleştiren string interpolation sözdizimidir.

```
216         int atSignIndex = normalizedEmail.IndexOf('@');
217         bool hasTextBeforeAtSign = atSignIndex > 0;
218         bool hasTextAfterAtSign = atSignIndex < normalizedEmail.Length - 1;
219         bool hasSingleAtSign = atSignIndex == normalizedEmail.LastIndexOf('@');
```

Basit biçim kontrolü: `@` işaretinden önce ve sonra metin olmalı ve tek bir `@` bulunmalı. İlk ve son `@` konumu eşitse tek demektir.

```
221         if (hasTextBeforeAtSign == false || hasTextAfterAtSign == false || hasSingleAtSign == false)
222         {
223             throw new DomainRuleViolationException(
224                 "The email address is not valid.");
225         }
```

Biçim koşullarından biri sağlanmazsa e-posta geçersiz sayılır ve kural ihlali fırlatılır.

```
227         return normalizedEmail;
```

Doğrulanmış, normalize edilmiş e-posta döndürülür.

```
228     }
```

ƒ ValidateAndNormalizeDisplayName — Görünen adı kontrol edip boşluklarını temizler.

```
230     private static string ValidateAndNormalizeDisplayName(string displayName)
```

Görünen ad için yardımcı doğrulama metodu.

```
231     {
232         if (string.IsNullOrWhiteSpace(displayName))
233         {
234             throw new DomainRuleViolationException(
235                 "A display name cannot be empty.");
236         }
```

Görünen ad boş olamaz; boşsa kural ihlali fırlatılır.

```
238         string normalizedDisplayName = displayName.Trim();
239
240         if (normalizedDisplayName.Length > MaximumDisplayNameLength)
241         {
242             throw new DomainRuleViolationException(
243                 $"A display name cannot exceed {MaximumDisplayNameLength} characters.");
244         }
245
246         return normalizedDisplayName;
```

Boşluklar temizlenir, 100 karakter sınırı kontrol edilir ve temiz değer döndürülür.

```
247     }
```

ƒ ValidatePasswordHash — Şifre hash'inin boş olmadığını kontrol eder.

```
249     private static string ValidatePasswordHash(string passwordHash)
```

Domain hash'in nasıl üretildiğini bilmez; sadece boş olmamasını garanti eder. Hash'leme Infrastructure'ın işidir.

```
250     {
```

```
        if (string.IsNullOrWhiteSpace(passwordHash))
        {
            throw new DomainRuleViolationException(
                "A password hash cannot be empty.");
        }

        return passwordHash;
    }
```

Hash boşsa kural ihlali fırlatılır; değilse aynen döndürülür.

ƒ ValidateGoogleSubjectId — Google kimliğini kontrol edip boşluklarını temizler.

```
    private static string ValidateGoogleSubjectId(string googleSubjectId)
    {
        if (string.IsNullOrWhiteSpace(googleSubjectId))
        {
            throw new DomainRuleViolationException(
                "A Google account id cannot be empty.");
        }

        string normalizedGoogleSubjectId = googleSubjectId.Trim();

        if (normalizedGoogleSubjectId.Length > MaximumGoogleSubjectIdLength)
        {
            throw new DomainRuleViolationException(
                $"A Google account id cannot exceed {MaximumGoogleSubjectIdLength} characters.");
        }

        return normalizedGoogleSubjectId;
    }
```

Google hesabı kimliği için yardımcı doğrulama metodu.

Google kimliği boş olamaz; boşsa kural ihlali fırlatılır.

Boşluklar temizlenir, 255 karakter sınırı kontrol edilir ve temiz değer döndürülür.

ƒ ValidateRole — Rol değerinin enum'da tanımlı olup olmadığını kontrol eder.

```
    private static UserRole ValidateRole(UserRole role)
    {
        bool isDefinedRole = Enum.IsDefined(typeof(UserRole), role);

        if (isDefinedRole == false)
        {
            throw new DomainRuleViolationException(
                "The selected user role is not supported.");
        }

        return role;
    }
}
```

Enum'lar aslında sayıdır; `(UserRole)99` gibi geçersiz bir değerin sisteme girmesini bu metot engeller.

`Enum.IsDefined` verilen değerin `UserRole` içinde tanımlı (1 veya 2) olup olmadığını söyler.

Tanımsız rolse kural ihlali fırlatılır; geçerliyse rol aynen döndürülür.

Domain `todo.Domain/Enums/UserRole.cs`

> Sistemdeki kullanıcı rollerini tanımlayan enum. Domain katmanındadır, çünkü rol iş kuralıdır ve hiçbir dış kütüphaneye bağlı değildir.

```
namespace TodoApi.Domain.Enums;
```

Dosya kapsamlı namespace: Domain katmanının enum'ları burada toplanır.

ƒ enum UserRole — Kullanıcının yetki seviyesini (User veya Admin) belirten sabit değerler kümesi.

```
public enum UserRole
{
    User = 1,
    Admin = 2
}
```

`enum` isimlendirilmiş sabit sayılar kümesidir; kodda `2` yerine `UserRole.Admin` yazmak okunabilirliği ve güvenliği artırır.

Değerler açıkça 1 ve 2 verilmiştir; veritabanında int olarak saklandığı için sıralama değişse bile kayıtlı veriler bozulmaz. 0 hiçbir role karşılık gelmez, atanmamış değer geçerli rol sayılmaz.

Application `todo.Application/Abstractions/Authentication/IPasswordHasher.cs`

> Şifre hash'leme ve doğrulama portu ile doğrulama sonucunu anlatan enum. Uygulaması Infrastructure'da ASP.NET Core'un şifre hasher'ını kullanır.

```
namespace TodoApi.Application.Abstractions.Authentication;
```

Kimlik doğrulama portlarının isim alanı.

ƒ interface IPasswordHasher — Şifreyi hash'leyen ve girilen şifreyi hash ile karşılaştıran servisin sözleşmesi.

```
public interface IPasswordHasher
{
```

`interface` : Application hangi algoritmanın kullanıldığını bilmez, sadece bu sözleşmeyi çağırır.

ƒ HashPassword — Düz şifreden veritabanına yazılacak güvenli hash üretir.

```
    string HashPassword(string password);
```

Düz şifreyi alır, saklanabilir hash döndürür. Şifrenin kendisi hiçbir zaman veritabanına yazılmaz.

ƒ VerifyPassword — Girilen şifrenin kayıtlı hash ile eşleşip eşleşmediğini söyler.

```
    PasswordVerificationOutcome VerifyPassword(string passwordHash, string providedPassword);
}
```

Kayıtlı hash ile kullanıcının girdiği şifreyi karşılaştırır; sonucu basit `bool` yerine zengin bir enum olarak döndürür.

ƒ enum PasswordVerificationOutcome — Şifre doğrulamasının üç olası sonucu.

```
10 public enum PasswordVerificationOutcome
11 {
12     Failed = 0,
13     Succeeded = 1,
14     SucceededRehashNeeded = 2
15 }
```

`enum`: isimlendirilmiş sabit değerler kümesi. Aynı dosyada port ile birlikte tanımlanmıştır.

`Failed`: şifre yanlış. `Succeeded`: şifre doğru.

`SucceededRehashNeeded`: şifre doğru ama hash eski ayarlarla yapılmış; giriş use case'i hash'i güncel algoritmayla yeniler.

Infrastructure `todo.Infrastructure/Services/Authentication/AspNetCorePasswordHasher.cs`

> Application katmanındaki `IPasswordHasher` arayüzünün Infrastructure uygulaması. Şifreleri ASP.NET Core Identity'nin PBKDF2 tabanlı `PasswordHasher` sınıfıyla hash'ler ve doğrular.

```
1 using System;
2 using Microsoft.AspNetCore.Identity;
3 using Microsoft.Extensions.Options;
4 using TodoApi.Application.Abstractions.Authentication;
5 using TodoApi.Domain.Entities;
```

Gerekli isim alanları: Identity (hazır şifre hash'leyici), Options (ayar nesnesi), Application arayüzü ve Domain'deki `User` sınıfı.

```
7 namespace TodoApi.Infrastructure.Services.Authentication;
```

Dosya kapsamlı namespace: bu dosyadaki tüm tipler Infrastructure katmanının kimlik doğrulama servislerine aittir.

ƒ class AspNetCorePasswordHasher — Şifre hash'leme ve doğrulamayı Identity kütüphanesine devreden altyapı servisi.

```
9  public sealed class AspNetCorePasswordHasher : IPasswordHasher
10 {
11     public const int IterationCount = 210_000;
13     private readonly PasswordHasher<User> _passwordHasher = new PasswordHasher<User>(
14         Options.Create(new PasswordHasherOptions { IterationCount = IterationCount }));
```

`sealed` sınıf kalıtılamaz. `IPasswordHasher` uygular; böylece Application katmanı Identity kütüphanesini hiç tanımadan şifre işleyebilir.

`const` derleme anında sabitlenen değerdir. 210.000 PBKDF2 tekrarı, kaba kuvvet saldırılarını yavaşlatmak için güncel güvenlik önerisine uygundur.

Identity'nin `PasswordHasher<User>` nesnesi tekrar sayısı ayarıyla bir kez oluşturulur. `Options.Create` ayarı DI olmadan sarmalar; `readonly` alan sonradan değiştirilemez.

ƒ HashPassword — Düz şifreyi alıp tuzlanmış PBKDF2 hash metnine çevirir.

```
16     public string HashPassword(string password)
17     {
18         if (string.IsNullOrEmpty(password))
19         {
20             throw new ArgumentException("A password cannot be empty.", nameof(password));
21         }
23         string passwordHash = _passwordHasher.HashPassword(null!, password);
24
25         return passwordHash;
26     }
```

Kayıt ve şifre yenileme sırasında çağrılır; düz şifre asla veritabanına yazılmaz, yalnızca bu metodun ürettiği hash saklanır.

Boş şifre programlama hatasıdır; `ArgumentException` fırlatılır. `nameof(password)` parametre adını metin olarak verir, yeniden adlandırmada otomatik güncellenir.

Identity rastgele tuz üretip hash'i hesaplar. Kullanıcı nesnesi gerekmediği için `null!` verilir; `!` derleyiciye 'null uyarısı verme' der.

ƒ VerifyPassword — Girilen şifreyi kayıtlı hash ile karşılaştırır; başarılı, başarılı-yeniden-hash veya başarısız döner.

```
28     public PasswordVerificationOutcome VerifyPassword(string passwordHash, string providedPassword)
29     {
30         if (string.IsNullOrEmpty(passwordHash) || string.IsNullOrEmpty(providedPassword))
31         {
32             return PasswordVerificationOutcome.Failed;
33         }
35         PasswordVerificationResult result;
37         try
38         {
39             result = _passwordHasher.VerifyHashedPassword(null!, passwordHash, providedPassword);
40         }
41         catch (FormatException)
42         {
43             return PasswordVerificationOutcome.Failed;
44         }
46         if (result == PasswordVerificationResult.Success)
47         {
48             return PasswordVerificationOutcome.Succeeded;
49         }
51         if (result == PasswordVerificationResult.SuccessRehashNeeded)
52         {
53             return PasswordVerificationOutcome.SucceededRehashNeeded;
54         }
```

Giriş sırasında çağrılır. Sonuç Identity tipi yerine bizim `PasswordVerificationOutcome` enum'umuzdur; böylece üst katman kütüphaneye bağımlı kalmaz.

Hash veya girilen şifre boşsa (ör. sadece Google ile giriş yapan hesap) doğrudan başarısız sayılır, istisna fırlatılmaz.

Identity'nin doğrulama sonucunu tutacak değişken önceden tanımlanır; değeri `try` bloğunun içinde atanacak.

Kayıtlı hash bozuk formatta ise Identity `FormatException` fırlatır; bunu yakalayıp sunucu hatası yerine sadece 'giriş başarısız' döneriz.

Şifre doğru ve hash güncel ayarlarla üretilmişse başarılı sonucu döner.

Şifre doğru ama hash eski/zayıf ayarlarla üretilmişse 'yeniden hash gerekli' döner; üst katman şifreyi yeni ayarlarla tekrar hash'leyip kaydeder.

```
        return PasswordVerificationOutcome.Failed;
    }
}
```

Diğer tüm durumlarda şifre yanlıştır; başarısız sonucu döner.

Application `todo.Application/Features/Authentication/Common/SessionIssuer.cs`

> Tüm giriş yolları (şifre, Google, yenileme) için ortak oturum üretici: yeni refresh token oluşturup hash'ini kaydeder, JWT access token üretir ve hepsini `AuthSession` olarak paketler.

```
using System;
using System.Threading;
using System.Threading.Tasks;
```

Temel .NET tipleri: `DateTime`, `TimeProvider`, `Task` ve `CancellationToken`.

```
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Application.Contracts;
using TodoApi.Domain.Entities;
```

Token üretici ve depo portları, dışarı dönen DTO'lar ve `User`, `RefreshToken` Domain varlıkları.

```
namespace TodoApi.Application.Features.Authentication.Common;
```

Birden fazla use case'in paylaştığı ortak kimlik doğrulama sınıflarının isim alanı.

ƒ class SessionIssuer — Kullanıcıya yeni access ve refresh token çifti üreten ortak servis.

```
public class SessionIssuer
```

Oturum açma mantığı tek yerde toplanır; böylece her giriş yolu aynı güvenli kuralları uygular (tekrar eden kod yok).

```
{
```

```
    private readonly IAccessTokenGenerator _accessTokenGenerator;
    private readonly ISecureTokenGenerator _secureTokenGenerator;
    private readonly IRefreshTokenRepository _refreshTokenRepository;
    private readonly RefreshTokenSettings _refreshTokenSettings;
    private readonly TimeProvider _timeProvider;
```

`private readonly` bağımlılıklar: JWT üretici, güvenli rastgele token üretici, token deposu, refresh token ayarları ve saat.

ƒ SessionIssuer(...) — Beş bağımlılığı DI ile alan yapıcı metot.

```
    public SessionIssuer(
        IAccessTokenGenerator accessTokenGenerator,
        ISecureTokenGenerator secureTokenGenerator,
        IRefreshTokenRepository refreshTokenRepository,
        RefreshTokenSettings refreshTokenSettings,
        TimeProvider timeProvider)
```

Yapıcı, bağımlılıkları DI konteynerinden alır; `RefreshTokenSettings` config'ten okunup tekil (singleton) olarak kaydedilir.

```
    {
        if (accessTokenGenerator == null)
        {
            throw new ArgumentNullException(nameof(accessTokenGenerator));
        }

        if (secureTokenGenerator == null)
        {
            throw new ArgumentNullException(nameof(secureTokenGenerator));
        }

        if (refreshTokenRepository == null)
        {
            throw new ArgumentNullException(nameof(refreshTokenRepository));
        }
```

Her bağımlılık için null kontrolü; eksikse `ArgumentNullException` fırlatılır.

```
        if (refreshTokenSettings == null)
        {
            throw new ArgumentNullException(nameof(refreshTokenSettings));
        }

        if (timeProvider == null)
        {
            throw new ArgumentNullException(nameof(timeProvider));
        }
```

Ayarlar ve saat için de aynı null kontrolü yapılır.

```
        _accessTokenGenerator = accessTokenGenerator;
        _secureTokenGenerator = secureTokenGenerator;
        _refreshTokenRepository = refreshTokenRepository;
        _refreshTokenSettings = refreshTokenSettings;
        _timeProvider = timeProvider;
    }
```

Doğrulanan bağımlılıklar özel alanlara atanır.

ƒ IssueAsync — Yeni refresh token kaydeder, JWT üretir ve `AuthSession` döndürür; isteğe bağlı eski token'ı döndürür.

```
    public async Task<AuthSession> IssueAsync(
        User user,
        CancellationToken cancellationToken,
        RefreshToken? replacedToken = null)
```

`RefreshToken? replacedToken = null`: isteğe bağlı parametre. Sadece yenileme akışı eski token'ı verir; giriş akışları vermez.

```
    {
        if (user == null)
        {
            throw new ArgumentNullException(nameof(user));
        }
```

Kullanıcı verilmemişse programlama hatasıdır; hemen `ArgumentNullException` fırlatılır.

```
        DateTime nowUtc = _timeProvider.GetUtcNow().UtcDateTime;
        DateTime refreshTokenExpiresAtUtc = nowUtc.Add(_refreshTokenSettings.Lifetime);
```

Şu anki UTC zaman alınır ve ayarlardaki süre (7 gün) eklenerek refresh token'ın bitiş zamanı hesaplanır.

```
        string refreshTokenValue = _secureTokenGenerator.GenerateToken();
        string refreshTokenHash = _secureTokenGenerator.ComputeHash(refreshTokenValue);
```

Kriptografik olarak rastgele bir token üretilir ve SHA-256 hash'i hesaplanır. Düz değer sadece istemciye gider, veritabanına hash yazılır.

```
        RefreshToken refreshToken = new RefreshToken(
            user.Id,
            refreshTokenHash,
            nowUtc,
            refreshTokenExpiresAtUtc);
```

Kullanıcı kimliği, hash, oluşturulma ve bitiş zamanıyla yeni `RefreshToken` Domain nesnesi oluşturulur.

```
        if (replacedToken != null)
        {
            replacedToken.Revoke(nowUtc, refreshTokenHash);
        }
```

Yenilemede eski token iptal edilir ve yerine geçen token'ın hash'i kaydedilir (rotation). Bu bağ, hırsızlık tespitindeki 30 saniyelik toleransı mümkün kılar.

```
        await _refreshTokenRepository.AddAsync(refreshToken, cancellationToken);
        await _refreshTokenRepository.SaveChangesAsync(cancellationToken);
```

Yeni token depoya eklenir ve tüm değişiklikler (eski token iptali dahil) tek seferde kaydedilir.

```
        AccessToken accessToken = _accessTokenGenerator.GenerateToken(user);
        AuthResultDto result = new AuthResultDto(accessToken, new UserDto(user));
```

15 dakikalık JWT access token üretilir; token ve kullanıcı bilgisi `AuthResultDto` içinde birleştirilir.

```
        AuthSession session = new AuthSession(result, refreshTokenValue, refreshTokenExpiresAtUtc);

        return session;
    }
}
```

Sonuç, düz refresh token değeri ve bitiş zamanıyla `AuthSession`'a paketlenip döndürülür; controller refresh token'ı HttpOnly cookie'ye yazar.

Application `todo.Application/Features/Authentication/Common/AuthSession.cs`

> Bir giriş veya yenileme sonucunu taşıyan değişmez paket: JSON gövdesine gidecek `AuthResultDto` ile cookie'ye yazılacak refresh token ve bitiş zamanı.

```
using System;
using TodoApi.Application.Contracts;
```

`DateTime` için `System`, `AuthResultDto` için Contracts isim alanı kullanıma açılır.

```
namespace TodoApi.Application.Features.Authentication.Common;
```

Ortak kimlik doğrulama sınıflarının isim alanı.

ƒ class AuthSession — Access token sonucu ile refresh token'ı birlikte taşıyan değişmez sınıf.

```
public sealed class AuthSession
{
    private readonly AuthResultDto _result;
    private readonly string _refreshToken;
    private readonly DateTime _refreshTokenExpiresAtUtc;
```

`sealed` sınıf: miras alınamaz. Use case'ten controller'a oturum bilgisini taşır.

Değişmez alanlar: istemciye dönecek sonuç, düz refresh token değeri ve onun bitiş zamanı.

ƒ AuthSession(...) — Sonucu, refresh token'ı ve bitiş zamanını doğrulayıp saklayan yapıcı.

```
    public AuthSession(AuthResultDto result, string refreshToken, DateTime refreshTokenExpiresAtUtc)
    {
        if (result == null)
        {
            throw new ArgumentNullException(nameof(result));
        }

        if (string.IsNullOrWhiteSpace(refreshToken))
        {
            throw new ArgumentException("A refresh token cannot be empty.", nameof(refreshToken));
        }

        _result = result;
        _refreshToken = refreshToken;
        _refreshTokenExpiresAtUtc = refreshTokenExpiresAtUtc;
    }
```

Yapıcı üç değeri alır; nesne bir kez oluşur ve sonra değişmez.

Sonuç null veya refresh token boşsa hata fırlatılır; geçersiz bir oturum nesnesi hiç oluşamaz.

Doğrulanan değerler özel alanlara kaydedilir.

```
    public AuthResultDto Result
    {
        get
        {
            return _result;
        }
    }
```

Sadece okunabilen (`get`) `Result` özelliği; access token ve kullanıcı bilgisini döndürür.

```
    public string RefreshToken
    {
        get
        {
            return _refreshToken;
        }
    }
```

`RefreshToken` özelliği düz token değerini verir; controller bunu HttpOnly cookie'ye yazar, JSON gövdesine koymaz.

```
    public DateTime RefreshTokenExpiresAtUtc
    {
        get
        {
            return _refreshTokenExpiresAtUtc;
        }
    }
}
```

`RefreshTokenExpiresAtUtc` cookie'nin ne zaman sona ereceğini belirlemek için kullanılır.

Application `todo.Application/Features/Authentication/Common/RefreshTokenSettings.cs`

> Refresh token'ın ömrünü tutan küçük ayar sınıfı. Değer Infrastructure'da config'teki `Jwt:RefreshTokenLifetimeDays` okunarak oluşturulur.

```
using System;
```

`TimeSpan` ve hata tipleri için `System` isim alanı.

```
namespace TodoApi.Application.Features.Authentication.Common;
```

Ortak kimlik doğrulama sınıflarının isim alanı.

ƒ class RefreshTokenSettings — Refresh token ömrünü doğrulanmış şekilde taşıyan ayar sınıfı.

```
public sealed class RefreshTokenSettings
```

`sealed` ayar sınıfı; Application katmanı config dosyasını bilmez, sadece bu nesneyi alır.

```
{
```

```
    private readonly TimeSpan _lifetime;
```

Token ömrü `TimeSpan` (zaman aralığı) olarak saklanır; değiştirilemez.

ƒ RefreshTokenSettings(TimeSpan) — Ömrün pozitif olduğunu kontrol edip saklayan yapıcı.

```
    public RefreshTokenSettings(TimeSpan lifetime)
```

Yapıcı ömrü parametre olarak alır.

```
    {
        if (lifetime <= TimeSpan.Zero)
        {
            throw new ArgumentOutOfRangeException(
                nameof(lifetime),
                "A refresh token lifetime must be positive.");
        }
```

Sıfır veya negatif ömür anlamsızdır; `ArgumentOutOfRangeException` ile uygulama başlarken hatalı config yakalanır.

```
        _lifetime = lifetime;
```

Geçerli değer özel alana kaydedilir.

```
    }
    public TimeSpan Lifetime
    {
        get
        {
            return _lifetime;
        }
    }
}
```

Sadece okunabilen `Lifetime` özelliği; `SessionIssuer` bitiş zamanını hesaplarken kullanır.

Application `todo.Application/Abstractions/Authentication/IAccessTokenGenerator.cs`

> JWT access token üretme portu. Application ne istediğini tanımlar; gerçek JWT imzalama Infrastructure'daki `JwtAccessTokenGenerator` içinde yapılır (dependency inversion).

```
using TodoApi.Domain.Entities;
```

Parametre tipi olan `User` Domain varlığı için isim alanı.

```
namespace TodoApi.Application.Abstractions.Authentication;
```

Kimlik doğrulama portlarının isim alanı.

ƒ interface IAccessTokenGenerator — Kullanıcı için access token üreten servisin sözleşmesi.

```
public interface IAccessTokenGenerator
```

`interface`: sadece metot imzalarını tanımlayan sözleşme; gövde yoktur, başka bir sınıf bunu uygular (implement eder).

```
{
```

ƒ GenerateToken — Verilen kullanıcı için imzalı, kısa ömürlü access token döndürür.

```
    AccessToken GenerateToken(User user);
```

Kullanıcıyı alır, içinde kimlik ve rol bilgisi olan 15 dakikalık JWT'yi `AccessToken` olarak döndürür.

```
}
```

Application `todo.Application/Abstractions/Authentication/AccessToken.cs`

> `IAccessTokenGenerator` portunun döndürdüğü sonuç: üretilen JWT metni ve bitiş zamanı. Port ile birlikte Abstractions klasöründe durur.

```
using System;
```

`DateTime` ve `ArgumentException` için `System` isim alanı.

```
namespace TodoApi.Application.Abstractions.Authentication;
```

Kimlik doğrulama portlarının isim alanı.

ƒ class AccessToken — JWT değerini ve bitiş zamanını taşıyan değişmez sınıf.

```
public sealed class AccessToken
```

`sealed`, değişmez (immutable) küçük bir değer sınıfı.

```
{
    private readonly string _value;
    private readonly DateTime _expiresAtUtc;
```

Token metni ve bitiş zamanı; `readonly` olduğu için oluşturulduktan sonra değişmez.

ƒ AccessToken(string, DateTime) — Boş olmayan token değerini ve bitiş zamanını saklayan yapıcı.

```
    public AccessToken(string value, DateTime expiresAtUtc)
```

Yapıcı token metnini ve UTC bitiş zamanını alır.

```
    {
        if (string.IsNullOrWhiteSpace(value))
        {
            throw new ArgumentException("An access token cannot be empty.", nameof(value));
        }
```

Boş token kabul edilmez; `ArgumentException` fırlatılır.

```
        _value = value;
        _expiresAtUtc = expiresAtUtc;
    }
```

Değerler özel alanlara kaydedilir.

```
    public string Value
    {
        get
        {
            return _value;
        }
    }
```

`Value` özelliği: imzalı JWT metni.

```
    public DateTime ExpiresAtUtc
    {
        get
        {
            return _expiresAtUtc;
        }
    }
}
```

`ExpiresAtUtc` özelliği: token'ın geçersiz olacağı an (yaklaşık 15 dakika sonra); istemci ne zaman yenileyeceğini buradan bilir.

Infrastructure `todo.Infrastructure/Services/Authentication/JwtAccessTokenGenerator.cs`

> Giriş başarılı olunca kullanıcıya verilen kısa ömürlü JWT erişim token'ını üretir. `IAccessTokenGenerator` arayüzünün HS256 imzalı Infrastructure uygulamasıdır.

```
using System;
using System.Collections.Generic;
using System.Text;
using Microsoft.IdentityModel.JsonWebTokens;
using Microsoft.IdentityModel.Tokens;
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Domain.Entities;
```

Gerekli isim alanları: JWT üretimi için `Microsoft.IdentityModel` paketleri, metni byte'a çevirmek için `System.Text`, Application arayüzü ve `User` entity'si.

```
namespace TodoApi.Infrastructure.Services.Authentication;
```

Dosya kapsamlı namespace: sınıf Infrastructure katmanının kimlik doğrulama servisleri içindedir.

ƒ class JwtAccessTokenGenerator — Kullanıcı bilgilerinden imzalı JWT erişim token'ı üreten altyapı servisi.

```
public sealed class JwtAccessTokenGenerator : IAccessTokenGenerator
{
```

`sealed` sınıf `IAccessTokenGenerator` arayüzünü uygular; Application katmanı JWT kütüphanesini bilmeden token isteyebilir.

```
    public const string RoleClaimType = "role";
```

Rol claim'inin adı sabit olarak tutulur; API tarafındaki yetkilendirme de aynı `"role"` adını okuyarak Admin/User ayrımı yapar.

```
    private readonly JwtSettings _jwtSettings;
    private readonly TimeProvider _timeProvider;
```

JWT ayarları (anahtar, issuer, audience, süre) ve zaman sağlayıcısı alanları. `TimeProvider` testlerde sahte saat verebilmek için kullanılır.

ƒ JwtAccessTokenGenerator — Ayarları ve zaman sağlayıcısını DI'dan alıp saklar.

```
    public JwtAccessTokenGenerator(JwtSettings jwtSettings, TimeProvider timeProvider)
    {
        if (jwtSettings == null)
        {
            throw new ArgumentNullException(nameof(jwtSettings));
        }

        if (timeProvider == null)
        {
            throw new ArgumentNullException(nameof(timeProvider));
        }
```

Kurucu metot; bağımlılıklar Dependency Injection tarafından verilir.

Bağımlılıklar null gelirse hemen `ArgumentNullException` fırlatılır; hata, token üretimi sırasında değil başlangıçta açıkça görünür.

```
        _jwtSettings = jwtSettings;
        _timeProvider = timeProvider;
    }
```

Doğrulanan bağımlılıklar `readonly` alanlara atanır.

ƒ GenerateToken — Verilen kullanıcı için claim'leri hazırlar, HS256 ile imzalar ve token ile bitiş zamanını döner.

```
    public AccessToken GenerateToken(User user)
    {
        if (user == null)
        {
            throw new ArgumentNullException(nameof(user));
        }
```

Giriş, kayıt ve token yenileme akışlarında çağrılan ana metot.

Kullanıcı null ise token üretilemez; programlama hatası olarak istisna fırlatılır.

```
        DateTime issuedAtUtc = _timeProvider.GetUtcNow().UtcDateTime;
        DateTime expiresAtUtc = issuedAtUtc.AddMinutes(_jwtSettings.AccessTokenLifetimeMinutes);
```

Şu anki UTC zaman alınır ve ayardaki dakika (varsayılan 15) eklenerek bitiş zamanı hesaplanır. Kısa ömür, çalınan token'ın zararını sınırlar.

```
        Dictionary<string, object> claims = new Dictionary<string, object>
        {
            { JwtRegisteredClaimNames.Sub, user.Id.ToString() },
            { JwtRegisteredClaimNames.Email, user.Email },
            { JwtRegisteredClaimNames.Name, user.DisplayName },
            { RoleClaimType, user.Role.ToString() },
            { JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString() }
        };
```

Token içine yazılacak claim'ler: sub (kullanıcı id), email, name, role ve jti (her token'a özgü benzersiz kimlik, `Guid.NewGuid()`).

```
        byte[] signingKeyBytes = Encoding.UTF8.GetBytes(_jwtSettings.SigningKey);
        SymmetricSecurityKey signingKey = new SymmetricSecurityKey(signingKeyBytes);
        SigningCredentials signingCredentials = new SigningCredentials(
            signingKey,
            SecurityAlgorithms.HmacSha256);
```

Gizli imza anahtarı byte dizisine çevrilip simetrik anahtar yapılır; HMAC-SHA256 (HS256) algoritmasıyla imzalama bilgisi hazırlanır.

```
        SecurityTokenDescriptor tokenDescriptor = new SecurityTokenDescriptor
        {
            Issuer = _jwtSettings.Issuer,
            Audience = _jwtSettings.Audience,
            Claims = claims,
            IssuedAt = issuedAtUtc,
            NotBefore = issuedAtUtc,
            Expires = expiresAtUtc,
            SigningCredentials = signingCredentials
        };
```

Token tanımı: kim üretti (Issuer), kim için (Audience), claim'ler, üretim, geçerlilik başlangıcı ve bitiş zamanı ile imza bilgisi.

```
        JsonWebTokenHandler tokenHandler = new JsonWebTokenHandler();
        string tokenValue = tokenHandler.CreateToken(tokenDescriptor);
```

`JsonWebTokenHandler` tanımı kullanarak imzalı JWT metnini (header.payload.signature) oluşturur.

```
        AccessToken accessToken = new AccessToken(tokenValue, expiresAtUtc);

        return accessToken;
    }
}
```

Token metni ve bitiş zamanı Application'daki `AccessToken` nesnesine sarılıp döndürülür; istemci ne zaman yenileme yapacağını bilir.

Application `todo.Application/Abstractions/Authentication/ISecureTokenGenerator.cs`

> Refresh token için rastgele değer üretme ve hash'leme portu. Infrastructure'daki `SecureTokenGenerator` 32 rastgele bayt ve SHA-256 kullanır.

```
namespace TodoApi.Application.Abstractions.Authentication;
```

Kimlik doğrulama portlarının isim alanı.

ƒ interface ISecureTokenGenerator — Güvenli rastgele token üreten ve hash'leyen servisin sözleşmesi.

```
public interface ISecureTokenGenerator
```

`interface`: kriptografi detayları Infrastructure'da kalır, Application sadece ne istediğini söyler.

```
{
```

ƒ GenerateToken — Tahmin edilemez, URL'de güvenli rastgele bir token metni döndürür.

```
    string GenerateToken();
```

Kriptografik rastgele sayı üreteciyle tahmin edilemez bir refresh token değeri üretir.

ƒ ComputeHash — Token'ın SHA-256 hash'ini hesaplar.

```
    string ComputeHash(string token);
```

Veritabanına sadece hash yazılır; böylece veritabanı sızsa bile gerçek token'lar ele geçirilemez. Aramalar da hash ile yapılır.

```
}
```

Infrastructure `todo.Infrastructure/Services/Authentication/SecureTokenGenerator.cs`

> Refresh token'ları kriptografik olarak güvenli rastgele sayı üreteciyle oluşturur ve veritabanında saklanacak SHA-256 hash'lerini hesaplar.

```
using System;
using System.Security.Cryptography;
using System.Text;
using TodoApi.Application.Abstractions.Authentication;
```

`System.Security.Cryptography` güvenli rastgele sayı ve SHA-256 için, `System.Text` metni byte'a çevirmek için, Application arayüzü de uygulanacak sözleşme için eklenir.

```
namespace TodoApi.Infrastructure.Services.Authentication;
```

Dosya kapsamlı namespace: Infrastructure kimlik doğrulama servisleri.

ƒ class SecureTokenGenerator — Güvenli rastgele refresh token üreten ve hash'leyen altyapı servisi.

```
public sealed class SecureTokenGenerator : ISecureTokenGenerator
```

`sealed` sınıf `ISecureTokenGenerator` arayüzünü uygular; Application katmanı kripto detaylarını bilmez.

```
{
    private const int TokenByteCount = 32;
```

`private const`: sadece bu sınıfın gördüğü sabit. 32 byte = 256 bit rastgelelik, tahmin edilmesi pratikte imkânsızdır.

ƒ GenerateToken — 32 byte güvenli rastgele veri üretip URL-güvenli Base64 metne çevirir.

```
    public string GenerateToken()
    {
        byte[] tokenBytes = RandomNumberGenerator.GetBytes(TokenByteCount);

        string token = Convert.ToBase64String(tokenBytes)
            .TrimEnd('=')
            .Replace('+', '-')
            .Replace('/', '_');

        return token;
    }
```

Yeni refresh token üretir; bu ham değer yalnızca istemciye gönderilir, veritabanına yazılmaz.

`RandomNumberGenerator` işletim sisteminin kriptografik üretecini kullanır; `Random` sınıfının aksine sonuçları tahmin edilemez.

Byte'lar Base64 metne çevrilir; `=` dolgu atılır, `+` ve `/` URL'de sorun çıkardığı için `-` ve `_` ile değiştirilir (Base64Url).

ƒ ComputeHash — Token metninin SHA-256 hash'ini hex metin olarak döner.

```
    public string ComputeHash(string token)
    {
        if (string.IsNullOrEmpty(token))
        {
            throw new ArgumentException("A token cannot be empty.", nameof(token));
        }
```

Veritabanına ham token yerine bu hash yazılır; veritabanı sızsa bile saldırgan geçerli token elde edemez.

Boş token hash'lenmez; programlama hatası olarak `ArgumentException` fırlatılır.

```
        byte[] hashBytes = SHA256.HashData(Encoding.UTF8.GetBytes(token));

        return Convert.ToHexString(hashBytes);
    }
}
```

Token UTF-8 byte'lara çevrilip SHA-256 ile hash'lenir ve büyük harfli hex metne dönüştürülerek döndürülür. Aynı token hep aynı hash'i verir, arama yapılabilir.

Application `todo.Application/Abstractions/Persistence/IRefreshTokenRepository.cs`

> Refresh token kayıtlarına erişim portu. Oturum açma, yenileme, çıkış ve toplu iptal işlemleri bu sözleşme üzerinden yapılır; uygulaması Infrastructure'dadır.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
```

`DateTime`, `IReadOnlyList`, `CancellationToken` ve `Task` tipleri için isim alanları.

```
using TodoApi.Domain.Entities;
```

`RefreshToken` Domain varlığı için isim alanı.

```
namespace TodoApi.Application.Abstractions.Persistence;
```

Veri erişim portlarının isim alanı.

ƒ interface IRefreshTokenRepository — Refresh token kayıtlarını bulma, ekleme ve kaydetme sözleşmesi.

```
public interface IRefreshTokenRepository
{
```

`interface`: Application token'ların nasıl saklandığını bilmez, sadece bu işlemleri ister.

ƒ GetByTokenHashAsync — Hash değerine göre refresh token kaydını getirir, yoksa null.

```
    Task<RefreshToken?> GetByTokenHashAsync(string tokenHash, CancellationToken cancellationToken);
```

Arama düz token ile değil SHA-256 hash ile yapılır; veritabanında düz token hiç bulunmaz.

ƒ GetActiveForUserAsync — Kullanıcının iptal edilmemiş ve süresi dolmamış token'larını getirir.

```
    Task<IReadOnlyList<RefreshToken>> GetActiveForUserAsync(
        int userId,
        DateTime nowUtc,
        CancellationToken cancellationToken);
```

`nowUtc` parametresi süresi dolmuşları elemek için verilir. Toplu iptalde (hırsızlık veya hesap bağlama) kullanılır.

ƒ AddAsync — Yeni refresh token kaydını ekler.

```
    Task AddAsync(RefreshToken refreshToken, CancellationToken cancellationToken);
```

Yeni token'ı takibe alır; `SaveChangesAsync` ile kalıcı olur.

ƒ SaveChangesAsync — Bekleyen token değişikliklerini veritabanına yazar.

```
    Task SaveChangesAsync(CancellationToken cancellationToken);
```

Ekleme ve iptal gibi değişiklikleri veritabanına yazar.

```
}
```

Infrastructure `todo.Infrastructure/Persistence/RefreshTokenRepository.cs`

> Refresh token kayıtlarını (hash halinde) veritabanında arayan ve kaydeden repository. `IRefreshTokenRepository` arayüzünü EF Core ile uygular.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Domain.Entities;
```

Gerekli isim alanları: koleksiyonlar, LINQ, asenkron işlemler, EF Core, Application arayüzü ve `RefreshToken` entity'si.

```
namespace TodoApi.Infrastructure.Persistence;
```

Dosya kapsamlı namespace: Infrastructure katmanının veri erişim bölümü.

ƒ class RefreshTokenRepository — Refresh token verisine EF Core üzerinden erişen repository sınıfı.

```
public sealed class RefreshTokenRepository : IRefreshTokenRepository
{
```

`IRefreshTokenRepository` uygular; oturum yenileme ve çıkış akışları bu sınıf üzerinden token kayıtlarına ulaşır.

```
    private readonly TodoDbContext _todoDbContext;
```

Veritabanı bağlamı alanı; tüm sorgular bunun üzerinden yapılır.

ƒ RefreshTokenRepository — DbContext'i DI'dan alıp saklar.

```
    public RefreshTokenRepository(TodoDbContext todoDbContext)
    {
        if (todoDbContext == null)
        {
            throw new ArgumentNullException(nameof(todoDbContext));
        }

        _todoDbContext = todoDbContext;
    }
```

Kurucu metot; `TodoDbContext` DI tarafından verilir.

DbContext null ise açık bir istisna fırlatılır; değilse `readonly` alana atanır.

ƒ GetByTokenHashAsync — Hash değerine göre refresh token kaydını bulur; yoksa null döner.

```
    public async Task<RefreshToken?> GetByTokenHashAsync(
        string tokenHash,
        CancellationToken cancellationToken)
    {
```

İstemciden gelen ham token önce hash'lenir, sonra bu metotla aranır; veritabanında ham token hiç bulunmaz.

```
30          RefreshToken? refreshToken = await _todoDbContext.RefreshTokens.FirstOrDefaultAsync(
31              existingToken => existingToken.TokenHash == tokenHash,
32              cancellationToken);
33
34          return refreshToken;
35      }
```

`TokenHash` eşleşen ilk kayıt getirilir. Bu sütunda benzersiz indeks olduğundan arama hızlıdır ve en fazla bir sonuç vardır.

ƒ GetActiveForUserAsync — Kullanıcının iptal edilmemiş ve süresi dolmamış tüm refresh token'larını döner.

```
37      public async Task<IReadOnlyList<RefreshToken>> GetActiveForUserAsync(
38          int userId,
39          DateTime nowUtc,
40          CancellationToken cancellationToken)
41      {
42          IQueryable<RefreshToken> refreshTokenQuery = _todoDbContext.RefreshTokens;
43
44          refreshTokenQuery = refreshTokenQuery.Where(refreshToken =>
45              refreshToken.UserId == userId &&
46              refreshToken.RevokedAtUtc == null &&
47              refreshToken.ExpiresAtUtc > nowUtc);
48
49          List<RefreshToken> refreshTokens = await refreshTokenQuery.ToListAsync(cancellationToken);
50
51          IReadOnlyList<RefreshToken> result = refreshTokens;
52
53          return result;
54      }
```

Çıkışta veya Google hesabı bağlanınca kullanıcının tüm açık oturumlarını iptal etmek için kullanılır. `nowUtc` dışarıdan verilir, test edilebilir.

Henüz çalışmamış bir `IQueryable` sorgusu başlatılır.

Koşullar: bu kullanıcıya ait, iptal tarihi boş ve bitiş zamanı şu andan sonra olan kayıtlar. EF Core bunu SQL WHERE'e çevirir.

Sorgu çalıştırılıp liste alınır ve salt okunur arayüz tipinde döndürülür. Takip açık kalır, çünkü bu kayıtlar sonra iptal edilip kaydedilecek.

ƒ AddAsync — Yeni refresh token kaydını EF Core takibine ekler.

```
56      public async Task AddAsync(RefreshToken refreshToken, CancellationToken cancellationToken)
57      {
58          if (refreshToken == null)
59          {
60              throw new ArgumentNullException(nameof(refreshToken));
61          }
62
63          await _todoDbContext.RefreshTokens.AddAsync(refreshToken, cancellationToken);
64      }
```

Her girişte ve token yenilemede yeni token kaydı eklemek için çağrılır.

Token null ise açık bir istisna fırlatılır.

Kayıt 'eklenecek' olarak işaretlenir; veritabanına `SaveChangesAsync` çağrılınca yazılır.

ƒ SaveChangesAsync — Bekleyen tüm değişiklikleri veritabanına yazar.

```
66      public async Task SaveChangesAsync(CancellationToken cancellationToken)
67      {
68          await _todoDbContext.SaveChangesAsync(cancellationToken);
69      }
70  }
```

Eski token'ın iptali ve yenisinin eklenmesi gibi değişiklikler tek çağrıda kaydedilir.

DbContext'teki tüm takip edilen değişiklikler tek veritabanı işleminde kaydedilir.

Domain `todo.Domain/Entities/RefreshToken.cs`

> Kullanıcının uzun ömürlü oturumunu temsil eden refresh token entity'si. Ham token değil yalnızca SHA-256 hash'i saklanır; iptal ve rotasyon kuralları burada korunur.

```
1   using TodoApi.Domain.Exceptions;
2
3   namespace TodoApi.Domain.Entities;
4
5   public class RefreshToken
6   {
7       private int _id;
8       private int _userId;
9       private string _tokenHash = string.Empty;
10      private DateTime _createdAtUtc;
11      private DateTime _expiresAtUtc;
12      private DateTime? _revokedAtUtc;
13      private string? _replacedByTokenHash;
14
15      public RefreshToken(int userId, string tokenHash, DateTime createdAtUtc, DateTime expiresAtUtc)
16      {
17          if (userId <= 0)
18          {
19              throw new DomainRuleViolationException(
20                  "A refresh token must belong to a saved user.");
21          }
```

Domain'e ait `DomainRuleViolationException` için isim alanı; Domain katmanı başka hiçbir dış kütüphaneye bağlı değildir.

Dosya kapsamlı namespace: Domain entity'leri.

ƒ class RefreshToken — Bir kullanıcıya ait, süreli ve iptal edilebilir refresh token kaydı.

Zengin domain modeli: alanlar dışarıdan doğrudan değiştirilemez, sadece kurucu ve metotlar üzerinden kurallı şekilde değişir.

Değerleri tutan `private` alanlar (backing field). `DateTime?` ve `string?` null olabilir: henüz iptal edilmemiş token'da bu alanlar boştur.

ƒ RefreshToken — Kuralları kontrol ederek geçerli bir refresh token nesnesi oluşturur.

Kurucu; EF Core da veritabanından okurken parametre adları property adlarıyla eşleştiği için bu kurucuyu kullanabilir.

Token kayıtlı bir kullanıcıya ait olmalı; id 0 veya negatifse domain kural ihlali fırlatılır.

```
        if (string.IsNullOrWhiteSpace(tokenHash))
        {
            throw new DomainRuleViolationException(
                "A refresh token hash cannot be empty.");
        }
```

Token hash'i boş olamaz; aksi halde token hiçbir zaman bulunamaz ve kayıt anlamsız olur.

```
        if (expiresAtUtc <= createdAtUtc)
        {
            throw new DomainRuleViolationException(
                "A refresh token must expire after it is created.");
        }
```

Bitiş zamanı oluşturulma zamanından sonra olmalı; zaten süresi dolmuş token oluşturulması engellenir.

```
        UserId = userId;
        TokenHash = tokenHash;
        CreatedAtUtc = createdAtUtc;
        ExpiresAtUtc = expiresAtUtc;
    }
```

Tüm kontroller geçince değerler `private set` property'ler üzerinden atanır.

ƒ Id — Veritabanının ürettiği birincil anahtar.

```
    public int Id
    {
        get
        {
            return _id;
        }
        private set
        {
            _id = value;
        }
    }
```

`private set`: herkes okuyabilir ama sadece sınıfın kendisi (ve EF Core) değer atayabilir. Id'yi veritabanı belirler.

ƒ UserId — Token'ın sahibi olan kullanıcının id'si.

```
    public int UserId
    {
        get
        {
            return _userId;
        }
        private set
        {
            _userId = value;
        }
    }
```

Kullanıcı tablosuna yabancı anahtar; kullanıcı silinirse token'ları da silinir (cascade).

ƒ TokenHash — Ham token'ın SHA-256 hash'i.

```
    public string TokenHash
    {
        get
        {
            return _tokenHash;
        }
        private set
        {
            _tokenHash = value;
        }
    }
```

Veritabanında sadece hash durur; veritabanı sızsa bile saldırgan geçerli bir token elde edemez.

ƒ CreatedAtUtc — Token'ın oluşturulma zamanı (UTC).

```
    public DateTime CreatedAtUtc
    {
        get
        {
            return _createdAtUtc;
        }
        private set
        {
            _createdAtUtc = value;
        }
    }
```

Saat dilimi karışıklığını önlemek için tüm zamanlar UTC olarak saklanır.

ƒ ExpiresAtUtc — Token'ın geçerliliğinin bittiği zaman (UTC).

```
    public DateTime ExpiresAtUtc
    {
        get
        {
            return _expiresAtUtc;
        }
        private set
        {
            _expiresAtUtc = value;
        }
    }
```

Bu zamandan sonra token oturum yenilemek için kullanılamaz.

ƒ RevokedAtUtc — Token'ın iptal edildiği zaman; iptal edilmediyse null.

```
    public DateTime? RevokedAtUtc
    {
        get
        {
            return _revokedAtUtc;
        }
        private set
        {
            _revokedAtUtc = value;
        }
    }
```

Çıkış yapınca veya token yenilenince doldurulur; dolu olması token'ın artık geçersiz olduğunu gösterir.

ƒ ReplacedByTokenHash — Bu token'ın yerine geçen yeni token'ın hash'i.

```
    public string? ReplacedByTokenHash
    {
        get
        {
            return _replacedByTokenHash;
        }
        private set
        {
            _replacedByTokenHash = value;
        }
    }
```

Token rotasyonunda zinciri izlemeye yarar; iptal edilmiş eski token tekrar kullanılırsa çalınma şüphesi anlaşılabilir.

ƒ IsRevoked — Token iptal edildiyse true döner.

```
    public bool IsRevoked
    {
        get
        {
            return RevokedAtUtc != null;
        }
    }
```

Hesaplanan property: veritabanında ayrı sütunu yoktur, `RevokedAtUtc` değerinden türetilir.

ƒ IsActive — Token iptal edilmemiş ve süresi dolmamışsa true döner.

```
    public bool IsActive(DateTime nowUtc)
    {
        return IsRevoked == false && nowUtc < ExpiresAtUtc;
    }
```

Şu anki zaman parametre olarak alınır; böylece metot saate bağımlı olmaz ve testte kolayca denenir.

İki koşul birlikte aranır: iptal edilmemiş olmalı ve şu an bitiş zamanından önce olmalı.

ƒ Revoke — Token'ı iptal eder; isteğe bağlı olarak yerine geçen token'ın hash'ini kaydeder.

```
    public void Revoke(DateTime nowUtc, string? replacedByTokenHash = null)
    {
        if (IsRevoked)
        {
            return;
        }

        RevokedAtUtc = nowUtc;
        ReplacedByTokenHash = replacedByTokenHash;
    }
}
```

`string? replacedByTokenHash = null` : varsayılan değerli opsiyonel parametre; çıkışta verilmez, yenilemede yeni hash verilir.

Zaten iptal edilmişse hiçbir şey yapılmaz; ilk iptal zamanı ve zincir bilgisi korunur (idempotent davranış).

İptal zamanı ve varsa yerine geçen token hash'i kaydedilir; token artık aktif sayılmaz.

Application `todo.Application/Contracts/AuthResultDto.cs`

> Başarılı giriş veya yenilemeden sonra istemciye JSON olarak dönen sonuç: access token, bitiş zamanı ve kullanıcı bilgisi. Refresh token bunun içinde değildir, cookie ile gider.

```
using System;
using TodoApi.Application.Abstractions.Authentication;
```

`DateTime` için `System`, `AccessToken` tipi için kimlik doğrulama portları isim alanı.

```
namespace TodoApi.Application.Contracts;
```

Dışarıya açılan veri sözleşmelerinin (DTO) isim alanı.

ƒ class AuthResultDto — Giriş sonucunu istemciye taşıyan değişmez DTO.

```
public class AuthResultDto
{
    private readonly string _accessToken;
    private readonly DateTime _expiresAtUtc;
    private readonly UserDto _user;
```

DTO (Data Transfer Object): katmanlar arası veya API cevabı olarak sadece veri taşıyan sınıf.

Değişmez alanlar: JWT metni, bitiş zamanı ve kullanıcı bilgisi.

ƒ AuthResultDto(AccessToken, UserDto) — Access token ve kullanıcı DTO'sundan sonucu oluşturan yapıcı.

```
    public AuthResultDto(AccessToken accessToken, UserDto user)
    {
```

Yapıcı, port sonucu olan `AccessToken` ile `UserDto`'yu alır.

```
        if (accessToken == null)
        {
            throw new ArgumentNullException(nameof(accessToken));
        }

        if (user == null)
        {
            throw new ArgumentNullException(nameof(user));
        }

        _accessToken = accessToken.Value;
        _expiresAtUtc = accessToken.ExpiresAtUtc;
        _user = user;
    }
    public string AccessToken
    {
        get
        {
            return _accessToken;
        }
    }
    public DateTime ExpiresAtUtc
    {
        get
        {
            return _expiresAtUtc;
        }
    }
    public UserDto User
    {
        get
        {
            return _user;
        }
    }
}
```

İki parametre için null kontrolü; eksikse `ArgumentNullException`.

`AccessToken` nesnesi düz alanlara açılır; JSON'da `accessToken` ve `expiresAtUtc` olarak görünür.

Sadece okunabilen `AccessToken` özelliği; istemci bunu `Authorization: Bearer` başlığında gönderir.

`ExpiresAtUtc`: access token'ın bitiş zamanı; Angular tarafı yenileme zamanını bununla bilir.

`User`: giriş yapan kullanıcının özet bilgisi.

Application `todo.Application/Contracts/UserDto.cs`

> Kullanıcının dışarıya gösterilebilir bilgilerini taşıyan DTO. Domain'deki `User` doğrudan dönülmez; şifre hash'i gibi hassas veriler böylece asla dışarı çıkmaz.

```
using System;
using TodoApi.Domain.Entities;
using TodoApi.Domain.Enums;

namespace TodoApi.Application.Contracts;
```

`DateTime` için `System`, ayrıca `User` varlığı ve `UserRole` enum'u için Domain isim alanları.

Veri sözleşmelerinin (DTO) isim alanı.

ƒ class UserDto — Kullanıcının güvenle paylaşılabilen bilgilerini taşıyan değişmez DTO.

```
public class UserDto
{
    private readonly int _id;
    private readonly string _email;
    private readonly string _displayName;
    private readonly UserRole _role;
    private readonly bool _hasPassword;
    private readonly bool _hasGoogleLogin;
    private readonly DateTime _createdAtUtc;
```

API cevaplarında kullanıcıyı temsil eden veri sınıfı.

Değişmez alanlar: kimlik, e-posta, ad, rol, şifresi var mı, Google bağlı mı, oluşturulma zamanı. Şifre hash'i ve Google subject id bilinçli olarak yok.

ƒ UserDto(User) — Domain `User` nesnesinden güvenli alanları kopyalayan yapıcı.

```
    public UserDto(User user)
    {
        if (user == null)
        {
            throw new ArgumentNullException(nameof(user));
        }

        _id = user.Id;
        _email = user.Email;
        _displayName = user.DisplayName;
        _role = user.Role;
        _hasPassword = user.HasPassword;
        _hasGoogleLogin = user.HasGoogleLogin;
        _createdAtUtc = user.CreatedAtUtc;
    }
    public int Id
    {
        get
        {
            return _id;
        }
    }
```

Yapıcı bir `User` alır ve gerekli alanları kopyalar (mapping).

Kullanıcı null ise `ArgumentNullException` fırlatılır.

Domain nesnesindeki değerler tek tek DTO alanlarına kopyalanır; `HasPassword` ve `HasGoogleLogin` sadece evet/hayır bilgisidir.

Sadece okunabilen `Id` özelliği.

```
    public string Email
    {
        get
        {
            return _email;
        }
    }
```

`Email` özelliği: normalize edilmiş e-posta.

```
    public string DisplayName
    {
        get
        {
            return _displayName;
        }
    }
```

`DisplayName` özelliği: ekranda gösterilecek ad.

```
    public UserRole Role
    {
        get
        {
            return _role;
        }
    }
```

`Role` özelliği: `User` veya `Admin`; Angular admin menüsünü göstermek için kullanabilir.

```
    public bool HasPassword
    {
        get
        {
            return _hasPassword;
        }
    }
```

`HasPassword`: hesabın şifreyle girişi var mı?

```
    public bool HasGoogleLogin
    {
        get
        {
            return _hasGoogleLogin;
        }
    }
```

`HasGoogleLogin`: hesap bir Google hesabına bağlı mı?

```
    public DateTime CreatedAtUtc
    {
        get
        {
            return _createdAtUtc;
        }
    }
}
```

`CreatedAtUtc`: hesabın oluşturulduğu UTC zaman.

Application `todo.Application/Exceptions/AuthenticationFailedException.cs`

> Kimlik doğrulama başarısız olduğunda (yanlış şifre, geçersiz Google token, geçersiz oturum) fırlatılan uygulama hatası. API'deki `ApplicationExceptionHandler` bunu 401 cevabına çevirir.

```
using System;
```

Temel `Exception` sınıfı için `System` isim alanı.

```
namespace TodoApi.Application.Exceptions;
```

Application katmanına özel hata tiplerinin isim alanı.

ƒ class AuthenticationFailedException — Giriş veya oturum doğrulaması başarısız olunca kullanılan özel hata tipi.

```
public sealed class AuthenticationFailedException : Exception
```

`: Exception` miras demektir. Özel tip sayesinde API bu hatayı tanıyıp 401 Unauthorized döndürebilir.

```
{
```

ƒ AuthenticationFailedException(string) — Mesajı alıp temel `Exception` sınıfına ileten yapıcı.

```
    public AuthenticationFailedException(string message) : base(message)
```

`: base(message)` mesajı üst sınıfın yapıcısına iletir; ek bir iş yapılmaz.

```
    {
    }
}
```

Application `todo.Application/Features/Authentication/RefreshSession/RefreshSessionUseCase.cs`

> Süresi dolan 15 dakikalık access token'ı, cookie'deki refresh token ile yeniler. Her yenilemede refresh token döndürülür (rotation) ve çalınmış token tekrar kullanılırsa tüm oturumlar kapatılır.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
```

Temel .NET tipleri: `TimeSpan`, `DateTime`, listeler, `Task` ve `CancellationToken`.

```
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Application.Exceptions;
using TodoApi.Application.Features.Authentication.Common;
using TodoApi.Domain.Entities;
```

Port arayüzleri, hata tipleri, ortak oturum sınıfları ve Domain varlıkları (`User`, `RefreshToken`).

```
namespace TodoApi.Application.Features.Authentication.RefreshSession;
```

Bu use case'in isim alanı.

ƒ class RefreshSessionUseCase — Refresh token ile oturumu yenileyen ve token hırsızlığını tespit eden use case.

```
public class RefreshSessionUseCase
```

Oturum yenileme senaryosunu yöneten use case sınıfı.

```
{
```

```
    private const string InvalidSessionMessage = "The session has expired. Please sign in again.";
```

Tüm hata durumlarında aynı mesaj kullanılır; saldırgana token'ın neden reddedildiği hakkında ipucu verilmez.

```
    private static readonly TimeSpan ConcurrentRefreshGracePeriod = TimeSpan.FromSeconds(30);
```

`static readonly`: tüm nesnelerce paylaşılan sabit değer. 30 saniyelik tolerans, aynı anda gelen iki meşru yenileme isteğini hırsızlık sanmamak içindir.

```
    private readonly IRefreshTokenRepository _refreshTokenRepository;
    private readonly IUserRepository _userRepository;
    private readonly ISecureTokenGenerator _secureTokenGenerator;
    private readonly SessionIssuer _sessionIssuer;
    private readonly TimeProvider _timeProvider;
```

Bağımlılıkları tutan değişmez alanlar: token ve kullanıcı depoları, hash üretici, oturum üretici, saat.

ƒ RefreshSessionUseCase(...) — Bağımlılıkları DI ile alan yapıcı metot.

```
    public RefreshSessionUseCase(
        IRefreshTokenRepository refreshTokenRepository,
        IUserRepository userRepository,
        ISecureTokenGenerator secureTokenGenerator,
        SessionIssuer sessionIssuer,
        TimeProvider timeProvider)
```

Yapıcı, beş bağımlılığı DI konteynerinden alır.

```
    {
```

```
        if (refreshTokenRepository == null)
        {
            throw new ArgumentNullException(nameof(refreshTokenRepository));
        }

        if (userRepository == null)
        {
            throw new ArgumentNullException(nameof(userRepository));
        }

        if (secureTokenGenerator == null)
        {
            throw new ArgumentNullException(nameof(secureTokenGenerator));
        }
```

Her bağımlılık için null kontrolü; eksik kayıt varsa uygulama başlarken hemen hata verir.

```
        if (sessionIssuer == null)
        {
            throw new ArgumentNullException(nameof(sessionIssuer));
        }

        if (timeProvider == null)
        {
            throw new ArgumentNullException(nameof(timeProvider));
        }
```

Kalan bağımlılıklar için aynı null kontrolü ve `ArgumentNullException`.

```
        _refreshTokenRepository = refreshTokenRepository;
        _userRepository = userRepository;
        _secureTokenGenerator = secureTokenGenerator;
        _sessionIssuer = sessionIssuer;
        _timeProvider = timeProvider;
```

Doğrulanan bağımlılıklar özel alanlara atanır.

```
    }
```

ƒ ExecuteAsync — Refresh token'ı doğrular, döndürür ve yeni oturum üretir.

```
    public async Task<AuthSession> ExecuteAsync(
        string? refreshTokenValue,
        CancellationToken cancellationToken = default)
```

`string?` token'ın null gelebileceğini belirtir (cookie yoksa). Sonuç olarak yeni `AuthSession` döner.

```
    {
```

```
        if (string.IsNullOrWhiteSpace(refreshTokenValue))
        {
            throw new AuthenticationFailedException(InvalidSessionMessage);
        }
```

Token hiç gelmemişse veya boşsa oturum geçersiz sayılır ve 401'e dönüşen hata fırlatılır.

```
        string tokenHash = _secureTokenGenerator.ComputeHash(refreshTokenValue);
        RefreshToken? storedToken = await _refreshTokenRepository.GetByTokenHashAsync(
            tokenHash,
            cancellationToken);
```

Veritabanında token'ın kendisi değil SHA-256 hash'i tutulur. Gelen değerin hash'i hesaplanıp bu hash ile arama yapılır; veritabanı sızsa bile token'lar kullanılamaz.

```
        if (storedToken == null)
        {
            throw new AuthenticationFailedException(InvalidSessionMessage);
        }
```

Bu hash'e sahip kayıt yoksa token sahte veya çok eski demektir; giriş reddedilir.

```
        DateTime nowUtc = _timeProvider.GetUtcNow().UtcDateTime;
```

Tüm zaman karşılaştırmaları için şu anki UTC zaman bir kez alınır.

```
        if (storedToken.IsRevoked)
```

Token daha önce iptal edilmiş (örneğin döndürülmüş) ama yine de kullanılmaya çalışılıyor.

```
        {
```

```
            bool wasJustRotated =
                storedToken.ReplacedByTokenHash != null &&
                storedToken.RevokedAtUtc.HasValue &&
                nowUtc - storedToken.RevokedAtUtc.Value <= ConcurrentRefreshGracePeriod;
```

Yeni bir token ile değiştirilmiş ve iptali 30 saniyeden yeni ise bu muhtemelen eşzamanlı meşru istektir. `HasValue` nullable tarihin dolu olup olmadığını kontrol eder.

```
            if (wasJustRotated == false)
            {
                await RevokeAllSessionsAsync(storedToken.UserId, nowUtc, cancellationToken);
            }
```

Tolerans dışında eski token kullanımı hırsızlık işaretidir; kullanıcının tüm oturumları iptal edilir.

```
            throw new AuthenticationFailedException(InvalidSessionMessage);
```
Her iki durumda da bu istek reddedilir; iptal edilmiş token asla yeni oturum vermez.

```
        }
```

```
        if (storedToken.IsActive(nowUtc) == false)
        {
            throw new AuthenticationFailedException(InvalidSessionMessage);
        }
```
İptal edilmemiş ama süresi (7 gün) dolmuş token da reddedilir.

```
        User? user = await _userRepository.GetByIdAsync(storedToken.UserId, cancellationToken);

        if (user == null)
        {
            throw new AuthenticationFailedException(InvalidSessionMessage);
        }
```
Token'ın sahibi olan kullanıcı yüklenir; silinmişse oturum geçersiz sayılır.

```
        AuthSession session = await _sessionIssuer.IssueAsync(user, cancellationToken, storedToken);

        return session;
    }
```
Yeni token çifti üretilir; eski token `replacedToken` olarak verilir ki iptal edilip yenisine bağlansın (rotation).

ƒ RevokeAllSessionsAsync — Hırsızlık şüphesinde kullanıcının tüm aktif oturumlarını iptal edip kaydeder.

```
    private async Task RevokeAllSessionsAsync(
        int userId,
        DateTime nowUtc,
        CancellationToken cancellationToken)
    {
        IReadOnlyList<RefreshToken> activeTokens = await _refreshTokenRepository.GetActiveForUserAsync(
            userId,
            nowUtc,
            cancellationToken);
```
Sadece bu sınıfın kullandığı özel asenkron yardımcı metot; kullanıcı kimliği ve zamanı alır.

Kullanıcının hâlâ aktif tüm refresh token'ları getirilir.

```
        foreach (RefreshToken activeToken in activeTokens)
        {
            activeToken.Revoke(nowUtc);
        }
```
`foreach` ile her token tek tek iptal edilir; saldırgan ve gerçek kullanıcı tekrar giriş yapmak zorunda kalır.

```
        await _refreshTokenRepository.SaveChangesAsync(cancellationToken);
```
İptaller hemen kaydedilir; çünkü ardından hata fırlatılacak ve başka kayıt noktası olmayacak.

```
    }
}
```

Application `todo.Application/Features/Authentication/Logout/LogoutUseCase.cs`

> Çıkış yapıldığında cookie'deki refresh token'ı veritabanında iptal eder; böylece o oturum bir daha yenilenemez.

```
using System;
using System.Threading;
using System.Threading.Tasks;
```
Temel .NET tipleri: `TimeProvider`, `Task` ve `CancellationToken`.

```
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Domain.Entities;
```
Port arayüzleri (`ISecureTokenGenerator`, `IRefreshTokenRepository`) ve `RefreshToken` Domain varlığı.

```
namespace TodoApi.Application.Features.Authentication.Logout;
```
Bu use case'in isim alanı.

ƒ class LogoutUseCase — Refresh token'ı iptal ederek oturumu kapatan use case.

```
public class LogoutUseCase
{
    private readonly IRefreshTokenRepository _refreshTokenRepository;
    private readonly ISecureTokenGenerator _secureTokenGenerator;
    private readonly TimeProvider _timeProvider;
```
Çıkış senaryosunu yöneten use case sınıfı.

`private readonly` bağımlılık alanları: token deposu, hash üretici ve saat.

ƒ LogoutUseCase(...) — Bağımlılıkları DI ile alan yapıcı metot.

```
    public LogoutUseCase(
        IRefreshTokenRepository refreshTokenRepository,
        ISecureTokenGenerator secureTokenGenerator,
        TimeProvider timeProvider)
    {
        if (refreshTokenRepository == null)
        {
            throw new ArgumentNullException(nameof(refreshTokenRepository));
        }

        if (secureTokenGenerator == null)
        {
            throw new ArgumentNullException(nameof(secureTokenGenerator));
        }

        if (timeProvider == null)
        {
            throw new ArgumentNullException(nameof(timeProvider));
        }
```
Yapıcı üç bağımlılığı DI konteynerinden alır.

Her bağımlılık için null kontrolü; null ise `ArgumentNullException` fırlatılır.

```
        _refreshTokenRepository = refreshTokenRepository;
        _secureTokenGenerator = secureTokenGenerator;
        _timeProvider = timeProvider;
    }
```
Bağımlılıklar özel alanlara kaydedilir.

ƒ ExecuteAsync — Verilen refresh token'ı bulup iptal eder; yoksa sessizce çıkar.

```
    public async Task ExecuteAsync(
        string? refreshTokenValue,
        CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrWhiteSpace(refreshTokenValue))
        {
            return;
        }

        string tokenHash = _secureTokenGenerator.ComputeHash(refreshTokenValue);
        RefreshToken? storedToken = await _refreshTokenRepository.GetByTokenHashAsync(
            tokenHash,
            cancellationToken);

        if (storedToken == null || storedToken.IsRevoked)
        {
            return;
        }

        storedToken.Revoke(_timeProvider.GetUtcNow().UtcDateTime);

        await _refreshTokenRepository.SaveChangesAsync(cancellationToken);
    }
}
```

Değer döndürmeyen asenkron metot (`Task`). Token null gelebilir (`string?`).

Token yoksa yapacak bir şey yok; `return` ile sessizce çıkılır. Çıkış her zaman başarılı görünmeli.

Token'ın SHA-256 hash'i hesaplanır ve veritabanında bu hash ile kayıt aranır; düz token hiç saklanmaz.

`||` (veya): kayıt yoksa ya da zaten iptal edilmişse işlem gereksizdir, metot biter.

Domain metodu token'ı şu anki UTC zamanla iptal edilmiş olarak işaretler.

Değişiklik veritabanına kaydedilir; artık bu token ile oturum yenilenemez.

Application `todo.Application/Features/Authentication/GetCurrentUser/GetCurrentUserUseCase.cs`

> Giriş yapmış kullanıcının bilgilerini ("ben kimim?") döndürür. Kullanıcı kimliğini JWT'den okuyan `ICurrentUser` portunu kullanır.

```
using System;
using System.Threading;
using System.Threading.Tasks;
```

Temel .NET tipleri: `ArgumentNullException`, `Task` ve `CancellationToken`.

```
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Application.Contracts;
using TodoApi.Domain.Entities;
```

Port arayüzleri, dışarıya dönen `UserDto` sözleşmesi ve `User` Domain varlığı.

```
namespace TodoApi.Application.Features.Authentication.GetCurrentUser;
```

Bu use case'in isim alanı.

ƒ class GetCurrentUserUseCase — Oturumdaki kullanıcının profil bilgisini getiren use case.

```
public class GetCurrentUserUseCase
{
    private readonly IUserRepository _userRepository;
    private readonly ICurrentUser _currentUser;
```

Mevcut kullanıcıyı getirme senaryosunu yöneten sınıf.

Kullanıcı deposu ve o anki isteğin kullanıcısını veren `ICurrentUser` portu.

ƒ GetCurrentUserUseCase(...) — Depo ve mevcut kullanıcı bağımlılıklarını alan yapıcı.

```
    public GetCurrentUserUseCase(IUserRepository userRepository, ICurrentUser currentUser)
    {
        if (userRepository == null)
        {
            throw new ArgumentNullException(nameof(userRepository));
        }

        if (currentUser == null)
        {
            throw new ArgumentNullException(nameof(currentUser));
        }

        _userRepository = userRepository;
        _currentUser = currentUser;
    }
```

Yapıcı iki bağımlılığı DI konteynerinden alır.

Her iki bağımlılık için null kontrolü yapılır.

Bağımlılıklar özel alanlara kaydedilir.

ƒ ExecuteAsync — Token'daki kullanıcı kimliğiyle kullanıcıyı bulup `UserDto` döndürür.

```
    public async Task<UserDto?> ExecuteAsync(CancellationToken cancellationToken = default)
    {
        User? user = await _userRepository.GetByIdAsync(_currentUser.UserId, cancellationToken);

        if (user == null)
        {
            return null;
        }

        UserDto response = new UserDto(user);

        return response;
    }
}
```

`Task<UserDto?>`: asenkron çalışır, kullanıcı bulunamazsa null dönebilir.

`_currentUser.UserId` JWT'deki `sub` bilgisinden gelir; kullanıcı bu kimlikle veritabanından yüklenir.

Kullanıcı silinmişse null döner; controller bunu 401 Unauthorized cevabına çevirir.

Domain varlığı doğrudan dışarı verilmez; sadece gereken alanları içeren `UserDto`'ya dönüştürülüp döndürülür.

Application `todo.Application/Abstractions/Authentication/ICurrentUser.cs`

> O anki isteği yapan kullanıcıyı soran port. Application HTTP'yi bilmez; API katmanındaki `HttpContextCurrentUser` bilgiyi JWT claim'lerinden okur.

```
1  namespace TodoApi.Application.Abstractions.Authentication;
```

Kimlik doğrulama portlarının isim alanı.

ƒ interface ICurrentUser — Giriş yapmış kullanıcının kimliğini ve admin olup olmadığını veren sözleşme.

```
3  public interface ICurrentUser
```

`interface`: uygulaması başka katmanda olan sözleşme. Testlerde sahte (`FakeCurrentUser`) bir sürümü kullanılabilir.

```
4  {
```

```
5      int UserId { get; }
```

`{ get; }` sadece okunabilen özellik demektir. `UserId` JWT'deki `sub` claim'inden gelen kullanıcı kimliğidir.

```
7      bool IsAdmin { get; }
```

`IsAdmin`: kullanıcının token'ında Admin rolü var mı? Yetki kontrollerinde kullanılır.

```
8  }
```

API `todo.API/Authentication/HttpContextCurrentUser.cs`

> Application katmanındaki `ICurrentUser` arayüzünün API katmanındaki uygulaması. Giriş yapan kullanıcının id'sini ve admin olup olmadığını JWT içindeki claim'lerden okur.

```
1  using System.Security.Claims;
2  using Microsoft.IdentityModel.JsonWebTokens;
3  using TodoApi.Application.Abstractions.Authentication;
4  using TodoApi.Domain.Enums;
```

Claim (token içindeki bilgi) tipleri, JWT standart claim adları, Application arayüzü ve `UserRole` enum'u içeri alınır.

```
6  namespace TodoApi.API.Authentication;
```

Dosyanın ad alanı.

ƒ HttpContextCurrentUser — Mevcut HTTP isteğindeki doğrulanmış kullanıcıdan id ve admin bilgisini okuyan ICurrentUser uygulaması.

```
8  public sealed class HttpContextCurrentUser : ICurrentUser
```

Arayüz Application'da, uygulaması API'de durur; böylece use case'ler HTTP'yi bilmeden kullanıcıyı öğrenir (bağımlılık tersine çevirme).

```
9  {
```

```
10     private readonly IHttpContextAccessor _httpContextAccessor;
```

`IHttpContextAccessor` o anki HTTP isteğine (HttpContext) her yerden erişmeyi sağlayan servistir.

ƒ HttpContextCurrentUser (constructor) — HttpContext erişim servisini DI ile alır ve null değilse saklar.

```
12     public HttpContextCurrentUser(IHttpContextAccessor httpContextAccessor)
```

Kurucu metot; erişim servisi Dependency Injection ile gelir.

```
13     {
14         if (httpContextAccessor == null)
15         {
16             throw new ArgumentNullException(nameof(httpContextAccessor));
17         }
```

Bağımlılık null gelirse hemen hata fırlatır; hata geç ve anlaşılmaz bir yerde çıkmasın diye. `nameof` parametre adını metne çevirir.

```
19         _httpContextAccessor = httpContextAccessor;
```

Servisi alana kaydeder.

```
20     }
```

ƒ UserId — Access token'daki `sub` claim'inden kullanıcı id'sini okuyup sayıya çevirir.

```
22     public int UserId
```

Sadece okunabilir (`get`) özellik; giriş yapan kullanıcının veritabanı id'sini verir.

```
23     {
24         get
```

`get` bloğu: özellik her okunduğunda bu kod çalışır.

```
25         {
26             ClaimsPrincipal principal = GetAuthenticatedPrincipal();
```

Doğrulanmış kullanıcıyı (ClaimsPrincipal) alır; giriş yoksa yardımcı metot hata fırlatır.

```
27             string? subjectClaimValue = principal.FindFirstValue(JwtRegisteredClai
   mNames.Sub);
```

JWT'nin standart `sub` (subject) claim'ini okur; token üretilirken kullanıcı id'si buraya yazılır.

```
28             bool isValidUserId = int.TryParse(subjectClaimValue, out int userId);
```

`int.TryParse` metni sayıya çevirmeyi dener, başarısızsa istisna yerine false döner; sonuç `out` ile `userId`'ye yazılır.

```
30             if (isValidUserId == false)
31             {
32                 throw new InvalidOperationException(
33                     "The access token does not contain a valid user id.");
34             }
```

Id geçersizse bu bir programlama/yapılandırma hatasıdır; `InvalidOperationException` ile açıkça bildirilir.

```
36             return userId;
```

Geçerli kullanıcı id'sini döner.

```
37         }
38     }
```

ƒ IsAdmin — Token'daki rol claim'ine bakarak kullanıcının admin olup olmadığını döner.

```
40     public bool IsAdmin
```

Kullanıcının admin olup olmadığını veren salt okunur özellik.

```
41     {
42         get
```

Özellik okunduğunda çalışan `get` bloğu.

```
43         {
44             ClaimsPrincipal principal = GetAuthenticatedPrincipal();
45
46             return principal.IsInRole(UserRole.Admin.ToString());
```

Doğrulanmış kullanıcıyı alır ve `IsInRole` ile rol claim'inde "Admin" olup olmadığını kontrol eder.

```
47         }
48     }
```

ƒ GetAuthenticatedPrincipal — O anki istekte doğrulanmış kullanıcıyı döner; yoksa hata fırlatır.

```
    private ClaimsPrincipal GetAuthenticatedPrincipal()
    {
        HttpContext? httpContext = _httpContextAccessor.HttpContext;

        if (httpContext == null || httpContext.User.Identity?.IsAuthenticated != true)
        {
            throw new InvalidOperationException(
                "The current user was requested outside of an authenticated request.");
        }

        return httpContext.User;
    }
}
```

İki özelliğin ortak kullandığı özel yardımcı metot.

O anki HttpContext'i alır; istek dışında (ör. arka plan işi) null olabilir, bu yüzden `HttpContext?`.

Context yoksa veya kullanıcı doğrulanmamışsa hata fırlatır. `?.` null ise durup null döner; `!= true` null'ı da yakalar.

Doğrulanmış kullanıcının claim'lerini taşıyan `ClaimsPrincipal` nesnesini döner.

Application `todo.Application/Features/Authentication/RegisterWithPassword/RegisterWithPasswordCommand.cs`

Şifreyle kayıt use case'inin girdi nesnesi. Controller HTTP isteğinden bu komutu oluşturur.

```
namespace TodoApi.Application.Features.Authentication.RegisterWithPassword;
```

Kayıt özelliğinin ad alanı; ilgili command, validator ve use case aynı klasördedir.

ƒ RegisterWithPasswordCommand — Kayıt için e-posta, görünen ad ve şifreyi taşıyan değişmez komut.

```
public sealed class RegisterWithPasswordCommand
{
    public string Email { get; init; } = string.Empty;

    public string DisplayName { get; init; } = string.Empty;

    public string Password { get; init; } = string.Empty;
}
```

Use case'e verilecek veriyi taşıyan `sealed` sınıf; rol alanı içermez, rol her zaman User olur.

E-posta adresi; `init` sayesinde sadece nesne oluşturulurken atanabilir, sonradan değiştirilemez.

Kullanıcının görünen adı.

Düz şifre; use case içinde hash'lenir.

Application `todo.Application/Features/Authentication/RegisterWithPassword/RegisterWithPasswordCommandValidator.cs`

Kayıt komutu için FluentValidation kuralları: geçerli e-posta, dolu ad ve güçlü şifre. Kurallar ihlal edilirse use case 400 hatasıyla durur.

```
using System.Linq;
using FluentValidation;

namespace TodoApi.Application.Features.Authentication.RegisterWithPassword;
```

LINQ (`Any` gibi koleksiyon sorguları) ve FluentValidation içeri alınır.

Kayıt özelliğinin ad alanı.

ƒ RegisterWithPasswordCommandValidator — Kayıt komutundaki e-posta, ad ve şifre için doğrulama kurallarını tanımlar.

```
public sealed class RegisterWithPasswordCommandValidator : AbstractValidator<RegisterWithPasswordCommand>
{
    public const int PasswordMinimumLength = 8;
    public const int PasswordMaximumLength = 128;
```

`AbstractValidator<RegisterWithPasswordCommand>`'dan türeyen doğrulayıcı sınıf.

Şifre uzunluk sınırları `public const` olarak tutulur; başka yerler (ör. testler) aynı değeri kullanabilir.

ƒ RegisterWithPasswordCommandValidator (constructor) — Tüm kayıt kurallarını kurucu içinde kaydeder.

```
    public RegisterWithPasswordCommandValidator()
    {
        RuleFor(command => command.Email)
            .NotEmpty()
            .EmailAddress()
            .MaximumLength(256)
            .WithName("E-posta");

        RuleFor(command => command.DisplayName)
            .NotEmpty()
            .MaximumLength(100)
            .WithName("Ad");

        RuleFor(command => command.Password)
            .NotEmpty()
            .Length(PasswordMinimumLength, PasswordMaximumLength)
            .Must(password => password.Any(char.IsLetter) && password.Any(char.IsDigit))
            .WithMessage("Şifre en az bir harf ve bir rakam içermelidir.")
            .WithName("Şifre");
    }
}
```

Kurallar kurucu metotta tanımlanır.

E-posta dolu, geçerli e-posta biçiminde ve en fazla 256 karakter olmalı; hata mesajında "E-posta" adı kullanılır.

Görünen ad dolu olmalı ve en fazla 100 karakter olabilir.

Şifre dolu olmalı ve 8 ile 128 karakter arasında olmalı; üst sınır aşırı uzun şifreyle sunucuyu yorma saldırısını önler.

`Must` özel kural: şifrede en az bir harf ve bir rakam olmalı. `Any(char.IsLetter)` her karakteri kontrol eder; hata mesajı Türkçe verilir.

Application `todo.Application/Features/Authentication/RegisterWithPassword/RegisterWithPasswordUseCase.cs`

E-posta ve şifreyle yeni kullanıcı kaydının iş akışı: doğrular, e-posta tekrarını kontrol eder, şifreyi hash'ler, kullanıcıyı kaydeder ve oturum açar. Rol her zaman

User'dır.

```
using System;
using System.Threading;
using System.Threading.Tasks;
```

Temel .NET tipleri, iptal belirteci ve asenkron `Task` tipleri içeri alınır.

```
using FluentValidation;
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Application.Common.Validation;
using TodoApi.Application.Exceptions;
using TodoApi.Application.Features.Authentication.Common;
using TodoApi.Domain.Entities;
```

FluentValidation, Application arayüzleri, doğrulama yardımcısı, özel istisnalar, ortak oturum sınıfları ve Domain'deki `User` varlığı içeri alınır.

```
namespace TodoApi.Application.Features.Authentication.RegisterWithPassword;
```

Kayıt özelliğinin ad alanı.

ƒ RegisterWithPasswordUseCase — Şifreyle kayıt senaryosunu yöneten use case sınıfı.

```
public class RegisterWithPasswordUseCase
```

Tek bir iş senaryosunu (şifreyle kayıt) temsil eden use case sınıfı.

```
{
```

```
    private readonly IUserRepository _userRepository;
    private readonly IPasswordHasher _passwordHasher;
    private readonly SessionIssuer _sessionIssuer;
    private readonly IValidator<RegisterWithPasswordCommand> _validator;
```

`private readonly` bağımlılıklar: kullanıcı deposu, şifre hash'leyici, oturum üretici ve komut doğrulayıcı.

ƒ RegisterWithPasswordUseCase (constructor) — Bağımlılıkları DI ile alır, null kontrolü yapıp alanlara kaydeder.

```
    public RegisterWithPasswordUseCase(
        IUserRepository userRepository,
        IPasswordHasher passwordHasher,
        SessionIssuer sessionIssuer,
        IValidator<RegisterWithPasswordCommand> validator)
```

Kurucu metot; parametreler Dependency Injection ile gelir.

```
    {
        if (userRepository == null)
        {
            throw new ArgumentNullException(nameof(userRepository));
        }

        if (passwordHasher == null)
        {
            throw new ArgumentNullException(nameof(passwordHasher));
        }

        if (sessionIssuer == null)
        {
            throw new ArgumentNullException(nameof(sessionIssuer));
        }

        if (validator == null)
        {
            throw new ArgumentNullException(nameof(validator));
        }
```

Her bağımlılık null mı diye kontrol edilir; eksikse hemen `ArgumentNullException` fırlatılır. `nameof` parametre adını metne çevirir.

```
        _userRepository = userRepository;
        _passwordHasher = passwordHasher;
        _sessionIssuer = sessionIssuer;
        _validator = validator;
    }
```

Bağımlılıklar `readonly` alanlara atanır.

ƒ ExecuteAsync — Komutu doğrular, e-posta tekrarını engeller, kullanıcıyı oluşturup kaydeder ve oturum döner.

```
    public async Task<AuthSession> ExecuteAsync(
        RegisterWithPasswordCommand command,
        CancellationToken cancellationToken = default)
```

Asenkron ana metot; başarılı olursa yeni kullanıcının `AuthSession` nesnesini döner.

```
    {
        await _validator.ValidateOrThrowAsync(command, cancellationToken);
```

Önce FluentValidation kuralları çalışır; geçersizse istisna fırlar ve API 400 döner.

```
        string normalizedEmail = EmailNormalizer.Normalize(command.Email);
```

E-posta küçük harfe çevrilip kırpılır; `Ali@x.com` ve `ali@x.com` aynı hesap sayılır.

```
        User? existingUser = await _userRepository.GetByEmailAsync(
            normalizedEmail,
            cancellationToken);
```

Bu e-postayla kayıtlı kullanıcı var mı diye veritabanına bakılır.

```
        if (existingUser != null)
        {
            throw new EmailAlreadyRegisteredException();
        }
```

E-posta zaten kayıtlıysa `EmailAlreadyRegisteredException` fırlatılır; API bunu 409 Conflict yanıtına çevirir.

```
        string passwordHash = _passwordHasher.HashPassword(command.Password);
```

Şifre PBKDF2 ile tuzlanarak hash'lenir; veritabanına asla düz şifre yazılmaz.

```
        User newUser = User.RegisterWithPassword(
            normalizedEmail,
            command.DisplayName,
            passwordHash);
```

Domain'deki fabrika metodu kullanıcıyı oluşturur ve rolü her zaman User yapar; admin sadece Google ile doğrulanmış e-postayla olunabilir.

```
        await _userRepository.AddAsync(newUser, cancellationToken);
        await _userRepository.SaveChangesAsync(cancellationToken);
```

Yeni kullanıcı depoya eklenir ve değişiklikler veritabanına kaydedilir; kullanıcı böylece bir Id alır.

```
        AuthSession session = await _sessionIssuer.IssueAsync(newUser, cancellationToken);

        return session;
```

Kayıttan hemen sonra oturum açılır: `SessionIssuer` access token ve refresh token üretir, sonuç controller'a döner.

```
    }
}
```

Application `todo.Application/Exceptions/EmailAlreadyRegisteredException.cs`

> Şifreyle kayıt olurken e-posta zaten kullanılıyorsa fırlatılan uygulama hatası. `ApplicationExceptionHandler` bunu 409 Conflict cevabına çevirir.

```
using System;
```

Temel `Exception` sınıfı için `System` isim alanı.

```
namespace TodoApi.Application.Exceptions;
```

Application katmanına özel hata tiplerinin isim alanı.

ƒ class EmailAlreadyRegisteredException — E-posta adresi zaten kayıtlıyken kullanılan özel hata tipi.

```
public sealed class EmailAlreadyRegisteredException : Exception
```

`sealed` ve `Exception`'dan türeyen özel hata; ayrı tip olması ona özel HTTP koduna (409) eşlenmesini sağlar.

```
{
```

ƒ EmailAlreadyRegisteredException() — Sabit hata mesajıyla nesneyi oluşturan parametresiz yapıcı.

```
    public EmailAlreadyRegisteredException()
        : base("An account with this email address already exists.")
```

Parametresiz yapıcı; `: base(...)` ile her zaman aynı sabit mesajı üst sınıfa iletir.

```
    {
    }
}
```

Application `todo.Application/Features/Authentication/LoginWithGoogle/LoginWithGoogleCommand.cs`

> Google ile giriş isteğinin taşıdığı veriyi (Google'ın verdiği ID token) tutan komut nesnesi. Controller bunu doldurup `LoginWithGoogleUseCase`'e verir.

```
namespace TodoApi.Application.Features.Authentication.LoginWithGoogle;
```

Dosya-kapsamlı `namespace`: bu dosyadaki tüm tipler bu isim alanına ait olur, ekstra süslü parantez gerekmez.

ƒ class LoginWithGoogleCommand — Google girişi için istemciden gelen ID token'ı taşıyan basit veri sınıfı.

```
public sealed class LoginWithGoogleCommand
```

`sealed`: bu sınıftan miras alınamaz. Komut sadece veri taşır, davranış içermez.

```
{
    public string IdToken { get; init; } = string.Empty;
```

`IdToken`: Google'ın tarayıcıda verdiği imzalı kimlik belgesi. `init` sadece nesne oluşturulurken atanabilir demek; boş string varsayılanı null riskini önler.

```
}
```

Application `todo.Application/Features/Authentication/LoginWithGoogle/LoginWithGoogleCommandValidator.cs`

> Google giriş komutunun biçimsel doğrulamasını FluentValidation ile yapar. Token'ın gerçekten geçerli olup olmadığına değil, boş veya aşırı uzun olmadığına bakar.

```
using FluentValidation;
```

FluentValidation kütüphanesini kullanıma açar; kurallar akıcı (zincirleme) bir sözdizimiyle yazılır.

```
namespace TodoApi.Application.Features.Authentication.LoginWithGoogle;
```

Komutla aynı isim alanında durur; özellik (feature) klasörü yapısı korunur.

ƒ class LoginWithGoogleCommandValidator — `LoginWithGoogleCommand` için doğrulama kurallarını tanımlayan sınıf.

```
public sealed class LoginWithGoogleCommandValidator : AbstractValidator<LoginWithGoogleCommand>
```

`AbstractValidator<T>`'den miras alır; `<LoginWithGoogleCommand>` hangi tipin doğrulanacağını söyleyen generic parametredir.

```
{
    private const int IdTokenMaximumLength = 4096;
```

`private const`: derleme anında sabit bir sayı. 4096 karakter üstü token reddedilir; aşırı büyük girdiyle sunucuyu yormayı engeller.

ƒ LoginWithGoogleCommandValidator() — Yapıcı metot; doğrulama kurallarını kaydeder.

```
    public LoginWithGoogleCommandValidator()
    {
        RuleFor(command => command.IdToken)
            .NotEmpty()
            .MaximumLength(IdTokenMaximumLength)
            .WithName("Google kimlik bilgisi");
    }
}
```

`IdToken` boş olamaz ve 4096 karakteri aşamaz. `WithName` hata mesajında alan adının Türkçe görünmesini sağlar.

Application `todo.Application/Features/Authentication/LoginWithGoogle/LoginWithGoogleUseCase.cs`

> Google ile giriş iş akışını yönetir: token'ı doğrular, kullanıcıyı bulur, bağlar veya oluşturur, gerekirse Admin yapar ve yeni oturum açar. Application katmanında durur, dış dünyaya sadece arayüzler (port) üzerinden dokunur.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
using FluentValidation;
```

Temel .NET tipleri (`TimeProvider`, listeler, `Task`, `CancellationToken`) ve FluentValidation kullanıma açılır.

```
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Application.Common.Validation;
using TodoApi.Application.Exceptions;
using TodoApi.Application.Features.Authentication.Common;
using TodoApi.Domain.Entities;
using TodoApi.Domain.Enums;
```

Projenin kendi katmanları: port arayüzleri, doğrulama yardımcısı, hata tipleri, ortak oturum sınıfları ve Domain varlıkları ile enum'ları.

```
namespace TodoApi.Application.Features.Authentication.LoginWithGoogle;
```

Bu use case'in isim alanı; klasör yapısıyla birebir aynıdır.

ƒ class LoginWithGoogleUseCase — Google ile giriş senaryosunun tamamını yürüten use case sınıfı.

```
public class LoginWithGoogleUseCase
```

Use case sınıfı: tek bir iş senaryosunu (Google ile giriş) baştan sona yönetir.

```
{
```

```
    private readonly IUserRepository _userRepository;
    private readonly IRefreshTokenRepository _refreshTokenRepository;
    private readonly IGoogleTokenValidator _googleTokenValidator;
    private readonly IRoleAssignmentPolicy _roleAssignmentPolicy;
    private readonly SessionIssuer _sessionIssuer;
    private readonly IValidator<LoginWithGoogleCommand> _validator;
    private readonly TimeProvider _timeProvider;
```

`private readonly` alanlar: sadece yapıcıda atanabilir, sonra değişmez. Hepsi arayüz ya da yardımcı sınıf; somut veritabanı veya Google kodu burada yok.

ƒ LoginWithGoogleUseCase(...) — Bağımlılıkları Dependency Injection ile alan yapıcı metot.

```
    public LoginWithGoogleUseCase(
        IUserRepository userRepository,
        IRefreshTokenRepository refreshTokenRepository,
        IGoogleTokenValidator googleTokenValidator,
        IRoleAssignmentPolicy roleAssignmentPolicy,
        SessionIssuer sessionIssuer,
        IValidator<LoginWithGoogleCommand> validator,
        TimeProvider timeProvider)
```

Yapıcı, ihtiyaç duyduğu yedi bağımlılığı DI konteynerinden parametre olarak alır. `TimeProvider` saati test edilebilir yapmak için enjekte edilir.

```
    {
```

```
        if (userRepository == null)
        {
            throw new ArgumentNullException(nameof(userRepository));
        }

        if (refreshTokenRepository == null)
        {
            throw new ArgumentNullException(nameof(refreshTokenRepository));
        }

        if (googleTokenValidator == null)
        {
            throw new ArgumentNullException(nameof(googleTokenValidator));
        }

        if (roleAssignmentPolicy == null)
        {
            throw new ArgumentNullException(nameof(roleAssignmentPolicy));
        }
```

Her bağımlılık null mı diye kontrol edilir; null ise `ArgumentNullException` fırlatılır. `nameof` parametre adını güvenli şekilde metne çevirir.

```
        if (sessionIssuer == null)
        {
            throw new ArgumentNullException(nameof(sessionIssuer));
        }

        if (validator == null)
        {
            throw new ArgumentNullException(nameof(validator));
        }

        if (timeProvider == null)
        {
            throw new ArgumentNullException(nameof(timeProvider));
        }
```

Kalan bağımlılıklar için aynı null kontrolü. Hatalı kurulum çalışma anında geç değil, nesne oluşurken hemen fark edilir.

```
        _userRepository = userRepository;
        _refreshTokenRepository = refreshTokenRepository;
        _googleTokenValidator = googleTokenValidator;
        _roleAssignmentPolicy = roleAssignmentPolicy;
        _sessionIssuer = sessionIssuer;
        _validator = validator;
        _timeProvider = timeProvider;
```

Kontrolden geçen bağımlılıklar `_` ile başlayan özel alanlara kaydedilir.

```
    }
```

ƒ ExecuteAsync — Google token'ını doğrular, kullanıcıyı bulur/oluşturur ve yeni oturum döndürür.

```
    public async Task<AuthSession> ExecuteAsync(
        LoginWithGoogleCommand command,
        CancellationToken cancellationToken = default)
```

`async Task<AuthSession>`: asenkron çalışır ve sonunda `AuthSession` döner. `CancellationToken = default` parametresi verilmezse varsayılan değer kullanılır; istek iptalini taşır.

```
    {
```

```
        await _validator.ValidateOrThrowAsync(command, cancellationToken);
```

`await` işlemin bitmesini bloklamadan bekler. Komut kurallara uymuyorsa doğrulama hatası fırlatılır ve akış durur.

```
        GoogleUserInfo? googleUser = await _googleTokenValidator.ValidateAsync(
            command.IdToken,
            cancellationToken);
```

Token, Infrastructure'daki Google doğrulayıcısına gönderilir. `GoogleUserInfo?` içindeki `?` sonucun null olabileceğini belirtir (geçersiz token).

```
        if (googleUser == null)
        {
            throw new AuthenticationFailedException("The Google sign-in token is not valid.");
        }
```

Token geçersizse `AuthenticationFailedException` fırlatılır; API bunu 401 Unauthorized cevabına çevirir.

```
        if (googleUser.IsEmailVerified == false)
        {
            throw new AuthenticationFailedException(
                "The Google account email address has not been verified.");
        }
```

Google e-postayı doğrulamamışsa giriş reddedilir. Doğrulanmamış e-postaya güvenmek başkasının hesabını ele geçirmeye yol açabilir.

```
        string normalizedEmail = EmailNormalizer.Normalize(googleUser.Email);
```

E-posta normalize edilir (kırpma, küçük harf); böylece aynı adres farklı yazımlarla iki hesap oluşturamaz.

```
        UserRole configuredRole = _roleAssignmentPolicy.GetRoleForEmail(normalizedEmail);
```

Rol politikası, config'teki admin e-posta listesine bakarak bu adresin `Admin` mi `User` mı olacağını söyler.

```
        User? user = await _userRepository.GetByGoogleSubjectIdAsync(
            googleUser.SubjectId,
            cancellationToken);
```

Önce Google'ın değişmez kullanıcı kimliği (subject id) ile arama yapılır; e-posta değişse bile hesap doğru bulunur.

```
        if (user == null)
        {
            user = await _userRepository.GetByEmailAsync(normalizedEmail, cancellationToken);
```

Bu Google hesabına bağlı kullanıcı yoksa, bağlanacak ya da yeni oluşturulacak demektir.

Aynı e-postayla daha önce açılmış (örneğin şifreli) bir hesap var mı diye bakılır.

```
            if (user != null)
            {
                bool hadPasswordBeforeLinking = user.HasPassword;
                user.LinkGoogleAccount(googleUser.SubjectId);
```

Hesap bulunduysa, bağlamadan önce şifresi olup olmadığı not edilir; çünkü bağlama işlemi şifreyi silecek.

Domain metodu Google kimliğini hesaba bağlar ve eski şifreyi siler. Bu, saldırganın önceden şifreyle hesap açıp beklemesine (pre-hijacking) karşı korumadır.

```
                if (hadPasswordBeforeLinking)
                {
                    await RevokeExistingSessionsAsync(user.Id, cancellationToken);
                }
            }
```

Hesabın eskiden şifresi varsa, o şifreyle açılmış tüm oturumlar iptal edilir; saldırganın elindeki oturum da düşer.

```
            else
            {
                string displayName = ChooseDisplayName(googleUser, normalizedEmail);
                user = User.RegisterWithGoogle(
                    normalizedEmail,
                    displayName,
                    googleUser.SubjectId,
                    configuredRole);
                await _userRepository.AddAsync(user, cancellationToken);
            }
        }
```

Hiç hesap yoksa yeni kullanıcı açılacak; önce görünen ad seçilir.

Domain'deki fabrika metodu Google'a bağlı yeni kullanıcıyı config'ten gelen rolle oluşturur.

Yeni kullanıcı depoya eklenir; henüz veritabanına yazılmaz, `SaveChangesAsync` ile yazılacak.

```
        if (configuredRole == UserRole.Admin && user.IsAdmin == false)
        {
            user.PromoteToAdmin();
        }
```

Config onu Admin olarak listeliyorsa ama hesap henüz Admin değilse yükseltilir. Domain sadece Google ile doğrulanmış hesapların Admin olmasına izin verir.

```
        await _userRepository.SaveChangesAsync(cancellationToken);
```

Tüm değişiklikler tek seferde kaydedilir. Depolar aynı `DbContext`'i paylaştığı için iptal edilen oturumlar da burada yazılır.

```
        AuthSession session = await _sessionIssuer.IssueAsync(user, cancellationToken);

        return session;
    }
```

Ortak `SessionIssuer` yeni access ve refresh token üretir; oluşan oturum çağırana döndürülür.

ƒ RevokeExistingSessionsAsync — Kullanıcının tüm aktif refresh token'larını iptal eder.

```
    private async Task RevokeExistingSessionsAsync(int userId, CancellationToken cancellationToken)
    {
        DateTime nowUtc = _timeProvider.GetUtcNow().UtcDateTime;
        IReadOnlyList<RefreshToken> activeTokens = await _refreshTokenRepository.GetActiveForUserAsync(
            userId,
            nowUtc,
            cancellationToken);
        foreach (RefreshToken activeToken in activeTokens)
        {
            activeToken.Revoke(nowUtc);
        }
    }
```

`private async Task`: sadece bu sınıf içinden çağrılan, değer döndürmeyen asenkron yardımcı metot.

Şu anki UTC zaman `TimeProvider`'dan alınır; testlerde sahte saat verilebilir.

Kullanıcının hâlâ geçerli refresh token'ları getirilir. `IReadOnlyList` sadece okunabilen, değiştirilemeyen liste demektir.

Her aktif token iptal edilir. Kayıt burada yapılmaz; `ExecuteAsync` içindeki `SaveChangesAsync` hepsini birlikte kaydeder.

ƒ ChooseDisplayName — Yeni kullanıcı için görünen adı Google adından veya e-postadan seçer.

```
    private static string ChooseDisplayName(GoogleUserInfo googleUser, string normalizedEmail)
    {
```

`private static`: nesne alanlarına ihtiyaç duymayan, sadece girdilerle çalışan yardımcı metot.

```
        if (string.IsNullOrWhiteSpace(googleUser.Name) == false)
        {
            return googleUser.Name;
        }

        int atSignIndex = normalizedEmail.IndexOf('@');

        return normalizedEmail.Substring(0, atSignIndex);
    }
}
```

Google bir ad verdiyse doğrudan o kullanılır.

Ad yoksa e-postanın `@` işaretinden önceki kısmı görünen ad olarak kullanılır.

Application `todo.Application/Abstractions/Authentication/IGoogleTokenValidator.cs`

> Google ID token'ını doğrulama portu. Gerçek imza ve hedef kitle (audience) kontrolü Infrastructure'daki `GoogleIdTokenValidator` içinde yapılır.

```
using System.Threading;
using System.Threading.Tasks;

namespace TodoApi.Application.Abstractions.Authentication;
```

`CancellationToken` ve `Task` tipleri için isim alanları.

Kimlik doğrulama portlarının isim alanı.

ƒ interface IGoogleTokenValidator — Google token'ını doğrulayıp kullanıcı bilgisini çıkaran servisin sözleşmesi.

```
public interface IGoogleTokenValidator
{
```

`interface`: Application Google kütüphanesine bağımlı olmasın diye sadece sözleşme burada tanımlanır.

ƒ ValidateAsync — Token geçerliyse `GoogleUserInfo`, değilse null döndürür.

```
    Task<GoogleUserInfo?> ValidateAsync(string idToken, CancellationToken cancellationToken);
}
```

`Task<GoogleUserInfo?>`: asenkron çalışır; token geçersizse istisna yerine null döner, use case bunu kontrol eder.

Application `todo.Application/Abstractions/Authentication/GoogleUserInfo.cs`

> Google token doğrulayıcısının (port) döndürdüğü kullanıcı bilgisi. Application katmanı Google kütüphanesini görmez, sadece bu sade sınıfı görür.

```
using System;

namespace TodoApi.Application.Abstractions.Authentication;
```

`ArgumentException` için `System` isim alanı.

Kimlik doğrulama portlarının isim alanı.

ƒ class GoogleUserInfo — Doğrulanmış Google token'ından çıkan kullanıcı bilgilerini taşıyan değişmez sınıf.

```
public sealed class GoogleUserInfo
{
    private readonly string _subjectId;
    private readonly string _email;
    private readonly bool _isEmailVerified;
    private readonly string? _name;
```

`sealed` ve değişmez veri sınıfı; Google'a özgü tipleri Application'a sızdırmamak için vardır.

Subject id, e-posta, e-posta doğrulandı mı bilgisi ve ad. `string?` adın boş (null) gelebileceğini belirtir.

ƒ GoogleUserInfo(...) — Zorunlu alanları kontrol edip Google bilgilerini saklayan yapıcı.

```
    public GoogleUserInfo(string subjectId, string email, bool isEmailVerified, string? name)
    {
        if (string.IsNullOrWhiteSpace(subjectId))
        {
            throw new ArgumentException("A Google subject id cannot be empty.", nameof(subjectId));
        }

        if (string.IsNullOrWhiteSpace(email))
        {
            throw new ArgumentException("A Google email cannot be empty.", nameof(email));
        }

        _subjectId = subjectId;
        _email = email;
        _isEmailVerified = isEmailVerified;
        _name = name;
    }
```

Yapıcı dört değeri alır; ad isteğe bağlıdır.

Subject id ve e-posta zorunludur; boşsa `ArgumentException` fırlatılır.

Değerler özel alanlara kaydedilir.

```
    public string SubjectId
    {
        get
        {
            return _subjectId;
        }
    }

    public string Email
    {
        get
        {
            return _email;
        }
    }
```

`SubjectId`: Google'ın kullanıcıya verdiği değişmez kimlik; hesap bağlama bu değerle yapılır.

`Email`: Google hesabının e-posta adresi.

```
    public bool IsEmailVerified
    {
        get
        {
            return _isEmailVerified;
        }
    }
```

`IsEmailVerified`: Google e-postayı doğruladı mı? Doğrulanmamışsa giriş reddedilir.

```
    public string? Name
    {
        get
        {
            return _name;
        }
    }
}
```

`Name`: Google'daki görünen ad; yoksa null döner ve e-postadan ad türetilir.

Infrastructure **todo.Infrastructure/Services/Authentication/GoogleIdTokenValidator.cs**

> Tarayıcıdan gelen Google ID token'ını `Google.Apis.Auth` kütüphanesiyle doğrular ve içinden kullanıcı bilgilerini çıkarır. `IGoogleTokenValidator` arayüzünün Infrastructure uygulamasıdır.

```
using System;
using System.Threading;
using System.Threading.Tasks;
using Google.Apis.Auth;
using Microsoft.Extensions.Logging;
using TodoApi.Application.Abstractions.Authentication;
```

Gerekli isim alanları: asenkron işlemler, Google'ın resmi doğrulama kütüphanesi (`Google.Apis.Auth`), loglama ve Application arayüzü.

```
namespace TodoApi.Infrastructure.Services.Authentication;
```

Dosya kapsamlı namespace: Infrastructure kimlik doğrulama servisleri.

ƒ **class GoogleIdTokenValidator** — Google ID token'ını imza, süre ve audience açısından doğrulayan altyapı servisi.

```
public sealed class GoogleIdTokenValidator : IGoogleTokenValidator
```

`sealed` sınıf `IGoogleTokenValidator` uygular; Application katmanı Google kütüphanesine bağımlı olmaz, testlerde sahtesi kullanılabilir.

```
{
    private readonly GoogleAuthenticationSettings _googleSettings;
    private readonly ILogger<GoogleIdTokenValidator> _logger;
```

Google ayarları (client id) ve loglayıcı alanları. `ILogger<T>` log kayıtlarına sınıf adını otomatik ekler.

ƒ **GoogleIdTokenValidator** — Google ayarlarını ve loglayıcıyı DI'dan alıp saklar.

```
    public GoogleIdTokenValidator(
        GoogleAuthenticationSettings googleSettings,
        ILogger<GoogleIdTokenValidator> logger)
    {
        if (googleSettings == null)
        {
            throw new ArgumentNullException(nameof(googleSettings));
        }

        if (logger == null)
        {
            throw new ArgumentNullException(nameof(logger));
        }

        _googleSettings = googleSettings;
        _logger = logger;
    }
```

Kurucu metot; parametreler okunabilirlik için alt alta yazılmıştır.

Bağımlılıklar null ise uygulama başlarken açık bir `ArgumentNullException` ile durur.

Doğrulanan bağımlılıklar `readonly` alanlara atanır.

ƒ **ValidateAsync** — Token'ı doğrular; geçerliyse Google kullanıcı bilgisini, değilse null döner.

```
    public async Task<GoogleUserInfo?> ValidateAsync(
        string idToken,
        CancellationToken cancellationToken)
    {
        if (_googleSettings.IsEnabled == false)
        {
            _logger.LogWarning("Google sign-in was attempted, but no Google client id is configured.");
            return null;
        }

        GoogleJsonWebSignature.ValidationSettings validationSettings =
            new GoogleJsonWebSignature.ValidationSettings
            {
                Audience = new[] { _googleSettings.ClientId }
            };

        GoogleJsonWebSignature.Payload payload;

        try
        {
            payload = await GoogleJsonWebSignature.ValidateAsync(idToken, validationSettings);
        }
```

`async Task<GoogleUserInfo?>`: asenkron çalışır, `?` sonucun null olabileceğini belirtir. `CancellationToken` istek iptalinde işi durdurmaya yarar.

Client id ayarlanmamışsa Google girişi kapalıdır; uyarı loglanır ve null dönülür, yani giriş reddedilir.

Doğrulama ayarı: token'ın audience değeri bizim client id'miz olmalı. Böylece başka bir uygulama için üretilmiş Google token'ı kabul edilmez.

Doğrulanmış token içeriğini tutacak değişken önceden tanımlanır.

Kütüphane Google'ın açık anahtarlarıyla imzayı, süreyi, issuer'ı ve audience'ı kontrol eder; hepsi geçerliyse içeriği (payload) döner.

```
        catch (InvalidJwtException exception)
        {
            _logger.LogWarning(exception, "A Google sign-in token was rejected.");
            return null;
        }

        if (string.IsNullOrWhiteSpace(payload.Subject) || string.IsNullOrWhiteSpace(payload.Email))
        {
            return null;
        }

        GoogleUserInfo googleUser = new GoogleUserInfo(
            payload.Subject,
            payload.Email,
            payload.EmailVerified,
            payload.Name);

        return googleUser;
    }
}
```

Token geçersizse `InvalidJwtException` fırlar; hata uyarı olarak loglanır ve null dönülür. Sahte token sunucu hatasına yol açmaz.

Google kimliği (Subject) veya e-posta boşsa kullanıcıyı tanımlayamayız; güvenli tarafta kalıp null döneriz.

Gerekli alanlar (Google kimliği, e-posta, e-posta doğrulandı mı, ad) Application'daki `GoogleUserInfo` nesnesine kopyalanıp döndürülür.

Infrastructure `todo.Infrastructure/Services/Authentication/GoogleAuthenticationSettings.cs`

> Google girişi için gereken client id ayarını yapılandırmadan okuyup tutan küçük ayar sınıfı. Client id yoksa Google girişi kapalı kabul edilir.

```
using System;
using Microsoft.Extensions.Configuration;
```

`System` istisnalar için, `Microsoft.Extensions.Configuration` appsettings ve user secrets değerlerini okumak için eklenir.

```
namespace TodoApi.Infrastructure.Services.Authentication;
```

Dosya kapsamlı namespace: Infrastructure kimlik doğrulama servisleri.

ƒ class GoogleAuthenticationSettings — Google client id ayarını değiştirilemez şekilde tutan ayar nesnesi.

```
public sealed class GoogleAuthenticationSettings
{
    public const string SectionName = "Authentication:Google";

    private readonly string? _clientId;
```

`sealed` sınıf; ayarlar oluşturulduktan sonra değiştirilemez, bu da uygulama boyunca tutarlılık sağlar.

Yapılandırmadaki bölüm yolu sabiti: `Authentication:Google`. İki nokta, JSON'daki iç içe nesneyi ifade eder.

`string?` nullable metin: client id hiç ayarlanmamış olabilir. `readonly` alan sadece kurucuda atanabilir.

ƒ GoogleAuthenticationSettings — Client id değerini temizleyip saklar; boşsa null olarak tutar.

```
    public GoogleAuthenticationSettings(string? clientId)
    {
        if (string.IsNullOrWhiteSpace(clientId))
        {
            _clientId = null;
        }
        else
        {
            _clientId = clientId.Trim();
        }
    }
```

Kurucu metot; dışarıdan gelen değeri normalize eder.

Değer boş veya sadece boşluksa null saklanır; değilse baş/son boşlukları `Trim` ile temizlenir. Kopyala-yapıştır hatalarını önler.

ƒ ClientId — Saklanan Google client id değerini döner.

```
    public string? ClientId
    {
        get
        {
            return _clientId;
        }
    }
```

Sadece `get` olan property: dışarıdan okunabilir ama değiştirilemez.

ƒ IsEnabled — Client id varsa Google girişinin açık olduğunu bildirir.

```
    public bool IsEnabled
    {
        get
        {
            return _clientId != null;
        }
    }
```

Hesaplanan property: client id null değilse true döner; doğrulayıcı bu değere bakarak girişi reddeder veya devam eder.

ƒ FromConfiguration — Yapılandırmadan Google bölümünü okuyup ayar nesnesi üreten statik fabrika metodu.

```
    public static GoogleAuthenticationSettings FromConfiguration(IConfiguration configuration)
    {
        if (configuration == null)
        {
            throw new ArgumentNullException(nameof(configuration));
        }
```

`static` metot nesne olmadan sınıf üzerinden çağrılır (`GoogleAuthenticationSettings.FromConfiguration(...)`); DI kaydı sırasında kullanılır.

Yapılandırma nesnesi null ise açık bir istisna fırlatılır.

```
        IConfigurationSection section = configuration.GetSection(SectionName);

        GoogleAuthenticationSettings settings = new GoogleAuthenticationSettings(section["ClientId"]);

        return settings;
    }
}
```

`Authentication:Google` bölümü alınır, içindeki `ClientId` anahtarı okunur ve yeni ayar nesnesi oluşturulup döndürülür. Değer yoksa indeksleyici null verir.

Application `todo.Application/Abstractions/Authentication/IRoleAssignmentPolicy.cs`

> Bir e-postanın hangi rolü alacağına karar veren politika portu. Uygulaması (`ConfiguredRoleAssignmentPolicy`) admin e-posta listesini config'ten okur.

```
using TodoApi.Domain.Enums;
```

Dönüş tipi olan `UserRole` enum'u için Domain isim alanı.

```
namespace TodoApi.Application.Abstractions.Authentication;
```

Kimlik doğrulama portlarının isim alanı.

ƒ interface IRoleAssignmentPolicy — E-postaya göre kullanıcı rolünü belirleyen politikanın sözleşmesi.

```
public interface IRoleAssignmentPolicy
```

`interface` : admin listesi koda gömülmez; config'ten okunur ve Application sadece bu sözleşmeyi bilir.

```
{
```

ƒ GetRoleForEmail — E-posta admin listesindeyse `Admin` , değilse `User` döndürür.

```
    UserRole GetRoleForEmail(string email);
```

Normalize edilmiş e-postayı alır ve rol döndürür. Admin'e yükseltme yine de sadece Google ile doğrulanmış hesaplarda gerçekleşir.

```
}
```

Infrastructure `todo.Infrastructure/Services/Authentication/ConfiguredRoleAssignmentPolicy.cs`

> Yapılandırmadaki admin e-posta listesine bakarak bir e-postanın Admin mi User mı olacağını söyler. Google ile giriş akışında rol belirlemek için kullanılır.

```
using System;
using System.Collections.Generic;
using Microsoft.Extensions.Configuration;
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Domain.Enums;
```

Gerekli isim alanları: koleksiyonlar (`HashSet` , `List`), yapılandırma okuma, Application arayüzü ve Domain'deki `UserRole` enum'u.

```
namespace TodoApi.Infrastructure.Services.Authentication;
```

Dosya kapsamlı namespace: Infrastructure kimlik doğrulama servisleri.

ƒ class ConfiguredRoleAssignmentPolicy — Admin e-posta listesine göre rol belirleyen politika sınıfı.

```
public sealed class ConfiguredRoleAssignmentPolicy : IRoleAssignmentPolicy
```

`IRoleAssignmentPolicy` uygular; admin listesinin nereden geldiğini Application katmanı bilmez, sadece rol sorar.

```
{
    public const string AdminEmailsSectionName = "Authorization:AdminEmails";
```

Admin e-postalarının yapılandırmadaki yolu: `Authorization:AdminEmails` (JSON'da bir dizi).

```
    private readonly HashSet<string> _adminEmails;
```

`HashSet` tekrarsız küme yapısıdır; `Contains` araması çok hızlıdır. Admin e-postaları burada tutulur.

ƒ ConfiguredRoleAssignmentPolicy — Gelen e-posta listesini temizleyip büyük/küçük harf duyarsız bir kümeye doldurur.

```
    public ConfiguredRoleAssignmentPolicy(IEnumerable<string> adminEmails)
```

Kurucu metot; `IEnumerable<string>` herhangi bir e-posta koleksiyonunu kabul eder.

```
    {
        if (adminEmails == null)
        {
            throw new ArgumentNullException(nameof(adminEmails));
        }
```

Liste null ise açık bir `ArgumentNullException` fırlatılır.

```
        _adminEmails = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
```

`StringComparer.OrdinalIgnoreCase` ile küme kurulur; `Patron@x.com` ile `patron@x.com` aynı kabul edilir.

```
        foreach (string adminEmail in adminEmails)
        {
            if (string.IsNullOrWhiteSpace(adminEmail) == false)
            {
                _adminEmails.Add(adminEmail.Trim());
            }
        }
    }
```

Her e-posta gezilir; boş olmayanlar `Trim` ile boşluklarından temizlenip kümeye eklenir. Hatalı yapılandırma satırları sessizce atlanır.

ƒ GetRoleForEmail — E-posta admin listesindeyse Admin, değilse User rolünü döner.

```
    public UserRole GetRoleForEmail(string email)
```

Google ile giriş yapan kullanıcının hangi rolü alacağını belirler; Admin yalnızca Google'ın doğruladığı hesaplara verilebilir.

```
    {
        if (string.IsNullOrWhiteSpace(email))
        {
            return UserRole.User;
        }
```

E-posta boşsa güvenli varsayılan olarak her zaman User döner.

```
        bool isAdminEmail = _adminEmails.Contains(email.Trim());

        if (isAdminEmail)
        {
            return UserRole.Admin;
        }
```

Temizlenmiş e-posta admin kümesinde aranır; bulunursa `UserRole.Admin` döner.

```
47          return UserRole.User;
48      }
```

Listede yoksa normal kullanıcı rolü `UserRole.User` döner.

ƒ FromConfiguration — Yapılandırmadan admin e-posta dizisini okuyup politika nesnesi üreten statik fabrika metodu.

```
50      public static ConfiguredRoleAssignmentPolicy FromConfiguration(IConfiguration
configuration)
51      {
52          if (configuration == null)
53          {
54              throw new ArgumentNullException(nameof(configuration));
55          }
```

`static` fabrika metodu; DI kaydında yapılandırmadan nesne üretmek için kullanılır.

Yapılandırma nesnesi null ise istisna fırlatılır.

```
57          List<string> adminEmails = new List<string>();
58
59          foreach (IConfigurationSection child in configuration.GetSection(AdminEmai
lsSectionName).GetChildren())
60          {
61              if (child.Value != null)
62              {
63                  adminEmails.Add(child.Value);
64              }
65          }
```

`GetChildren` dizinin her elemanını ayrı bölüm olarak verir; değeri null olmayanlar listeye eklenir.

```
67          ConfiguredRoleAssignmentPolicy policy = new ConfiguredRoleAssignmentPolicy
(adminEmails);
68
69          return policy;
70      }
71  }
```

Toplanan e-postalarla yeni politika nesnesi oluşturulup döndürülür; temizleme işi kurucuda yapılır.

Bölüm 3 — Görev oluşturma (ve hata yolları)

Kullanıcı 'Görev ekle' butonuna basar. İstek Angular'dan çıkar, API'de doğrulanır, sahiplik kontrol edilir, Domain kurallarından geçer, veritabanına yazılır ve 201 ile geri döner. Bir şey ters giderse hata, türüne göre doğru handler'a düşer ve kullanıcıya Türkçe bir mesaj olarak ulaşır.

Akış adım adım

1. `createTodo()`: form kontrol edilir, yerel saat UTC'ye çevrilir.
   `todo.Web/src/app/pages/todos/todo-page.ts`
2. `POST /api/todo-lists/{id}/todos`; interceptor Bearer token ekler.
   `todo.Web/src/app/services/todo-api.service.ts`
3. `Create` : `CreateTodoRequest` → `CreateTodoCommand`.
   `todo.API/Controllers/TodosController.cs`
4. Mekanik doğrulama (boş/uzun başlık, tarih kuralları). Hata → 400.
   `todo.Application/Features/Todos/CreateTodo/CreateTodoCommandValidator.cs`
5. Liste yüklenir, sahiplik kontrol edilir (yabancıysa 404).
   `todo.Application/Features/Todos/CreateTodo/CreateTodoUseCase.cs`
6. `AddTodo`: aynı isim kuralı (iş kuralı). Hata → 409.
   `todo.Domain/Entities/TodoList.cs`
7. `SaveChangesAsync`: INSERT; veritabanı hataları çevrilir (409 / 503).
   `todo.Infrastructure/Persistence/TodoDbContext.cs`
8. Entity dışarı verilmez, DTO'ya kopyalanır.
   `todo.Application/Contracts/TodoItemDto.cs`
9. `CreatedAtAction` → 201 + Location header + JSON.
   `todo.API/Controllers/TodosController.cs`
10. Hata olduysa: handler'lar ProblemDetails + traceId üretir.
    `todo.API/ExceptionHandling/ProblemDetailsWriter.cs`
11. Başarılıysa görev listeye eklenir; hata varsa `showError` Türkçe mesaj gösterir.
    `todo.Web/src/app/pages/todos/todo-page.ts`

Bu bölümde açıklanan dosyalar (kullanım sırasıyla): todo.models.ts, todo-page.ts, todo-api.service.ts, TodosController.cs, CreateTodoRequest.cs, UpdateTodoRequest.cs, SetTodoCompletionStatusRequest.cs, CreateTodoCommand.cs, CreateTodoCommandValidator.cs, SharedValidationRules.cs, ValidatorExtensions.cs, RequestValidationException.cs, CreateTodoUseCase.cs, ITodoListRepository.cs, TodoListRepository.cs, TodoListAccessPolicy.cs, TodoList.cs, TodoItem.cs, TodoPriority.cs, NameComparison.cs, DomainRuleViolationException.cs, DuplicateNameException.cs, TodoDbContext.cs, DatabaseErrorTranslator.cs, ConflictException.cs, DataAccessException.cs, ConcurrencyConflictException.cs, TodoItemDto.cs, ValidationExceptionHandler.cs, DomainExceptionHandler.cs, ApplicationExceptionHandler.cs, DatabaseExceptionHandler.cs, UnhandledExceptionHandler.cs, ProblemDetailsWriter.cs

Web (Angular) `todo.Web/src/app/models/todo.models.ts`

> Frontend'in API ile konuştuğu veri şekillerini (liste, görev, istek gövdeleri) TypeScript tipleri olarak tanımlar. Backend DTO'larının Angular tarafındaki aynasıdır.

ƒ type TodoPriority — Görev önceliğinin alabileceği üç sabit değeri tanımlayan birleşim tipi.

```ts
export type TodoPriority = 'Low' | 'Normal' | 'High';
```

`type` ile birleşim (union) tipi: öncelik yalnızca 'Low', 'Normal' veya 'High' olabilir; yanlış yazım derlemede yakalanır. Backend enum adlarıyla birebir aynıdır.

ƒ interface TodoList — API'den gelen bir yapılacaklar listesinin şeklini tanımlar.

```ts
export interface TodoList {
  id: number;
  name: string;
  createdAtUtc: string;
}
```

`interface` yalnızca derleme zamanında var olan bir şekil sözleşmesidir; çalışırken koda dönüşmez. `export` başka dosyaların kullanabilmesini sağlar.

Listenin kimliği, adı ve oluşturulma zamanı. Tarih JSON'da ISO metni olarak geldiği için `string`; `Utc` eki saatin UTC olduğunu hatırlatır.

ƒ interface TodoItem — API'nin döndürdüğü tek bir görevin tüm alanlarını tanımlar (backend TodoItemDto karşılığı).

```ts
export interface TodoItem {
```

Bir görevin frontend'deki şekli; backend'deki `TodoItemDto` ile alan adları aynıdır çünkü JSON camelCase olarak gelir.

```
  id: number;
  todoListId: number;
  title: string;
```

Görev kimliği, bağlı olduğu listenin kimliği ve zorunlu başlık.

```
  description: string | null;
  isCompleted: boolean;
```

`string | null`: açıklama boş bırakılabilir, API null gönderir. `isCompleted` görevin tamamlanıp tamamlanmadığını tutar.

```
  createdAtUtc: string;
  dueAtUtc: string | null;
  priority: TodoPriority;
```

Oluşturulma ve son tarih UTC metin olarak gelir; son tarih isteğe bağlı olduğu için null olabilir. Öncelik yukarıdaki `TodoPriority` tipini kullanır.

```
}
```

ƒ interface TodoListRequest — Liste oluşturma ve yeniden adlandırma isteklerinin gövdesini tanımlar.

```
export interface TodoListRequest {
```

Liste oluştururken ve adını değiştirirken API'ye gönderilen istek gövdesi.

```
  name: string;
```

Sadece liste adı gönderilir; id ve tarih sunucu tarafında belirlenir.

```
}
```

ƒ interface TodoRequest — Görev oluşturma ve güncelleme isteklerinde API'ye gönderilen gövdeyi tanımlar.

```
export interface TodoRequest {
```

Görev oluşturma (POST) ve güncelleme (PUT) için ortak istek gövdesi; id, tamamlanma ve oluşturulma tarihi içermez çünkü onları sunucu yönetir.

```
  title: string;
  description: string | null;
  dueAtUtc: string | null;
  priority: TodoPriority;
```

Başlık zorunlu; açıklama ve son tarih null olabilir. Son tarih tarayıcıda UTC'ye çevrilip `dueAtUtc` olarak gönderilir.

```
}
```

Web (Angular) `todo.Web/src/app/pages/todos/todo-page.ts`

> Ana yapılacaklar sayfasının bileşenidir: listeleri ve görevleri yükler, oluşturma/düzenleme/silme işlemlerini yönetir ve API hatalarını kullanıcıya anlaşılır mesajlara çevirir. Görev oluşturma akışının frontend tarafındaki başlangıç noktasıdır.

```
import { DatePipe } from '@angular/common';
import { HttpErrorResponse } from '@angular/common/http';
import { ChangeDetectorRef, Component, OnInit, inject } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { RouterLink } from '@angular/router';
import { firstValueFrom } from 'rxjs';
```

Angular parçaları içe aktarılır: tarih biçimleyen `DatePipe`, HTTP hata tipi, bileşen araçları, `ngModel` için `FormsModule`, yönlendirme linki ve Observable'ı Promise'e çeviren `firstValueFrom`.

```
import { NzButtonModule } from 'ng-zorro-antd/button';
import { NzCheckboxModule } from 'ng-zorro-antd/checkbox';
import { NzInputModule } from 'ng-zorro-antd/input';
import { NzMessageService } from 'ng-zorro-antd/message';
import { NzPopconfirmModule } from 'ng-zorro-antd/popconfirm';
import { NzSelectModule } from 'ng-zorro-antd/select';
import { NzTagModule } from 'ng-zorro-antd/tag';
```

ng-zorro-antd arayüz kütüphanesinden buton, onay kutusu, input, bildirim mesajı, silme onayı (popconfirm), açılır seçim ve etiket modülleri alınır.

```
import { TodoItem, TodoList, TodoPriority, TodoRequest } from '../../models/todo.models';
import { AuthService } from '../../services/auth.service';
import { TodoApiService } from '../../services/todo-api.service';
```

Veri tipleri ile oturum servisi `AuthService` ve tüm HTTP çağrılarını yapan `TodoApiService` içe aktarılır.

```
const CONCURRENCY_CONFLICT_PROBLEM_TYPE = 'urn:todo:problem:concurrency-conflict';
```

API'nin eşzamanlılık çakışmasında ProblemDetails içinde döndürdüğü `type` değeri; başka sekmede değişen kaydı tanımak için sabit olarak tutulur.

```
@Component({
  selector: 'app-todo-page',
```

`@Component` dekoratörü sınıfı bir Angular bileşeni yapar; `selector` bileşenin HTML'deki etiket adıdır.

```
  imports: [
    DatePipe,
    RouterLink,
    FormsModule,
    NzButtonModule,
    NzCheckboxModule,
    NzInputModule,
    NzPopconfirmModule,
    NzSelectModule,
    NzTagModule,
  ],
```

Standalone bileşen olduğu için şablonda kullanılan pipe, direktif ve ng-zorro modülleri burada tek tek `imports` listesine yazılır.

```
  templateUrl: './todo-page.html',
  styleUrl: './todo-page.scss',
})
```

Görünüm `todo-page.html`, stiller `todo-page.scss` dosyasından gelir.

ƒ class TodoPage — Liste ve görev ekranının tüm durumunu ve kullanıcı işlemlerini yöneten sayfa bileşeni.

```
export class TodoPage implements OnInit {
```

`implements OnInit` sınıfın `ngOnInit` yaşam döngüsü metodunu yazmayı taahhüt ettiğini belirtir.

```
  private readonly todoApi = inject(TodoApiService);
  protected readonly auth = inject(AuthService);
  private readonly messages = inject(NzMessageService);
  private readonly changeDetector = inject(ChangeDetectorRef);
```

`inject()` ile servisler alınır. `auth` şablonda kullanıldığı için `protected`; `ChangeDetectorRef` zoneless Angular'da ekranı elle yenilemek için gerekir.

```
  todoLists: TodoList[] = [];
  todos: TodoItem[] = [];
  selectedList: TodoList | null = null;
```

Ekrandaki veriler: tüm listeler, seçili listenin görevleri ve seçili liste; `TodoList | null` hiçbir liste seçili olmayabileceğini söyler.

```
  isLoadingLists = false;
  isLoadingTodos = false;
  isSavingList = false;
  isSavingTodo = false;
  isApiAvailable = false;
```

Yükleniyor/kaydediliyor bayrakları butonları kilitlemek ve spinner göstermek için; `isApiAvailable` başlıktaki bağlantı noktasını yönetir.

```
  newListName = '';
  isRenamingList = false;
  renamedListName = '';
```

Yeni liste adı ve yeniden adlandırma formunun alanları; şablonda `[(ngModel)]` ile iki yönlü bağlanır.

```
  isCreatingTodo = false;
  newTodoTitle = '';
  newTodoDescription = '';
  newTodoDueAtLocal = '';
  newTodoPriority: TodoPriority = 'Normal';
```

Görev oluşturma formunun alanları: form açık mı, başlık, açıklama, yerel saatli son tarih metni ve varsayılanı 'Normal' olan öncelik.

```
  editingTodoId: number | null = null;
  editedTodoTitle = '';
  editedTodoDescription = '';
  editedTodoDueAtLocal = '';
  editedTodoPriority: TodoPriority = 'Normal';
```

Düzenleme formu: hangi görevin düzenlendiği (`null` ise hiçbiri) ve formdaki geçici değerler; asıl görev kaydedilene kadar değişmez.

ƒ ngOnInit — Bileşen ilk açıldığında listeleri yüklemeyi başlatır.

```
  ngOnInit(): void {
```

Angular bileşeni oluşturduktan sonra bu metodu bir kez çağırır.

```
    void this.loadTodoLists();
```

`void` operatörü dönen Promise'i bilerek beklemediğimizi belirtir; yükleme arka planda sürer.

```
  }
```

ƒ logout — Çıkış butonuna basılınca oturumu kapatır.

```
  logout(): void {
```

Şablondaki 'Çıkış yap' butonunun tıklama işleyicisi.

```
    void this.auth.logout();
```

Asıl çıkış mantığı `AuthService`'tedir; Promise sonucu beklenmez.

```
  }
```

ƒ loadTodoLists — Listeleri API'den çeker, uygun listeyi seçer ve onun görevlerini yükler.

```
  async loadTodoLists(preferredListId?: number): Promise<void> {
```

`async` metot `Promise` döndürür ve içinde `await` kullanılabilir. `preferredListId?` isteğe bağlı parametredir: yükleme sonrası seçilecek liste.

```
    this.isLoadingLists = true;
```

Yükleme bayrağı açılır ki arayüz yükleniyor durumunu göstersin.

```
    try {
      this.todoLists = await firstValueFrom(this.todoApi.getTodoLists());
      this.isApiAvailable = true;
```

`try` bloğu hataları yakalamak içindir. `firstValueFrom` Observable'ın ilk değerini Promise'e çevirir, `await` yanıt gelene kadar bekler; başarılıysa API erişilebilir sayılır.

```
      if (this.todoLists.length === 0) {
        this.selectedList = null;
        this.todos = [];
        return;
      }
```

Hiç liste yoksa seçim ve görevler temizlenip metottan erken çıkılır; `finally` bloğu yine de çalışır.

```
      const currentListId = preferredListId ?? this.selectedList?.id;
```

`??` soldaki değer null/undefined ise sağdakini alır; `?.` seçili liste null ise hata vermeden undefined döner. Önce tercih edilen, yoksa mevcut liste.

```
      const matchingList = this.todoLists.find((list) => list.id === currentListId);
      const listToSelect = matchingList ?? this.todoLists[0];
```

Bu kimlikteki liste hâlâ var mı diye aranır; silinmiş ya da bulunamamışsa ilk liste seçilir.

```
      if (listToSelect !== undefined) {
        this.selectedList = listToSelect;
        await this.loadTodos(listToSelect.id);
      }
```

Seçilecek liste varsa (dizi boş değilse) seçili yapılır ve görevleri beklenerek yüklenir.

```
    } catch (error: unknown) {
      this.showError(error);
```

`catch` her türlü hatayı yakalar; tipi `unknown` olduğu için ortak `showError` metoduna devredilir.

```
    } finally {
      this.isLoadingLists = false;
      this.changeDetector.markForCheck();
    }
```

`finally` başarı da hata da olsa çalışır: bayrak kapanır. `markForCheck` zoneless Angular'a bu bileşeni yeniden çizmesini söyler.

```
  }
```

ƒ selectList — Kullanıcının tıkladığı listeyi seçer, açık formları kapatır ve görevlerini yükler.

```
  async selectList(todoList: TodoList): Promise<void> {
```

Sol menüde bir listeye tıklandığında çağrılır.

```
    this.selectedList = todoList;
    this.isRenamingList = false;
    this.isCreatingTodo = false;
    this.editingTodoId = null;
```

Liste seçilir; yeniden adlandırma, görev oluşturma ve düzenleme formları kapatılır ki eski listeye ait yarım işlem kalmasın.

```
    await this.loadTodos(todoList.id);
```

Yeni listenin görevleri API'den beklenerek yüklenir.

```
  }
```

ƒ createList — Formdaki adla yeni liste oluşturur ve listeleri yeniden yükleyip onu seçer.

```
  async createList(): Promise<void> {
```

Yeni liste formu gönderildiğinde çalışır.

```
    const name = this.newListName.trim();
```

`trim()` baştaki ve sondaki boşlukları siler; sadece boşluk yazılmış ad boş sayılır.

```
    if (name.length === 0) {
      this.messages.warning('Liste adı boş olamaz.');
      return;
    }
```

Ad boşsa API'ye hiç gitmeden uyarı gösterilir. Bu istemci tarafı kontrol hızlı geri bildirim içindir; asıl doğrulama backend'dedir.

121	`this.isSavingList = true;`	Kaydetme bayrağı açılır; buton tekrar tıklanmasın diye kilitlenir.
123–126	`try { const createdList = await firstValueFrom( this.todoApi.createTodoList({ name }), );`	`{ name }` gövdesiyle POST isteği atılır ve oluşturulan liste beklenir.
128–130	`this.newListName = ''; this.messages.success('Liste oluşturuldu.'); await this.loadTodoLists(createdList.id);`	Form temizlenir, başarı mesajı gösterilir ve listeler yeni listenin kimliği tercih edilerek yeniden yüklenir; böylece yeni liste seçili olur.
131–132	`} catch (error: unknown) { this.showError(error, 'Bu isimde bir listen zaten var.');`	Hata olursa `showError` çağrılır; 409 gelirse aynı isimli liste var demektir ve özel mesaj gösterilir.
133–136	`} finally { this.isSavingList = false; this.changeDetector.markForCheck(); }`	Her durumda kaydetme bayrağı kapanır ve ekran yenilenmesi istenir.
137	`}`	

ƒ startRenamingList — Seçili listenin adını düzenleme moduna geçirir.

139	`startRenamingList(): void {`	'Yeniden adlandır' butonuna basılınca çalışır.
140–142	`if (this.selectedList === null) { return; }`	Seçili liste yoksa yapacak bir şey yoktur, metottan çıkılır (guard clause).
144–145	`this.renamedListName = this.selectedList.name; this.isRenamingList = true;`	Input mevcut adla doldurulur ve düzenleme modu açılır.
146	`}`	

ƒ saveListName — Seçili listenin yeni adını API'ye kaydeder ve ekrandaki listeyi günceller.

148	`async saveListName(): Promise<void> {`	Yeniden adlandırma formu kaydedildiğinde çalışır.
149–151	`if (this.selectedList === null) { return; }`	Seçili liste yoksa erken çıkılır; bu kontrol TypeScript'e de aşağıda `selectedList`'in null olmadığını kanıtlar.
153–154	`const todoListId = this.selectedList.id; const name = this.renamedListName.trim();`	Liste kimliği yerel değişkene alınır (await sırasında seçim değişse bile doğru liste kullanılsın) ve ad kırpılır.
156–159	`if (name.length === 0) { this.messages.warning('Liste adı boş olamaz.'); return; }`	Boş ad için uyarı verilip API çağrısı yapılmaz.
161	`this.isSavingList = true;`	İstek sürerken kaydetme bayrağı açılır.
163–166	`try { const renamedList = await firstValueFrom( this.todoApi.renameTodoList(todoListId, { name }), );`	PUT isteğiyle yeni ad gönderilir, sunucunun döndürdüğü güncel liste beklenir.
168–171	`this.selectedList = renamedList; this.todoLists = this.todoLists.map((list) => list.id === renamedList.id ? renamedList : list, );`	Seçili liste ve dizideki karşılığı güncellenir. `map` yeni dizi üretir; sadece eşleşen kimlik değiştirilir, diğerleri aynen kalır.
172–173	`this.isRenamingList = false; this.messages.success('Liste adı güncellendi.');`	Düzenleme modu kapanır ve başarı mesajı gösterilir.
174–175	`} catch (error: unknown) { this.showError(error, 'Bu isimde bir listen zaten var.');`	Hata `showError`'a gider; 409 ise aynı isimli liste uyarısı gösterilir.
176–179	`} finally { this.isSavingList = false; this.changeDetector.markForCheck(); }`	Bayrak kapanır, ekran yenilenmesi istenir.
180	`}`	

ƒ deleteList — Seçili listeyi içindeki görevlerle birlikte siler ve listeleri yeniden yükler.

182	`async deleteList(): Promise<void> {`	Popconfirm onayından sonra çağrılır.
183–185	`if (this.selectedList === null) { return; }`	Seçili liste yoksa erken çıkılır.
187	`const deletedListId = this.selectedList.id;`	Silinecek listenin kimliği saklanır.
189–192	`try { await firstValueFrom(this.todoApi.deleteTodoList(deletedListId)); this.messages.success('Liste ve içindeki görevler silindi.'); await this.loadTodoLists();`	DELETE isteği beklenir, başarı mesajı gösterilir ve listeler yeniden yüklenir; silinen liste bulunamayacağı için ilk liste seçilir.
193–197	`} catch (error: unknown) { this.showError(error); } finally { this.changeDetector.markForCheck(); }`	Hata ortak yöntemle gösterilir; her durumda ekran yenilenmesi istenir.
198	`}`	

ƒ loadTodos — Verilen listenin görevlerini API'den çekip `todos` dizisine yazar.

200	`async loadTodos(todoListId: number): Promise<void> {`	Liste seçildiğinde ya da listeler yüklendiğinde görevleri getirir.
201	`this.isLoadingTodos = true;`	Görev yükleniyor bayrağı açılır.
203–204	`try { this.todos = await firstValueFrom(this.todoApi.getTodos(todoListId));`	GET isteğiyle görevler çekilir ve diziye atanır.

```
    } catch (error: unknown) {
      this.todos = [];
      this.showError(error);
```
Hata olursa eski listenin görevleri ekranda kalmasın diye dizi boşaltılır ve hata gösterilir.

```
    } finally {
      this.isLoadingTodos = false;
      this.changeDetector.markForCheck();
    }
  }
```
Yükleme bayrağı kapanır, ekran yeniden çizilir.

ƒ createTodo — Formdaki bilgilerle seçili listeye yeni görev oluşturur; Görev oluşturma akışının frontend girişidir.

```
  async createTodo(): Promise<void> {
```
Görev oluşturma formu gönderildiğinde şablondan çağrılır; akışın frontend tarafı buradan başlar.

```
    if (this.selectedList === null) {
      return;
    }
```
Görev bir listeye ait olmak zorunda; seçili liste yoksa işlem yapılmaz.

```
    const todoListId = this.selectedList.id;
    const title = this.newTodoTitle.trim();
```
Hedef liste kimliği saklanır ve başlıktaki gereksiz boşluklar kırpılır.

```
    if (title.length === 0) {
      this.messages.warning('Görev başlığı boş olamaz.');
      return;
    }
```
Başlık boşsa sunucuya gitmeden uyarı verilir. Backend FluentValidation da aynı kuralı ayrıca uygular.

```
    this.isSavingTodo = true;
```
Kaydetme bayrağı açılır; 'Ekle' butonu yükleniyor durumuna geçer ve çift gönderim engellenir.

```
    try {
      const request = this.createTodoRequest(
        title,
        this.newTodoDescription,
        this.newTodoDueAtLocal,
        this.newTodoPriority,
      );
```
Form alanları `createTodoRequest` ile API'nin beklediği `TodoRequest` gövdesine çevrilir: açıklama kırpılır, yerel tarih UTC'ye dönüştürülür.

```
      const createdTodo = await firstValueFrom(
        this.todoApi.createTodo(todoListId, request),
      );
```
POST isteği gönderilir ve sunucunun kaydettiği görev (id ve oluşturulma tarihiyle) beklenir.

```
      this.todos = [...this.todos, createdTodo];
```
Spread `[...]` mevcut diziyi açıp sonuna yeni görevi ekleyerek yeni bir dizi üretir; listeyi baştan yüklemeye gerek kalmaz.

```
      this.newTodoTitle = '';
      this.newTodoDescription = '';
      this.newTodoDueAtLocal = '';
      this.newTodoPriority = 'Normal';
      this.isCreatingTodo = false;
      this.messages.success('Görev oluşturuldu.');
```
Form alanları varsayılan değerlere sıfırlanır, form kapanır ve başarı mesajı gösterilir.

```
    } catch (error: unknown) {
      this.showError(error, 'Bu listede aynı isimli bir görev zaten var.');
```
Hata `showError`'a gider; 409 gelirse aynı listede aynı başlıklı görev olduğu söylenir.

```
    } finally {
      this.isSavingTodo = false;
      this.changeDetector.markForCheck();
    }
  }
```
Her durumda kaydetme bayrağı kapanır ve zoneless Angular'a ekranı güncellemesi söylenir.

ƒ startEditingTodo — Bir görevi düzenleme moduna alır ve formu görevin mevcut değerleriyle doldurur.

```
  startEditingTodo(todo: TodoItem): void {
```
Görev satırındaki 'Düzenle' butonuyla çağrılır.

```
    this.editingTodoId = todo.id;
    this.editedTodoTitle = todo.title;
```
Hangi görevin düzenlendiği işaretlenir ve başlık forma kopyalanır.

```
    this.editedTodoDescription = todo.description ?? '';
```
Açıklama null ise input'a boş metin konur (`??` operatörü).

```
    this.editedTodoDueAtLocal = this.toLocalDateTimeValue(todo.dueAtUtc);
    this.editedTodoPriority = todo.priority;
```
UTC son tarih `datetime-local` input'unun beklediği yerel saat biçimine çevrilir; öncelik de kopyalanır.

```
  }
```

ƒ cancelEditingTodo — Görev düzenlemeyi kaydetmeden iptal eder.

```
  cancelEditingTodo(): void {
```
'Vazgeç' butonuyla çağrılır.

```
    this.editingTodoId = null;
```
Düzenlenen görev kimliği temizlenir; form kapanır ve görev değişmeden kalır.

```
  }
```

ƒ saveTodo — Düzenleme formundaki değişiklikleri API'ye gönderir ve ekrandaki görevi günceller.

```
  async saveTodo(todo: TodoItem): Promise<void> {
```
Düzenleme formu kaydedildiğinde çalışır; hangi görevin güncelleneceği parametre olarak gelir.

```
    if (this.selectedList === null) {
      return;
    }
```
Seçili liste yoksa erken çıkılır.

```
    const todoListId = this.selectedList.id;
    const title = this.editedTodoTitle.trim();
```
Liste kimliği saklanır, düzenlenen başlık kırpılır.

```
    if (title.length === 0) {
      this.messages.warning('Görev başlığı boş olamaz.');
      return;
    }
```
Başlık boşsa uyarı verilir, istek gönderilmez.

```
    this.isSavingTodo = true;
```
Kaydetme bayrağı açılır.

282 `try {` 283 `const request = this.createTodoRequest(` 284 `title,` 285 `this.editedTodoDescription,` 286 `this.editedTodoDueAtLocal,` 287 `this.editedTodoPriority,` 288 `);`	Düzenleme formundaki alanlardan aynı yardımcı metotla `TodoRequest` gövdesi hazırlanır.
289 `const updatedTodo = await firstValueFrom(` 290 `this.todoApi.updateTodo(todoListId, todo.id, request),` 291 `);`	PUT isteğiyle görev güncellenir ve güncel hali beklenir.
293 `this.replaceTodo(updatedTodo);` 294 `this.editingTodoId = null;` 295 `this.messages.success('Görev güncellendi.');`	Dizideki eski görev sunucudan gelen yenisiyle değiştirilir, düzenleme kapanır ve başarı mesajı gösterilir.
296 `} catch (error: unknown) {` 297 `this.showError(error, 'Bu listede aynı isimli bir görev zaten var.');`	Hata olursa ortak yöntem çalışır; 409 aynı isimli görev ya da eşzamanlılık çakışması olabilir, `showError` ikisini ayırt eder.
298 `} finally {` 299 `this.isSavingTodo = false;` 300 `this.changeDetector.markForCheck();` 301 `}`	Bayrak kapanır, ekran yenilenmesi istenir.
302 `}`	
ƒ setTodoCompletion — Onay kutusu değişince görevin tamamlanma durumunu API'de günceller.	
304 `async setTodoCompletion(todo: TodoItem, isCompleted: boolean): Promise<void> {`	Görevin onay kutusu işaretlenince ya da kaldırılınca yeni değerle çağrılır.
305 `if (this.selectedList === null) {` 306 `return;` 307 `}`	Seçili liste yoksa erken çıkılır.
309 `const todoListId = this.selectedList.id;`	Liste kimliği saklanır.
311 `try {` 312 `const updatedTodo = await firstValueFrom(` 313 `this.todoApi.setTodoCompletionStatus(` 314 `todoListId,` 315 `todo.id,` 316 `isCompleted,` 317 `),` 318 `);`	PATCH isteğiyle sadece tamamlanma durumu gönderilir ve güncel görev beklenir.
320 `this.replaceTodo(updatedTodo);`	Ekrandaki görev sunucunun döndürdüğü haliyle değiştirilir.
321 `} catch (error: unknown) {` 322 `this.showError(error);` 323 `} finally {` 324 `this.changeDetector.markForCheck();` 325 `}`	Hata ortak yöntemle gösterilir; her durumda ekran yenilenmesi istenir.
326 `}`	
ƒ deleteTodo — Görevi API'de siler ve ekrandaki diziden çıkarır.	
328 `async deleteTodo(todo: TodoItem): Promise<void> {`	Silme onayından (popconfirm) sonra çağrılır.
329 `if (this.selectedList === null) {` 330 `return;` 331 `}`	Seçili liste yoksa erken çıkılır.
333 `const todoListId = this.selectedList.id;`	Liste kimliği saklanır.
335 `try {` 336 `await firstValueFrom(this.todoApi.deleteTodo(todoListId, todo.id));` 337 `this.todos = this.todos.filter((existingTodo) => existingTodo.id !== todo.id);` 338 `this.messages.success('Görev silindi.');`	DELETE isteği beklenir; `filter` silinen görev hariç yeni bir dizi üretir ve başarı mesajı gösterilir.
339 `} catch (error: unknown) {` 340 `this.showError(error);` 341 `} finally {` 342 `this.changeDetector.markForCheck();` 343 `}`	Hata ortak yöntemle gösterilir; her durumda ekran yenilenmesi istenir.
344 `}`	
ƒ completedTodoCount — Seçili listedeki tamamlanmış görev sayısını hesaplar.	
346 `completedTodoCount(): number {`	Şablondaki 'x / y tamamlandı' sayacı için kullanılır.
347 `return this.todos.filter((todo) => todo.isCompleted).length;`	Sadece `isCompleted` true olan görevler süzülür ve sayıları döndürülür.
348 `}`	
ƒ priorityLabel — Öncelik değerini ekranda gösterilecek Türkçe metne çevirir.	
350 `priorityLabel(priority: TodoPriority): string {`	API'deki İngilizce öncelik değerini kullanıcıya gösterilecek Türkçe etikete çevirir.
351 `if (priority === 'High') {` 352 `return 'Yüksek';` 353 `}`	'High' için 'Yüksek' döner.
355 `if (priority === 'Low') {` 356 `return 'Düşük';` 357 `}`	'Low' için 'Düşük' döner.
359 `return 'Normal';`	Geriye kalan tek değer 'Normal' olduğu için varsayılan olarak o döner.
360 `}`	

ƒ priorityColor — Öncelik değerine göre ng-zorro etiket rengini seçer.

```
362  priorityColor(priority: TodoPriority): string {
```
Şablondaki `nz-tag` bileşeninin `nzColor` değerini üretir.

```
363    if (priority === 'High') {
364      return 'red';
365    }
```
Yüksek öncelik kırmızı gösterilir.

```
367    if (priority === 'Low') {
368      return 'blue';
369    }
```
Düşük öncelik mavi gösterilir.

```
371    return 'gold';
```
Normal öncelik altın sarısı gösterilir.

```
372  }
```

ƒ createTodoRequest — Form alanlarından API'ye gönderilecek `TodoRequest` nesnesini üretir.

```
374  private createTodoRequest(
375    title: string,
376    description: string,
377    dueAtLocal: string,
378    priority: TodoPriority,
379  ): TodoRequest {
```
`private` yardımcı metot; oluşturma ve güncelleme aynı gövdeyi kullandığı için kod tekrarını önler. Dönüş tipi `TodoRequest`.

```
380    return {
381      title,
```
Nesne döndürülür; `title` kısaltılmış yazımdır, yani `title: title`.

```
382      description: description.trim().length > 0 ? description.trim() : null,
```
Üçlü operatör: açıklama kırpıldıktan sonra doluysa gönderilir, boşsa null gönderilir; veritabanına boş metin yazılmaz.

```
383      dueAtUtc: this.toUtcDateTimeValue(dueAtLocal),
384      priority,
385    };
```
Yerel son tarih UTC ISO metnine çevrilir (boşsa null); öncelik olduğu gibi eklenir.

```
386  }
```

ƒ minimumDueAtLocal — Tarih input'unun seçilebilecek en erken değerini (şu an) yerel biçimde verir.

```
388  minimumDueAtLocal(): string {
```
Şablonda `[min]` ile bağlanır; geçmiş bir son tarih seçilmesini tarayıcı düzeyinde engeller.

```
389    return this.toLocalDateTimeValue(new Date().toISOString());
```
Şimdiki zaman önce UTC ISO metnine, sonra `datetime-local` biçimindeki yerel saate çevrilir.

```
390  }
```

ƒ toUtcDateTimeValue — `datetime-local` input'undaki yerel saati UTC ISO metnine çevirir.

```
392  private toUtcDateTimeValue(localDateTime: string): string | null {
```
Kullanıcının yerel saatle seçtiği tarihi sunucunun beklediği UTC biçimine dönüştürür.

```
393    if (localDateTime.length === 0) {
394      return null;
395    }
```
Tarih seçilmemişse son tarih yok demektir, null döner.

```
397    return new Date(localDateTime).toISOString();
```
`new Date('2026-10-08T14:30')` metni yerel saat olarak yorumlar; `toISOString` onu UTC'ye çevirip 'Z' ekli metin üretir.

```
398  }
```

ƒ toLocalDateTimeValue — API'den gelen UTC tarihi `datetime-local` input'unun beklediği yerel 'YYYY-MM-DDTHH:mm' metnine çevirir.

```
400  private toLocalDateTimeValue(utcDateTime: string | null): string {
```
Ters dönüşüm: sunucudaki UTC tarihi input'ta gösterilecek yerel saate çevirir.

```
401    if (utcDateTime === null) {
402      return '';
403    }
```
Tarih yoksa input boş kalsın diye boş metin döner.

```
405    const date = new Date(utcDateTime);
```
UTC metni bir `Date` nesnesine çevrilir.

```
406    const localOffsetInMilliseconds = date.getTimezoneOffset() * 60_000;
407    const localDate = new Date(date.getTime() - localOffsetInMilliseconds);
```
`getTimezoneOffset` UTC ile yerel saat farkını dakika verir (Türkiye için -180). Milisaniyeye çevrilip çıkarılınca saat yerel saate kaydırılır.

```
409    return localDate.toISOString().slice(0, 16);
```
Kaydırılmış tarihin ISO metninin ilk 16 karakteri alınır: 'YYYY-MM-DDTHH:mm', yani `datetime-local` input biçimi.

```
410  }
```

ƒ replaceTodo — Dizideki aynı kimlikli görevi güncel haliyle değiştirir.

```
412  private replaceTodo(updatedTodo: TodoItem): void {
```
Güncelleme ve tamamlanma işlemlerinden sonra ekrandaki görevi yenilemek için kullanılır.

```
413    this.todos = this.todos.map((todo) =>
414      todo.id === updatedTodo.id ? updatedTodo : todo,
415    );
```
`map` ile yeni dizi üretilir: kimliği eşleşen görev yenisiyle değişir, diğerleri aynen kalır. Diziyi değiştirmek yerine yenisini atamak değişikliği belirgin kılar.

```
416  }
```

ƒ showError — API hatasını tipine ve durum koduna göre kullanıcıya uygun Türkçe mesaja çevirip gösterir.

```
418  private showError(error: unknown, conflictMessage?: string): void {
```
Tüm `catch` bloklarının kullandığı ortak hata yöneticisi. `conflictMessage?` isteğe bağlıdır: 409 durumunda gösterilecek özel metin.

```
419    let message = 'İstek tamamlanamadı.';
```
Hiçbir özel durum eşleşmezse gösterilecek varsayılan mesaj.

```
421    if (error instanceof HttpErrorResponse) {
```
`instanceof` hatanın gerçekten bir HTTP yanıt hatası olup olmadığını kontrol eder; değilse varsayılan mesaj kullanılır.

```
422      const problem = error.error as { detail?: string; title?: string; type?: str
ing } | null;
```
`error.error` sunucunun gövdesidir; RFC ProblemDetails alanları (`detail`, `title`, `type`) olabileceği için `as` ile bu şekle dönüştürülür.

```
      message = problem?.detail ?? problem?.title ?? `Sunucu yanıtı: ${error.status}`;
```
Önce `detail`, yoksa `title`, o da yoksa durum kodunu içeren metin kullanılır; `${}` şablon metin içine değer yerleştirir.

```
      if (error.status === 409 && problem?.type === CONCURRENCY_CONFLICT_PROBLEM_TYPE) {
        this.messages.warning('Bu kayıt başka bir sekmede ya da cihazda değiştirildi veya silindi. Liste yenilendi.');
        void this.loadTodoLists();
        return;
      }
```
409 ve eşzamanlılık çakışması tipi gelirse kayıt başka sekmede/cihazda değişmiş ya da silinmiştir: uyarı gösterilir, listeler yeniden yüklenir ve çıkılır.

```
      if (error.status === 409 && conflictMessage !== undefined) {
        message = conflictMessage;
      }
```
Diğer 409'lar (örneğin aynı isim) için çağıran metot özel mesaj verdiyse o kullanılır.

```
      if (error.status === 503) {
        message = 'Sunucu şu anda veritabanına ulaşamıyor. Biraz sonra tekrar dene.';
      }
```
503, backend'in veritabanına ulaşamadığını bildirdiği durumdur; kullanıcıya biraz sonra denemesi söylenir.

```
      if (error.status >= 500 && error.status !== 503) {
        message = 'Beklenmeyen bir hata oluştu. Sorun devam ederse yöneticine bildir.';
      }
```
Diğer 5xx hatalarında teknik ayrıntı gizlenir, genel bir mesaj gösterilir.

```
      if (error.status === 401) {
        return;
      }
```
401'de mesaj gösterilmez; `authInterceptor` oturumu yenilemeyi dener, başaramazsa oturumu kapatıp girişe yönlendirir.

```
      if (error.status === 0) {
        this.isApiAvailable = false;
        message = 'API sunucusuna bağlanılamadı. HerSeyiBaslat.bat ile uygulamayı aç.';
      }
```
Durum 0, sunucuya hiç ulaşılamadığı anlamına gelir (API kapalı/ağ hatası); bağlantı göstergesi kırmızıya döner ve nasıl açılacağı söylenir.

```
    }
    this.messages.error(message);
  }
}
```
Belirlenen mesaj ng-zorro'nun hata bildirimi olarak ekranda gösterilir.

Web (Angular) `todo.Web/src/app/services/todo-api.service.ts`

> Liste ve görev uç noktalarına giden tüm HTTP çağrılarını tek yerde toplayan Angular servisidir. Sayfa bileşenleri URL bilmez, sadece bu servisin metotlarını çağırır.

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
```
Angular'ın servis işaretleyicisi `Injectable`, bağımlılık almak için `inject`, HTTP istemcisi `HttpClient` ve RxJS'in `Observable` tipi içe aktarılır.

```
import { resolveApiUrl } from '../core/api-url';
import { TodoItem, TodoList, TodoListRequest, TodoRequest } from '../models/todo.models';
```
API taban adresini bulan yardımcı fonksiyon ve istek/yanıt tipleri (models dosyası) içe aktarılır.

```
@Injectable({
  providedIn: 'root',
})
```
`@Injectable` dekoratörü sınıfı DI sistemine tanıtır; `providedIn: 'root'` tüm uygulamada tek bir örnek (singleton) oluşturulmasını sağlar.

ƒ class TodoApiService — Listeler ve görevler için REST API çağrılarını yapan, Observable döndüren servis.

```
export class TodoApiService {
```
Servis sınıfı; bileşenler bunu `inject(TodoApiService)` ile alır.

```
  private readonly http = inject(HttpClient);
```
`inject(HttpClient)` constructor yazmadan HttpClient'ı alır. Bu istemci `authInterceptor`'dan geçtiği için Authorization başlığı otomatik eklenir.

```
  private readonly apiUrl = resolveApiUrl();
```
API adresi bir kez hesaplanır: pencerede `__TODO_API_URL__` varsa o, yoksa `http://localhost:7070/api` kullanılır.

ƒ getTodoLists — Kullanıcının tüm listelerini GET ile çeker.

```
  getTodoLists(): Observable<TodoList[]> {
    return this.http.get<TodoList[]>(`${this.apiUrl}/todo-lists`);
  }
```
Dönüş tipi `Observable<TodoList[]>`: istek abone olunana kadar gönderilmez, sonuç ileride akış olarak gelir. `GET /api/todo-lists` isteği; `<TodoList[]>` jenerik parametresi yanıtın tipini belirtir. Şablon metin (backtick) ile URL birleştirilir.

ƒ createTodoList — Yeni liste oluşturmak için POST isteği gönderir.

```
  createTodoList(request: TodoListRequest): Observable<TodoList> {
    return this.http.post<TodoList>(`${this.apiUrl}/todo-lists`, request);
  }
```
Gövde olarak `{ name }` içeren isteği alır, oluşan listeyi döndürür. `POST /api/todo-lists`; HttpClient nesneyi otomatik JSON'a çevirir. Aynı isimli liste varsa API 409 döner.

ƒ renameTodoList — Var olan bir listenin adını PUT ile değiştirir.

```
  renameTodoList(todoListId: number, request: TodoListRequest): Observable<TodoList> {
    return this.http.put<TodoList>(`${this.apiUrl}/todo-lists/${todoListId}`, request);
  }
```
Hangi listenin değişeceği `todoListId` ile, yeni ad `request` ile verilir. `PUT /api/todo-lists/{id}` isteği; güncellenmiş listeyi geri alır.

ƒ deleteTodoList — Bir listeyi ve içindeki görevleri DELETE ile siler.

```
  deleteTodoList(todoListId: number): Observable<void> {
```
Yanıt gövdesi olmadığı için dönüş tipi `Observable<void>`.

```
    return this.http.delete<void>(`${this.apiUrl}/todo-lists/${todoListId}`);
  }
```

`DELETE /api/todo-lists/{id}`; sunucu başarılıysa 204 No Content döner.

ƒ getTodos — Seçili listenin görevlerini GET ile çeker.

```
  getTodos(todoListId: number): Observable<TodoItem[]> {
    return this.http.get<TodoItem[]>(`${this.apiUrl}/todo-lists/${todoListId}/todos`);
  }
```

Belirli bir listenin görevlerini getirir.

`GET /api/todo-lists/{id}/todos`; görevler listenin alt kaynağı olduğu için URL iç içedir (backend `TodosController` rotası).

ƒ createTodo — Seçili listeye yeni görev ekler (POST); "Görev oluşturma" akışının HTTP adımıdır.

```
  createTodo(todoListId: number, request: TodoRequest): Observable<TodoItem> {
    return this.http.post<TodoItem>(`${this.apiUrl}/todo-lists/${todoListId}/todos`, request);
  }
```

Görev oluşturma akışının API çağrısı: liste kimliği ve `TodoRequest` gövdesi alır, oluşan `TodoItem`'ı döndürür.

`POST /api/todo-lists/{id}/todos`; backend `TodosController.Create` çalışır ve 201 Created ile yeni görevi döndürür.

ƒ updateTodo — Var olan bir görevin başlık, açıklama, tarih ve önceliğini PUT ile günceller.

```
  updateTodo(todoListId: number, todoId: number, request: TodoRequest): Observable<TodoItem> {
    return this.http.put<TodoItem>(
      `${this.apiUrl}/todo-lists/${todoListId}/todos/${todoId}`,
      request,
    );
  }
```

Hem liste hem görev kimliği gerekir çünkü görev URL'de listenin altındadır.

`PUT /api/todo-lists/{listId}/todos/{todoId}`; gövde olarak aynı `TodoRequest` gönderilir, güncel görev geri gelir.

ƒ setTodoCompletionStatus — Görevi tamamlandı/tamamlanmadı olarak işaretler (PATCH).

```
  setTodoCompletionStatus(
    todoListId: number,
    todoId: number,
    isCompleted: boolean,
  ): Observable<TodoItem> {
    return this.http.patch<TodoItem>(
      `${this.apiUrl}/todo-lists/${todoListId}/todos/${todoId}/completion`,
      { isCompleted },
    );
  }
```

Sadece tamamlanma durumunu değiştiren metot; parametreler okunaklılık için alt alta yazılmıştır.

`PATCH .../completion` kısmi güncellemedir; gövde `{ isCompleted }` kısaltılmış nesne yazımıdır, yani `{ isCompleted: isCompleted }`.

ƒ deleteTodo — Tek bir görevi DELETE ile siler.

```
  deleteTodo(todoListId: number, todoId: number): Observable<void> {
    return this.http.delete<void>(
      `${this.apiUrl}/todo-lists/${todoListId}/todos/${todoId}`,
    );
  }
}
```

Bir görevi silmek için liste ve görev kimliğini alır.

`DELETE /api/todo-lists/{listId}/todos/{todoId}`; başarılıysa 204, bulunamazsa 404 döner.

API `todo.API/Controllers/TodosController.cs`

> Görevlerle (todo) ilgili HTTP uçlarını sunan API katmanı controller'ı. İsteği alır, Application komutuna çevirir, use case'i çağırır ve sonucu uygun HTTP koduna dönüştürür.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
```

ASP.NET Core'un yetkilendirme (`[Authorize]`) ve MVC (controller, `ActionResult`) türlerini kullanabilmek için gerekli namespace'ler.

```
using TodoApi.API.Models.Requests;
using TodoApi.Application.Contracts;
using TodoApi.Application.Features.Todos.CreateTodo;
using TodoApi.Application.Features.Todos.DeleteTodo;
using TodoApi.Application.Features.Todos.GetTodoById;
using TodoApi.Application.Features.Todos.GetTodosInList;
using TodoApi.Application.Features.Todos.SetTodoCompletionStatus;
using TodoApi.Application.Features.Todos.UpdateTodo;
```

API'nin kendi istek modelleri, Application'daki DTO'lar ve her görev işlemi için ayrı use case/command klasörleri. Her özellik kendi klasöründe durur (feature-based yapı).

```
namespace TodoApi.API.Controllers;
```

Dosya kapsamlı namespace: bu dosyadaki her şey `TodoApi.API.Controllers` altındadır; ekstra süslü parantez gerekmez.

ƒ TodosController — Bir listeye ait görevleri listeleme, oluşturma, okuma, güncelleme, tamamlama ve silme uçlarını barındırır.

```
[ApiController]
```

`[ApiController]`: otomatik model doğrulama (geçersiz gövdede 400), `[FromBody]` çıkarımı gibi API davranışlarını açar.

```
[Authorize]
```

`[Authorize]`: tüm uçlar giriş yapmış kullanıcı ister; token yoksa istek controller'a ulaşmadan 401 döner.

```
[Route("api/todo-lists/{todoListId:int}/todos")]
```

Temel adres. `{todoListId:int}` route kısıtıdır: sadece tam sayı eşleşir. Görevler REST kuralına uygun olarak listenin altında iç içe kaynak olarak tanımlanır.

```
public sealed class TodosController : ControllerBase
```

`sealed` sınıftan türetilemez demektir. `ControllerBase` görünüm (view) içermeyen API controller'ları için `Ok`, `NotFound` gibi yardımcıları verir.

```
{
```

```
    private readonly CreateTodoUseCase _createTodoUseCase;
    private readonly GetTodosInListUseCase _getTodosInListUseCase;
    private readonly GetTodoByIdUseCase _getTodoByIdUseCase;
    private readonly UpdateTodoUseCase _updateTodoUseCase;
    private readonly SetTodoCompletionStatusUseCase _setTodoCompletionStatusUseCas
e;
    private readonly DeleteTodoUseCase _deleteTodoUseCase;
```

Her işlem için ayrı bir use case alanı. `readonly` olduğundan yalnızca constructor'da atanabilir; controller iş kuralı içermez, işi bu sınıflara devreder.

ƒ TodosController (constructor) — Dependency Injection ile altı use case'i alır ve alanlara saklar.

```
    public TodosController(
        CreateTodoUseCase createTodoUseCase,
        GetTodosInListUseCase getTodosInListUseCase,
        GetTodoByIdUseCase getTodoByIdUseCase,
        UpdateTodoUseCase updateTodoUseCase,
        SetTodoCompletionStatusUseCase setTodoCompletionStatusUseCase,
        DeleteTodoUseCase deleteTodoUseCase)
```

Constructor parametreleri DI konteynerinden otomatik gelir; controller nesneleri kendisi `new` ile yaratmaz, bu sayede test edilebilir ve gevşek bağlı olur.

```
    {
        _createTodoUseCase = createTodoUseCase;
        _getTodosInListUseCase = getTodosInListUseCase;
        _getTodoByIdUseCase = getTodoByIdUseCase;
        _updateTodoUseCase = updateTodoUseCase;
        _setTodoCompletionStatusUseCase = setTodoCompletionStatusUseCase;
        _deleteTodoUseCase = deleteTodoUseCase;
    }
```

Gelen use case örnekleri ilgili `readonly` alanlara atanır; metotlar bunları kullanır.

ƒ GetForList — Bir listedeki tüm görevleri döner; liste yoksa veya kullanıcıya ait değilse 404.

```
    [HttpGet]
```

`[HttpGet]` : bu metot temel adrese gelen GET isteklerini karşılar (GET api/todo-lists/5/todos).

```
    public async Task<ActionResult<IReadOnlyList<TodoItemDto>>> GetForList(
        [FromRoute] int todoListId,
        CancellationToken cancellationToken)
```

`async Task<ActionResult<T>>` : asenkron çalışır, ya veri (T) ya da HTTP sonucu döner. `[FromRoute]` değeri URL'den alır. `CancellationToken` istemci bağlantıyı keserse işi iptal etmeye yarar.

```
    {
        IReadOnlyList<TodoItemDto>? todos = await _getTodosInListUseCase.ExecuteAs
ync(
            todoListId,
            cancellationToken);
```

`await` ile use case çağrılır; beklerken thread serbest kalır. `?` sonucun null olabileceğini belirtir.

```
        if (todos == null)
        {
            return NotFound();
        }
```

null, liste yok ya da başkasına ait demektir. İkisinde de 404 döner ki saldırgan başkasının listesinin var olduğunu öğrenemesin.

```
        return Ok(todos);
    }
```

Başarılı durumda 200 OK ile görev DTO listesi JSON olarak döner.

ƒ Create — İstek modelini CreateTodoCommand'a çevirip yeni görev oluşturur; başarıda 201 Created döner.

```
    [HttpPost]
```

`[HttpPost]` : yeni kaynak oluşturmak için POST isteğini karşılar (POST api/todo-lists/5/todos).

```
    public async Task<ActionResult<TodoItemDto>> Create(
        [FromRoute] int todoListId,
        [FromBody] CreateTodoRequest request,
        CancellationToken cancellationToken)
```

Liste kimliği URL'den, görev bilgileri `[FromBody]` ile JSON gövdesinden `CreateTodoRequest` olarak okunur.

```
    {
        CreateTodoCommand command = new CreateTodoCommand
        {
            TodoListId = todoListId,
            Title = request.Title,
            Description = request.Description,
            DueAtUtc = request.DueAtUtc,
            Priority = request.Priority
        };
```

Object initializer ile API istek modeli Application komutuna eşlenir. Böylece API modelleri Application katmanına hiç girmez; katmanlar birbirinden bağımsız kalır.

```
        TodoItemDto? createdTodo = await _createTodoUseCase.ExecuteAsync(
            command,
            cancellationToken);
```

Use case çalıştırılır: doğrulama, sahiplik kontrolü ve kayıt orada yapılır. Doğrulama hatası olursa exception fırlar ve global handler 400 döner.

```
        if (createdTodo == null)
        {
            return NotFound();
        }
```

Liste bulunamazsa veya kullanıcıya ait değilse use case null döner; controller bunu 404'e çevirir.

```
        return CreatedAtAction(
            nameof(GetById),
            new { todoListId, todoId = createdTodo.Id },
            createdTodo);
    }
```

`CreatedAtAction` 201 döner ve `Location` başlığına yeni görevin adresini yazar. `nameof(GetById)` metot adını derleme zamanında güvenle string yapar; anonim nesne route değerlerini verir.

ƒ GetById — Listedeki tek bir görevi kimliğiyle getirir; bulunamazsa 404.

```
    [HttpGet("{todoId:int}")]
```

`[HttpGet("{todoId:int}")]` : temel adrese `/5` gibi bir tam sayı eklenen GET isteklerini karşılar.

```
    public async Task<ActionResult<TodoItemDto>> GetById(
        [FromRoute] int todoListId,
        [FromRoute] int todoId,
        CancellationToken cancellationToken)
```

Hem liste hem görev kimliği URL'den alınır; görevin gerçekten o listeye ait olduğu use case içinde kontrol edilir.

```
    {
```

```
        TodoItemDto? todo = await _getTodoByIdUseCase.ExecuteAsync(
            todoListId,
            todoId,
            cancellationToken);
```

Use case çağrılır; liste/görev yoksa veya erişim yoksa null döner.

```
        if (todo == null)
        {
            return NotFound();
        }

        return Ok(todo);
    }
```

null ise 404, değilse 200 OK ile görev DTO'su döner.

ƒ Update — Görevin başlık, açıklama, bitiş tarihi ve önceliğini günceller; başarıda 200 OK.

```
    [HttpPut("{todoId:int}")]
```

`[HttpPut]`: kaynağın tamamını güncelleme isteği (PUT api/todo-lists/5/todos/7).

```
    public async Task<ActionResult<TodoItemDto>> Update(
        [FromRoute] int todoListId,
        [FromRoute] int todoId,
        [FromBody] UpdateTodoRequest request,
        CancellationToken cancellationToken)
    {
        UpdateTodoCommand command = new UpdateTodoCommand
        {
            TodoListId = todoListId,
            TodoId = todoId,
            Title = request.Title,
            Description = request.Description,
            DueAtUtc = request.DueAtUtc,
            Priority = request.Priority
        };
```

Route'tan iki kimlik, gövdeden `UpdateTodoRequest` alınır.

İstek modeli `UpdateTodoCommand`'a eşlenir. Geçmiş tarih kontrolü use case'te yalnızca tarih gerçekten değiştiyse yapılır; böylece süresi geçmiş görevin başlığı düzenlenebilir.

```
        TodoItemDto? updatedTodo = await _updateTodoUseCase.ExecuteAsync(
            command,
            cancellationToken);
```

Güncelleme use case'i çalıştırılır; kurallar her şeyden önce kontrol edilir, sonra değişiklik yapılır.

```
        if (updatedTodo == null)
        {
            return NotFound();
        }

        return Ok(updatedTodo);
    }
```

Bulunamazsa 404, başarılıysa güncel görev 200 OK ile döner.

ƒ SetCompletionStatus — Görevi tamamlandı veya tamamlanmadı olarak işaretler; sadece bu alanı değiştirir.

```
    [HttpPatch("{todoId:int}/completion")]
```

`[HttpPatch]`: kaynağın bir kısmını değiştirir. `/completion` alt yolu sadece tamamlanma durumunu hedefler.

```
    public async Task<ActionResult<TodoItemDto>> SetCompletionStatus(
        [FromRoute] int todoListId,
        [FromRoute] int todoId,
        [FromBody] SetTodoCompletionStatusRequest request,
        CancellationToken cancellationToken)
    {
        if (request.IsCompleted.HasValue == false)
        {
            return BadRequest();
        }
```

Gövdeden yalnızca `IsCompleted` alanını taşıyan `SetTodoCompletionStatusRequest` okunur.

Ek güvenlik: değer gönderilmemişse (null) 400 BadRequest. Normalde `[Required]` + `[ApiController]` bunu zaten yakalar.

```
        TodoItemDto? updatedTodo = await _setTodoCompletionStatusUseCase.ExecuteAsync(
            todoListId,
            todoId,
            request.IsCompleted.Value,
            cancellationToken);
```

`.Value` ile nullable bool'un gerçek değeri alınır ve use case'e iletilir; yukarıdaki kontrol sayesinde güvenlidir.

```
        if (updatedTodo == null)
        {
            return NotFound();
        }

        return Ok(updatedTodo);
    }
```

Bulunamazsa 404, başarılıysa güncel görev 200 OK ile döner.

ƒ Delete — Görevi siler; başarıda 204 No Content, bulunamazsa 404 döner.

```
    [HttpDelete("{todoId:int}")]
```

`[HttpDelete]`: kaynağı silme isteği (DELETE api/todo-lists/5/todos/7).

```
    public async Task<IActionResult> Delete(
        [FromRoute] int todoListId,
        [FromRoute] int todoId,
        CancellationToken cancellationToken)
    {
        bool todoWasDeleted = await _deleteTodoUseCase.ExecuteAsync(
            todoListId,
            todoId,
            cancellationToken);
```

Dönüş tipi `IActionResult`: gövdede veri dönmeyecek, sadece durum kodu döneceği için generic tip gerekmez.

Use case silme işlemini yapar ve başarılı olup olmadığını `bool` olarak bildirir.

```
        if (todoWasDeleted == false)
        {
            return NotFound();
        }

        return NoContent();
    }
}
```

Silinemediyse (yok veya erişim yok) 404; silindiyse 204 No Content, REST'te gövdesiz başarılı silme yanıtıdır.

API `todo.API/Models/Requests/CreateTodoRequest.cs`

> Yeni görev oluşturma isteğinde istemcinin gönderdiği JSON gövdesinin API katmanındaki karşılığı. Controller bunu Application komutuna çevirir.

```
using TodoApi.Domain.Enums;
```

`TodoPriority` enum'u Domain katmanında tanımlı; öncelik alanı için kullanılır.

```
namespace TodoApi.API.Models.Requests;
```

Dosya kapsamlı namespace: istek modelleri API katmanının `Models.Requests` klasöründe toplanır.

ƒ CreateTodoRequest — POST ile gelen yeni görev verisini (başlık, açıklama, bitiş tarihi, öncelik) taşır.

```
public sealed class CreateTodoRequest
```

`sealed` sınıf; sadece veri taşır, iş kuralı yoktur. Model binding JSON'u bu sınıfa doldurur.

```
{
```

```
    public string Title { get; set; } = string.Empty;
```

Görev başlığı. `= string.Empty` varsayılanı null olmasını önler; boş olup olmadığını validator kontrol eder.

```
    public string? Description { get; set; }
```

Açıklama isteğe bağlıdır; `string?` null olabileceğini gösterir.

```
    public DateTime? DueAtUtc { get; set; }
```

Bitiş tarihi (UTC) isteğe bağlıdır. `DateTime?` nullable demektir; gönderilmezse görevin son tarihi olmaz.

```
    public TodoPriority Priority { get; set; } = TodoPriority.Normal;
```

Öncelik; gönderilmezse `Normal` varsayılır. Geçersiz enum değeri validator'da `IsInEnum` ile yakalanır.

```
}
```

API `todo.API/Models/Requests/UpdateTodoRequest.cs`

> Görev güncelleme (PUT) isteğinin JSON gövdesini temsil eden API modeli. Controller bunu UpdateTodoCommand'a eşler.

```
using TodoApi.Domain.Enums;
```

Öncelik alanı için Domain'deki `TodoPriority` enum'u kullanılır.

```
namespace TodoApi.API.Models.Requests;
```

Dosya kapsamlı namespace; API istek modellerinin yeri.

ƒ UpdateTodoRequest — PUT ile gelen güncel görev bilgilerini taşır; görev kimliği gövdede değil URL'dedir.

```
public sealed class UpdateTodoRequest
```

Create isteğiyle aynı alanlara sahip ama ayrı sınıf; ileride iki işlem farklılaşırsa birbirini etkilemez.

```
{
```

```
    public string Title { get; set; } = string.Empty;
```

Yeni başlık; varsayılan boş string, kontrolü validator yapar.

```
    public string? Description { get; set; }
```

Yeni açıklama; null olabilir.

```
    public DateTime? DueAtUtc { get; set; }
```

Yeni bitiş tarihi; null gönderilirse tarih kaldırılır. Geçmiş tarih kontrolü sadece tarih değiştiyse uygulanır.

```
    public TodoPriority Priority { get; set; } = TodoPriority.Normal;
```

Yeni öncelik; gönderilmezse `Normal` olur.

```
}
```

API `todo.API/Models/Requests/SetTodoCompletionStatusRequest.cs`

> Görevi tamamlandı/tamamlanmadı yapan PATCH isteğinin gövde modeli. Sadece tek bir alan taşır.

```
using System.ComponentModel.DataAnnotations;
```

`[Required]` gibi Data Annotation özniteliklerini kullanmak için gerekli namespace.

```
namespace TodoApi.API.Models.Requests;
```

Dosya kapsamlı namespace; API istek modelleri.

ƒ SetTodoCompletionStatusRequest — PATCH isteğinde görevin yeni tamamlanma durumunu taşır.

```
public sealed class SetTodoCompletionStatusRequest
```

Sadece tamamlanma durumunu değiştiren küçük, amaca özel istek modeli.

```
{
```

```
    [Required]
    public bool? IsCompleted { get; set; }
```

`bool?` kullanılır çünkü düz `bool` gönderilmeyince sessizce false olurdu. `[Required]` alan eksikse `[ApiController]` sayesinde otomatik 400 döndürür.

```
}
```

Application `todo.Application/Features/Todos/CreateTodo/CreateTodoCommand.cs`

> Görev oluşturma use case'inin girdi modeli. Controller HTTP isteğini buna çevirir; Application katmanı yalnızca bu komutu tanır.

```
using System;
using TodoApi.Domain.Enums;
```

`DateTime` için `System`, öncelik enum'u için Domain'in `Enums` namespace'i.

```
namespace TodoApi.Application.Features.Todos.CreateTodo;
```

Komut, validator ve use case aynı `CreateTodo` özellik klasöründe durur.

ƒ CreateTodoCommand — Yeni görev için gereken tüm verileri (liste kimliği dahil) taşıyan değişmez komut nesnesi.

```
6  public sealed class CreateTodoCommand
```

API modelinden bağımsız Application sözleşmesi; böylece Application, ASP.NET'e veya HTTP'ye bağımlı olmaz.

```
7  {
8      public int TodoListId { get; init; }
```

Görevin ekleneceği listenin kimliği (URL'den gelir). `init` yalnızca nesne oluşturulurken atanabilir, sonra değiştirilemez.

```
10     public string Title { get; init; } = string.Empty;
```

Görev başlığı; varsayılan boş string. Boş ve uzunluk kontrolü `CreateTodoCommandValidator`'da yapılır.

```
12     public string? Description { get; init; }
```

İsteğe bağlı açıklama.

```
14     public DateTime? DueAtUtc { get; init; }
```

İsteğe bağlı bitiş tarihi (UTC); geçmişte veya 10 yıldan ileride olamaz.

```
16     public TodoPriority Priority { get; init; } = TodoPriority.Normal;
```

Görev önceliği; varsayılanı `Normal`.

```
17 }
```

Application `todo.Application/Features/Todos/CreateTodo/CreateTodoCommandValidator.cs`

> CreateTodoCommand için FluentValidation kurallarını tanımlar. Mekanik (biçimsel) hataları use case iş yapmadan önce yakalar.

```
1  using FluentValidation;
2  using TodoApi.Application.Common.Validation;
```

FluentValidation kütüphanesi ve projenin ortak kural uzantıları (`SharedValidationRules`) içeri alınır.

```
4  namespace TodoApi.Application.Features.Todos.CreateTodo;
```

Validator, ait olduğu komutla aynı özellik klasöründe bulunur.

ƒ CreateTodoCommandValidator — Yeni görev komutundaki liste kimliği, başlık, açıklama, bitiş tarihi ve önceliği doğrular.

```
6  public sealed class CreateTodoCommandValidator : AbstractValidator<CreateTodoCommand>
```

`AbstractValidator<CreateTodoCommand>`: generic taban sınıf; hangi tipi doğrulayacağını `<...>` içinde belirtiriz.

```
7  {
```

ƒ CreateTodoCommandValidator (constructor) — Kuralları kaydeder; tarih kontrolü için TimeProvider alır.

```
8      public CreateTodoCommandValidator(TimeProvider timeProvider)
```

Kurallar constructor içinde tanımlanır. `TimeProvider` şimdiki zamanı verir; testlerde sahte saat verilebilsin diye `DateTime.UtcNow` yerine enjekte edilir.

```
9      {
10         if (timeProvider == null)
11         {
12             throw new ArgumentNullException(nameof(timeProvider));
13         }
```

Bağımlılık null gelirse hemen `ArgumentNullException` fırlatılır; hata kaynağında yakalanır. `nameof` parametre adını güvenle string yapar.

```
15         RuleFor(command => command.TodoListId).EntityId();
```

`RuleFor` hangi özelliğin doğrulanacağını lambda ile seçer. `EntityId()` ortak uzantıdır: kimlik 0'dan büyük olmalı.

```
16         RuleFor(command => command.Title).TodoTitle();
17         RuleFor(command => command.Description).TodoDescription();
```

Başlık boş olamaz ve en fazla 120 karakter; açıklama en fazla 1000 karakter. Kurallar ortak dosyada olduğu için Create ve Update aynı sınırları kullanır.

```
18         RuleFor(command => command.DueAtUtc).NewTodoDueDate(timeProvider);
```

Yeni görev kuralı: bitiş tarihi geçmişte olamaz (5 dakika saat toleransı) ve 10 yıldan ileride olamaz.

```
19         RuleFor(command => command.Priority).TodoPriority();
```

Öncelik, `TodoPriority` enum'unda tanımlı bir değer olmalı.

```
20     }
21 }
```

Application `todo.Application/Common/Validation/SharedValidationRules.cs`

> Birden fazla validator'ın kullandığı ortak FluentValidation kurallarını ve sınır değerlerini tek yerde toplar. Böylece Create ve Update aynı kuralları paylaşır.

```
1  using FluentValidation;
2  using TodoApi.Domain.Enums;
```

FluentValidation türleri ve öncelik kuralı için Domain'deki `TodoPriority` enum'u.

```
4  namespace TodoApi.Application.Common.Validation;
```

Ortak doğrulama kodu Application katmanının `Common.Validation` klasöründe durur.

ƒ SharedValidationRules — Liste adı, görev başlığı, açıklama, öncelik, bitiş tarihi ve kimlik için yeniden kullanılabilir kurallar.

```
6  public static class SharedValidationRules
```

`static class`: örneği oluşturulamaz; extension metotlar yalnızca static sınıflarda tanımlanabilir.

```
7  {
8      public const int TodoListNameMaximumLength = 80;
9      public const int TodoTitleMaximumLength = 120;
10     public const int TodoDescriptionMaximumLength = 1000;
11     public const int TodoDueDateMaximumYearsAhead = 10;
```

Sınır değerleri `const` sabitler: liste adı 80, başlık 120, açıklama 1000 karakter, bitiş tarihi en fazla 10 yıl ileri. Sihirli sayılar kod içine dağılmaz.

```
12     public const string TodoDueDateInThePastMessage = "'Bitiş tarihi' geçmişte olamaz.";
13     public const string TodoDueDateTooFarAheadMessage = "'Bitiş tarihi' en fazla 10 yıl sonrası olabilir.";
```

Tarih hataları için Türkçe mesajlar. `public` oldukları için UpdateTodoUseCase gibi yerler de aynı mesajı kullanabilir.

```
15     public static readonly TimeSpan TodoDueDateClockTolerance = TimeSpan.FromMinutes(5);
```

5 dakikalık saat toleransı: istemci ile sunucu saati biraz farklı olsa bile "şimdi" seçilen tarih geçmiş sayılmaz. `TimeSpan` sabit olamadığı için `static readonly`.

ƒ TodoListName — Liste adı boş olamaz ve en fazla 80 karakter olabilir kuralını uygular.

```
17     public static IRuleBuilderOptions<T, string> TodoListName<T>(this IRuleBuilder<T, string> rule)
```

Extension metot: `this IRuleBuilder<T, string>` sayesinde `RuleFor(x => x.Name).TodoListName()` şeklinde çağrılır. `<T>` her komut tipinde çalışmasını sağlar.

```
    {
        return rule
            .NotEmpty()
            .MaximumLength(TodoListNameMaximumLength)
            .WithName("Liste adı");
    }
```

`NotEmpty` boş/null'u reddeder, `MaximumLength` uzunluğu sınırlar, `WithName` hata mesajında görünen alan adını Türkçe yapar.

ƒ TodoTitle — Görev başlığı boş olamaz ve en fazla 120 karakter olabilir.

```
    public static IRuleBuilderOptions<T, string> TodoTitle<T>(this IRuleBuilder<T, string> rule)
    {
        return rule
            .NotEmpty()
            .MaximumLength(TodoTitleMaximumLength)
            .WithName("Görev adı");
    }
```

Görev başlığı için ortak kural; Create ve Update validator'ları bunu kullanır.

Zorunlu, en fazla 120 karakter; mesajlarda alan adı "Görev adı" olarak görünür.

ƒ TodoDescription — Açıklamanın en fazla 1000 karakter olmasını sağlar; boş bırakılabilir.

```
    public static IRuleBuilderOptions<T, string?> TodoDescription<T>(this IRuleBuilder<T, string?> rule)
    {
        return rule
            .MaximumLength(TodoDescriptionMaximumLength)
            .WithName("Açıklama");
    }
```

Tip `string?` olduğu için null açıklamalara da uygulanabilir.

`NotEmpty` yok, yani açıklama isteğe bağlı; sadece uzunluk sınırı var.

ƒ TodoPriority — Önceliğin enum'da tanımlı geçerli bir değer olmasını kontrol eder.

```
    public static IRuleBuilderOptions<T, TodoPriority> TodoPriority<T>(this IRuleBuilder<T, TodoPriority> rule)
    {
        return rule
            .IsInEnum()
            .WithName("Öncelik");
    }
```

Metot adı enum adıyla aynı; C# dönüş tipinde enum'u, çağrıda metodu anlar.

`IsInEnum` JSON'dan gelen 99 gibi tanımsız sayısal değerleri reddeder; C# enum'ları her int'i kabul edebildiği için gereklidir.

ƒ TodoDueDate — Bitiş tarihinin 10 yıldan ileride olmamasını kontrol eder; geçmiş kontrolü yapmaz.

```
    public static IRuleBuilderOptions<T, DateTime?> TodoDueDate<T>(
        this IRuleBuilder<T, DateTime?> rule,
        TimeProvider timeProvider)
    {
        return rule
            .Must(dueAtUtc => dueAtUtc == null
                || IsTodoDueDateTooFarAhead(dueAtUtc.Value, timeProvider.GetUtcNow().UtcDateTime) == false)
            .WithMessage(TodoDueDateTooFarAheadMessage);
    }
```

Hem yeni hem güncellenen görevde geçerli üst sınır kuralı. Şimdiki zaman `TimeProvider` parametresinden alınır.

`Must` özel koşul yazmayı sağlar: tarih null ise geçerli, değilse fazla ileride olmamalı. `WithMessage` hata mesajını belirler.

ƒ NewTodoDueDate — Yeni görev için tarih geçmişte olamaz ve 10 yıldan ileride olamaz kurallarını birleştirir.

```
    public static IRuleBuilderOptions<T, DateTime?> NewTodoDueDate<T>(
        this IRuleBuilder<T, DateTime?> rule,
        TimeProvider timeProvider)
    {
        return rule
            .Must(dueAtUtc => dueAtUtc == null
                || IsTodoDueDateInThePast(dueAtUtc.Value, timeProvider.GetUtcNow().UtcDateTime) == false)
            .WithMessage(TodoDueDateInThePastMessage)
            .TodoDueDate(timeProvider);
    }
```

Sadece yeni görevlerde kullanılır; güncellemede geçmiş kontrolü use case'te, tarih değiştiyse yapılır.

Önce geçmiş tarih kontrolü ve mesajı, ardından zincirlenen `TodoDueDate` ile üst sınır kontrolü uygulanır.

ƒ IsTodoDueDateInThePast — Tarih, şimdi eksi 5 dakikadan önceyse true döner.

```
    public static bool IsTodoDueDateInThePast(DateTime dueAtUtc, DateTime nowUtc)
    {
        return dueAtUtc < nowUtc - TodoDueDateClockTolerance;
    }
```

Public yardımcı; Update use case'i de aynı hesabı kullanabilsin diye ayrı metot.

Tolerans çıkarılmış şimdiki zamandan önceki tarih geçmiş sayılır.

ƒ IsTodoDueDateTooFarAhead — Tarih şimdiden 10 yıl sonrasını aşıyorsa true döner.

```
    public static bool IsTodoDueDateTooFarAhead(DateTime dueAtUtc, DateTime nowUtc)
    {
        return dueAtUtc > nowUtc.AddYears(TodoDueDateMaximumYearsAhead);
    }
```

Üst sınır hesabı için yardımcı metot.

`AddYears(10)` ile sınır hesaplanır; daha ileri tarihler reddedilir.

ƒ EntityId — Veritabanı kimliğinin pozitif olmasını şart koşar.

```
    public static IRuleBuilderOptions<T, int> EntityId<T>(this IRuleBuilder<T, int> rule)
    {
        return rule.GreaterThan(0);
    }
}
```

Tüm int kimlikler için ortak kural.

`GreaterThan(0)` : 0 veya negatif kimlik geçersizdir, veritabanına gereksiz sorgu atılmaz.

Application `todo.Application/Common/Validation/ValidatorExtensions.cs`

> Her use case'in başında kullanılan "doğrula, geçersizse exception fırlat" adımını tek bir extension metotta toplar.

```
using System.Threading;
using System.Threading.Tasks;
```

`CancellationToken` ve `Task` türleri için gerekli namespace'ler.

```
using FluentValidation;
using FluentValidation.Results;
```

FluentValidation'ın `IValidator` arayüzü ve `ValidationResult` sonuç tipi.

```
using TodoApi.Application.Exceptions;
```

Projenin kendi `RequestValidationException` sınıfı.

```
namespace TodoApi.Application.Common.Validation;
```

Ortak doğrulama yardımcılarının namespace'i.

ƒ ValidatorExtensions — IValidator'a ValidateOrThrowAsync metodunu ekleyen static uzantı sınıfı.

```
public static class ValidatorExtensions
{
```

Extension metot barındırdığı için `static` sınıf.

ƒ ValidateOrThrowAsync — Komutu doğrular; hata varsa RequestValidationException fırlatır, yoksa sessizce biter.

```
    public static async Task ValidateOrThrowAsync<TCommand>(
        this IValidator<TCommand> validator,
        TCommand command,
        CancellationToken cancellationToken)
```

`this IValidator<TCommand>` extension metottur: `_validator.ValidateOrThrowAsync(...)` diye çağrılır. `<TCommand>` generic olduğundan her komut tipiyle çalışır.

```
    {
        ValidationResult result = await validator.ValidateAsync(command, cancellat
ionToken);
```

`await` ile asenkron doğrulama yapılır; sonuç tüm hataları içeren `ValidationResult` nesnesidir.

```
        if (result.IsValid == false)
        {
            throw new RequestValidationException(result.Errors);
        }
    }
}
```

Geçersizse hatalar exception'a sarılıp fırlatılır. API'deki `ValidationExceptionHandler` bunu yakalayıp Türkçe mesajlarla 400 döner.

Application `todo.Application/Exceptions/RequestValidationException.cs`

> Doğrulama hatalarını alan adı -> mesajlar sözlüğü olarak taşıyan Application exception'ı. API katmanı bunu 400 Bad Request yanıtına çevirir.

```
using System;
using System.Collections.Generic;
using System.Linq;
```

`Exception`, koleksiyon türleri ve LINQ (`GroupBy`, `ToDictionary`) için gerekli namespace'ler.

```
using FluentValidation.Results;
```

FluentValidation'ın tek bir hatayı temsil eden `ValidationFailure` tipi.

```
namespace TodoApi.Application.Exceptions;
```

Uygulamaya özel exception'lar `Exceptions` klasöründe toplanır.

ƒ RequestValidationException — İstek doğrulama hatalarını alan bazında gruplayıp taşıyan exception.

```
public sealed class RequestValidationException : Exception
{
```

`Exception`'dan türer; `sealed` olduğu için ondan başka sınıf türetilemez.

```
    private readonly IReadOnlyDictionary<string, string[]> _errors;
```

Hatalar salt okunur sözlükte tutulur: anahtar alan adı, değer o alanın mesaj dizisi.

ƒ RequestValidationException (constructor) — Hata listesini alır, alan adına göre gruplayıp sözlüğe dönüştürür.

```
    public RequestValidationException(IEnumerable<ValidationFailure> failures)
        : base("One or more validation errors occurred.")
```

`: base(...)` üst sınıf `Exception`'ın constructor'ını genel bir mesajla çağırır.

```
    {
        if (failures == null)
        {
            throw new ArgumentNullException(nameof(failures));
        }
```

Hata listesi null gelirse programlama hatasıdır; hemen `ArgumentNullException` fırlatılır.

```
        _errors = failures
            .GroupBy(failure => failure.PropertyName)
            .ToDictionary(
                group => group.Key,
                group => group.Select(failure => failure.ErrorMessage).Distinct().
ToArray());
    }
```

LINQ: `GroupBy` hataları alan adına göre gruplar, `ToDictionary` her grubu alan -> mesaj dizisine çevirir. `Distinct` tekrar eden mesajları atar.

```
    public IReadOnlyDictionary<string, string[]> Errors
    {
        get
        {
            return _errors;
        }
    }
}
```

Sözlüğü dışarıya sadece okunabilir şekilde açan özellik; `ValidationExceptionHandler` yanıtı oluştururken bunu kullanır.

Application `todo.Application/Features/Todos/CreateTodo/CreateTodoUseCase.cs`

> Görev oluşturma iş akışını yöneten Application use case'i: doğrular, listeyi yükler, sahipliği kontrol eder, görevi ekler, kaydeder ve DTO döner.

```cs
using System;
using System.Threading;
using System.Threading.Tasks;
using FluentValidation;
```

Temel .NET türleri (`ArgumentNullException`, `Task`, `CancellationToken`) ve FluentValidation'ın `IValidator` arayüzü.

```cs
using TodoApi.Application.Abstractions.Authentication;
using TodoApi.Application.Abstractions.Persistence;
```

Application'ın kendi arayüzleri: oturumdaki kullanıcı (`ICurrentUser`) ve repository. Gerçek uygulamaları Infrastructure/API'dedir (bağımlılığın tersine çevrilmesi).

```cs
using TodoApi.Application.Authorization;
using TodoApi.Application.Common.Validation;
using TodoApi.Application.Contracts;
using TodoApi.Domain.Entities;
```

Erişim politikası, `ValidateOrThrowAsync` uzantısı, dönüş DTO'su ve Domain varlıkları (`TodoList`, `TodoItem`).

```cs
namespace TodoApi.Application.Features.Todos.CreateTodo;
```

Use case, komut ve validator ile aynı özellik klasöründe.

ƒ CreateTodoUseCase — Bir listeye yeni görev ekleme senaryosunu baştan sona yürüten sınıf.

```cs
public class CreateTodoUseCase
{
    private readonly ITodoListRepository _todoListRepository;
    private readonly ICurrentUser _currentUser;
    private readonly IValidator<CreateTodoCommand> _validator;
```

Her kullanıcı senaryosu için bir use case sınıfı: controller ince kalır, iş akışı burada okunur.

Bağımlılıklar arayüz olarak tutulur; bu sayede testte sahte (fake) repository veya kullanıcı verilebilir.

ƒ CreateTodoUseCase (constructor) — Repository, mevcut kullanıcı ve validator'ı DI ile alır, null kontrolü yapıp saklar.

```cs
    public CreateTodoUseCase(
        ITodoListRepository todoListRepository,
        ICurrentUser currentUser,
        IValidator<CreateTodoCommand> validator)
    {
        if (todoListRepository == null)
        {
            throw new ArgumentNullException(nameof(todoListRepository));
        }

        if (currentUser == null)
        {
            throw new ArgumentNullException(nameof(currentUser));
        }

        if (validator == null)
        {
            throw new ArgumentNullException(nameof(validator));
        }

        _todoListRepository = todoListRepository;
        _currentUser = currentUser;
        _validator = validator;
    }
```

Üç bağımlılık constructor injection ile gelir.

Her bağımlılık için null kontrolü; eksik kayıt varsa uygulama ilk kullanımda net bir hata ile durur (fail fast).

Kontrolden geçen bağımlılıklar alanlara atanır.

ƒ ExecuteAsync — Komutu doğrular, erişimi kontrol eder, görevi ekleyip kaydeder; erişim yoksa null döner.

```cs
    public async Task<TodoItemDto?> ExecuteAsync(
        CreateTodoCommand command,
        CancellationToken cancellationToken = default)
    {
        await _validator.ValidateOrThrowAsync(command, cancellationToken);

        TodoList? todoList = await _todoListRepository.GetByIdWithTodosAsync(
            command.TodoListId,
            cancellationToken);

        if (todoList == null)
        {
            return null;
        }

        bool canAccessTodoList = TodoListAccessPolicy.CanAccess(todoList, _currentUser);

        if (canAccessTodoList == false)
        {
            return null;
        }

        TodoItem newTodo = todoList.AddTodo(
            command.Title,
            command.Description,
            command.DueAtUtc,
            command.Priority);

        await _todoListRepository.SaveChangesAsync(cancellationToken);

        TodoItemDto response = new TodoItemDto(newTodo);

        return response;
```

`Task<TodoItemDto?>`: asenkron, null dönebilir. `= default` iptal token'ını isteğe bağlı parametre yapar.

İlk adım doğrulama: hata varsa `RequestValidationException` fırlar ve veritabanına hiç gidilmez; API bunu 400'e çevirir.

Liste, görevleriyle birlikte yüklenir; aynı başlıklı görev kontrolü için mevcut görevler gerekir.

Liste yoksa null döner; controller 404 verir.

Sahiplik kontrolü `TodoListAccessPolicy`'de: kullanıcı listenin sahibi veya admin olmalı.

Başkasının listesi için de null (404) dönülür, 403 değil; böylece saldırgan listenin var olduğunu öğrenemez.

Görev Domain metodu `AddTodo` ile eklenir; varlık kendi kurallarını (ör. listede benzersiz başlık) kendisi uygular.

Değişiklikler tek seferde veritabanına yazılır; DB hataları `TodoDbContext.SaveChangesAsync` içinde anlamlı hatalara çevrilir.

Entity yerine DTO döner; hatırlatma deneme sayacı gibi iç alanlar API yanıtına sızmaz.

```
78     }
79 }
```

Application `todo.Application/Abstractions/Persistence/ITodoListRepository.cs`

> Application katmanının liste verisine erişmek için ihtiyaç duyduğu sözleşme. Gerçek EF Core uygulaması Infrastructure'daki TodoListRepository'dir.

```
1  using System.Collections.Generic;
2  using System.Threading;
3  using System.Threading.Tasks;
```

Koleksiyon, iptal token'ı ve `Task` türleri için namespace'ler.

```
4  using TodoApi.Domain.Entities;
```

Repository Domain varlığı `TodoList` ile çalışır.

```
6  namespace TodoApi.Application.Abstractions.Persistence;
```

Kalıcılık (persistence) soyutlamaları Application'ın `Abstractions` klasöründe; Application EF Core'u hiç tanımaz.

ƒ ITodoListRepository — Liste okuma, ekleme, silme ve kaydetme işlemlerinin arayüzü.

```
8  public interface ITodoListRepository
```

`interface`: sadece ne yapılacağını söyler, nasıl yapılacağını değil. Testlerde sahte uygulama verilebilir.

```
9  {
```

ƒ GetAllByOwnerAsync — Bir kullanıcıya ait tüm listeleri döner.

```
10     Task<IReadOnlyList<TodoList>> GetAllByOwnerAsync(
11         int ownerUserId,
12         CancellationToken cancellationToken);
```

Sahibine göre listeler; `IReadOnlyList` dönerek çağıranın listeyi değiştirmesini engeller.

ƒ GetByIdWithTodosAsync — Listeyi görevleriyle birlikte kimliğe göre getirir; yoksa null.

```
14     Task<TodoList?> GetByIdWithTodosAsync(int todoListId, CancellationToken cancel
   lationToken);
```

Görev işlemleri bunu kullanır; `TodoList?` liste bulunamazsa null döneceğini belirtir.

ƒ ExistsWithNameForOwnerAsync — Kullanıcının aynı adda başka listesi var mı kontrol eder.

```
16     Task<bool> ExistsWithNameForOwnerAsync(
17         int ownerUserId,
18         string name,
19         int? excludedTodoListId,
20         CancellationToken cancellationToken);
```

`excludedTodoListId` yeniden adlandırmada listenin kendisini kontrolden hariç tutmak içindir; `int?` olduğundan oluşturmada null verilir.

ƒ AddAsync — Yeni listeyi kaydedilmek üzere ekler.

```
22     Task AddAsync(TodoList todoList, CancellationToken cancellationToken);
```

Ekler ama hemen yazmaz; yazma `SaveChangesAsync` ile yapılır.

ƒ Delete — Listeyi silinmek üzere işaretler.

```
24     void Delete(TodoList todoList);
```

Senkron çünkü sadece bellekte işaretler; veritabanına gitmez.

ƒ SaveChangesAsync — Bekleyen tüm değişiklikleri veritabanına yazar.

```
26     Task SaveChangesAsync(CancellationToken cancellationToken);
```

Use case işini bitirince tüm değişiklikler tek seferde kaydedilir.

```
27 }
```

Infrastructure `todo.Infrastructure/Persistence/TodoListRepository.cs`

> ITodoListRepository arayüzünün EF Core ile yazılmış gerçek uygulaması. Application'ın istediği sorguları veritabanı sorgularına çevirir.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Threading;
5  using System.Threading.Tasks;
```

Temel .NET türleri: koleksiyonlar, LINQ, iptal token'ı ve `Task`.

```
6  using Microsoft.EntityFrameworkCore;
```

EF Core: `Include`, `AsNoTracking`, `ToListAsync`, `AnyAsync` gibi veritabanı uzantıları buradan gelir.

```
7  using TodoApi.Application.Abstractions.Persistence;
8  using TodoApi.Domain.Entities;
```

Uygulanan arayüz (Application) ve Domain varlığı. Infrastructure, Application'a bağımlıdır; tersi değil.

```
10 namespace TodoApi.Infrastructure.Persistence;
```

Veritabanı erişim kodu Infrastructure katmanının `Persistence` klasöründe.

ƒ TodoListRepository — Liste verisini EF Core ile okuyup yazan repository.

```
12 public sealed class TodoListRepository : ITodoListRepository
```

`: ITodoListRepository` arayüzü uygular; DI, Application'a bu sınıfı verir.

```
13 {
14     private readonly TodoDbContext _todoDbContext;
```

EF Core'un veritabanı oturumu; tüm sorgular ve değişiklik takibi bunun üzerinden yapılır.

ƒ TodoListRepository (constructor) — DbContext'i DI ile alır, null kontrolü yapıp saklar.

```
16     public TodoListRepository(TodoDbContext todoDbContext)
```

`TodoDbContext` istek başına (scoped) DI'dan gelir; aynı istekteki tüm repository'ler aynı oturumu paylaşır.

```
17     {
18         if (todoDbContext == null)
19         {
20             throw new ArgumentNullException(nameof(todoDbContext));
21         }
22
23         _todoDbContext = todoDbContext;
```

DbContext null ise hemen hata; değilse alana atanır.

```
24     }
```

ƒ GetAllByOwnerAsync — Kullanıcının tüm listelerini kimliğe göre sıralı ve salt okunur getirir.

```
26      public async Task<IReadOnlyList<TodoList>> GetAllByOwnerAsync(
27          int ownerUserId,
28          CancellationToken cancellationToken)
```

Sahip kimliğine göre listeleri asenkron getirir.

```
29      {
```

```
30          IQueryable<TodoList> todoListQuery = _todoDbContext.TodoLists;
```

`IQueryable` sorguyu henüz çalıştırmaz; sonraki adımlarla SQL adım adım inşa edilir.

```
32          todoListQuery = todoListQuery.AsNoTracking();
```

`AsNoTracking`: sadece okuma yapılacağı için EF değişiklik takibini kapatır; daha hızlı ve az bellek kullanır.

```
33          todoListQuery = todoListQuery.Where(todoList => todoList.OwnerUserId == ow
    nerUserId);
34          todoListQuery = todoListQuery.OrderBy(todoList => todoList.Id);
```

`Where` sadece kullanıcının listelerini süzer (SQL WHERE), `OrderBy` kimliğe göre sıralar (ORDER BY).

```
36          List<TodoList> todoLists = await todoListQuery.ToListAsync(cancellationTok
    en);
```

`ToListAsync` sorguyu veritabanında asıl çalıştırdığı yerdir.

```
38          IReadOnlyList<TodoList> result = todoLists;
39
40          return result;
```

Liste `IReadOnlyList` olarak döndürülür; çağıran onu değiştiremez.

```
41      }
```

ƒ GetByIdWithTodosAsync — Listeyi görevleriyle birlikte kimliğe göre getirir; yoksa null döner.

```
43      public async Task<TodoList?> GetByIdWithTodosAsync(
44          int todoListId,
45          CancellationToken cancellationToken)
```

Görev oluşturma, güncelleme ve silme use case'leri bu metodu kullanır.

```
46      {
```

```
47          IQueryable<TodoList> todoListQuery = _todoDbContext.TodoLists;
```

Listeler tablosundan sorgu başlatılır.

```
49          todoListQuery = todoListQuery.Include(todoList => todoList.Todos);
```

`Include` ilişkili görevleri aynı sorguda yükler (JOIN). Takip açık kalır, çünkü listeye görev eklenip kaydedilecek.

```
51          TodoList? todoList = await todoListQuery.FirstOrDefaultAsync(
52              todoList => todoList.Id == todoListId,
53              cancellationToken);
```

`FirstOrDefaultAsync` kimliği eşleşen ilk listeyi döner, yoksa null. Sahiplik burada değil, use case'te kontrol edilir.

```
55          return todoList;
```

Bulunan liste (veya null) döndürülür.

```
56      }
```

ƒ ExistsWithNameForOwnerAsync — Kullanıcının aynı adda listesi olup olmadığını, isteğe bağlı bir listeyi hariç tutarak kontrol eder.

```
58      public async Task<bool> ExistsWithNameForOwnerAsync(
59          int ownerUserId,
60          string name,
61          int? excludedTodoListId,
62          CancellationToken cancellationToken)
```

Liste adı benzersizliği için kullanılır; yeniden adlandırmada listenin kendisi hariç tutulur.

```
63      {
```

```
64          IQueryable<TodoList> todoListQuery = _todoDbContext.TodoLists;
65
66          todoListQuery = todoListQuery.Where(todoList =>
67              todoList.OwnerUserId == ownerUserId && todoList.Name == name);
```

Aynı sahip ve aynı ada sahip listeler süzülür.

```
69          if (excludedTodoListId.HasValue)
70          {
71              int excludedId = excludedTodoListId.Value;
72              todoListQuery = todoListQuery.Where(todoList => todoList.Id != exclude
    dId);
73          }
```

Hariç tutulacak kimlik verildiyse o liste sonuçtan çıkarılır. Değer önce yerel değişkene alınır ki sorgu sade SQL'e çevrilsin.

```
75          bool exists = await todoListQuery.AnyAsync(cancellationToken);
76
77          return exists;
```

`AnyAsync` kayıtları getirmeden sadece var mı yok mu diye bakar (SQL EXISTS); hızlıdır.

```
78      }
```

ƒ AddAsync — Yeni listeyi DbContext'e ekler; kayıt SaveChangesAsync ile yapılır.

```
80      public async Task AddAsync(
81          TodoList todoList,
82          CancellationToken cancellationToken)
```

Yeni listeyi değişiklik takibine "eklenecek" olarak alır.

```
83      {
```

```
84          if (todoList == null)
85          {
86              throw new ArgumentNullException(nameof(todoList));
87          }
```

Null liste eklenmeye çalışılırsa hemen hata.

```
89          await _todoDbContext.TodoLists.AddAsync(todoList, cancellationToken);
```

EF'e listeyi eklemesi söylenir; INSERT ancak `SaveChangesAsync` çağrılınca çalışır.

```
90      }
```

ƒ Delete — Listeyi silinmek üzere işaretler.

```
92      public void Delete(TodoList todoList)
```

Senkron metot; sadece bellekteki takip durumunu değiştirir.

```
93      {
```

```
94          if (todoList == null)
95          {
96              throw new ArgumentNullException(nameof(todoList));
97          }
```

Null kontrolü.

```
99          _todoDbContext.TodoLists.Remove(todoList);
```

`Remove` listeyi silinecek olarak işaretler; DELETE kaydetme sırasında çalışır.

```
100     }
```

ƒ SaveChangesAsync — Takip edilen tüm değişiklikleri veritabanına yazar.

```
102     public async Task SaveChangesAsync(CancellationToken cancellationToken)
```
Application'ın kaydetme isteğini DbContext'e iletir.

```
103     {
104         await _todoDbContext.SaveChangesAsync(cancellationToken);
```
Tüm eklemeler, güncellemeler, silmeler tek işlemde yazılır. `TodoDbContext.SaveChangesAsync` DB hatalarını anlamlı uygulama hatalarına çevirir.

```
105     }
106 }
```

Application `todo.Application/Authorization/TodoListAccessPolicy.cs`

> Bir kullanıcının bir listeye erişip erişemeyeceğine karar veren tek merkezi kural. Tüm liste ve görev use case'leri bunu kullanır.

```
1 using System;
2 using TodoApi.Application.Abstractions.Authentication;
3 using TodoApi.Domain.Entities;
```
`ArgumentNullException` için `System`, kullanıcı arayüzü `ICurrentUser` ve Domain'deki `TodoList` varlığı.

```
5 namespace TodoApi.Application.Authorization;
```
Yetkilendirme kuralları Application katmanının `Authorization` klasöründe.

ƒ TodoListAccessPolicy — Liste erişim kuralını tek yerde tutan static politika sınıfı.

```
7 public static class TodoListAccessPolicy
```
Durum tutmadığı için `static`; kural tek yerde olduğundan değişince her use case otomatik güncellenir.

```
8 {
```

ƒ CanAccess — Kullanıcı admin ise veya listenin sahibiyse true döner.

```
9     public static bool CanAccess(TodoList todoList, ICurrentUser currentUser)
```
Liste ve mevcut kullanıcıyı alıp erişim izni olup olmadığını `bool` olarak döner.

```
10    {
11        if (todoList == null)
12        {
13            throw new ArgumentNullException(nameof(todoList));
14        }
15
16        if (currentUser == null)
17        {
18            throw new ArgumentNullException(nameof(currentUser));
19        }
```
Parametreler null ise programlama hatasıdır; sessizce false dönmek yerine exception fırlatılır.

```
21        if (currentUser.IsAdmin)
22        {
23            return true;
24        }
```
Admin kullanıcılar tüm listelere erişebilir.

```
26        bool isOwner = todoList.IsOwnedBy(currentUser.UserId);
27
28        return isOwner;
```
Diğer kullanıcılar için sahiplik kontrolü Domain metodu `IsOwnedBy` ile yapılır. false ise use case null döner, controller 404 verir.

```
29    }
30 }
```

Domain `todo.Domain/Entities/TodoList.cs`

> Bir kullanıcının görev listesini temsil eden entity ve aggregate root. Görevler yalnızca bu sınıf üzerinden eklenir/yeniden adlandırılır; böylece "aynı listede aynı başlık olamaz" kuralı atlatılamaz.

```
1 using TodoApi.Domain.Common;
2 using TodoApi.Domain.Enums;
3 using TodoApi.Domain.Exceptions;
```
Kullanılan domain ad alanları: isim karşılaştırma yardımcısı (`Common`), öncelik enum'u (`Enums`) ve özel hatalar (`Exceptions`).

```
5 namespace TodoApi.Domain.Entities;
```
Dosyanın ad alanı: entity'ler `TodoApi.Domain.Entities` altında toplanır.

ƒ class TodoList — Görev listesini ve içindeki görevleri yöneten aggregate root sınıfı.

```
7 public class TodoList
```
Aggregate root: listeye ait görevlerin tüm değişiklikleri bu sınıftan geçer, kurallar tek yerde korunur.

```
8 {
9     private const int MaximumNameLength = 80;
```
Liste adı için en fazla karakter sayısı. `const` derleme anında sabitlenen, değişmeyen değerdir; DB'deki `HasMaxLength(80)` ile uyumludur.

```
11    private int _id;
12    private string _name = string.Empty;
13    private int _ownerUserId;
14    private DateTime _createdAtUtc;
```
Özelliklerin (property) değerlerini tutan `private` alanlar (field). Dışarıdan doğrudan erişilemez; kapsülleme (encapsulation) sağlar.

```
15    private readonly List<TodoItem> _todos = new List<TodoItem>();
```
Görevlerin tutulduğu iç liste. `readonly` alanın yeniden atanamayacağını söyler; liste içeriği yalnızca sınıfın metotlarıyla değişir. EF Core da bu alanı kullanır.

ƒ TodoList(string name, int ownerUserId) — Adı ve sahibi doğrulanmış yeni bir görev listesi oluşturur.

```
17    public TodoList(string name, int ownerUserId)
```
Kurucu: liste oluşturmak için ad ve sahip kullanıcının id'si zorunludur, geçersiz bir nesne hiç oluşamaz.

```
18    {
```

```
        string normalizedName = ValidateAndNormalizeName(name);
        Name = normalizedName;
```
Ad doğrulanır, boşlukları kırpılır ve `Name` özelliğine atanır; geçersizse hata fırlatılır.

```
        OwnerUserId = ValidateOwnerUserId(ownerUserId);
```
Sahip kullanıcı id'si pozitif mi kontrol edilip atanır.

```
        CreatedAtUtc = DateTime.UtcNow;
```
Oluşturulma zamanı UTC olarak kaydedilir; projede tüm tarihler UTC saklanır.

```
    }
    public int Id
    {
        get
        {
            return _id;
        }
        private set
        {
            _id = value;
        }
    }
```
`Id` özelliği: herkes okuyabilir ama `private set` sayesinde yalnızca sınıf içi (ve EF Core) değiştirebilir. Değeri DB üretir.

```
    public string Name
    {
        get
        {
            return _name;
        }
        private set
        {
            _name = value;
        }
    }
```
`Name` özelliği: dışarıdan okunur, değiştirmek için `Rename` metodu kullanılmalıdır; böylece doğrulama atlanamaz.

```
    public int OwnerUserId
    {
        get
        {
            return _ownerUserId;
        }
        private set
        {
            _ownerUserId = value;
        }
    }
```
`OwnerUserId`: listenin hangi kullanıcıya ait olduğunu tutar; sadece sınıf içinden atanır.

```
    public DateTime CreatedAtUtc
    {
        get
        {
            return _createdAtUtc;
        }
        private set
        {
            _createdAtUtc = value;
        }
    }
```
`CreatedAtUtc`: listenin oluşturulma anı (UTC); dışarıdan değiştirilemez.

```
    public IReadOnlyCollection<TodoItem> Todos
    {
        get
        {
            return _todos.AsReadOnly();
        }
    }
```
`Todos`: görevleri salt okunur (`IReadOnlyCollection`) döndürür. Dışarıdaki kod listeye doğrudan ekleme yapamaz, `AddTodo` kullanmak zorundadır.

ƒ IsOwnedBy — Verilen kullanıcının bu listenin sahibi olup olmadığını döndürür.

```
    public bool IsOwnedBy(int userId)
    {
        return OwnerUserId == userId;
    }
```
Sahiplik kontrolü; başka kullanıcının listesine erişimi engellemek için üst katmanlarda kullanılır.

ƒ Rename — Liste adını doğrulayıp değiştirir.

```
    public void Rename(string name)
    {
        string normalizedName = ValidateAndNormalizeName(name);
        Name = normalizedName;
    }
```
Yeni ad kurucudaki aynı doğrulamadan geçer, sonra `Name`'e atanır. Kullanıcı bazında ad tekrarını DB'deki unique index yakalar.

ƒ AddTodo — Listeye yeni görev ekler; başlığın listede benzersiz olmasını garanti eder.

```
    public TodoItem AddTodo(
        string title,
        string? description = null,
        DateTime? dueAtUtc = null,
        TodoPriority priority = TodoPriority.Normal)
    {
        TodoItem todo = new TodoItem(
            Id,
            title,
            description,
            dueAtUtc,
            priority);
```
Görev eklemenin tek yolu. `string?` null olabilir demektir; `= null` gibi değerler parametreyi isteğe bağlı yapar.

Yeni `TodoItem` oluşturulur; kurucusu `internal` olduğu için bunu sadece domain içi kod yapabilir. Alan doğrulamaları orada yapılır.

```
        EnsureTitleIsUnique(todo.Title, todoBeingRenamed: null);
```

Aynı başlıkta görev var mı kontrol edilir; varsa `DuplicateNameException` fırlatılır. `todoBeingRenamed: null` isimli argümandır.

```
        _todos.Add(todo);

        return todo;
```

Kontrolden geçen görev iç listeye eklenir ve çağırana döndürülür. EF Core kaydederken bunu yeni satır olarak ekler.

```
    }
```

ƒ RenameTodo — Listedeki bir görevin başlığını benzersizlik kuralını koruyarak değiştirir.

```
    public void RenameTodo(TodoItem todo, string title)
```

Görev yeniden adlandırma da liste üzerinden yapılır ki tekrar eden başlık kuralı atlatılamasın.

```
    {
        if (todo == null)
        {
            throw new ArgumentNullException(nameof(todo));
        }
```

Görev null verilirse `ArgumentNullException` fırlatılır; `nameof(todo)` parametre adını metin olarak verir.

```
        if (_todos.Contains(todo) == false)
        {
            throw new DomainRuleViolationException(
                "A todo can only be renamed through the list it belongs to.");
        }
```

Görev bu listeye ait değilse kural ihlali hatası fırlatılır; başka listenin görevi buradan değiştirilemez.

```
        EnsureTitleIsUnique(title, todoBeingRenamed: todo);

        todo.Rename(title);
```

Yeni başlık, görevin kendisi hariç diğerleriyle karşılaştırılır; sorun yoksa `TodoItem.Rename` (internal) çağrılır.

```
    }
```

ƒ RemoveTodo — Id'si verilen görevi listeden çıkarır; bulunamazsa false döner.

```
    public bool RemoveTodo(int todoId)
```

Görevi id ile silen metot; sonuç `bool` olarak silinip silinmediğini bildirir.

```
    {
        if (todoId <= 0)
        {
            throw new DomainRuleViolationException(
                "A saved todo must have a positive id before it can be removed by id.");
        }
```

Id pozitif değilse hata: henüz kaydedilmemiş (id'si 0 olan) görev id ile silinemez.

```
        TodoItem? todoToRemove = null;
```

Silinecek görevi tutacak değişken; `TodoItem?` başlangıçta null olabileceğini belirtir.

```
        foreach (TodoItem todo in _todos)
        {
            if (todo.Id == todoId)
            {
                todoToRemove = todo;
                break;
            }
        }
```

`foreach` listedeki her öğeyi sırayla dolaşır; id eşleşen görev bulununca saklanır ve `break` ile döngüden çıkılır.

```
        if (todoToRemove == null)
        {
            return false;
        }
```

Görev bulunamadıysa `false` döner; üst katman bunu 404 olarak yorumlayabilir.

```
        bool wasRemoved = _todos.Remove(todoToRemove);

        return wasRemoved;
```

Görev iç listeden çıkarılır ve sonuç döndürülür. EF Core ilişki ayarı sayesinde kayıt DB'den de silinir.

```
    }
```

ƒ EnsureTitleIsUnique — Başlık listede başka bir görevde varsa DuplicateNameException fırlatır.

```
    private void EnsureTitleIsUnique(string title, TodoItem? todoBeingRenamed)
```

Tekrar eden başlık kuralını uygulayan `private` yardımcı metot; sadece bu sınıf içinden çağrılır.

```
    {
        foreach (TodoItem existingTodo in _todos)
        {
            if (ReferenceEquals(existingTodo, todoBeingRenamed))
            {
                continue;
            }
```

Tüm görevler dolaşılır; yeniden adlandırılan görevin kendisi `ReferenceEquals` (aynı nesne mi) ile tespit edilip `continue` ile atlanır.

```
            if (NameComparison.AreSame(existingTodo.Title, title))
            {
                throw new DuplicateNameException(
                    "A todo with the same title already exists in this list.");
            }
```

`NameComparison.AreSame` Türkçe, harf duyarsız karşılaştırır; aynıysa `DuplicateNameException` fırlatılır (üstte 409 olur).

```
        }
    }
```

ƒ ValidateAndNormalizeName — Liste adını boş/uzunluk kurallarına göre doğrular ve kırpılmış halini döndürür.

```
    private static string ValidateAndNormalizeName(string name)
```

`static` metot nesne durumuna erişmez; sadece girdiyi doğrular.

```
    {
        if (string.IsNullOrWhiteSpace(name))
        {
            throw new DomainRuleViolationException(
                "A todo list name cannot be empty.");
        }
```

Ad null, boş ya da sadece boşluksa kural ihlali hatası fırlatılır.

```
        string normalizedName = name.Trim();
```

Baştaki ve sondaki boşluklar `Trim` ile temizlenir.

```
        if (normalizedName.Length > MaximumNameLength)
        {
            throw new DomainRuleViolationException(
                $"A todo list name cannot exceed {MaximumNameLength} character
s.");
        }
```

Kırpılmış ad 80 karakteri aşarsa hata; `$"..."` metin içine değişken yerleştirme (string interpolation) sözdizimidir.

```
        return normalizedName;
```

Geçerli ve normalize edilmiş ad döndürülür.

```
    }
```

ƒ ValidateOwnerUserId — Sahip kullanıcı id'sinin pozitif olduğunu doğrular.

```
    private static int ValidateOwnerUserId(int ownerUserId)
```

Sahip id'sini kontrol eden statik yardımcı metot.

```
    {
```

```
        if (ownerUserId <= 0)
        {
            throw new DomainRuleViolationException(
                "A todo list must belong to a saved user.");
        }

        return ownerUserId;
```

Id 0 veya negatifse liste kayıtlı bir kullanıcıya bağlanamaz, hata fırlatılır; aksi halde id aynen döndürülür.

```
    }
}
```

Domain `todo.Domain/Entities/TodoItem.cs`

> Tek bir görevi (başlık, açıklama, bitiş tarihi, öncelik, tamamlanma ve hatırlatma bilgileri) temsil eden entity. Oluşturma ve yeniden adlandırma yalnızca TodoList üzerinden yapılır; kendi alan kurallarını kendisi doğrular.

```
using TodoApi.Domain.Enums;
using TodoApi.Domain.Exceptions;
```

Öncelik enum'u ve domain hataları için gerekli ad alanları.

```
namespace TodoApi.Domain.Entities;
```

Dosyanın ad alanı: `TodoApi.Domain.Entities`.

ƒ class TodoItem — Bir görevi ve onun kurallarını (doğrulama, tamamlama, hatırlatma) içeren entity sınıfı.

```
public class TodoItem
```

Görev entity'si; TodoList aggregate'inin bir parçasıdır.

```
{
```

```
    private const int MaximumTitleLength = 120;
    private const int MaximumDescriptionLength = 1000;
```

Başlık (120) ve açıklama (1000) için en fazla uzunluklar; DB'deki `HasMaxLength` ayarlarıyla aynıdır.

```
    private int _id;
    private int _todoListId;
```

Görevin kendi id'si ve bağlı olduğu listenin id'si (foreign key) için private alanlar.

```
    private string _title = string.Empty;
    private string? _description;
    private bool _isCompleted;
    private DateTime _createdAtUtc;
    private DateTime? _dueAtUtc;
    private TodoPriority _priority = TodoPriority.Normal;
```

Görev verilerini tutan private alanlar. `string?` ve `DateTime?` null olabilir demektir; öncelik varsayılan olarak `Normal`.

```
    private DateTime? _reminderSentAtUtc;
    private int _reminderAttemptCount;
    private DateTime? _reminderLastAttemptAtUtc;
```

Hatırlatma alanları: e-posta gönderilme zamanı, başarısız deneme sayısı ve son deneme zamanı. Tekrar deneme ve vazgeçme mantığı bunlarla çalışır.

ƒ TodoItem (constructor) — Tüm alanları doğrulayarak yeni bir görev nesnesi oluşturur.

```
    internal TodoItem(
        int todoListId,
        string title,
        string? description,
        DateTime? dueAtUtc,
        TodoPriority priority)
```

`internal` kurucu: sadece aynı projeden (Domain) çağrılabilir. Böylece görev yalnızca `TodoList.AddTodo` ile yaratılır.

```
    {
```

```
        if (todoListId < 0)
        {
            throw new DomainRuleViolationException(
                "A todo list id cannot be negative.");
        }
```

Liste id'si negatifse hata. 0'a izin verilir çünkü liste henüz kaydedilmemişse id'si 0 olabilir.

```
        TodoListId = todoListId;
```

Görevin bağlı olduğu liste id'si atanır.

```
        string normalizedTitle = ValidateAndNormalizeTitle(title);
        Title = normalizedTitle;
```

Başlık doğrulanır, kırpılır ve atanır.

```
        string? normalizedDescription = ValidateAndNormalizeDescription(descriptio
n);
        Description = normalizedDescription;
```

Açıklama doğrulanır; boşsa null olur, değilse kırpılmış hali atanır.

```
        DateTime? validatedDueAtUtc = ValidateDueAtUtc(dueAtUtc);
        DueAtUtc = validatedDueAtUtc;
```

Bitiş tarihi UTC mi kontrol edilip atanır.

```
        TodoPriority validatedPriority = ValidatePriority(priority);
        Priority = validatedPriority;
```

Önceliğin tanımlı bir enum değeri olduğu doğrulanıp atanır.

```
        IsCompleted = false;
```

Yeni görev her zaman tamamlanmamış olarak başlar.

```
        CreatedAtUtc = DateTime.UtcNow;
```

Oluşturulma zamanı UTC olarak kaydedilir.

```
    }
```

```
    public int Id
    {
        get
        {
            return _id;
        }
        private set
        {
            _id = value;
        }
    }
```

`Id` özelliği: herkes okuyabilir, `private set` ile yalnızca sınıf içi ve EF Core yazabilir. Değeri DB üretir.

```
    public int TodoListId
    {
        get
        {
            return _todoListId;
        }
        private set
        {
            _todoListId = value;
        }
    }
```

`TodoListId`: görevin hangi listeye ait olduğunu gösteren foreign key.

```
    public string Title
    {
        get
        {
            return _title;
        }
        private set
        {
            _title = value;
        }
    }
```

`Title`: görev başlığı; değiştirmek için liste üzerinden `RenameTodo` kullanılır.

```
    public string? Description
    {
        get
        {
            return _description;
        }
        private set
        {
            _description = value;
        }
    }
```

`Description`: isteğe bağlı açıklama; `ChangeDescription` ile değiştirilir.

```
    public bool IsCompleted
    {
        get
        {
            return _isCompleted;
        }
        private set
        {
            _isCompleted = value;
        }
    }
```

`IsCompleted`: görev tamamlandı mı; `MarkCompleted` / `MarkIncomplete` ile değişir.

```
    public DateTime CreatedAtUtc
    {
        get
        {
            return _createdAtUtc;
        }
        private set
        {
            _createdAtUtc = value;
        }
    }
```

`CreatedAtUtc`: oluşturulma anı, sonradan değiştirilemez.

```
    public DateTime? DueAtUtc
    {
        get
        {
            return _dueAtUtc;
        }
        private set
        {
            _dueAtUtc = value;
        }
    }
```

`DueAtUtc`: isteğe bağlı bitiş tarihi; `SetDueAtUtc` ile değişir.

```
    public TodoPriority Priority
    {
        get
        {
            return _priority;
        }
        private set
        {
            _priority = value;
        }
    }
```

`Priority`: görev önceliği; `ChangePriority` ile değişir.

```
    public DateTime? ReminderSentAtUtc
    {
        get
        {
            return _reminderSentAtUtc;
        }
        private set
        {
            _reminderSentAtUtc = value;
        }
    }
```

`ReminderSentAtUtc`: hatırlatma e-postası başarıyla gönderildiyse zamanı; null ise henüz gönderilmedi.

```
    public int ReminderAttemptCount
    {
        get
        {
            return _reminderAttemptCount;
        }
        private set
        {
            _reminderAttemptCount = value;
        }
    }
```

`ReminderAttemptCount`: başarısız gönderim denemesi sayısı; en fazla deneme sınırı için kullanılır.

```
    public DateTime? ReminderLastAttemptAtUtc
    {
        get
        {
            return _reminderLastAttemptAtUtc;
        }
        private set
        {
            _reminderLastAttemptAtUtc = value;
        }
    }
```

`ReminderLastAttemptAtUtc`: son başarısız denemenin zamanı; tekrar denemeden önce beklenecek süreyi hesaplamak için.

ƒ Rename — Görev başlığını doğrulayıp değiştirir; sadece TodoList çağırabilir.

```
    internal void Rename(string title)
    {
        string normalizedTitle = ValidateAndNormalizeTitle(title);
        Title = normalizedTitle;
    }
```

`internal` olduğu için API katmanı bunu doğrudan çağıramaz; benzersizlik kontrolü yapan `TodoList.RenameTodo` üzerinden gelinir.

ƒ ChangeDescription — Görev açıklamasını doğrulayıp günceller.

```
    public void ChangeDescription(string? description)
    {
        string? normalizedDescription = ValidateAndNormalizeDescription(description);
        Description = normalizedDescription;
    }
```

Açıklama doğrulanır ve atanır; boş metin null'a çevrilir.

ƒ SetDueAtUtc — Bitiş tarihini değiştirir; tarih değiştiyse hatırlatma durumunu sıfırlar.

```
    public void SetDueAtUtc(DateTime? dueAtUtc)
```

Bitiş tarihini ayarlayan metot; null verilirse tarih kaldırılır.

```
    {
```

```
        DateTime? validatedDueAtUtc = ValidateDueAtUtc(dueAtUtc);
```

Yeni tarihin UTC olduğu doğrulanır.

```
        if (validatedDueAtUtc != DueAtUtc)
        {
            ReminderSentAtUtc = null;
            ReminderAttemptCount = 0;
            ReminderLastAttemptAtUtc = null;
        }
```

Tarih gerçekten değiştiyse hatırlatma bilgileri sıfırlanır; böylece yeni tarih için hatırlatma tekrar gönderilebilir.

```
        DueAtUtc = validatedDueAtUtc;
```

Yeni bitiş tarihi atanır.

```
    }
```

ƒ NeedsReminder — Görev için şimdi hatırlatma e-postası gönderilmesi gerekip gerekmediğini söyler.

```
    public bool NeedsReminder(DateTime nowUtc, TimeSpan reminderLeadTime)
```

Arka plan işi (Hangfire) bu metotla hangi görevlere e-posta atılacağına karar verir.

```
    {
```

```
        if (IsCompleted || ReminderSentAtUtc.HasValue || DueAtUtc.HasValue == false)
        {
            return false;
        }
```

Görev tamamlandıysa, hatırlatma zaten gönderildiyse veya bitiş tarihi yoksa hatırlatma gerekmez. `HasValue` nullable değerin dolu olup olmadığını söyler.

```
        DateTime dueAtUtc = DueAtUtc.Value;
```

`.Value` ile nullable tarihin gerçek değeri alınır (yukarıda dolu olduğu garanti edildi).

```
        if (dueAtUtc <= nowUtc)
        {
            return false;
        }
```

Bitiş tarihi geçmişse artık hatırlatma anlamsızdır, `false` döner.

```
        return dueAtUtc - nowUtc <= reminderLeadTime;
```

Kalan süre hatırlatma öncesi süreden (ör. 1 saat) küçük veya eşitse `true`; iki tarih farkı `TimeSpan` verir.

```
    }
```

ƒ MarkReminderSent — Hatırlatmanın başarıyla gönderildiğini zaman damgasıyla işaretler.

```
    public void MarkReminderSent(DateTime nowUtc)
    {
        ReminderSentAtUtc = nowUtc;
    }
```

Gönderim zamanı kaydedilir; böylece aynı görev için tekrar e-posta atılmaz.

ƒ RecordFailedReminderAttempt — Başarısız gönderim denemesini sayar ve zamanını kaydeder.

```
    public void RecordFailedReminderAttempt(DateTime nowUtc)
    {
        ReminderAttemptCount++;
        ReminderLastAttemptAtUtc = nowUtc;
    }
```

Deneme sayısı `++` ile bir artırılır ve son deneme zamanı yazılır; tekrar deneme mantığı bunlara bakar.

ƒ CanRetryReminder — Başarısız hatırlatmanın şimdi tekrar denenip denenemeyeceğini söyler.

```
    public bool CanRetryReminder(DateTime nowUtc, int maximumAttempts, TimeSpan retryDelay)
    {
        if (ReminderAttemptCount >= maximumAttempts)
        {
            return false;
        }

        if (ReminderLastAttemptAtUtc.HasValue == false)
        {
            return true;
        }

        return nowUtc - ReminderLastAttemptAtUtc.Value >= retryDelay;
    }
```

En fazla deneme sayısı ve denemeler arası bekleme süresi parametre olarak alınır.

Deneme sınırına ulaşıldıysa vazgeçilir, `false` döner.

Daha önce hiç başarısız deneme yoksa hemen denenebilir.

Son denemeden bu yana bekleme süresi geçtiyse tekrar denenebilir; böylece mail sunucusu sürekli zorlanmaz.

ƒ ChangePriority — Görev önceliğini doğrulayıp değiştirir.

```
    public void ChangePriority(TodoPriority priority)
    {
        TodoPriority validatedPriority = ValidatePriority(priority);
        Priority = validatedPriority;
    }
```

Yeni öncelik tanımlı bir değer mi kontrol edilir ve atanır.

ƒ MarkCompleted — Görevi tamamlandı olarak işaretler.

```
    public void MarkCompleted()
    {
        IsCompleted = true;
    }
```

`IsCompleted` true yapılır; tamamlanan görevlere hatırlatma gönderilmez.

ƒ MarkIncomplete — Görevi tekrar tamamlanmamış duruma getirir.

```
    public void MarkIncomplete()
    {
        IsCompleted = false;
    }
```

`IsCompleted` false yapılır.

ƒ ValidateAndNormalizeTitle — Başlığı boş/uzunluk kurallarına göre doğrular ve kırpılmış halini döndürür.

```
    private static string ValidateAndNormalizeTitle(string title)
    {
        if (string.IsNullOrWhiteSpace(title))
        {
            throw new DomainRuleViolationException(
                "A todo title cannot be empty.");
        }

        string normalizedTitle = title.Trim();

        if (normalizedTitle.Length > MaximumTitleLength)
        {
            throw new DomainRuleViolationException(
                $"A todo title cannot exceed {MaximumTitleLength} characters.");
        }

        return normalizedTitle;
    }
```

`private static`: sadece bu sınıf içinden çağrılır ve nesne durumuna erişmez.

Başlık boş veya sadece boşluksa kural ihlali hatası fırlatılır.

Baştaki ve sondaki boşluklar temizlenir.

120 karakteri aşan başlık reddedilir; mesajda `$"..."` ile sınır değeri yazılır.

Geçerli başlık döndürülür.

ƒ ValidateAndNormalizeDescription — Açıklamayı doğrular; boşsa null, değilse kırpılmış halini döndürür.

```
    private static string? ValidateAndNormalizeDescription(string? description)
    {
        if (string.IsNullOrWhiteSpace(description))
        {
            return null;
        }

        string normalizedDescription = description.Trim();

        if (normalizedDescription.Length > MaximumDescriptionLength)
        {
            throw new DomainRuleViolationException(
                $"A todo description cannot exceed {MaximumDescriptionLength} characters.");
        }

        return normalizedDescription;
    }
```

Açıklama isteğe bağlı olduğu için hem girdi hem çıktı `string?`.

Boş açıklama hata değildir; DB'de boş metin yerine null saklansın diye null döner.

Açıklama kırpılır.

1000 karakteri aşan açıklama reddedilir.

Geçerli açıklama döndürülür.

ƒ ValidateDueAtUtc — Bitiş tarihi verildiyse UTC olduğunu doğrular.

```
310     private static DateTime? ValidateDueAtUtc(DateTime? dueAtUtc)
311     {
312         if (dueAtUtc.HasValue)
313         {
314             DateTime providedDueAtUtc = dueAtUtc.Value;
316             if (providedDueAtUtc.Kind != DateTimeKind.Utc)
317             {
318                 throw new DomainRuleViolationException(
319                     "A todo due date must be expressed in UTC.");
320             }
321         }
323         return dueAtUtc;
324     }
```

Tarih kurallarını kontrol eden yardımcı metot.

Tarih doluysa gerçek değeri alınır.

`Kind` tarihin türünü (Utc/Local/Unspecified) söyler; UTC değilse hata. Saat dilimi karışıklıklarını domain seviyesinde engeller.

Tarih (veya null) aynen döndürülür.

ƒ ValidatePriority — Önceliğin enum'da tanımlı bir değer olduğunu doğrular.

```
326     private static TodoPriority ValidatePriority(TodoPriority priority)
327     {
328         bool isDefinedPriority = Enum.IsDefined(typeof(TodoPriority), priority);
330         if (isDefinedPriority == false)
331         {
332             throw new DomainRuleViolationException(
333                 "The selected todo priority is not supported.");
334         }
336         return priority;
337     }
338 }
```

Öncelik kontrolü yapan yardımcı metot.

`Enum.IsDefined` değerin enum'da gerçekten var olup olmadığına bakar; C#'ta enum'a her sayı atanabildiği için gereklidir (ör. 0 veya 99).

Tanımsız öncelik gelirse kural ihlali hatası fırlatılır.

Geçerli öncelik döndürülür.

Domain `todo.Domain/Enums/TodoPriority.cs`

> Görev önceliğini (Düşük, Normal, Yüksek) tanımlayan enum. Veritabanında int olarak saklanır ve görev oluştururken öncelik bu değerlerden seçilir.

```
1 namespace TodoApi.Domain.Enums;
```

Ad alanı: domain'deki sabit değer kümeleri (enum'lar) `TodoApi.Domain.Enums` altında durur.

ƒ enum TodoPriority — Görevin öncelik seviyelerini tanımlayan sabit değer kümesi.

```
3 public enum TodoPriority
4 {
5     Low = 1,
6     Normal = 2,
7     High = 3
8 }
```

`enum`, isimlendirilmiş sabit sayılardan oluşan bir türdür; sihirli sayılar yerine okunabilir isimler kullanmayı sağlar.

Değerler 1'den başlatılır; böylece varsayılan 0 değeri geçersiz kalır ve `Enum.IsDefined` kontrolüyle yakalanır. DB'de bu sayılar saklanır.

Domain `todo.Domain/Common/NameComparison.cs`

> İki ismin/başlığın aynı sayılıp sayılmayacağına karar veren yardımcı sınıf. Türkçe kültür ve büyük/küçük harf duyarsız karşılaştırma yaparak SQL Server'ın Turkish_CI_AS kuralıyla aynı sonucu verir.

```
1 using System.Globalization;
3 namespace TodoApi.Domain.Common;
```

`System.Globalization`, kültüre (dile) özgü karşılaştırma araçlarını (`CultureInfo`, `CompareInfo`) içerir.

Ad alanı: domain genelinde paylaşılan yardımcılar `TodoApi.Domain.Common` altında durur.

ƒ class NameComparison — İsimleri Türkçe kurallarına göre, harf büyüklüğünü önemsemeden karşılaştıran yardımcı sınıf.

```
5 public static class NameComparison
6 {
7     private static readonly CompareInfo TurkishCompareInfo = CultureInfo.GetCulturalInfo("tr-TR").CompareInfo;
```

`static class` örneği oluşturulamaz, sadece statik üyeler içerir; durum tutmayan yardımcı fonksiyonlar için kullanılır.

tr-TR kültürünün karşılaştırıcısı (`CompareInfo`) bir kez alınıp saklanır. Türkçe kuralı sayesinde "i/İ" ve "ı/I" doğru eşleşir, DB collation'ıyla uyumlu olur.

ƒ AreSame — İki metni kırpıp Türkçe kurallarla, büyük/küçük harf duyarsız karşılaştırır.

```
9     public static bool AreSame(string first, string second)
10    {
11        return TurkishCompareInfo.Compare(first.Trim(), second.Trim(), CompareOptions.IgnoreCase) == 0;
12    }
13 }
```

İki metin aynı isim sayılıyorsa `true` döndüren metot.

Baştaki/sondaki boşluklar `Trim` ile atılır, `IgnoreCase` ile harf büyüklüğü yok sayılır; sonuç 0 ise metinler eşit kabul edilir.

Domain `todo.Domain/Exceptions/DomainRuleViolationException.cs`

> Domain katmanında bir iş kuralı çiğnendiğinde (boş başlık, çok uzun isim, UTC olmayan tarih gibi) fırlatılan özel hata sınıfı. Üst katmanlar bunu yakalayıp

kullanıcıya anlamlı bir hata (400) döndürür.

```
1 namespace TodoApi.Domain.Exceptions;
```

Dosyanın ad alanı (namespace): domain'e ait tüm özel hatalar `TodoApi.Domain.Exceptions` altında toplanır.

ƒ class DomainRuleViolationException — İş kuralı ihlallerini temsil eden, türetilemez özel exception sınıfı.

```
2 public sealed class DomainRuleViolationException : Exception
3 {
```

`Exception` sınıfından türeyen özel hata. `sealed` bu sınıftan başka sınıf türetilemeyeceği anlamına gelir; tasarım netliği sağlar.

ƒ DomainRuleViolationException(string message) — Hata mesajını alıp temel Exception sınıfına ileten kurucu.

```
4     public DomainRuleViolationException(string message) : base(message)
5     {
6     }
7 }
```

Kurucu metot; `: base(message)` mesajı üst sınıf `Exception`'a iletir, böylece `exception.Message` ile okunabilir. Gövde boştur çünkü ekstra iş yoktur.

Domain `todo.Domain/Exceptions/DuplicateNameException.cs`

Aynı listede aynı başlıkta ikinci bir görev eklenmeye çalışıldığında domain'in fırlattığı özel hata. Üst katmanda 409 Conflict cevabına çevrilir.

```
1 namespace TodoApi.Domain.Exceptions;
```

Ad alanı: domain hatalarının toplandığı `TodoApi.Domain.Exceptions`.

ƒ class DuplicateNameException — Tekrarlanan isim/başlık durumunu temsil eden özel exception sınıfı.

```
3 public sealed class DuplicateNameException : Exception
4 {
```

`Exception`'dan türeyen `sealed` (türetilemez) sınıf. Ayrı bir tür olması, üst katmanın bu hatayı ayırt edip 409 döndürmesini sağlar.

ƒ DuplicateNameException(string message) — Hata mesajını temel Exception sınıfına ileten kurucu.

```
5     public DuplicateNameException(string message) : base(message)
6     {
7     }
8 }
```

Kurucu; `: base(message)` ile mesajı `Exception` sınıfına iletir. Başka iş yapmadığı için gövde boştur.

Infrastructure `todo.Infrastructure/Persistence/TodoDbContext.cs`

EF Core'un veritabanı oturumu: tabloları (DbSet), sütun kurallarını, indeksleri ve ilişkileri tanımlar; kaydetme hatalarını anlamlı hatalara çevirir. Görev oluşturulduğunda kayıt bu sınıf üzerinden SQL Server'a yazılır.

```
1 using System;
2 using System.Collections.Generic;
3 using System.Threading;
4 using System.Threading.Tasks;
```

Temel .NET ad alanları: `DateTime`, koleksiyonlar, iptal belirteci (`CancellationToken`) ve asenkron `Task`.

```
5 using Microsoft.EntityFrameworkCore;
6 using Microsoft.EntityFrameworkCore.Metadata.Builders;
7 using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
```

EF Core ad alanları: `DbContext`, yapılandırma builder'ları ve değer dönüştürücüler (`ValueConverter`).

```
8 using TodoApi.Domain.Entities;
9 using TodoApi.Domain.Enums;
```

Tablolara eşlenecek domain entity'leri ve enum'lar.

```
11 namespace TodoApi.Infrastructure.Persistence;
```

Ad alanı: veritabanı altyapı kodu `TodoApi.Infrastructure.Persistence`.

ƒ class TodoDbContext — Uygulamanın EF Core veritabanı bağlamı; tablo, ilişki ve kaydetme davranışını tanımlar.

```
13 public sealed class TodoDbContext : DbContext
14 {
15     private static readonly ValueConverter<DateTime, DateTime> UtcDateTimeConverte
   r =
16         new ValueConverter<DateTime, DateTime>(
17             value => value,
18             value => DateTime.SpecifyKind(value, DateTimeKind.Utc));
```

`DbContext`'ten türer; EF Core ile veritabanı arasındaki köprüdür. `sealed` türetilmesini engeller.

`ValueConverter`: yazarken tarihi aynen bırakır, okurken `SpecifyKind` ile türünü Utc yapar. SQL Server tarih türünü saklamadığı için gereklidir.

ƒ TodoDbContext(DbContextOptions<TodoDbContext> options) — Bağlantı ayarlarını alıp temel DbContext'e ileten kurucu.

```
20     public TodoDbContext(DbContextOptions<TodoDbContext> options)
21         : base(options)
22     {
23     }
```

Bağlantı dizesi gibi ayarlar dependency injection ile gelir ve `: base(options)` ile EF Core'a verilir.

```
25     public DbSet<TodoList> TodoLists
26     {
27         get
28         {
29             return Set<TodoList>();
30         }
31     }
```

`DbSet<TodoList>`: TodoLists tablosunu temsil eder; sorgu ve ekleme bunun üzerinden yapılır. `Set<T>()` EF'in o türe ait kümesini döndürür.

```
33     public DbSet<TodoItem> TodoItems
34     {
35         get
36         {
37             return Set<TodoItem>();
38         }
39     }
```

`TodoItems` tablosuna erişim sağlayan DbSet.

```
    public DbSet<User> Users
    {
        get
        {
            return Set<User>();
        }
    }
```

`Users` tablosuna erişim sağlayan DbSet.

```
    public DbSet<RefreshToken> RefreshTokens
    {
        get
        {
            return Set<RefreshToken>();
        }
    }
```

`RefreshTokens` tablosuna erişim sağlayan DbSet (oturum yenileme token'ları).

ƒ SaveChangesAsync — Değişiklikleri kaydeder; DB hatalarını uygulama hatalarına çevirir.

```
    public override async Task<int> SaveChangesAsync(
        bool acceptAllChangesOnSuccess,
        CancellationToken cancellationToken = default)
    {
        try
        {
            return await base.SaveChangesAsync(acceptAllChangesOnSuccess, cancellationToken);
        }
        catch (DbUpdateException exception)
        {
            throw DatabaseErrorTranslator.Translate(exception);
        }
    }
```

`override` temel sınıftaki metodu yeniden yazar. Diğer `SaveChangesAsync` aşırı yüklemeleri de sonunda buraya gelir.

Asıl kaydetme işi `base.SaveChangesAsync` ile EF Core'a bırakılır; `await` işlem bitene kadar bekler, thread'i bloklamaz.

Kaydetme hatası yakalanır ve `DatabaseErrorTranslator` ile 409/503 anlamlı hatalara çevrilip yeniden fırlatılır.

ƒ OnModelCreating — EF Core modelini (tablolar, sütunlar, indeksler, ilişkiler) yapılandırır.

```
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        ConfigureUser(modelBuilder);
        ConfigureTodoList(modelBuilder);
        ConfigureTodoItem(modelBuilder);
        ConfigureRefreshToken(modelBuilder);
        ConfigureUserToTodoListRelationship(modelBuilder);
        ConfigureTodoListToTodoItemRelationship(modelBuilder);
    }
```

EF Core modeli oluştururken bir kez çağırır; tablo yapısı ve migration'lar buradaki ayarlardan üretilir.

Önce temel sınıfın varsayılan yapılandırması çalıştırılır.

Yapılandırma okunabilirlik için entity ve ilişki başına ayrı küçük metotlara bölünmüştür.

ƒ ConfigureUser — Users tablosunun sütunlarını, indekslerini ve yoksayılan alanlarını tanımlar.

```
    private static void ConfigureUser(ModelBuilder modelBuilder)
    {
        EntityTypeBuilder<User> userEntity = modelBuilder.Entity<User>();

        userEntity.ToTable("Users");
        userEntity.HasKey(user => user.Id);

        PropertyBuilder<int> userIdProperty = userEntity.Property(user => user.Id);

        userIdProperty.ValueGeneratedOnAdd();

        PropertyBuilder<string> userEmailProperty = userEntity.Property(user => user.Email);

        userEmailProperty.HasMaxLength(256);
        userEmailProperty.IsRequired();

        userEntity.HasIndex(user => user.Email).IsUnique();

        PropertyBuilder<string> userDisplayNameProperty =
            userEntity.Property(user => user.DisplayName);

        userDisplayNameProperty.HasMaxLength(100);
        userDisplayNameProperty.IsRequired();

        PropertyBuilder<string?> userPasswordHashProperty =
            userEntity.Property(user => user.PasswordHash);

        userPasswordHashProperty.HasMaxLength(500);
        userPasswordHashProperty.IsRequired(false);

        PropertyBuilder<string?> userGoogleSubjectIdProperty =
            userEntity.Property(user => user.GoogleSubjectId);

        userGoogleSubjectIdProperty.HasMaxLength(255);
        userGoogleSubjectIdProperty.IsRequired(false);

        userEntity.HasIndex(user => user.GoogleSubjectId).IsUnique();
```

Kullanıcı entity'sinin veritabanı eşlemesi.

`Entity<User>()` yapılandırıcıyı alır; tablo adı `Users`, birincil anahtar (`HasKey`) `Id` olarak belirlenir.

`ValueGeneratedOnAdd`: Id değeri ekleme sırasında DB tarafından (identity) üretilir.

E-posta en fazla 256 karakter ve zorunlu (`NOT NULL`).

`HasIndex(...).IsUnique()`: e-posta için benzersiz indeks; aynı e-postayla iki hesap açılamaz.

Görünen ad en fazla 100 karakter ve zorunlu.

Şifre hash'i isteğe bağlı (`IsRequired(false)`); Google ile giriş yapanların şifresi olmayabilir.

Google hesap kimliği (subject id) isteğe bağlı, en fazla 255 karakter.

Aynı Google hesabı iki kullanıcıya bağlanamasın diye benzersiz indeks.

121	`        PropertyBuilder<UserRole> userRoleProperty = userEntity.Property(user => user.Role);`	`HasConversion<int>()`: rol enum'u DB'de sayı olarak saklanır; zorunludur.
122		
123	`        userRoleProperty.HasConversion<int>();`	
124	`        userRoleProperty.IsRequired();`	
126	`        PropertyBuilder<DateTime> userCreatedAtUtcProperty =`	Oluşturulma zamanı UTC dönüştürücüsüyle eşlenir; okunurken Utc olarak işaretlenir.
127	`            userEntity.Property(user => user.CreatedAtUtc);`	
128		
129	`        userCreatedAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
130	`        userCreatedAtUtcProperty.IsRequired();`	
132	`        userEntity.Ignore(user => user.HasPassword);`	`Ignore`: hesaplanan özellikler (`HasPassword` vb.) tabloya sütun olarak eklenmez.
133	`        userEntity.Ignore(user => user.HasGoogleLogin);`	
134	`        userEntity.Ignore(user => user.IsAdmin);`	
135	`    }`	
	ƒ ConfigureTodoList — TodoLists tablosunun sütunlarını ve benzersiz indeksini tanımlar.	
137	`    private static void ConfigureTodoList(ModelBuilder modelBuilder)`	Görev listesi entity'sinin veritabanı eşlemesi.
138	`    {`	
139	`        EntityTypeBuilder<TodoList> todoListEntity = modelBuilder.Entity<TodoList>();`	Tablo adı `TodoLists`, birincil anahtar `Id`.
140		
141	`        todoListEntity.ToTable("TodoLists");`	
142	`        todoListEntity.HasKey(todoList => todoList.Id);`	
144	`        PropertyBuilder<int> todoListIdProperty =`	Liste Id'si eklemede DB tarafından üretilir.
145	`            todoListEntity.Property(todoList => todoList.Id);`	
146		
147	`        todoListIdProperty.ValueGeneratedOnAdd();`	
149	`        PropertyBuilder<string> todoListNameProperty =`	Liste adı en fazla 80 karakter ve zorunlu; domain'deki `MaximumNameLength` ile aynı.
150	`            todoListEntity.Property(todoList => todoList.Name);`	
151		
152	`        todoListNameProperty.HasMaxLength(80);`	
153	`        todoListNameProperty.IsRequired();`	
155	`        PropertyBuilder<DateTime> todoListCreatedAtUtcProperty =`	Oluşturulma zamanı zorunlu ve UTC dönüştürücülü.
156	`            todoListEntity.Property(todoList => todoList.CreatedAtUtc);`	
157		
158	`        todoListCreatedAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
159	`        todoListCreatedAtUtcProperty.IsRequired();`	
161	`        PropertyBuilder<int> todoListOwnerUserIdProperty =`	Sahip kullanıcı id'si zorunlu; her liste bir kullanıcıya aittir.
162	`            todoListEntity.Property(todoList => todoList.OwnerUserId);`	
163		
164	`        todoListOwnerUserIdProperty.IsRequired();`	
166	`        todoListEntity`	(OwnerUserId, Name) bileşik benzersiz indeks: aynı kullanıcı aynı adlı iki liste açamaz. Eşzamanlı isteklere karşı son savunma hattıdır.
167	`            .HasIndex(todoList => new { todoList.OwnerUserId, todoList.Name })`	
168	`            .IsUnique();`	
169	`    }`	
	ƒ ConfigureTodoItem — TodoItems tablosunun sütunlarını, varsayılanlarını ve indekslerini tanımlar.	
171	`    private static void ConfigureTodoItem(ModelBuilder modelBuilder)`	Görev entity'sinin veritabanı eşlemesi.
172	`    {`	
173	`        EntityTypeBuilder<TodoItem> todoItemEntity = modelBuilder.Entity<TodoItem>();`	Tablo adı `TodoItems`, birincil anahtar `Id`.
174		
175	`        todoItemEntity.ToTable("TodoItems");`	
176	`        todoItemEntity.HasKey(todoItem => todoItem.Id);`	
178	`        PropertyBuilder<int> todoItemIdProperty =`	Görev Id'si eklemede DB tarafından üretilir.
179	`            todoItemEntity.Property(todoItem => todoItem.Id);`	
180		
181	`        todoItemIdProperty.ValueGeneratedOnAdd();`	
183	`        PropertyBuilder<int> todoItemTodoListIdProperty =`	Bağlı liste id'si (foreign key) zorunlu; görev listesiz olamaz.
184	`            todoItemEntity.Property(todoItem => todoItem.TodoListId);`	
185		
186	`        todoItemTodoListIdProperty.IsRequired();`	
188	`        PropertyBuilder<string> todoItemTitleProperty =`	Başlık en fazla 120 karakter ve zorunlu.
189	`            todoItemEntity.Property(todoItem => todoItem.Title);`	
190		
191	`        todoItemTitleProperty.HasMaxLength(120);`	
192	`        todoItemTitleProperty.IsRequired();`	
194	`        PropertyBuilder<string?> todoItemDescriptionProperty =`	Açıklama en fazla 1000 karakter; `string?` olduğu için null olabilir.
195	`            todoItemEntity.Property(todoItem => todoItem.Description);`	
196		
197	`        todoItemDescriptionProperty.HasMaxLength(1000);`	
199	`        PropertyBuilder<bool> todoItemIsCompletedProperty =`	Tamamlanma durumu zorunlu (bit sütunu).
200	`            todoItemEntity.Property(todoItem => todoItem.IsCompleted);`	
201		
202	`        todoItemIsCompletedProperty.IsRequired();`	
204	`        PropertyBuilder<DateTime> todoItemCreatedAtUtcProperty =`	Oluşturulma zamanı zorunlu ve UTC dönüştürücülü.
205	`            todoItemEntity.Property(todoItem => todoItem.CreatedAtUtc);`	
206		
207	`        todoItemCreatedAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
208	`        todoItemCreatedAtUtcProperty.IsRequired();`	

210	`        PropertyBuilder<DateTime?> todoItemDueAtUtcProperty =`	Bitiş tarihi isteğe bağlı; nullable olsa da aynı UTC dönüştürücüsü uygulanır.
211	`            todoItemEntity.Property(todoItem => todoItem.DueAtUtc);`	
212		
213	`        todoItemDueAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
214	`        todoItemDueAtUtcProperty.IsRequired(false);`	
216	`        PropertyBuilder<TodoPriority> todoItemPriorityProperty =`	Öncelik enum'u DB'de int olarak saklanır (1, 2, 3) ve zorunludur.
217	`            todoItemEntity.Property(todoItem => todoItem.Priority);`	
218		
219	`        todoItemPriorityProperty.HasConversion<int>();`	
220	`        todoItemPriorityProperty.IsRequired();`	
222	`        PropertyBuilder<DateTime?> todoItemReminderSentAtUtcProperty =`	Hatırlatma gönderilme zamanı isteğe bağlı, UTC dönüştürücülü.
223	`            todoItemEntity.Property(todoItem => todoItem.ReminderSentAtUtc);`	
224		
225	`        todoItemReminderSentAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
226	`        todoItemReminderSentAtUtcProperty.IsRequired(false);`	
228	`        PropertyBuilder<int> todoItemReminderAttemptCountProperty =`	Deneme sayısı zorunlu; `HasDefaultValue(0)` sütuna DB varsayılanı ekler, migration sırasında eski satırlar 0 alır.
229	`            todoItemEntity.Property(todoItem => todoItem.ReminderAttemptCount);`	
230		
231	`        todoItemReminderAttemptCountProperty.IsRequired();`	
232	`        todoItemReminderAttemptCountProperty.HasDefaultValue(0);`	
234	`        PropertyBuilder<DateTime?> todoItemReminderLastAttemptAtUtcProperty =`	Son hatırlatma deneme zamanı isteğe bağlı, UTC dönüştürücülü.
235	`            todoItemEntity.Property(todoItem => todoItem.ReminderLastAttemptAtUtc);`	
236		
237	`        todoItemReminderLastAttemptAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
238	`        todoItemReminderLastAttemptAtUtcProperty.IsRequired(false);`	
240	`        todoItemEntity.HasIndex(todoItem => todoItem.DueAtUtc);`	`DueAtUtc` için normal indeks: hatırlatma işi yaklaşan görevleri hızlı bulsun diye.
242	`        todoItemEntity`	(TodoListId, Title) benzersiz indeks: aynı listede aynı başlık iki kez kaydedilemez; domain kontrolünü atlatan eşzamanlı istekleri yakalar.
243	`            .HasIndex(todoItem => new { todoItem.TodoListId, todoItem.Title })`	
244	`            .IsUnique();`	
245	`    }`	

ƒ ConfigureRefreshToken — RefreshTokens tablosunu ve kullanıcıyla ilişkisini tanımlar.

247	`    private static void ConfigureRefreshToken(ModelBuilder modelBuilder)`	Oturum yenileme token'larının veritabanı eşlemesi.
248	`    {`	
249	`        EntityTypeBuilder<RefreshToken> refreshTokenEntity = modelBuilder.Entity<RefreshToken>();`	Tablo adı `RefreshTokens`, birincil anahtar `Id`.
250		
251	`        refreshTokenEntity.ToTable("RefreshTokens");`	
252	`        refreshTokenEntity.HasKey(refreshToken => refreshToken.Id);`	
254	`        PropertyBuilder<int> refreshTokenIdProperty =`	Token Id'si eklemede DB tarafından üretilir.
255	`            refreshTokenEntity.Property(refreshToken => refreshToken.Id);`	
256		
257	`        refreshTokenIdProperty.ValueGeneratedOnAdd();`	
259	`        PropertyBuilder<string> tokenHashProperty =`	Token hash'i: 64 karakter sabit uzunluk (`IsFixedLength`), Unicode değil (`varchar`), zorunlu. Token'ın kendisi değil hash'i saklanır.
260	`            refreshTokenEntity.Property(refreshToken => refreshToken.TokenHash);`	
261		
262	`        tokenHashProperty.HasMaxLength(64);`	
263	`        tokenHashProperty.IsFixedLength();`	
264	`        tokenHashProperty.IsUnicode(false);`	
265	`        tokenHashProperty.IsRequired();`	
267	`        refreshTokenEntity.HasIndex(refreshToken => refreshToken.TokenHash).IsUnique();`	Hash için benzersiz indeks: hem tekrar engellenir hem token araması hızlanır.
269	`        PropertyBuilder<string?> replacedByTokenHashProperty =`	Yerine geçen token'ın hash'i: aynı format, isteğe bağlı. Token döndürme (rotation) zincirini izlemek için.
270	`            refreshTokenEntity.Property(refreshToken => refreshToken.ReplacedByTokenHash);`	
271		
272	`        replacedByTokenHashProperty.HasMaxLength(64);`	
273	`        replacedByTokenHashProperty.IsFixedLength();`	
274	`        replacedByTokenHashProperty.IsUnicode(false);`	
275	`        replacedByTokenHashProperty.IsRequired(false);`	
277	`        PropertyBuilder<DateTime> createdAtUtcProperty =`	Token oluşturulma zamanı zorunlu ve UTC dönüştürücülü.
278	`            refreshTokenEntity.Property(refreshToken => refreshToken.CreatedAtUtc);`	
279		
280	`        createdAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
281	`        createdAtUtcProperty.IsRequired();`	
283	`        PropertyBuilder<DateTime> expiresAtUtcProperty =`	Token son kullanma zamanı zorunlu ve UTC dönüştürücülü.
284	`            refreshTokenEntity.Property(refreshToken => refreshToken.ExpiresAtUtc);`	
285		
286	`        expiresAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
287	`        expiresAtUtcProperty.IsRequired();`	
289	`        PropertyBuilder<DateTime?> revokedAtUtcProperty =`	İptal edilme zamanı isteğe bağlı; null ise token iptal edilmemiştir.
290	`            refreshTokenEntity.Property(refreshToken => refreshToken.RevokedAtUtc);`	
291		
292	`        revokedAtUtcProperty.HasConversion(UtcDateTimeConverter);`	
293	`        revokedAtUtcProperty.IsRequired(false);`	
295	`        refreshTokenEntity.Ignore(refreshToken => refreshToken.IsRevoked);`	Hesaplanan `IsRevoked` özelliği tabloya sütun olarak eklenmez.

```
        refreshTokenEntity
            .HasOne<User>()
            .WithMany()
            .HasForeignKey(refreshToken => refreshToken.UserId)
            .OnDelete(DeleteBehavior.Cascade);
    }
```

Her token bir kullanıcıya bağlı (`HasOne/WithMany`, foreign key `UserId`). `OnDelete(DeleteBehavior.Cascade)`: kullanıcı silininca token'ları da silinir.

ƒ ConfigureUserToTodoListRelationship — Kullanıcı ile görev listeleri arasındaki bire-çok ilişkiyi tanımlar.

```
    private static void ConfigureUserToTodoListRelationship(ModelBuilder modelBuilder)
    {
        EntityTypeBuilder<TodoList> todoListEntity = modelBuilder.Entity<TodoList>();

        ReferenceNavigationBuilder<TodoList, User> ownerNavigation =
            todoListEntity.HasOne<User>();

        ReferenceCollectionBuilder<User, TodoList> ownerRelationship =
            ownerNavigation.WithMany();

        ownerRelationship.HasForeignKey(todoList => todoList.OwnerUserId);
        ownerRelationship.OnDelete(DeleteBehavior.Cascade);
    }
```

Kullanıcı-liste ilişkisinin yapılandırması.

TodoList entity yapılandırıcısı alınır.

Her listenin bir sahibi (`HasOne<User>`), her kullanıcının çok listesi (`WithMany`) vardır; entity'lerde gezinme özelliği tanımlanmaz.

Foreign key `OwnerUserId`; kullanıcı silininca listeleri de cascade ile silinir.

ƒ ConfigureTodoListToTodoItemRelationship — Liste ile görevleri arasındaki ilişkiyi ve private _todos alanının kullanımını tanımlar.

```
    private static void ConfigureTodoListToTodoItemRelationship(ModelBuilder modelBuilder)
    {
        EntityTypeBuilder<TodoList> todoListEntity = modelBuilder.Entity<TodoList>();

        CollectionNavigationBuilder<TodoList, TodoItem> todoItemsNavigation =
            todoListEntity.HasMany(todoList => todoList.Todos);

        ReferenceCollectionBuilder<TodoList, TodoItem> todoItemsRelationship =
            todoItemsNavigation.WithOne();

        todoItemsRelationship.HasForeignKey(todoItem => todoItem.TodoListId);
        todoItemsRelationship.OnDelete(DeleteBehavior.Cascade);

        NavigationBuilder<TodoList, IReadOnlyCollection<TodoItem>> todoListTodosNavigation =
            todoListEntity.Navigation(todoList => todoList.Todos);

        todoListTodosNavigation.HasField("_todos");
        todoListTodosNavigation.UsePropertyAccessMode(PropertyAccessMode.Field);
    }
}
```

Liste-görev ilişkisinin yapılandırması (aggregate yapısı).

TodoList entity yapılandırıcısı alınır.

Bir listenin çok görevi var (`HasMany(Todos)`), her görev bir listeye ait (`WithOne`).

Foreign key `TodoListId`; liste silininca görevleri de cascade ile silinir.

`Todos` gezinme özelliğinin ayar nesnesi alınır.

`HasField("_todos")` ve `UsePropertyAccessMode(PropertyAccessMode.Field)`: EF salt okunur `Todos` yerine private `_todos` listesini doğrudan doldurur; kapsülleme bozulmaz.

Infrastructure `todo.Infrastructure/Persistence/DatabaseErrorTranslator.cs`

> EF Core'un kaydetme sırasında fırlattığı teknik veritabanı hatalarını, uygulamanın anladığı anlamlı hatalara (409 çakışma, 503 erişim hatası) çeviren yardımcı sınıf. TodoDbContext.SaveChangesAsync içinden çağrılır.

```
using System;
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using TodoApi.Application.Exceptions;

namespace TodoApi.Infrastructure.Persistence;
```

Gerekli ad alanları: SQL Server hataları (`SqlException`), EF Core hataları (`DbUpdateException`) ve Application katmanındaki özel hata sınıfları.

Ad alanı: veritabanıyla ilgili altyapı kodu `TodoApi.Infrastructure.Persistence` altında.

ƒ class DatabaseErrorTranslator — Veritabanı hatalarını uygulama hatalarına çeviren statik yardımcı sınıf.

```
public static class DatabaseErrorTranslator
{
    private const int DuplicateKeyInUniqueIndex = 2601;
    private const int UniqueConstraintViolation = 2627;
```

`static class`: nesnesi oluşturulmaz, durum tutmaz; sadece çeviri fonksiyonları içerir.

SQL Server hata kodları: 2601 unique index'te tekrar eden anahtar, 2627 unique kısıt ihlali. Sihirli sayı yerine isimli sabitler kullanılır.

ƒ Translate — DbUpdateException'ı uygun uygulama hatasına (409 veya 503) dönüştürür.

```
    public static Exception Translate(DbUpdateException exception)
    {
        if (exception == null)
        {
            throw new ArgumentNullException(nameof(exception));
        }

        if (exception is DbUpdateConcurrencyException)
        {
            return new ConcurrencyConflictException(
                "The record was changed or deleted by another request.",
                exception);
        }
```

Hatayı fırlatmaz, döndürür; fırlatma işini çağıran taraf (`throw ...Translate(...)`) yapar.

Girdi null ise `ArgumentNullException` fırlatılır; programlama hatası erken yakalanır.

`is` tür kontrolüdür. Kayıt bu arada başka istek tarafından değiştirilmiş/silinmişse `ConcurrencyConflictException` (409) döner; asıl hata iç hata olarak saklanır.

```
27          if (exception.InnerException is SqlException sqlException && IsUniqueViola
    tion(sqlException.Number))
28          {
29              return new ConflictException(
30                  "The record conflicts with an existing record.",
31                  exception);
32          }
```

İç hata `SqlException` ise ve kodu 2601/2627 ise `ConflictException` (409) döner. Aynı anda gelen tekrar eden isimleri unique index burada yakalar.

```
34          return new DataAccessException(
35              "The changes could not be saved to the database.",
36              exception);
37      }
```

Diğer tüm kaydetme hataları `DataAccessException` (503) olur; kullanıcıya teknik detay sızdırılmaz, orijinal hata log için saklanır.

ƒ IsUniqueViolation — SQL hata kodunun benzersizlik ihlali olup olmadığını söyler.

```
39      public static bool IsUniqueViolation(int sqlErrorNumber)
40      {
41          return sqlErrorNumber == DuplicateKeyInUniqueIndex || sqlErrorNumber == Un
    iqueConstraintViolation;
42      }
43  }
```

Hata kodu 2601 veya 2627 ise `true` döner. `public` olduğu için başka yerler de aynı kontrolü kullanabilir.

Application `todo.Application/Exceptions/ConflictException.cs`

> Veritabanı tekillik (unique) kuralı bir yazmayı reddettiğinde fırlatılan uygulama exception'ı. Infrastructure SQL hatasını bu tipe çevirir, API'de DatabaseExceptionHandler 409 döner.

```
1  using System;
```

`Exception` temel sınıfı için `System` namespace'i.

```
3  namespace TodoApi.Application.Exceptions;
```

Application katmanının ortak exception'ları bu namespace'te toplanır; API ve Infrastructure buraya bağımlıdır, tersi olmaz.

ƒ ConflictException — Unique index/constraint ihlalini temsil eden exception sınıfı.

```
5  public sealed class ConflictException : Exception
```

`Exception`'dan türeyen `sealed` sınıf; SQL Server'a özgü hatayı katmandan bağımsız bir tipe dönüştürmek için vardır.

```
6  {
```

ƒ ConflictException — Mesaj ve asıl hatayı alıp temel Exception'a iletir.

```
7      public ConflictException(string message, Exception innerException)
8          : base(message, innerException)
```

`: base(...)` temel sınıf constructor'ını çağırır. `innerException` asıl `DbUpdateException`'ı saklar ki loglarda SQL detayı görünsün.

```
9      {
10     }
11 }
```

Application `todo.Application/Exceptions/DataAccessException.cs`

> Veritabanına yazma beklenmedik bir sebeple başarısız olduğunda (bağlantı yok, disk dolu vb.) kullanılan genel exception. API'de 503 yanıtına dönüşür.

```
1  using System;
```

`Exception` temel sınıfı için `System` namespace'i.

```
3  namespace TodoApi.Application.Exceptions;
```

Application katmanının ortak exception namespace'i.

ƒ DataAccessException — Genel veritabanı erişim hatasını temsil eden exception sınıfı.

```
5  public sealed class DataAccessException : Exception
```

Tekillik veya eşzamanlılık dışındaki tüm kaydetme hataları için `DatabaseErrorTranslator` bu tipi üretir.

```
6  {
```

ƒ DataAccessException — Mesaj ve asıl hatayı temel Exception'a iletir.

```
7      public DataAccessException(string message, Exception innerException)
8          : base(message, innerException)
```

`: base(message, innerException)` ile asıl EF Core hatası saklanır; kullanıcı görmez ama Error logunda tam detay bulunur.

```
9      {
10     }
11 }
```

Application `todo.Application/Exceptions/ConcurrencyConflictException.cs`

> Kaydedilmek istenen kayıt bu arada başka bir istek tarafından değiştirilmiş veya silinmişse fırlatılır. API'de `urn:todo:problem:concurrency-conflict` tipli 409 yanıtına dönüşür.

```
1  using System;
```

`Exception` temel sınıfı için `System` namespace'i.

```
3  namespace TodoApi.Application.Exceptions;
```

Application katmanının ortak exception namespace'i.

ƒ ConcurrencyConflictException — Eşzamanlı değişiklik çakışmasını temsil eden exception sınıfı.

```
5  public sealed class ConcurrencyConflictException : Exception
```

EF Core'un `DbUpdateConcurrencyException`'ını katmandan bağımsız bir tipe çevirmek için kullanılan `sealed` sınıf.

```
6  {
```

ƒ ConcurrencyConflictException — Mesaj ve asıl hatayı temel Exception'a iletir.

```
7      public ConcurrencyConflictException(string message, Exception innerException)
8          : base(message, innerException)
```

`: base(...)` ile mesaj ve orijinal EF Core hatası temel `Exception` sınıfına aktarılır.

```
9      {
10     }
11 }
```

Application `todo.Application/Contracts/TodoItemDto.cs`

> Görevin dışarıya (API yanıtına) gösterilecek alanlarını taşıyan DTO. Entity doğrudan dönülmez; iç alanlar sızmaz.

```
1  using System;
2  using TodoApi.Domain.Entities;
3  using TodoApi.Domain.Enums;
```

`DateTime` için `System`, kaynak entity `TodoItem` ve öncelik enum'u için Domain namespace'leri.

```
5  namespace TodoApi.Application.Contracts;
```

Application'ın dış dünyaya verdiği veri sözleşmeleri `Contracts` klasöründe.

ƒ TodoItemDto — Görevin API'ye açılan salt okunur kopyası.

```
7  public class TodoItemDto
```

DTO (Data Transfer Object): katmanlar arası veri taşıyan basit nesne. Entity değişse bile API sözleşmesi korunur.

```
8  {
```

```
9      private readonly int _id;
10     private readonly int _todoListId;
11     private readonly string _title;
12     private readonly string? _description;
13     private readonly bool _isCompleted;
14     private readonly DateTime _createdAtUtc;
15     private readonly DateTime? _dueAtUtc;
16     private readonly TodoPriority _priority;
```

Tüm alanlar `private readonly`: nesne oluştuktan sonra değiştirilemez (immutable). Hatırlatma deneme sayacı gibi iç alanlar bilerek yok.

ƒ TodoItemDto (constructor) — TodoItem entity'sinden gerekli alanları kopyalayarak DTO oluşturur.

```
18     public TodoItemDto(TodoItem todoItem)
```

Eşleme (mapping) constructor içinde yapılır; ayrıca bir mapping kütüphanesine gerek kalmaz.

```
19     {
20         if (todoItem == null)
21         {
22             throw new ArgumentNullException(nameof(todoItem));
23         }
```

Entity null gelirse hemen `ArgumentNullException` fırlatılır.

```
25         _id = todoItem.Id;
26         _todoListId = todoItem.TodoListId;
27         _title = todoItem.Title;
28         _description = todoItem.Description;
29         _isCompleted = todoItem.IsCompleted;
30         _createdAtUtc = todoItem.CreatedAtUtc;
31         _dueAtUtc = todoItem.DueAtUtc;
32         _priority = todoItem.Priority;
```

Entity'nin dışarı gösterilecek alanları tek tek DTO'ya kopyalanır: kimlik, liste kimliği, başlık, açıklama, durum, tarihler, öncelik.

```
33     }
35     public int Id
36     {
37         get
38         {
39             return _id;
40         }
41     }
```

Sadece `get` olan özellik: JSON'a `id` olarak yazılır, dışarıdan değiştirilemez.

```
43     public int TodoListId
44     {
45         get
46         {
47             return _todoListId;
48         }
49     }
```

Görevin ait olduğu listenin kimliği.

```
51     public string Title
52     {
53         get
54         {
55             return _title;
56         }
57     }
```

Görev başlığı.

```
59     public string? Description
60     {
61         get
62         {
63             return _description;
64         }
65     }
```

Açıklama; `string?` olduğundan null olabilir.

```
67     public bool IsCompleted
68     {
69         get
70         {
71             return _isCompleted;
72         }
73     }
```

Görev tamamlandı mı bilgisi.

```
75     public DateTime CreatedAtUtc
76     {
77         get
78         {
79             return _createdAtUtc;
80         }
81     }
```

Görevin oluşturulma zamanı (UTC).

```
    public DateTime? DueAtUtc
    {
        get
        {
            return _dueAtUtc;
        }
    }
```

Bitiş tarihi (UTC); tanımlanmadıysa null.

```
    public TodoPriority Priority
    {
        get
        {
            return _priority;
        }
    }
}
```

Görev önceliği; Program.cs'teki `JsonStringEnumConverter` sayesinde JSON'da "Normal" gibi metin olarak gönderilir.

API todo.API/ExceptionHandling/ValidationExceptionHandler.cs

> Mekanik (doğrulama) hatalarını yakalayıp 400 Bad Request ve alan bazlı hata listesi içeren ProblemDetails yanıtına çeviren ilk handler. Program.cs sırasında en başta çalışır.

```
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using TodoApi.Application.Exceptions;
```

`IExceptionHandler` arayüzü, `ValidationProblemDetails` sınıfı ve uygulamanın kendi `RequestValidationException` tipi için gerekli namespace'ler.

```
namespace TodoApi.API.ExceptionHandling;
```

Dosya kapsamlı namespace: bu dosyadaki her şey `TodoApi.API.ExceptionHandling` altında yer alır.

ƒ ValidationExceptionHandler — Doğrulama hatalarını 400 ProblemDetails yanıtına çeviren exception handler sınıfı.

```
public sealed class ValidationExceptionHandler : IExceptionHandler
{
```

`IExceptionHandler`, ASP.NET Core 8'in global hata yakalama arayüzüdür. `sealed` bu sınıftan miras alınamayacağını belirtir.

```
    private readonly ILogger<ValidationExceptionHandler> _logger;
```

Log yazmak için kullanılacak logger alanı; `ILogger<T>` log satırlarında kaynak olarak bu sınıfın adını gösterir.

ƒ ValidationExceptionHandler — Constructor; DI'dan gelen logger'ı alana kaydeder.

```
    public ValidationExceptionHandler(ILogger<ValidationExceptionHandler> logger)
    {
        _logger = logger;
    }
```

Constructor injection: DI container logger'ı otomatik verir, biz de alana saklarız.

ƒ TryHandleAsync — Hata doğrulama hatasıysa 400 yanıtı yazar ve true döner; değilse false döner.

```
    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext,
        Exception exception,
        CancellationToken cancellationToken)
    {
```

`ValueTask<bool>` döner: true ise hata işlendi demektir, false ise sıradaki handler'a (Domain) geçilir.

```
        if (exception is not RequestValidationException validationException)
        {
            return false;
        }
```

`is not X değişken` pattern'i: hata `RequestValidationException` değilse false döner; öyleyse `validationException` değişkenine atanır.

```
        _logger.LogInformation(
            "Validation failed for {RequestPath}: {InvalidFields}",
            httpContext.Request.Path,
            validationException.Errors.Keys);
```

Structured logging: `{RequestPath}` ve `{InvalidFields}` yer tutucuları Serilog'da ayrı aranabilir alanlar olur. Kullanıcı hatası olduğu için Information seviyesinde loglanır.

```
        Dictionary<string, string[]> errors = new Dictionary<string, string[]>(validationException.Errors);
```

Exception içindeki alan adı -> hata mesajları sözlüğünü kopyalar; `ValidationProblemDetails` bu tipi bekler.

```
        ValidationProblemDetails problemDetails = new ValidationProblemDetails(errors)
        {
            Status = StatusCodes.Status400BadRequest,
            Title = "One or more validation errors occurred.",
            Detail = errors.Values.SelectMany(messages => messages).FirstOrDefault()
        };
```

Hata sözlüğüyle `ValidationProblemDetails` oluşturulur; yanıtta `errors` alanı olarak döner. Durum kodu 400, başlık sabit bir metindir.

`SelectMany` tüm mesaj dizilerini tek listeye düzleştirir; ilk mesaj `Detail` olur ki ön yüz tek satırlık bir uyarı gösterebilsin.

```
        await ProblemDetailsWriter.WriteAsync(httpContext, problemDetails, cancellationToken);
```

Ortak yazıcı yanıtı `application/problem+json` olarak, `traceId` ve `instance` ekleyerek gönderir.

```
        return true;
    }
}
```

true dönerek hatanın işlendiğini bildirir; sonraki handler'lar çalışmaz.

API todo.API/ExceptionHandling/DomainExceptionHandler.cs

> İş kuralı (domain) hatalarını HTTP yanıtına çevirir: aynı isim varsa 409 Conflict, başka bir kural ihlalinde 400. Validation handler'dan sonra ikinci sırada çalışır.

```
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using TodoApi.Domain.Exceptions;
```

Handler arayüzü, `ProblemDetails` ve Domain katmanının exception tipleri (`DuplicateNameException`, `DomainRuleViolationException`) için using'ler.

```
namespace TodoApi.API.ExceptionHandling;
```

Dosya kapsamlı namespace bildirimi.

ƒ DomainExceptionHandler — Domain kural ihlallerini 409 veya 400 ProblemDetails yanıtına çeviren handler.

```
public sealed class DomainExceptionHandler : IExceptionHandler
```

`IExceptionHandler` uygulayan, miras alınamayan (`sealed`) handler sınıfı.

```
{
```

```
    private readonly ILogger<DomainExceptionHandler> _logger;
```

Bu sınıf adına log yazacak logger alanı.

ƒ DomainExceptionHandler — Constructor; logger'ı DI'dan alıp saklar.

```
    public DomainExceptionHandler(ILogger<DomainExceptionHandler> logger)
    {
        _logger = logger;
    }
```

Logger constructor injection ile gelir ve alana atanır.

ƒ TryHandleAsync — Domain hatasıysa uygun durum koduyla yanıt yazar, değilse false döner.

```
    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext,
        Exception exception,
        CancellationToken cancellationToken)
```

Her handler'daki ortak imza; false dönerse ASP.NET sıradaki handler'ı (Application) dener.

```
    {
```

```
        int statusCode;
        string title;
```

Durum kodu ve başlık aşağıdaki if/else'te hata türüne göre doldurulacak.

```
        if (exception is DuplicateNameException)
        {
            statusCode = StatusCodes.Status409Conflict;
            title = "The name is already in use.";
        }
```

Aynı isimde liste/görev varsa iş kuralı çakışmasıdır: 409 Conflict döner.

```
        else if (exception is DomainRuleViolationException)
        {
            statusCode = StatusCodes.Status400BadRequest;
            title = "The request violates a domain rule.";
        }
```

Diğer domain kural ihlalleri (ör. geçersiz durum geçişi) istemcinin hatalı isteğidir: 400 Bad Request.

```
        else
        {
            return false;
        }
```

Domain hatası değilse false döner; hata bir sonraki handler'a devredilir.

```
        _logger.LogInformation(
            "Business rule violated for {RequestPath}: {Reason}",
            httpContext.Request.Path,
            exception.Message);
```

Structured log: yol ve sebep ayrı alanlar olarak yazılır. Beklenen bir kullanıcı durumu olduğu için seviye Information.

```
        ProblemDetails problemDetails = new ProblemDetails
        {
            Status = statusCode,
            Title = title,
            Detail = exception.Message
        };
```

Standart RFC `ProblemDetails` nesnesi; domain mesajı kullanıcıya anlamlı olduğu için `Detail` olarak gösterilir.

```
        await ProblemDetailsWriter.WriteAsync(httpContext, problemDetails, cancellationToken);
```

Ortak yazıcı yanıta `traceId` ekler; böylece kullanıcıdaki kod log satırıyla eşleştirilebilir.

```
        return true;
```

Hata işlendi; zincir burada durur.

```
    }
}
```

API `todo.API/ExceptionHandling/ApplicationExceptionHandler.cs`

> Application katmanındaki kimlik doğrulama ve kayıt hatalarını (401, 409) ProblemDetails yanıtına çevirir. Zincirde Domain handler'dan sonra, Database handler'dan önce çalışır.

```
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using TodoApi.Application.Exceptions;
```

Handler arayüzü, `ProblemDetails` ve Application katmanı exception tipleri için using'ler.

```
namespace TodoApi.API.ExceptionHandling;
```

Dosya kapsamlı namespace bildirimi.

ƒ ApplicationExceptionHandler — Giriş başarısızlığı ve kayıtlı e-posta hatalarını HTTP yanıtına çeviren handler.

```
public sealed class ApplicationExceptionHandler : IExceptionHandler
```

`IExceptionHandler` uygulayan `sealed` handler sınıfı.

```
{
```

```
    private readonly ILogger<ApplicationExceptionHandler> _logger;
```

Bu sınıf için logger alanı.

ƒ ApplicationExceptionHandler — Constructor; logger'ı DI'dan alır.

```
    public ApplicationExceptionHandler(ILogger<ApplicationExceptionHandler> logger)
    {
        _logger = logger;
    }
```

Logger constructor injection ile alınır ve saklanır.

ƒ TryHandleAsync — Uygulama hatasıysa 401/409 yanıtı yazar, değilse false döner.

```
    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext,
        Exception exception,
        CancellationToken cancellationToken)
```

Ortak handler imzası; true işlendi, false sıradakine geç demektir.

```
    {
```

```
        int statusCode;
        string title;
```

Hata türüne göre doldurulacak durum kodu ve başlık değişkenleri.

```
        if (exception is AuthenticationFailedException)
        {
            statusCode = StatusCodes.Status401Unauthorized;
            title = "Authentication failed.";
```

Yanlış e-posta/şifre veya geçersiz token: 401 Unauthorized döner.

```
            _logger.LogWarning("Authentication failed for {RequestPath}.", httpCon
text.Request.Path);
```

Warning seviyesinde loglanır çünkü tekrarlayan başarısız girişler saldırı işareti olabilir; kimin denediği loglanmaz.

```
        }
        else if (exception is EmailAlreadyRegisteredException)
        {
            statusCode = StatusCodes.Status409Conflict;
            title = "The email address is already registered.";
```

Kayıtta e-posta zaten kullanılıyorsa 409 Conflict döner.

```
            _logger.LogInformation("Registration rejected because the email addres
s is already in use.");
```

Log satırına e-posta adresi yazılmaz; kişisel veri loglara sızmasın diye sadece olay kaydedilir.

```
        }
        else
        {
            return false;
        }
```

Bu iki tipten biri değilse false; hata Database handler'a geçer.

```
        ProblemDetails problemDetails = new ProblemDetails
        {
            Status = statusCode,
            Title = title,
            Detail = exception.Message
        };
```

Durum kodu, başlık ve exception mesajı ile standart `ProblemDetails` nesnesi oluşturulur.

```
        await ProblemDetailsWriter.WriteAsync(httpContext, problemDetails, cancell
ationToken);
```

Ortak yazıcıyla `traceId` eklenmiş JSON yanıt gönderilir.

```
        return true;
```

Hata işlendi; zincir durur.

```
    }
}
```

API todo.API/ExceptionHandling/DatabaseExceptionHandler.cs

> Veritabanı hatalarını HTTP yanıtına çevirir: tekillik ihlali 409, eşzamanlılık çakışması 409 (özel type ile), erişilemeyen veritabanı 503. İç detaylar kullanıcıya sızdırılmaz.

```
using System.Data.Common;
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using TodoApi.Application.Exceptions;
```

`DbException` (tüm ADO.NET veritabanı hatalarının atası), handler arayüzü, `ProblemDetails` ve Application exception tipleri için using'ler.

```
namespace TodoApi.API.ExceptionHandling;
```

Dosya kapsamlı namespace bildirimi.

ƒ DatabaseExceptionHandler — Veritabanı kaynaklı hataları 409 veya 503 ProblemDetails yanıtına çeviren handler.

```
public sealed class DatabaseExceptionHandler : IExceptionHandler
```

Zincirde dördüncü sıradaki `sealed` handler; Validation, Domain ve Application handler'ları hatayı almadıysa çalışır.

```
{
    public const string ConcurrencyConflictProblemType = "urn:todo:problem:concurr
ency-conflict";
```

Eşzamanlılık çakışmasının `type` değeri. Ön yüz bu sabit URN ile 'veri değişti, yenile' durumunu diğer 409'lardan ayırır.

```
    private readonly ILogger<DatabaseExceptionHandler> _logger;
```

Bu sınıf için logger alanı.

ƒ DatabaseExceptionHandler — Constructor; logger'ı DI'dan alır.

```
    public DatabaseExceptionHandler(ILogger<DatabaseExceptionHandler> logger)
    {
        _logger = logger;
    }
```

Logger constructor injection ile alınıp saklanır.

ƒ TryHandleAsync — Hata veritabanı kaynaklıysa uygun yanıtı yazar, değilse false döner.

```
    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext,
        Exception exception,
        CancellationToken cancellationToken)
    {
        ProblemDetails problemDetails;
```

Ortak handler imzası; false dönerse son çare Unhandled handler çalışır.

Her dalda farklı içerikle doldurulacak yanıt nesnesi; tek bir yazma çağrısı sonda yapılır.

```
        if (exception is ConflictException)
        {
            _logger.LogWarning(
                exception,
                "A database uniqueness rule rejected a write for {RequestPath}.",
                httpContext.Request.Path);
```

Unique index ihlali (`ConflictException`): Warning olarak loglanır, exception da eklenir ki SQL detayı logda görünsün.

```
            problemDetails = new ProblemDetails
            {
                Status = StatusCodes.Status409Conflict,
                Title = "The record conflicts with existing data.",
                Detail = exception.Message
            };
```

Kullanıcıya 409 Conflict döner; mesaj Infrastructure'da yazılmış genel bir metindir, SQL detayı içermez.

```
        }
```

```
        else if (exception is ConcurrencyConflictException)
        {
            _logger.LogWarning(
                "A concurrent request changed or deleted the record first for {RequestPath}.",
                httpContext.Request.Path);
```

Başka bir istek kaydı önce değiştirmiş/silmiş (EF Core concurrency token). Beklenen bir yarış durumu olduğu için Warning loglanır.

```
            problemDetails = new ProblemDetails
            {
                Type = ConcurrencyConflictProblemType,
                Status = StatusCodes.Status409Conflict,
                Title = "The record was changed by another request.",
                Detail = "The record was changed or deleted by another request. Reload and try again."
            };
        }
```

409 yanıtına özel `Type` eklenir; `Detail` kullanıcıya sayfayı yenileyip tekrar denemesini söyler.

```
        else if (exception is DataAccessException || ContainsDatabaseException(exception))
        {
            _logger.LogError(
                exception,
                "A database error occurred for {RequestPath}.",
                httpContext.Request.Path);
```

Infrastructure'ın sardığı `DataAccessException` ya da iç zincirde ham bir `DbException` varsa veritabanı erişim hatası sayılır.

Gerçek bir altyapı sorunu olduğu için Error seviyesinde, tüm exception detayıyla loglanır.

```
            problemDetails = new ProblemDetails
            {
                Status = StatusCodes.Status503ServiceUnavailable,
                Title = "The service is temporarily unavailable.",
                Detail = "Please try again in a moment."
            };
        }
```

Kullanıcıya 503 Service Unavailable ve genel bir mesaj döner; bağlantı dizesi veya SQL hatası asla sızdırılmaz.

```
        else
        {
            return false;
        }
```

Veritabanıyla ilgisi yoksa false; hata Unhandled handler'a kalır.

```
        await ProblemDetailsWriter.WriteAsync(httpContext, problemDetails, cancellationToken);
```

Seçilen `problemDetails` ortak yazıcıyla `traceId` eklenerek gönderilir.

```
        return true;
    }
```

Hata işlendi; zincir durur.

ƒ ContainsDatabaseException — Exception ve iç exception zincirinde bir DbException olup olmadığını kontrol eder.

```
    private static bool ContainsDatabaseException(Exception exception)
    {
        Exception? current = exception;

        while (current != null)
        {
            if (current is DbException)
            {
                return true;
            }

            current = current.InnerException;
        }

        return false;
    }
}
```

`private static`: sadece bu sınıf içinde kullanılan, nesne durumuna ihtiyaç duymayan yardımcı metot.

`Exception?` null olabilen tiptir; döngüde zincirde gezinmek için başlangıç noktası.

Zincirdeki her halka `DbException` mı diye bakılır; bulunursa hemen true döner.

Bir sonraki iç exception'a geçilir; zincir bitince `current` null olur ve döngü sonlanır.

Zincirde veritabanı hatası bulunamadı.

API todo.API/ExceptionHandling/UnhandledExceptionHandler.cs

> Zincirin son halkası: önceki handler'ların tanımadığı her beklenmeyen hatayı Error olarak loglar ve kullanıcıya detay vermeden 500 yanıtı döner.

```
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;
```

Handler arayüzü ve `ProblemDetails` için using'ler.

```
namespace TodoApi.API.ExceptionHandling;
```

Dosya kapsamlı namespace bildirimi.

ƒ UnhandledExceptionHandler — Tanınmayan tüm hataları 500 yanıtına çeviren son çare handler.

```
public sealed class UnhandledExceptionHandler : IExceptionHandler
{
    private readonly ILogger<UnhandledExceptionHandler> _logger;
```

Program.cs'te en son kaydedilen `sealed` handler; her zaman true döndüğü için zincirin sonudur.

Bu sınıf için logger alanı.

ƒ UnhandledExceptionHandler — Constructor; logger'ı DI'dan alır.

```
    public UnhandledExceptionHandler(ILogger<UnhandledExceptionHandler> logger)
    {
        _logger = logger;
    }
```

Logger constructor injection ile alınıp saklanır.

ƒ TryHandleAsync — Hatayı Error olarak loglar ve 500 ProblemDetails yanıtı yazar.

```
15      public async ValueTask<bool> TryHandleAsync(
16          HttpContext httpContext,
17          Exception exception,
18          CancellationToken cancellationToken)
```

Ortak handler imzası; burada tip kontrolü yoktur çünkü kalan her hata bu handler'ındır.

```
19      {
```

```
20          _logger.LogError(
21              exception,
22              "An unexpected error occurred for {RequestMethod} {RequestPath}.",
23              httpContext.Request.Method,
24              httpContext.Request.Path);
```

Error seviyesinde, tam exception (stack trace dahil) ile HTTP metodu ve yol loglanır; geliştirici sorunu buradan bulur.

```
26          ProblemDetails problemDetails = new ProblemDetails
27          {
28              Status = StatusCodes.Status500InternalServerError,
29              Title = "An unexpected error occurred.",
```

500 Internal Server Error ve genel bir başlık; iç hata mesajı kullanıcıya gösterilmez.

```
30              Detail = $"Please contact support with this reference: {ProblemDetails
    Writer.GetTraceId(httpContext)}"
31          };
```

`$"..."` string interpolation: `Detail` içine traceId yazılır. Kullanıcı destek ekibine bu kodu verir, log satırı bununla bulunur.

```
33          await ProblemDetailsWriter.WriteAsync(httpContext, problemDetails, cancell
    ationToken);
```

Ortak yazıcıyla JSON yanıt gönderilir.

```
35          return true;
```

Her zaman true; hata kesin olarak işlenmiş olur.

```
36      }
37  }
```

API todo.API/ExceptionHandling/ProblemDetailsWriter.cs

Tüm exception handler'ların kullandığı ortak yardımcı: ProblemDetails'e instance ve traceId ekleyip yanıtı application/problem+json olarak yazar. Aynı traceId log satırlarında da bulunur.

```
1  using System.Diagnostics;
2  using System.Text.Json;
3  using Microsoft.AspNetCore.Mvc;
```

`Activity` (dağıtık izleme/trace), `JsonSerializer` ve `ProblemDetails` için using'ler.

```
5  namespace TodoApi.API.ExceptionHandling;
```

Dosya kapsamlı namespace bildirimi.

ƒ ProblemDetailsWriter — ProblemDetails yanıtlarını tek tip biçimde yazan statik yardımcı sınıf.

```
7  public static class ProblemDetailsWriter
```

`static class`: nesnesi oluşturulmaz, sadece statik metotlar içerir; her handler doğrudan çağırır.

```
8  {
```

```
9      private static readonly JsonSerializerOptions JsonOptions =
10         new JsonSerializerOptions(JsonSerializerDefaults.Web);
```

`JsonSerializerDefaults.Web` camelCase alan adları üretir. Statik ve readonly tutulur ki her istekte yeniden oluşturulmasın.

ƒ GetTraceId — İsteğin izleme kimliğini (traceId) döndürür.

```
12     public static string GetTraceId(HttpContext httpContext)
```

Hem handler'lar hem Program.cs'teki `AddProblemDetails` ayarı bu metodu kullanır; böylece kimlik her yerde aynıdır.

```
13     {
```

```
14         return Activity.Current?.TraceId.ToString() ?? httpContext.TraceIdentifie
    r;
```

`?.` null ise durur, `??` null ise sağdakini kullanır: aktif trace varsa onun ID'si, yoksa ASP.NET'in istek kimliği döner.

```
15     }
```

ƒ WriteAsync — ProblemDetails'i tamamlayıp HTTP yanıtına JSON olarak yazar.

```
17     public static async Task WriteAsync(
18         HttpContext httpContext,
19         ProblemDetails problemDetails,
20         CancellationToken cancellationToken)
```

Yanıtı yazacak asenkron metot; `CancellationToken` istemci bağlantıyı keserse yazmayı iptal etmeye yarar.

```
21     {
```

```
22         int statusCode = problemDetails.Status ?? StatusCodes.Status500InternalSer
    verError;
```

`Status` boşsa (`int?` null) güvenli varsayılan olarak 500 kullanılır.

```
24         problemDetails.Instance ??= httpContext.Request.Path.ToString();
```

`??=` yalnızca null ise atar: `Instance` boşsa hatanın oluştuğu istek yolu yazılır.

```
25         problemDetails.Extensions["traceId"] = GetTraceId(httpContext);
```

`Extensions` sözlüğüne `traceId` eklenir; JSON'da üst düzey alan olarak görünür ve log satırındaki kimlikle eşleşir.

```
27         httpContext.Response.StatusCode = statusCode;
28         httpContext.Response.ContentType = "application/problem+json";
```

HTTP durum kodu ve RFC 7807/9457'nin öngördüğü `application/problem+json` içerik tipi ayarlanır.

```
30         await JsonSerializer.SerializeAsync(
31             httpContext.Response.Body,
32             problemDetails,
33             problemDetails.GetType(),
34             JsonOptions,
35             cancellationToken);
```

Nesne doğrudan yanıt gövdesine serileştirilir. `GetType()` gerçek tipi verir ki `ValidationProblemDetails`'in `errors` alanı kaybolmasın.

```
36     }
37 }
```

Bölüm 4 — Hatırlatma maili (arka plan işi)

Hiçbir kullanıcı tıklamadan çalışan akış. Hangfire her 5 dakikada bir işi tetikler; iş, 24 saat içinde bitecek görevleri bulur ve sahiplerine mail atar. Reddedilen mailler 15 dakika sonra tekrar denenir, 3 denemeden sonra bırakılır; mail sunucusu çökerse iş durur ve görevlere ceza yazılmaz.

Akış adım adım

1. Açılışta: SMTP ayarlıysa iş Hangfire'a cron `*/5 * * * *` ile kaydedilir.
   `todo.API/BackgroundJobs/BackgroundJobsSetup.cs`
2. Hangfire işi çağırır; aynı anda iki kopya çalışmaz.
   `todo.API/BackgroundJobs/TodoReminderJob.cs`
3. Adaylar istenir, her biri için mail denenir.
   `todo.Application/Features/Reminders/SendDueTodoReminders/SendDueTodoRemindersUseCase.cs`
4. Aday sorgusu: süresi yaklaşan, tamamlanmamış, hatırlatılmamış, deneme hakkı olan görevler.
   `todo.Infrastructure/Persistence/TodoReminderRepository.cs`
5. Mail metni hazırlanır (İstanbul saati, HTML-encode).
   `todo.Application/Features/Reminders/SendDueTodoReminders/TodoReminderEmailComposer.cs`
6. Gmail SMTP ile gönderim; hatalar 'reddedildi' / 'sunucu yok' diye ayrılır.
   `todo.Infrastructure/Services/Email/MailKitEmailSender.cs`
7. Sonuç işlenir: `MarkReminderSent` ya da `RecordFailedReminderAttempt`.
   `todo.Domain/Entities/TodoItem.cs`
8. Sonuç loglanır; sunucu çöktüyse iş hata verir, Hangfire panelinde görünür.
   `todo.API/BackgroundJobs/TodoReminderJob.cs`

Bu bölümde açıklanan dosyalar (kullanım sırasıyla): TodoReminderJob.cs, SendDueTodoRemindersUseCase.cs, ReminderSettings.cs, ReminderRunResult.cs, ITodoReminderRepository.cs, TodoReminderRepository.cs, TodoReminderEmailComposer.cs, IEmailSender.cs, EmailMessage.cs, EmailRejectedException.cs, EmailServiceUnavailableException.cs, MailKitEmailSender.cs, SmtpSettings.cs

API `todo.API/BackgroundJobs/TodoReminderJob.cs`

> Hangfire'ın 5 dakikada bir çalıştırdığı tekrarlayan iş. Hatırlatma use case'ini çağırır, sonucu loglar ve mail sunucusu çökmüşse exception fırlatarak Hangfire panelinde başarısız görünür.

```
1  using Hangfire;
2  using TodoApi.Application.Features.Reminders.SendDueTodoReminders;
```

Hangfire attribute'ları ve Application katmanındaki hatırlatma use case'i için using'ler.

```
4  namespace TodoApi.API.BackgroundJobs;
```

Arka plan işleri API projesinde durur; iş mantığı ise Application'dadır.

ƒ TodoReminderJob — Hatırlatma use case'ini Hangfire'dan tetikleyen ince sarmalayıcı sınıf.

```
6  public sealed class TodoReminderJob
```

İnce bir sarmalayıcı: Hangfire'a bağımlı kod burada kalır, Application katmanı Hangfire'ı bilmez.

```
7  {
8      public const string RecurringJobId = "todo-due-date-reminders";
```

Tekrarlayan işin sabit kimliği; `BackgroundJobsSetup` bu kimlikle işi kaydeder veya siler.

```
10     private readonly SendDueTodoRemindersUseCase _sendDueTodoRemindersUseCase;
11     private readonly ILogger<TodoReminderJob> _logger;
```

Çağrılacak use case ve logger alanları.

ƒ TodoReminderJob — Constructor; use case ve logger'ı DI'dan alır.

```
13     public TodoReminderJob(
14         SendDueTodoRemindersUseCase sendDueTodoRemindersUseCase,
15         ILogger<TodoReminderJob> logger)
```

Hangfire işi DI üzerinden oluşturduğu için bağımlılıklar constructor'a otomatik gelir.

```
16     {
17         _sendDueTodoRemindersUseCase = sendDueTodoRemindersUseCase;
18         _logger = logger;
19     }
```

Gelen bağımlılıklar alanlara atanır.

ƒ RunAsync — Hatırlatma turunu çalıştırır, sonucu loglar, mail sunucusu yoksa hata fırlatır.

```
21     [DisableConcurrentExecution(timeoutInSeconds: 10 * 60)]
```

`[DisableConcurrentExecution]`: aynı iş aynı anda iki kez çalışmasın diye kilit alır; kilit için en fazla 10 dakika bekler.

```
22     [AutomaticRetry(Attempts = 3)]
```

`[AutomaticRetry(Attempts = 3)]`: iş exception fırlatırsa Hangfire en fazla 3 kez yeniden dener.

```
23     public async Task RunAsync(CancellationToken cancellationToken)
```

Hangfire'ın çağırdığı asenkron metot; `CancellationToken` sunucu kapanırken işi durdurmak için kullanılır.

```
24     {
```

```
25          ReminderRunResult result = await _sendDueTodoRemindersUseCase.ExecuteAsync
    (cancellationToken);
```
Asıl iş use case'te yapılır; sonuç gönderilen, başarısız ve aday sayılarını taşır.

```
27          _logger.LogInformation(
28              "Reminder run finished: {SentCount} sent, {FailedCount} failed, {Candi
    dateCount} candidates.",
29              result.SentCount,
30              result.Failures.Count,
31              result.CandidateCount);
```
Structured log: `{SentCount}` gibi yer tutucular Serilog'da ayrı alanlar olur, sonra filtrelenebilir. Tur özeti Information seviyesinde yazılır.

```
33          foreach (ReminderFailure failure in result.Failures)
```
Her başarısız hatırlatma için ayrı log satırı yazılır.

```
34          {
```

```
35              if (failure.WillBeRetried)
36              {
37                  _logger.LogWarning(
38                      failure.Exception,
39                      "The reminder for todo {TodoId} (user {UserId}) could not be s
    ent and will be retried later.",
40                      failure.TodoId,
41                      failure.OwnerUserId);
```
Tekrar denenecekse Warning yeterlidir; exception, todo ve kullanıcı kimliği loga eklenir.

```
42              }
```

```
43              else
44              {
45                  _logger.LogError(
46                      failure.Exception,
47                      "The reminder for todo {TodoId} (user {UserId}) could not be s
    ent and will not be retried.",
48                      failure.TodoId,
49                      failure.OwnerUserId);
```
Deneme hakkı bittiyse Error seviyesinde loglanır çünkü bu kullanıcı artık hatırlatma almayacak.

```
50              }
51          }
```

```
53          if (result.EmailServiceUnavailable)
```
Use case mail sunucusuna ulaşılamadığını bildirdiyse tur erken durmuş demektir.

```
54          {
```

```
55              throw new InvalidOperationException(
56                  "The mail server is unavailable, so the reminder run stopped earl
    y. " +
57                  "Remaining reminders will be sent on a later run.");
```
Exception fırlatılır ki Hangfire işi başarısız göstersin ve yeniden denesin; görevlerin deneme sayısı bu durumda artırılmaz.

```
58          }
59      }
60  }
```

Application `todo.Application/Features/Reminders/SendDueTodoReminders/SendDueTodoRemindersUseCase.cs`

> Hatırlatma mailinin asıl iş mantığı: süresi 24 saat içinde dolacak görevleri bulur, her biri için mail gönderir, başarı/başarısızlığı görev üzerine kaydeder ve tur özetini döner.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Threading;
4  using System.Threading.Tasks;
```
Temel .NET tipleri: `TimeProvider`, koleksiyonlar, iptal token'ı ve `Task` için using'ler.

```
5  using TodoApi.Application.Abstractions.Email;
6  using TodoApi.Application.Abstractions.Persistence;
7  using TodoApi.Domain.Entities;
```
Mail gönderici soyutlaması (`IEmailSender`), hatırlatma deposu (`ITodoReminderRepository`) ve `TodoItem` entity'si için using'ler. Somut MailKit/EF Core sınıflarına bağımlılık yok.

```
9  namespace TodoApi.Application.Features.Reminders.SendDueTodoReminders;
```
Feature klasörü yapısı: hatırlatma özelliğine ait her şey bu namespace'te toplanır.

ƒ SendDueTodoRemindersUseCase — Vakti yaklaşan görevler için hatırlatma maili gönderen use case sınıfı.

```
11 public class SendDueTodoRemindersUseCase
```
Application katmanındaki use case; Hangfire işi bu sınıfı çağırır.

```
12 {
```

```
13     private readonly ITodoReminderRepository _todoReminderRepository;
14     private readonly IEmailSender _emailSender;
15     private readonly ReminderSettings _reminderSettings;
16     private readonly TimeProvider _timeProvider;
```
Depo, mail gönderici, ayarlar ve saat sağlayıcı alanları. `TimeProvider` sayesinde testlerde zaman sahte bir saatle kontrol edilebilir.

ƒ SendDueTodoRemindersUseCase — Constructor; bağımlılıkları null kontrolüyle alır ve saklar.

```
18     public SendDueTodoRemindersUseCase(
19         ITodoReminderRepository todoReminderRepository,
20         IEmailSender emailSender,
21         ReminderSettings reminderSettings,
22         TimeProvider timeProvider)
```
Dört bağımlılık DI container tarafından verilir.

```
23     {
```

```
        if (todoReminderRepository == null)
        {
            throw new ArgumentNullException(nameof(todoReminderRepository));
        }

        if (emailSender == null)
        {
            throw new ArgumentNullException(nameof(emailSender));
        }

        if (reminderSettings == null)
        {
            throw new ArgumentNullException(nameof(reminderSettings));
        }

        if (timeProvider == null)
        {
            throw new ArgumentNullException(nameof(timeProvider));
        }
```

Her bağımlılık null mı diye kontrol edilir; hatalı kurulum çalışma anında değil nesne oluşurken hemen fark edilsin. `nameof` parametre adını güvenle yazar.

```
        _todoReminderRepository = todoReminderRepository;
        _emailSender = emailSender;
        _reminderSettings = reminderSettings;
        _timeProvider = timeProvider;
    }
```

Kontrolden geçen bağımlılıklar alanlara atanır.

ƒ ExecuteAsync — Bir hatırlatma turunu çalıştırır ve gönderim sonuçlarını ReminderRunResult olarak döner.

```
    public async Task<ReminderRunResult> ExecuteAsync(CancellationToken cancellationToken = default)
    {
        DateTime nowUtc = _timeProvider.GetUtcNow().UtcDateTime;

        DateTime dueBeforeUtc = nowUtc.Add(_reminderSettings.LeadTime);

        DateTime lastFailedAttemptBeforeUtc = nowUtc.Subtract(_reminderSettings.RetryDelay);

        IReadOnlyList<TodoReminderCandidate> candidates =
            await _todoReminderRepository.GetTodosDueForReminderAsync(
                nowUtc,
                dueBeforeUtc,
                lastFailedAttemptBeforeUtc,
                _reminderSettings.MaximumAttempts,
                _reminderSettings.BatchSize,
                cancellationToken);

        int sentCount = 0;
        bool emailServiceUnavailable = false;
        List<ReminderFailure> failures = new List<ReminderFailure>();

        foreach (TodoReminderCandidate candidate in candidates)
        {
            cancellationToken.ThrowIfCancellationRequested();

            TodoItem todo = candidate.Todo;
            bool needsReminder = todo.NeedsReminder(nowUtc, _reminderSettings.LeadTime);
            bool canRetry = todo.CanRetryReminder(nowUtc, _reminderSettings.MaximumAttempts, _reminderSettings.RetryDelay);

            if (needsReminder == false || canRetry == false)
            {
                continue;
            }

            try
            {
                EmailMessage message = TodoReminderEmailComposer.Compose(
                    candidate,
                    _reminderSettings.DisplayTimeZone);

                await _emailSender.SendAsync(message, cancellationToken);
            }
            catch (EmailServiceUnavailableException exception)
            {
                failures.Add(new ReminderFailure(todo.Id, candidate.OwnerUserId, exception, willBeRetried: true));
                emailServiceUnavailable = true;
                break;
            }
            catch (Exception exception) when (exception is not OperationCanceledException)
            {
```

`Task<ReminderRunResult>` asenkron sonuç döner. `= default` token verilmezse iptal edilemeyen boş token kullanılır.

Şu anki zaman UTC olarak alınır; veritabanındaki tüm tarihler UTC tutulur.

Bu andan önce bitecek görevler aday sayılır: şimdi + öncelik süresi (varsayılan 24 saat).

Son başarısız denemesi bu andan önce olan görevler tekrar denenebilir: şimdi - bekleme süresi (15 dakika).

Depo filtreyi veritabanında uygular: tamamlanmamış, hatırlatılmamış, deneme sayısı 3'ten az, bitişe göre sıralı en fazla 50 görev getirir.

Tur boyunca tutulacak sayaç, mail sunucusu durumu bayrağı ve hata listesi.

Adaylar tek tek işlenir.

Uygulama kapanıyorsa döngü burada `OperationCanceledException` fırlatarak durur; iptal başarısızlık sayılmaz.

Domain metotları kuralları bir kez daha kontrol eder: görev hâlâ hatırlatma gerektiriyor mu ve yeniden deneme hakkı var mı.

`continue` döngünün bu adımını atlayıp sıradaki adaya geçer; koşul tutmayan görev için mail gönderilmez.

`try` bloğu: e-posta içeriği oluşturulur, saatler ayarlardaki Europe/Istanbul dilimine göre gösterilir.

Mail soyutlama üzerinden gönderilir; hata olursa aşağıdaki `catch` blokları devreye girer.

Mail sunucusu tamamen erişilemezse bu özel exception yakalanır.

Hata tekrar denenecek diye kaydedilir, bayrak açılır ve `break` ile tur tamamen durur; görevin deneme sayısı artırılmaz çünkü suç görevde değil.

`catch (X) when (...)` filtresi: iptal dışındaki tüm hataları yakalar, ör. alıcıyı reddeden `EmailRejectedException`. İptal böylece hata sayılmaz.

98	`                todo.RecordFailedReminderAttempt(_timeProvider.GetUtcNow().UtcDateTime);`	Görev üzerine başarısız deneme kaydedilir (sayaç artar, zaman yazılır) ve hemen veritabanına kaydedilir.
99	`                await _todoReminderRepository.SaveChangesAsync(cancellationToken);`	
101	`                bool willBeRetried = todo.ReminderAttemptCount < _reminderSettings.MaximumAttempts;`	Deneme sayısı hâlâ sınırın altındaysa tekrar denenecektir; hata bu bilgiyle listeye eklenir.
102	`                failures.Add(new ReminderFailure(todo.Id, candidate.OwnerUserId, exception, willBeRetried));`	
103	`                continue;`	Bir görevin hatası diğerlerini engellemez; döngü sıradaki adayla devam eder.
104	`            }`	
106	`            todo.MarkReminderSent(_timeProvider.GetUtcNow().UtcDateTime);`	Mail başarıyla gittiyse görev 'hatırlatıldı' olarak işaretlenir, kaydedilir ve sayaç artar. Her görev ayrı kaydedilir ki sonraki hata önceki başarıları silmesin.
107	`            await _todoReminderRepository.SaveChangesAsync(cancellationToken);`	
108	`            sentCount++;`	
109	`        }`	
111	`        return new ReminderRunResult(candidates.Count, sentCount, failures, emailServiceUnavailable);`	Aday sayısı, gönderilen sayısı, hatalar ve mail sunucusu durumu tek bir sonuç nesnesinde döner.
112	`    }`	
113	`}`	

Application `todo.Application/Features/Reminders/SendDueTodoReminders/ReminderSettings.cs`

> Hatırlatma özelliğinin ayarlarını (açık/kapalı, 24 saat öncelik, 5 dakikalık cron, batch 50, saat dilimi, 3 deneme, 15 dakika bekleme) doğrulanmış ve değiştirilemez biçimde taşır. Infrastructure appsettings'ten okuyup oluşturur.

1	`using System;`	`TimeSpan`, `TimeZoneInfo` ve exception tipleri için `System` namespace'i.
3	`namespace TodoApi.Application.Features.Reminders.SendDueTodoReminders;`	Hatırlatma feature'ının namespace'i.
	ƒ ReminderSettings — Hatırlatma ayarlarını değiştirilemez şekilde tutan ayar sınıfı.	
5	`public sealed class ReminderSettings`	`sealed` ayar sınıfı; oluşturulduktan sonra değerleri değiştirilemez.
6	`{`	
7	`    private readonly bool _isEnabled;`	`readonly` alanlar sadece constructor'da atanabilir; böylece ayarlar uygulama çalışırken yanlışlıkla değişmez.
8	`    private readonly TimeSpan _leadTime;`	
9	`    private readonly string _schedule;`	
10	`    private readonly int _batchSize;`	
11	`    private readonly TimeZoneInfo _displayTimeZone;`	
12	`    private readonly int _maximumAttempts;`	
13	`    private readonly TimeSpan _retryDelay;`	
	ƒ ReminderSettings — Ayarları alır, geçersiz değerleri reddeder ve alanlara atar.	
15	`    public ReminderSettings(`	Yedi ayar değeri constructor parametresi olarak alınır.
16	`        bool isEnabled,`	
17	`        TimeSpan leadTime,`	
18	`        string schedule,`	
19	`        int batchSize,`	
20	`        TimeZoneInfo displayTimeZone,`	
21	`        int maximumAttempts,`	
22	`        TimeSpan retryDelay)`	
23	`    {`	
24	`        if (leadTime <= TimeSpan.Zero)`	Öncelik süresi sıfır veya negatifse hiçbir görev aday olamaz; `ArgumentOutOfRangeException` ile uygulama açılışta durur.
25	`        {`	
26	`            throw new ArgumentOutOfRangeException(nameof(leadTime), "The reminder lead time must be positive.");`	
27	`        }`	
29	`        if (string.IsNullOrWhiteSpace(schedule))`	Cron ifadesi (ör. her 5 dakika `*/5 * * * *`) boş olamaz.
30	`        {`	
31	`            throw new ArgumentException("A reminder schedule is required.", nameof(schedule));`	
32	`        }`	
34	`        if (batchSize <= 0)`	Bir turda işlenecek görev sayısı pozitif olmalı.
35	`        {`	
36	`            throw new ArgumentOutOfRangeException(nameof(batchSize), "The batch size must be positive.");`	
37	`        }`	
39	`        if (maximumAttempts <= 0)`	Maksimum deneme sayısı pozitif olmalı.
40	`        {`	
41	`            throw new ArgumentOutOfRangeException(nameof(maximumAttempts), "The maximum attempt count must be positive.");`	
42	`        }`	
44	`        if (retryDelay <= TimeSpan.Zero)`	Yeniden deneme bekleme süresi pozitif olmalı.
45	`        {`	
46	`            throw new ArgumentOutOfRangeException(nameof(retryDelay), "The retry delay must be positive.");`	
47	`        }`	
49	`        _isEnabled = isEnabled;`	Geçerli değerler alanlara atanır; cron ifadesinin baş ve son boşlukları `Trim` ile temizlenir.
50	`        _leadTime = leadTime;`	
51	`        _schedule = schedule.Trim();`	
52	`        _batchSize = batchSize;`	
53	`        _displayTimeZone = displayTimeZone ?? throw new ArgumentNullException(nameof(displayTimeZone));`	`?? throw` kısayolu: saat dilimi null ise hemen `ArgumentNullException` fırlatılır, değilse atanır.

```
        _maximumAttempts = maximumAttempts;
        _retryDelay = retryDelay;
    }
```

Deneme sınırı ve bekleme süresi alanlara atanır.

ƒ IsEnabled — Hatırlatmaların açık olup olmadığını döner.

```
    public bool IsEnabled
    {
        get
        {
            return _isEnabled;
        }
    }
```

Sadece `get` içeren property: dışarıdan okunur ama değiştirilemez.

ƒ LeadTime — Bitişten ne kadar önce hatırlatılacağını döner.

```
    public TimeSpan LeadTime
    {
        get
        {
            return _leadTime;
        }
    }
```

Öncelik süresi; varsayılan 24 saat.

ƒ Schedule — Hangfire cron ifadesini döner.

```
    public string Schedule
    {
        get
        {
            return _schedule;
        }
    }
```

Hangfire'ın işi ne sıklıkla çalıştıracağını belirten cron ifadesi.

ƒ BatchSize — Bir turda işlenecek maksimum görev sayısını döner.

```
    public int BatchSize
    {
        get
        {
            return _batchSize;
        }
    }
```

Tek turda en fazla kaç görev işleneceği; varsayılan 50, mail sunucusunu boğmamak için sınırlıdır.

ƒ DisplayTimeZone — Maildeki saatlerin gösterileceği saat dilimini döner.

```
    public TimeZoneInfo DisplayTimeZone
    {
        get
        {
            return _displayTimeZone;
        }
    }
```

Maildeki tarih bu dilime göre yazılır; varsayılan Europe/Istanbul.

ƒ MaximumAttempts — Bir görev için maksimum gönderim denemesini döner.

```
    public int MaximumAttempts
    {
        get
        {
            return _maximumAttempts;
        }
    }
```

Bir görev için en fazla kaç kez mail denenir; varsayılan 3.

ƒ RetryDelay — Başarısız denemeden sonra beklenecek süreyi döner.

```
    public TimeSpan RetryDelay
    {
        get
        {
            return _retryDelay;
        }
    }
}
```

Başarısız denemeden sonra tekrar denemeden önce beklenecek süre; varsayılan 15 dakika.

Application `todo.Application/Features/Reminders/SendDueTodoReminders/ReminderRunResult.cs`

> Bir hatırlatma turunun özetini (aday, gönderilen, hatalar, mail sunucusu durumu) ve her başarısız gönderimin detayını taşıyan sonuç sınıfları. Hangfire işi bunlara bakarak log yazar.

```
using System;
using System.Collections.Generic;
```

`Exception` ve `IReadOnlyList` için using'ler.

```
namespace TodoApi.Application.Features.Reminders.SendDueTodoReminders;
```

Hatırlatma feature'ının namespace'i.

ƒ ReminderRunResult — Bir hatırlatma turunun sonucunu taşıyan değiştirilemez sınıf.

```
public sealed class ReminderRunResult
{
    private readonly int _candidateCount;
    private readonly int _sentCount;
    private readonly IReadOnlyList<ReminderFailure> _failures;
    private readonly bool _emailServiceUnavailable;
```

Use case'ten Hangfire işine dönen sonuç nesnesi.

`readonly` alanlar: sonuç oluşturulduktan sonra değiştirilemez. `IReadOnlyList` dışarıdan listeye eleman eklenmesini engeller.

ƒ ReminderRunResult — Tur sonuç değerlerini alıp alanlara atar.

```
13      public ReminderRunResult(
14          int candidateCount,
15          int sentCount,
16          IReadOnlyList<ReminderFailure> failures,
17          bool emailServiceUnavailable)
```

Dört sonuç değeri constructor'dan alınır.

```
18      {
```

```
19          _candidateCount = candidateCount;
20          _sentCount = sentCount;
```

Aday ve gönderilen sayıları atanır.

```
21          _failures = failures ?? throw new ArgumentNullException(nameof(failures));
```

`?? throw`: hata listesi null ise exception fırlatılır; boş liste kabul edilir, null kabul edilmez.

```
22          _emailServiceUnavailable = emailServiceUnavailable;
```

Mail sunucusunun erişilemez olup olmadığı bilgisi atanır.

```
23      }
```

ƒ CandidateCount — Turda bulunan aday görev sayısını döner.

```
25      public int CandidateCount
26      {
27          get
28          {
29              return _candidateCount;
30          }
31      }
```

Veritabanından gelen aday görev sayısı; sadece okunabilir property.

ƒ SentCount — Başarıyla gönderilen mail sayısını döner.

```
33      public int SentCount
34      {
35          get
36          {
37              return _sentCount;
38          }
39      }
```

Başarıyla gönderilen hatırlatma sayısı.

ƒ Failures — Başarısız gönderimlerin listesini döner.

```
41      public IReadOnlyList<ReminderFailure> Failures
42      {
43          get
44          {
45              return _failures;
46          }
47      }
```

Her başarısız gönderim için bir `ReminderFailure` içeren salt okunur liste.

ƒ EmailServiceUnavailable — Mail sunucusunun erişilemez olup olmadığını döner.

```
49      public bool EmailServiceUnavailable
50      {
51          get
52          {
53              return _emailServiceUnavailable;
54          }
55      }
```

true ise tur erken durmuştur; Hangfire işi bu durumda exception fırlatır.

```
56  }
```

ƒ ReminderFailure — Tek bir başarısız hatırlatma gönderiminin detayını tutan sınıf.

```
58  public sealed class ReminderFailure
```

Aynı dosyadaki ikinci sınıf: bir görev için başarısız gönderimin bilgisi.

```
59  {
```

```
60      private readonly int _todoId;
61      private readonly int _ownerUserId;
62      private readonly Exception _exception;
63      private readonly bool _willBeRetried;
```

Görev kimliği, sahibi, oluşan hata ve tekrar denenip denenmeyeceği bilgisi.

ƒ ReminderFailure — Hata bilgilerini alıp alanlara atar.

```
65      public ReminderFailure(int todoId, int ownerUserId, Exception exception, bool
    willBeRetried)
```

Dört değer constructor'dan alınır.

```
66      {
```

```
67          _todoId = todoId;
68          _ownerUserId = ownerUserId;
```

Görev ve kullanıcı kimlikleri atanır; log satırında hangi görevin başarısız olduğunu göstermek için kullanılır.

```
69          _exception = exception ?? throw new ArgumentNullException(nameof(exceptio
    n));
```

Exception zorunludur; null ise `ArgumentNullException` fırlatılır.

```
70          _willBeRetried = willBeRetried;
```

Tekrar denenecek mi bilgisi; log seviyesini (Warning veya Error) belirler.

```
71      }
```

ƒ TodoId — Başarısız görevin kimliğini döner.

```
73      public int TodoId
74      {
75          get
76          {
77              return _todoId;
78          }
79      }
```

Görev kimliği; sadece okunabilir property.

ƒ OwnerUserId — Görev sahibinin kullanıcı kimliğini döner.

```
    public int OwnerUserId
    {
        get
        {
            return _ownerUserId;
        }
    }
```

Görev sahibinin kimliği; e-posta yerine kimlik loglanır ki kişisel veri loga düşmesin.

ƒ Exception — Gönderim sırasında oluşan hatayı döner.

```
    public Exception Exception
    {
        get
        {
            return _exception;
        }
    }
```

Gönderim sırasında oluşan asıl hata; loga stack trace ile yazılır.

ƒ WillBeRetried — Gönderimin sonraki turda tekrar denenip denenmeyeceğini döner.

```
    public bool WillBeRetried
    {
        get
        {
            return _willBeRetried;
        }
    }
}
```

true ise sonraki turlarda tekrar denenecek, false ise deneme hakkı bitmiştir.

Application `todo.Application/Abstractions/Persistence/ITodoReminderRepository.cs`

> Hatırlatma işinin veritabanından ihtiyaç duyduğu sözleşmeyi (port) ve sorgu sonucunu taşıyan `TodoReminderCandidate` sınıfını tanımlar. Application katmanı EF Core'u bilmez, yalnızca bu arayüzü kullanır.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
using TodoApi.Domain.Entities;
```

Gerekli isim alanları: `Task`, liste tipleri, iptal belirteci ve `TodoItem` varlığının bulunduğu Domain katmanı.

```
namespace TodoApi.Application.Abstractions.Persistence;
```

Dosyanın isim alanı; kalıcılık (persistence) portları Application/Abstractions altında toplanır.

ƒ interface ITodoReminderRepository — Hatırlatma adaylarını getiren ve değişiklikleri kaydeden veritabanı portu.

```
public interface ITodoReminderRepository
```

Arayüz yalnızca sözleşmedir; gerçek kodu Infrastructure'daki `TodoReminderRepository` yazar. Buna bağımlılığın tersine çevrilmesi (dependency inversion) denir.

```
{
```

ƒ GetTodosDueForReminderAsync — Hatırlatma maili gönderilmesi gereken todo'ları sahip bilgileriyle birlikte döndürür.

```
    Task<IReadOnlyList<TodoReminderCandidate>> GetTodosDueForReminderAsync(
        DateTime nowUtc,
        DateTime dueBeforeUtc,
        DateTime lastFailedAttemptBeforeUtc,
        int maximumAttempts,
        int maxCount,
        CancellationToken cancellationToken);
```

Şimdiki zaman, son tarih sınırı, yeniden deneme sınırı, en fazla deneme sayısı ve parti büyüklüğü dışarıdan verilir; böylece kurallar use case'te kalır, sorgu sadece uygular.

ƒ SaveChangesAsync — Todo üzerinde yapılan hatırlatma işaretlerini veritabanına yazar.

```
    Task SaveChangesAsync(CancellationToken cancellationToken);
```

Mail gönderildikten veya başarısız olduktan sonra todo'ya yazılan durum bilgisini kalıcı hale getirmek için çağrılır.

```
}
```

ƒ class TodoReminderCandidate — Bir todo'yu, liste adını ve sahibinin iletişim bilgilerini bir arada taşıyan sonuç nesnesi.

```
public sealed class TodoReminderCandidate
```

`sealed` sınıf kalıtılamaz. Sorgu sonucunu use case'e taşıyan, sadece okunabilen basit bir veri nesnesidir.

```
{
    private readonly TodoItem _todo;
    private readonly string _listName;
    private readonly int _ownerUserId;
    private readonly string _ownerEmail;
    private readonly string _ownerDisplayName;
```

`readonly` alanlar yalnızca constructor içinde atanabilir; nesne oluştuktan sonra değişmez (immutable).

ƒ TodoReminderCandidate (constructor) — Aday nesnesini gelen değerlerle güvenli şekilde oluşturur.

```
    public TodoReminderCandidate(
        TodoItem todo,
        string listName,
        int ownerUserId,
        string ownerEmail,
        string ownerDisplayName)
    {
```

Constructor; todo, liste adı ve sahip bilgilerini parametre olarak alır.

```
        _todo = todo ?? throw new ArgumentNullException(nameof(todo));
        _listName = listName ?? string.Empty;
        _ownerUserId = ownerUserId;
        _ownerEmail = ownerEmail ?? throw new ArgumentNullException(nameof(ownerEmail));
        _ownerDisplayName = ownerDisplayName ?? string.Empty;
    }
    public TodoItem Todo
    {
        get
        {
            return _todo;
        }
    }
    public string ListName
    {
        get
        {
            return _listName;
        }
    }
    public int OwnerUserId
    {
        get
        {
            return _ownerUserId;
        }
    }
    public string OwnerEmail
    {
        get
        {
            return _ownerEmail;
        }
    }
    public string OwnerDisplayName
    {
        get
        {
            return _ownerDisplayName;
        }
    }
}
```

`??` null ise sağdakini kullanır. Todo ve e-posta zorunlu olduğu için null gelirse `ArgumentNullException` atılır; isimler boşsa boş metne çevrilir.

`Todo` özelliği: takip edilen `TodoItem` varlığını döndürür; use case hatırlatma alanlarını bunun üzerinden günceller.

`ListName` özelliği: todo'nun bulunduğu listenin adı, mail içeriğinde gösterilir.

`OwnerUserId` özelliği: liste sahibinin kullanıcı kimliği, loglama ve takip için kullanılır.

`OwnerEmail` özelliği: hatırlatma mailinin gönderileceği adres.

`OwnerDisplayName` özelliği: mailde kullanıcıya hitap etmek için görünen ad.

Infrastructure `todo.Infrastructure/Persistence/TodoReminderRepository.cs`

> `ITodoReminderRepository` portunun EF Core ile yazılmış gerçek uygulamasıdır. Hatırlatma zamanı gelen todo'ları liste ve sahip bilgileriyle tek sorguda çeker.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using TodoApi.Application.Abstractions.Persistence;
using TodoApi.Domain.Entities;
```

Gerekli isim alanları: LINQ, EF Core (`ToListAsync` için), uygulanacak port ve Domain varlıkları.

```
namespace TodoApi.Infrastructure.Persistence;
```

Infrastructure katmanının kalıcılık isim alanı; veritabanı ile konuşan kodlar burada durur.

ƒ class TodoReminderRepository — Hatırlatma adaylarını EF Core sorgusuyla veritabanından getiren repository.

```
public sealed class TodoReminderRepository : ITodoReminderRepository
{
    private readonly TodoDbContext _todoDbContext;
```

`: ITodoReminderRepository` bu sınıfın Application'daki sözleşmeyi uyguladığını söyler; DI bu arayüz istendiğinde bu sınıfı verir.

Veritabanına erişim için kullanılan EF Core `DbContext` nesnesi.

ƒ TodoReminderRepository (constructor) — DI'dan gelen `TodoDbContext` nesnesini alır ve saklar.

```
    public TodoReminderRepository(TodoDbContext todoDbContext)
    {
        _todoDbContext = todoDbContext ?? throw new ArgumentNullException(nameof(todoDbContext));
    }
```

DbContext constructor ile enjekte edilir; null gelirse hemen `ArgumentNullException` atılarak hata erken yakalanır.

ƒ GetTodosDueForReminderAsync — Kurallara uyan todo'ları son tarihe göre sıralayıp en fazla `maxCount` kadarını döndürür.

```
    public async Task<IReadOnlyList<TodoReminderCandidate>> GetTodosDueForReminderAsync(
        DateTime nowUtc,
        DateTime dueBeforeUtc,
        DateTime lastFailedAttemptBeforeUtc,
        int maximumAttempts,
        int maxCount,
        CancellationToken cancellationToken)
    {
```

Portun metodunu uygular; `async` olduğu için veritabanı beklenirken thread serbest kalır.

```
        var rows = await (
            from todo in _todoDbContext.TodoItems
            join todoList in _todoDbContext.TodoLists on todo.TodoListId equals todoList.Id
            join owner in _todoDbContext.Users on todoList.OwnerUserId equals owner.Id
            where todo.IsCompleted == false
                && todo.ReminderSentAtUtc == null
                && todo.DueAtUtc != null
                && todo.DueAtUtc > nowUtc
                && todo.DueAtUtc <= dueBeforeUtc
                && todo.ReminderAttemptCount < maximumAttempts
                && (todo.ReminderLastAttemptAtUtc == null || todo.ReminderLastAttemptAtUtc <= lastFailedAttemptBeforeUtc)
            orderby todo.DueAtUtc
            select new
            {
                Todo = todo,
                ListName = todoList.Name,
                OwnerUserId = owner.Id,
                OwnerEmail = owner.Email,
                OwnerDisplayName = owner.DisplayName
            })
            .Take(maxCount)
            .ToListAsync(cancellationToken);

        List<TodoReminderCandidate> candidates = new List<TodoReminderCandidate>();

        foreach (var row in rows)
        {
            candidates.Add(new TodoReminderCandidate(
                row.Todo,
                row.ListName,
                row.OwnerUserId,
                row.OwnerEmail,
                row.OwnerDisplayName));
        }

        return candidates;
    }
```

LINQ sorgu sözdizimi: `from` ana tabloyu seçer, `join ... equals` ile todo listeye, liste de sahibine bağlanır. EF Core bunu tek bir SQL JOIN'e çevirir.

`where` filtreleri: tamamlanmamış, daha önce hatırlatılmamış, son tarihi olan ve son tarihi şimdi ile öncelik süresi sınırı arasında olan todo'lar.

Deneme sayısı en fazla deneme sınırının altında olmalı; böylece sürekli reddedilen bir todo sonsuza kadar denenmez.

Hiç denenmemiş ya da son başarısız denemesi yeniden deneme bekleme süresinden eski olanlar alınır; bu sayede başarısızlık sonrası hemen tekrar denenmez.

`orderby` son tarihi en yakın olanları öne alır; parti sınırlıysa en acil todo'lar önce işlenir.

`select new { ... }` isimsiz (anonymous) bir tip oluşturur; sadece gereken alanlar seçilir, tipin adını `var` ile derleyici bilir.

`Take(maxCount)` sonucu parti büyüklüğüyle sınırlar (SQL'de TOP/LIMIT). `ToListAsync` sorguyu çalıştırıp listeyi asenkron getirir.

Sonuçların dönüştürüleceği boş `TodoReminderCandidate` listesi oluşturulur.

Her isimsiz satır, Application katmanının tanıdığı `TodoReminderCandidate` nesnesine çevrilir; anonim tip metot dışına taşınamadığı için bu gereklidir.

Hazırlanan aday listesi `IReadOnlyList` olarak döndürülür.

ƒ SaveChangesAsync — Takip edilen todo değişikliklerini veritabanına kaydeder.

```
    public async Task SaveChangesAsync(CancellationToken cancellationToken)
    {
        await _todoDbContext.SaveChangesAsync(cancellationToken);
    }
}
```

Sorgudan gelen todo'lar EF Core tarafından takip edilir; use case alanları değiştirince bu çağrı onları UPDATE olarak yazar.

Application `todo.Application/Features/Reminders/SendDueTodoReminders/TodoReminderEmailComposer.cs`

> Bir hatırlatma adayından Türkçe e-posta (konu, düz metin ve HTML gövde) üretir. Saat İstanbul dilimine çevrilir, kullanıcı verisi HTML'e eklenmeden önce encode edilir.

```
using System;
using System.Globalization;
using System.Net;
using System.Text;
```

`CultureInfo` (Türkçe biçimlendirme), `WebUtility` (HTML encode) ve `StringBuilder` için using'ler.

```
using TodoApi.Application.Abstractions.Email;
using TodoApi.Application.Abstractions.Persistence;
```

`EmailMessage` ve `TodoReminderCandidate` tipleri için Application soyutlama namespace'leri.

```
namespace TodoApi.Application.Features.Reminders.SendDueTodoReminders;
```

Hatırlatma feature'ının namespace'i.

ƒ TodoReminderEmailComposer — Hatırlatma e-postasının içeriğini oluşturan statik yardımcı sınıf.

```
public static class TodoReminderEmailComposer
```

`static class`: durum tutmaz, sadece girdiden çıktı üretir; bu yüzden kolayca test edilir.

```
{
    private static readonly CultureInfo TurkishCulture = CultureInfo.GetCultureInfo("tr-TR");
```

Türkçe kültür bir kez oluşturulup saklanır; tarih biçimlendirmede kullanılır.

ƒ Compose — Adaydan konu, düz metin ve HTML içeren EmailMessage oluşturur.

```
    public static EmailMessage Compose(TodoReminderCandidate candidate, TimeZoneInfo displayTimeZone)
    {
```

Aday ve gösterim saat dilimi alınır, gönderilmeye hazır `EmailMessage` döner.

```
        if (candidate == null)
        {
            throw new ArgumentNullException(nameof(candidate));
        }

        if (displayTimeZone == null)
        {
            throw new ArgumentNullException(nameof(displayTimeZone));
        }
```

Parametreler null mı kontrol edilir; hatalı çağrı hemen anlaşılır bir hatayla durur.

```
        DateTime dueAtUtc = candidate.Todo.DueAtUtc
            ?? throw new InvalidOperationException("A reminder can only be composed for a todo with a due date.");
```

`DueAtUtc` null olabilir (`DateTime?`); `?? throw` ile bitiş tarihi yoksa mail oluşturulmaz, çünkü bu durumda hatırlatma anlamsızdır.

```
        DateTime dueAtLocal = TimeZoneInfo.ConvertTimeFromUtc(dueAtUtc, displayTimeZone);
```

`TimeZoneInfo.ConvertTimeFromUtc` UTC zamanı verilen saat dilimine (Europe/Istanbul) çevirir; kullanıcı kendi yerel saatini görür.

```
        string dueAtText = dueAtLocal.ToString("dd.MM.yyyy HH:mm", TurkishCulture);
```

Tarih Türkçe kültürle gün.ay.yıl saat:dakika biçiminde metne çevrilir.

```
        string title = RemoveLineBreaks(candidate.Todo.Title);
        string subject = $"Hatırlatma: \"{title}\" görevinin bitişi yaklaşıyor";
```

Başlıktaki satır sonları temizlenir; konu satırında satır sonu olursa mail başlığı bozulabilir (header injection). Sonra konu metni oluşturulur.

```
        StringBuilder plainText = new StringBuilder();
```

`StringBuilder` çok parçalı metni verimli birleştirir; HTML desteklemeyen istemciler için düz metin gövde.

```
        plainText.AppendLine($"Merhaba {candidate.OwnerDisplayName},");
        plainText.AppendLine();
        plainText.AppendLine($"\"{candidate.ListName}\" listendeki \"{title}\" görevinin bitiş zamanı yaklaşıyor.");
        plainText.AppendLine($"Bitiş: {dueAtText}");
```

Selamlama, liste ve görev adı, bitiş zamanı satırları eklenir. `AppendLine()` boş satır ekler.

```
        if (string.IsNullOrWhiteSpace(candidate.Todo.Description) == false)
        {
            plainText.AppendLine($"Açıklama: {candidate.Todo.Description}");
        }
```

Açıklama doluysa ayrı satır olarak eklenir; boşsa atlanır.

```
        plainText.AppendLine();
        plainText.AppendLine("Bu e-posta Todo uygulaması tarafından otomatik olarak gönderildi.");
```

Boş satır ve otomatik gönderildiğini belirten alt bilgi eklenir.

```
        StringBuilder html = new StringBuilder();
        html.Append("<div style=\"font-family:Segoe UI,Arial,sans-serif;color:#26362e;line-height:1.6\">");
```

HTML gövde başlar; mail istemcileri harici CSS'i desteklemediği için stiller satır içi yazılır.

```
        html.Append($"<p>Merhaba {Encode(candidate.OwnerDisplayName)},</p>");
        html.Append($"<p><strong>{Encode(candidate.ListName)}</strong> listendeki ");
        html.Append($"<strong>{Encode(title)}</strong> görevinin bitiş zamanı yaklaşıyor.</p>");
        html.Append($"<p>Bitiş: <strong>{Encode(dueAtText)}</strong></p>");
```

Kullanıcıdan gelen her değer `Encode` ile HTML'e eklenir; görev adına `<script>` yazılsa bile etiket olarak çalışmaz, metin olarak görünür.

```
        if (string.IsNullOrWhiteSpace(candidate.Todo.Description) == false)
        {
            html.Append($"<p style=\"color:#536158\">{Encode(candidate.Todo.Description)}</p>");
        }
```

Açıklama doluysa o da encode edilerek gri renkli paragraf olarak eklenir.

```
        html.Append("<p style=\"color:#8c9790;font-size:12px\">Bu e-posta Todo uygulaması tarafından otomatik olarak gönderildi.</p>");
        html.Append("</div>");
```

Sabit alt bilgi ve kapanış etiketi; bunlar kullanıcı verisi olmadığından encode gerekmez.

```
        EmailMessage message = new EmailMessage(
            candidate.OwnerEmail,
            candidate.OwnerDisplayName,
            subject,
            plainText.ToString(),
            html.ToString());
```

Alıcı adresi, alıcı adı, konu, düz metin ve HTML gövde ile `EmailMessage` oluşturulur.

```
        return message;
    }
```

Hazır mesaj use case'e döner, gönderimi `IEmailSender` yapar.

ƒ Encode — Metni HTML'e güvenle eklenecek şekilde encode eder.

```
    private static string Encode(string value)
    {
        return WebUtility.HtmlEncode(value);
    }
```

Kısa isimli yardımcı; HTML oluşturan satırları okunur tutar.

`WebUtility.HtmlEncode` `<`, `>`, `&`, `"` gibi karakterleri `<` gibi güvenli karşılıklara çevirir; HTML enjeksiyonunu önler.

ƒ RemoveLineBreaks — Metindeki satır sonu karakterlerini boşlukla değiştirir.

```
    private static string RemoveLineBreaks(string value)
    {
        return value.Replace("\r", " ").Replace("\n", " ");
    }
}
```

Konu satırında kullanılacak metni tek satıra indiren yardımcı.

`\r` ve `\n` karakterleri boşlukla değiştirilir.

Application `todo.Application/Abstractions/Email/IEmailSender.cs`

> Mail gönderme işinin portudur. Application katmanı hangi kütüphanenin (MailKit) kullanıldığını bilmeden bu arayüz üzerinden mail gönderir.

```
using System.Threading;
using System.Threading.Tasks;
```

`CancellationToken` ve `Task` tipleri için gerekli isim alanları.

```
namespace TodoApi.Application.Abstractions.Email;
```

E-posta ile ilgili portların toplandığı Application isim alanı.

ƒ interface IEmailSender — Mail gönderme yeteneğini soyutlayan port; uygulaması Infrastructure'daki `MailKitEmailSender`.

```
6  public interface IEmailSender
```

Arayüz sayesinde testlerde sahte bir gönderici kullanılabilir, kütüphane değişse de Application kodu değişmez.

```
7  {
8      bool IsConfigured { get; }
```

`IsConfigured`: SMTP ayarları eksiksiz mi? Eksikse hatırlatma işi mail göndermeye hiç kalkışmaz.

ƒ SendAsync — Verilen `EmailMessage` nesnesini asenkron olarak gönderir.

```
10     Task SendAsync(EmailMessage message, CancellationToken cancellationToken);
```

Hata durumunda `EmailRejectedException` veya `EmailServiceUnavailableException` atabilir; iptal belirteci uzun bağlantıların durdurulmasını sağlar.

```
11 }
```

Application `todo.Application/Abstractions/Email/EmailMessage.cs`

> Gönderilecek bir mailin alıcı, konu ve gövde bilgilerini taşıyan değişmez (immutable) veri nesnesidir. Use case bunu oluşturur, `IEmailSender` gönderir.

```
1  using System;
```

`ArgumentException` için `System` isim alanı.

```
3  namespace TodoApi.Application.Abstractions.Email;
```

E-posta portlarıyla aynı Application isim alanı.

ƒ class EmailMessage — Alıcı, konu, düz metin ve HTML gövdeyi taşıyan mail modeli.

```
5  public sealed class EmailMessage
```

`sealed` sınıf; MailKit'e özgü hiçbir tip içermez, böylece Application katmanı kütüphaneden bağımsız kalır.

```
6  {
7      private readonly string _toAddress;
8      private readonly string _toName;
9      private readonly string _subject;
10     private readonly string _plainTextBody;
11     private readonly string _htmlBody;
```

Değerler `readonly` alanlarda tutulur; oluşturulduktan sonra değiştirilemez.

ƒ EmailMessage (constructor) — Mail bilgilerini doğrulayarak nesneyi oluşturur.

```
13     public EmailMessage(
14         string toAddress,
15         string toName,
16         string subject,
17         string plainTextBody,
18         string htmlBody)
```

Alıcı adresi, alıcı adı, konu, düz metin gövde ve HTML gövde parametre olarak alınır.

```
19     {
20         if (string.IsNullOrWhiteSpace(toAddress))
21         {
22             throw new ArgumentException("A recipient address is required.", nameof
(toAddress));
23         }
```

Alıcı adresi boş ya da sadece boşluksa nesne hiç oluşturulmaz; geçersiz mail baştan engellenir.

```
25         if (string.IsNullOrWhiteSpace(subject))
26         {
27             throw new ArgumentException("A subject is required.", nameof(subjec
t));
28         }
```

Konu da zorunludur; boşsa `ArgumentException` atılır ve hangi parametrenin hatalı olduğu `nameof` ile belirtilir.

```
30         _toAddress = toAddress;
31         _toName = toName ?? string.Empty;
32         _subject = subject;
33         _plainTextBody = plainTextBody ?? string.Empty;
34         _htmlBody = htmlBody ?? string.Empty;
```

Değerler alanlara atanır; isteğe bağlı olanlar null gelirse boş metne çevrilir, böylece sonradan null kontrolü gerekmez.

```
35     }
37     public string ToAddress
38     {
39         get
40         {
41             return _toAddress;
42         }
43     }
```

`ToAddress` özelliği: alıcının e-posta adresi.

```
45     public string ToName
46     {
47         get
48         {
49             return _toName;
50         }
51     }
```

`ToName` özelliği: alıcının görünen adı.

```
53     public string Subject
54     {
55         get
56         {
57             return _subject;
58         }
59     }
```

`Subject` özelliği: mailin konu satırı.

```
61     public string PlainTextBody
62     {
63         get
64         {
65             return _plainTextBody;
66         }
67     }
```

`PlainTextBody` özelliği: HTML göstermeyen istemciler için düz metin gövde.

```
    public string HtmlBody
    {
        get
        {
            return _htmlBody;
        }
    }
}
```

`HtmlBody` özelliği: biçimlendirilmiş HTML gövde.

Application `todo.Application/Abstractions/Email/EmailRejectedException.cs`

Mail sunucusunun belirli bir mesajı veya alıcıyı reddettiğini anlatan özel hata tipidir. Hatırlatma işi bunu yakalayınca sadece o todo için başarısız deneme sayar.

```
using System;
```

`Exception` temel sınıfı için `System` isim alanı.

```
namespace TodoApi.Application.Abstractions.Email;
```

Hata tipi port ile aynı Application isim alanında durur; böylece use case MailKit'i bilmeden yakalayabilir.

ƒ class EmailRejectedException — Sunucu bu mesajı/alıcıyı kabul etmediğinde atılan hata.

```
public sealed class EmailRejectedException : Exception
```

`: Exception` ile standart hata sınıfından türetilir. Ayrı tip olması, use case'in bu durumu diğer hatalardan ayırmasını sağlar.

```
{
```

ƒ EmailRejectedException (constructor) — Mesajı ve asıl (iç) hatayı alıp temel sınıfa iletir.

```
    public EmailRejectedException(string message, Exception innerException)
        : base(message, innerException)
```

`: base(...)` temel `Exception` constructor'ını çağırır; `innerException` asıl SMTP hatasını loglar için saklar.

```
    {
    }
}
```

Application `todo.Application/Abstractions/Email/EmailServiceUnavailableException.cs`

Mail sunucusuna ulaşılamadığını veya oturumun reddedildiğini anlatan özel hata tipidir. Hatırlatma işi bunu görünce çalışmayı durdurur ve todo'ları cezalandırmaz.

```
using System;
```

`Exception` temel sınıfı için `System` isim alanı.

```
namespace TodoApi.Application.Abstractions.Email;
```

E-posta portlarıyla aynı Application isim alanı.

ƒ class EmailServiceUnavailableException — Bağlantı, kimlik doğrulama veya protokol hatasında atılan hata.

```
public sealed class EmailServiceUnavailableException : Exception
```

Sorun todo'da değil sunucudadır; bu yüzden deneme sayısı artırılmaz, çalışma bir sonraki tura bırakılır.

```
{
```

ƒ EmailServiceUnavailableException (constructor) — Mesajı ve asıl hatayı alıp temel `Exception` sınıfına iletir.

```
    public EmailServiceUnavailableException(string message, Exception innerExcepti
on)
        : base(message, innerException)
```

`: base(message, innerException)` asıl hatayı içinde saklar; loglarda kök neden görülebilir.

```
    {
    }
}
```

Infrastructure `todo.Infrastructure/Services/Email/MailKitEmailSender.cs`

`IEmailSender` portunun MailKit kütüphanesiyle yazılmış uygulamasıdır. Maili SMTP (ör. smtp.gmail.com:587, StartTls) üzerinden gönderir ve SMTP hatalarını Application'ın anladığı iki hata tipine çevirir.

```
using System;
using System.Threading;
using System.Threading.Tasks;
```

Temel .NET isim alanları: `TimeSpan`, iptal belirteci ve `Task`.

```
using MailKit.Net.Smtp;
using MailKit.Security;
using MimeKit;
```

MailKit (SMTP istemcisi ve güvenlik seçenekleri) ile MimeKit (mail mesajı oluşturma) kütüphaneleri.

```
using TodoApi.Application.Abstractions.Email;
```

Uygulanacak port, `EmailMessage` ve özel hata tipleri Application katmanından gelir.

```
namespace TodoApi.Infrastructure.Services.Email;
```

Dış servislerle konuşan kodların bulunduğu Infrastructure isim alanı.

ƒ class MailKitEmailSender — MailKit ile SMTP üzerinden mail gönderen `IEmailSender` uygulaması.

```
public sealed class MailKitEmailSender : IEmailSender
```

MailKit'e bağımlılık sadece bu sınıfta durur; kütüphane değişirse yalnızca bu dosya değişir.

```
{
```

```
    private static readonly TimeSpan SendTimeout = TimeSpan.FromSeconds(30);
```

`static readonly` sabit: sunucu yanıt vermezse işlem en fazla 30 saniye bekler, iş sonsuza kadar asılı kalmaz.

```
    private readonly SmtpSettings _smtpSettings;
```

Host, port, güvenlik ve kimlik bilgilerini içeren SMTP ayarları.

ƒ MailKitEmailSender (constructor) — DI'dan gelen `SmtpSettings` nesnesini alır ve saklar.

```
    public MailKitEmailSender(SmtpSettings smtpSettings)
    {
        _smtpSettings = smtpSettings ?? throw new ArgumentNullException(nameof(smtpSettings));
    }
```

Ayarlar null gelirse hemen hata atılır; uygulama yanlış kurulumla sessizce çalışmaz.

```
    public bool IsConfigured
    {
        get
        {
            return _smtpSettings.IsConfigured;
        }
    }
```

`IsConfigured` özelliği: kararı `SmtpSettings` nesnesine bırakır; ayarlar eksikse hatırlatma işi gönderim yapmaz.

ƒ SendAsync — Mesajı MIME formatına çevirip SMTP sunucusuna bağlanarak gönderir, hataları özel tiplere çevirir.

```
    public async Task SendAsync(EmailMessage message, CancellationToken cancellationToken)
```

Portun gönderme metodunu uygular; `async` ile ağ beklenirken thread bloklanmaz.

```
    {
```

```
        if (message == null)
        {
            throw new ArgumentNullException(nameof(message));
        }
```

Mesaj null ise programlama hatasıdır; `ArgumentNullException` ile hemen bildirilir.

```
        if (_smtpSettings.IsConfigured == false)
        {
            throw new InvalidOperationException("SMTP settings are not configured.");
        }
```

Ayarlar eksikse göndermeye çalışmak anlamsızdır; `InvalidOperationException` atılır.

```
        MimeMessage mimeMessage = new MimeMessage();
        mimeMessage.From.Add(new MailboxAddress(_smtpSettings.FromName, _smtpSettings.FromAddress));
        mimeMessage.To.Add(new MailboxAddress(message.ToName, message.ToAddress));
        mimeMessage.Subject = message.Subject;
```

MimeKit ile mail nesnesi kurulur: gönderen, alıcı ve konu eklenir. `MailboxAddress` ad ile adresi birlikte tutar.

```
        BodyBuilder bodyBuilder = new BodyBuilder
        {
            TextBody = message.PlainTextBody,
            HtmlBody = message.HtmlBody
        };
```

`BodyBuilder` nesne başlatıcı (`{ ... }`) ile kurulur; hem düz metin hem HTML gövde verilir, istemci uygun olanı gösterir.

```
        mimeMessage.Body = bodyBuilder.ToMessageBody();
```

Gövde MIME parçalarına çevrilip mesaja eklenir. Mesaj try bloğundan önce kurulur; bozuk bir adres sunucu kesintisi sanılmaz.

```
        using SmtpClient smtpClient = new SmtpClient();
        smtpClient.Timeout = (int)SendTimeout.TotalMilliseconds;
```

`using` bildirimi: metot bitince `smtpClient` otomatik kapatılıp (Dispose) kaynaklar serbest bırakılır. Zaman aşımı milisaniye olarak ayarlanır.

```
        try
        {
            await smtpClient.ConnectAsync(
                _smtpSettings.Host,
                _smtpSettings.Port,
                ToSecureSocketOptions(_smtpSettings.Security),
                cancellationToken);
```

Sunucuya host ve port ile bağlanılır; güvenlik türü (StartTls vb.) MailKit'in anladığı değere çevrilir.

```
            if (_smtpSettings.Username != null && _smtpSettings.Password != null)
            {
                await smtpClient.AuthenticateAsync(
                    _smtpSettings.Username,
                    _smtpSettings.Password,
                    cancellationToken);
            }
```

Kullanıcı adı ve şifre varsa giriş yapılır; Gmail'de şifre user secrets'taki uygulama şifresidir. Yerel test sunucusunda giriş gerekmeyebilir.

```
            await smtpClient.SendAsync(mimeMessage, cancellationToken);
            await smtpClient.DisconnectAsync(quit: true, cancellationToken);
```

Mesaj gönderilir, ardından `quit: true` ile sunucuya QUIT komutu yollanarak bağlantı düzgünce kapatılır.

```
        }
        catch (SmtpCommandException exception) when (IsRejectionOfThisMessage(exception))
        {
            throw new EmailRejectedException(
                $"The mail server rejected the message ({(int)exception.StatusCode}).",
                exception);
        }
```

`catch (X) when (...)` sadece koşul doğruysa yakalar. Sunucu alıcıyı veya mesajı reddettiyse `EmailRejectedException` atılır; o todo için başarısız deneme sayılır.

```
        catch (Exception exception) when (cancellationToken.IsCancellationRequested == false)
        {
            throw new EmailServiceUnavailableException(
                "The mail server could not be reached or refused the session.",
                exception);
        }
```

Diğer tüm bağlantı, kimlik doğrulama ve protokol hataları `EmailServiceUnavailableException` olur; çalışma durur, todo cezalandırılmaz. İptal istendiyse sarılmaz, iptal olduğu gibi yükselir.

```
    }
```

ƒ IsRejectionOfThisMessage — SMTP hatasının bu mesaja/alıcıya özel bir ret olup olmadığını söyler.

```
    private static bool IsRejectionOfThisMessage(SmtpCommandException exception)
    {
        return exception.ErrorCode == SmtpErrorCode.RecipientNotAccepted
            || exception.ErrorCode == SmtpErrorCode.MessageNotAccepted;
    }
```

Sadece `RecipientNotAccepted` ve `MessageNotAccepted` kodları mesaja özel kabul edilir; diğer komut hataları sunucu sorunu sayılır.

ƒ ToSecureSocketOptions — Projenin `SmtpSecurity` enum değerini MailKit'in `SecureSocketOptions` değerine çevirir.

```
96      private static SecureSocketOptions ToSecureSocketOptions(SmtpSecurity securit
        y)
97      {
98          if (security == SmtpSecurity.SslOnConnect)
99          {
100             return SecureSocketOptions.SslOnConnect;
101         }
103         if (security == SmtpSecurity.None)
104         {
105             return SecureSocketOptions.None;
106         }
108         return SecureSocketOptions.StartTls;
109     }
110 }
```

Kendi enum'umuzu kullanmak, ayarların MailKit tipine bağımlı olmamasını sağlar.

`SslOnConnect`: bağlantı baştan şifreli açılır (genelde port 465).

`None`: şifresiz bağlantı; `SmtpSettings` bunu sadece yerel makinedeki sunucu için izin verir.

Varsayılan `StartTls`: düz bağlanıp sonra şifrelemeye geçer; Gmail'in 587 portu böyle çalışır.

Infrastructure `todo.Infrastructure/Services/Email/SmtpSettings.cs`

> SMTP bağlantı ayarlarını (`Email:Smtp` bölümü) okuyup doğrulayan ve değişmez şekilde tutan sınıftır. `MailKitEmailSender` bu ayarlarla sunucuya bağlanır.

```
1  using System;
2  using Microsoft.Extensions.Configuration;
4  namespace TodoApi.Infrastructure.Services.Email;
```

`System` ve ayar dosyalarını/user secrets'ı okumak için `IConfiguration` isim alanı.

E-posta altyapısının Infrastructure isim alanı.

ƒ enum SmtpSecurity — SMTP bağlantısının şifreleme türlerini listeler.

```
6  public enum SmtpSecurity
7  {
8      StartTls = 1,
10     SslOnConnect = 2,
12     None = 3
13 }
```

`enum` adlandırılmış sabit sayılar kümesidir; metin yerine tip güvenli seçenek kullanılır.

`StartTls = 1`: düz bağlanıp şifrelemeye geçer; Gmail 587 için varsayılan seçenek.

`SslOnConnect = 2`: bağlantı en baştan SSL ile şifrelenir.

`None = 3`: şifresiz; sadece yerel test sunucusu için izinlidir. Değerler 1'den başlar, böylece 0 geçersiz kalır.

ƒ class SmtpSettings — SMTP ayarlarını doğrulayıp tutan değişmez ayar sınıfı.

```
15 public sealed class SmtpSettings
16 {
17     public const string SectionName = "Email:Smtp";
19     private readonly string _host;
20     private readonly int _port;
21     private readonly SmtpSecurity _security;
22     private readonly string? _username;
23     private readonly string? _password;
24     private readonly string _fromAddress;
25     private readonly string _fromName;
```

`sealed` sınıf; ayarlar oluşturulurken doğrulanır, sonra değiştirilemez.

`const` derleme zamanı sabiti: ayarların okunduğu yapılandırma bölümünün adı `Email:Smtp`.

Ayar değerleri `readonly` alanlarda tutulur. `string?` bu değerin null olabileceğini belirtir (kullanıcı adı ve şifre isteğe bağlı).

ƒ SmtpSettings (constructor) — Ayar değerlerini temizleyip atar ve geçersiz kombinasyonları reddeder.

```
27     public SmtpSettings(
28         string host,
29         int port,
30         SmtpSecurity security,
31         string? username,
32         string? password,
33         string? fromAddress,
34         string? fromName)
35     {
36         _host = host?.Trim() ?? string.Empty;
37         _port = port;
38         _security = security;
39         _username = string.IsNullOrWhiteSpace(username) ? null : username.Trim();
40         _password = string.IsNullOrEmpty(password) ? null : password;
42         _fromAddress = string.IsNullOrWhiteSpace(fromAddress) ? _username ?? strin
   g.Empty : fromAddress.Trim();
43         _fromName = string.IsNullOrWhiteSpace(fromName) ? "Todo" : fromName.Trim
   ();
45         if (port <= 0 || port > 65535)
46         {
47             throw new InvalidOperationException("The Email:Smtp:Port setting must
   be a valid port number.");
48         }
```

Host, port, güvenlik türü, kimlik bilgileri ve gönderen bilgileri parametre olarak alınır.

Host baştaki/sondaki boşluklardan temizlenir (`?.Trim()` null ise atlar), port ve güvenlik türü olduğu gibi saklanır.

`koşul ? a : b` üçlü operatörü: boş kullanıcı adı ve şifre null yapılır. Şifre kırpılmaz çünkü boşluk şifrenin parçası olabilir.

Gönderen adresi verilmemişse kullanıcı adı kullanılır (Gmail'de aynıdır); gönderen adı boşsa varsayılan olarak "Todo" olur.

Port 1-65535 aralığında değilse uygulama açılırken hata verir; yanlış ayar erken fark edilir.

```
        if (security == SmtpSecurity.None && IsLocalHost(_host) == false && _host.Length > 0)
        {
            throw new InvalidOperationException(
                "Unencrypted SMTP is only allowed for a server on this machine. Use StartTls or SslOnConnect.");
        }
    }
```

Şifresiz SMTP yalnızca yerel makinedeki sunucuya izinlidir; uzak sunucuya şifre ağda açık gitmesin diye reddedilir.

```
    public string Host
    {
        get
        {
            return _host;
        }
    }
```

`Host` özelliği: SMTP sunucusunun adresi, ör. smtp.gmail.com.

```
    public int Port
    {
        get
        {
            return _port;
        }
    }
```

`Port` özelliği: bağlanılacak port numarası, ör. 587.

```
    public SmtpSecurity Security
    {
        get
        {
            return _security;
        }
    }
```

`Security` özelliği: seçilen şifreleme türü.

```
    public string? Username
    {
        get
        {
            return _username;
        }
    }
```

`Username` özelliği: SMTP kullanıcı adı, yoksa null.

```
    public string? Password
    {
        get
        {
            return _password;
        }
    }
```

`Password` özelliği: SMTP şifresi (Gmail uygulama şifresi), user secrets'tan gelir; yoksa null.

```
    public string FromAddress
    {
        get
        {
            return _fromAddress;
        }
    }
```

`FromAddress` özelliği: mailin gönderen adresi.

```
    public string FromName
    {
        get
        {
            return _fromName;
        }
    }
```

`FromName` özelliği: alıcının gördüğü gönderen adı.

```
    public bool IsConfigured
    {
        get
        {
            bool hasCredentialsIfNeeded = _username == null || _password != null;

            return _host.Length > 0 && _fromAddress.Length > 0 && hasCredentialsIfNeeded;
        }
    }
```

`IsConfigured`: host ve gönderen adresi dolu olmalı; kullanıcı adı varsa şifre de olmalı. Biri eksikse mail gönderimi atlanır.

ƒ FromConfiguration — `Email:Smtp` bölümünü okuyup doğrulanmış bir `SmtpSettings` nesnesi üretir.

```
    public static SmtpSettings FromConfiguration(IConfiguration configuration)
    {
        if (configuration == null)
        {
            throw new ArgumentNullException(nameof(configuration));
        }

        IConfigurationSection section = configuration.GetSection(SectionName);

        string portText = section["Port"] ?? "587";
        bool portIsNumber = int.TryParse(portText, out int port);
```

`static` fabrika metodu: nesne oluşturmadan sınıf üzerinden çağrılır, yapılandırmayı nesneye çevirir.

Yapılandırma nesnesi null ise `ArgumentNullException` atılır.

`GetSection` ile `Email:Smtp` bölümü alınır; appsettings, ortam değişkenleri ve user secrets birleşik okunur.

Port verilmemişse "587" varsayılır. `int.TryParse` metni sayıya çevirmeyi dener, hata atmaz; sonucu `out int port` değişkenine yazar.

```
        if (portIsNumber == false)
        {
            throw new InvalidOperationException("The Email:Smtp:Port setting must be a whole number.");
        }

        string securityText = section["Security"] ?? nameof(SmtpSecurity.StartTls);
        bool securityIsKnown = Enum.TryParse(securityText, ignoreCase: true, out SmtpSecurity security);

        if (securityIsKnown == false || Enum.IsDefined(typeof(SmtpSecurity), security) == false)
        {
            throw new InvalidOperationException(
                "The Email:Smtp:Security setting must be StartTls, SslOnConnect or None.");
        }

        return new SmtpSettings(
            section["Host"] ?? string.Empty,
            port,
            security,
            section["Username"],
            section["Password"],
            section["FromAddress"],
            section["FromName"]);
    }
```

Port metni sayı değilse anlaşılır bir mesajla hata atılır.

Güvenlik türü yoksa `StartTls` varsayılır. `Enum.TryParse` metni büyük/küçük harf duyarsız şekilde enum değerine çevirmeyi dener.

`Enum.IsDefined` de kontrol edilir; çünkü "7" gibi bir sayı da parse olur ama tanımlı bir seçenek değildir.

Okunan değerlerle yeni `SmtpSettings` oluşturulur; constructor kalan doğrulamaları yapar.

ƒ IsLocalHost — Host adının bu makineyi (localhost, 127.0.0.1, ::1) gösterip göstermediğini söyler.

```
    private static bool IsLocalHost(string host)
    {
        return host.Equals("localhost", StringComparison.OrdinalIgnoreCase)
            || host == "127.0.0.1"
            || host == "::1";
    }
}
```

Şifresiz SMTP'ye izin verip vermemeye karar vermek için kullanılır; `OrdinalIgnoreCase` harf büyüklüğünü önemsemez.

Ek — Aynı kalıpla çalışan diğer dosyalar

Bu dosyalar yukarıdaki akışlarda anlatılan kalıpların aynısını kullanır (request → command → validator → use case → repository → DTO). Satır satır tekrar anlatmak yerine tek cümleyle özetlenmiştir.

API `todo.API/Controllers/TodoListsController.cs`	Liste CRUD uç noktalarını sunar; ince controller desenine uyar: isteği komuta çevirip use case'e verir, sonucu Ok/NotFound/CreatedAtAction'a dönüştürür.
API `todo.API/Controllers/AdminUsersController.cs`	Tüm kullanıcıları listeleyen admin uç noktası; AdminOnly politikasıyla korunur ve ince controller desenine göre işi GetUsersUseCase'e bırakır.
API `todo.API/Models/Requests/CreateTodoListRequest.cs`	Liste oluşturma isteğinin HTTP gövdesini temsil eden API modeli; request modeli ile Application komutunu ayırma desenini izler.
API `todo.API/Models/Requests/RenameTodoListRequest.cs`	Liste yeniden adlandırma isteğinin gövde modeli; liste kimliği rotadan gelir, controller ikisini birleştirip RenameTodoListCommand oluşturur.
Application `todo.Application/Features/TodoLists/CreateTodoList/CreateTodoListCommand.cs`	Yeni liste adını taşıyan değişmez (init) komut nesnesi; her özellik kendi klasöründe komut + validator + use case desenine uyar.
Application `todo.Application/Features/TodoLists/CreateTodoList/CreateTodoListCommandValidator.cs`	FluentValidation ile liste adını ortak TodoListName kuralıyla doğrular; doğrulama katmanını iş kurallarından ayırma desenini izler.
Application `todo.Application/Features/TodoLists/CreateTodoList/CreateTodoListUseCase.cs`	Komutu doğrular, aynı sahipte isim tekrarını DuplicateNameException ile engeller, listeyi kaydedip TodoListDto döndürür; standart use case akışı.
Application `todo.Application/Features/TodoLists/RenameTodoList/RenameTodoListCommand.cs`	Liste kimliği ve yeni adı taşıyan değişmez komut nesnesi; komut + validator + use case desenine uyar.
Application `todo.Application/Features/TodoLists/RenameTodoList/RenameTodoListCommandValidator.cs`	Liste kimliğini EntityId, adı TodoListName ortak kurallarıyla doğrulayan FluentValidation sınıfı; paylaşılan doğrulama kuralları desenini izler.
Application `todo.Application/Features/TodoLists/RenameTodoList/RenameTodoListUseCase.cs`	Doğrular, listeyi yükler, erişim politikasını uygular, isim tekrarını kontrol edip domain metoduyla yeniden adlandırır; erişim yoksa null (404) döner.
Application `todo.Application/Features/TodoLists/DeleteTodoList/DeleteTodoListUseCase.cs`	Listeyi yükleyip TodoListAccessPolicy ile erişimi kontrol eder ve siler; bulunamaz veya yetki yoksa false döner, controller 404 verir.
Application `todo.Application/Features/TodoLists/GetTodoListById/GetTodoListByIdUseCase.cs`	Tek listeyi getirir; başkasının listesi için null döndürerek varlığını gizler ve sonucu TodoListDto'ya çevirir (yükle-yetkilendir-DTO deseni).
Application `todo.Application/Features/TodoLists/GetTodoLists/GetTodoListsUseCase.cs`	Giriş yapan kullanıcının tüm listelerini sahibine göre çeker ve her birini TodoListDto'ya dönüştürür; okuma use case deseni.
Application `todo.Application/Features/Todos/UpdateTodo/UpdateTodoCommand.cs`	Todo güncellemesi için başlık, açıklama, son tarih ve öncelik taşıyan değişmez komut; komut + validator + use case desenine uyar.
Application `todo.Application/Features/Todos/UpdateTodo/UpdateTodoCommandValidator.cs`	Kimlikleri, başlığı, açıklamayı, son tarihi (TimeProvider ile) ve önceliği ortak FluentValidation kurallarıyla doğrular.
Application `todo.Application/Features/Todos/UpdateTodo/UpdateTodoUseCase.cs`	Doğrular, erişimi kontrol eder, todo'yu bulur, değişen son tarihin geçmişte olmadığını denetleyip domain metotlarıyla günceller ve TodoItemDto döndürür.
Application `todo.Application/Features/Todos/DeleteTodo/DeleteTodoUseCase.cs`	Listeyi yükleyip erişimi kontrol eder, todo'yu aggregate kökü TodoList.RemoveTodo ile siler; bulunamazsa false döner.
Application `todo.Application/Features/Todos/GetTodoById/GetTodoByIdUseCase.cs`	Liste üzerinden erişim politikası uygulanarak tek todo'yu bulur ve TodoItemDto döndürür; yoksa null ile 404'e yol açar.
Application `todo.Application/Features/Todos/GetTodosInList/GetTodosInListUseCase.cs`	Erişim izni olan listenin tüm todo'larını TodoItemDto listesine çevirir; yükle-yetkilendir-DTO okuma desenini izler.
Application `todo.Application/Features/Todos/SetTodoCompletionStatus/SetTodoCompletionStatusUseCase.cs`	Todo'yu domain metotları MarkCompleted/MarkIncomplete ile tamamlandı ya da tamamlanmadı yapar, kaydeder ve TodoItemDto döndürür.
Application `todo.Application/Features/Users/GetUsers/GetUsersUseCase.cs`	IUserRepository portundan tüm kullanıcıları alıp UserDto'ya çevirir; admin ekranı için basit okuma use case'i.

Application `todo.Application/Contracts/TodoListDto.cs`	Listenin kimlik, ad ve oluşturma zamanını dışarı taşıyan salt okunur DTO; domain varlığını API'ye doğrudan açmama desenini izler.
Infrastructure `todo.Infrastructure/Persistence/TodoDbContextDesignTimeFactory.cs`	EF Core migration araçlarının tasarım zamanında DbContext oluşturmasını sağlar; bağlantı dizesini appsettings veya ortam değişkenlerinden okur.
Web (Angular) `todo.Web/src/app/pages/admin/admin-users-page.ts`	Kullanıcıları listeleyen standalone Angular sayfası; durumu signal'larla tutar, 403 hatasında yetki mesajı gösterir, servis üzerinden API çağırır.
Web (Angular) `todo.Web/src/app/services/admin-api.service.ts`	HttpClient ile /admin/users uç noktasını çağıran root seviyesinde enjekte edilen servis; HTTP çağrılarını bileşenlerden ayırma desenini izler.